

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP

**MÔ HÌNH KHO DỮ LIỆU (DATA
WAREHOUSE) VỀ GIAO THÔNG PHỤC
VỤ CÔNG TÁC THỐNG KÊ, DỰ BÁO, HỖ
TRỢ RA QUYẾT ĐỊNH**

NGÀNH: KHOA HỌC MÁY TÍNH

CBHD1: PGS.TS. Trần Minh Quang

CBHD2: ThS. Bùi Tiến Đức

SVTH1: Đinh Nguyễn Việt Khiêm (2211565)

SVTH2: Nguyễn Thành Dũng (2210589)

SVTH3: Hồ Trần Ngọc Liêm (2211835)

TP. HỒ CHÍ MINH, THÁNG 05 NĂM 2026

Mục lục

Danh mục hình ảnh	8
Danh mục bảng biểu	10
1 Giới thiệu	11
1.1 Động cơ của đề tài	11
1.2 Mục tiêu	12
1.3 Phạm vi	13
1.4 Ý nghĩa	14
1.4.1 Ý nghĩa thực tiễn	14
1.4.2 Ý nghĩa khoa học	14
1.5 Đóng góp của đề tài	15
2 Quản lý dự án và phân công nhiệm vụ	17
2.1 Phương pháp quản lý dự án	17
2.2 Công cụ quản lý và Môi trường làm việc nhóm	17
2.3 Phân công vai trò và chi tiết nhiệm vụ	18
2.4 Tiến độ thực hiện đồ án	18
3 Kiến thức và công nghệ nền tảng	20
3.1 Nền tảng Kỹ thuật dữ liệu	20
3.1.1 Tổng quan về Kho dữ liệu (Data Warehouse) và OLAP	20
3.1.2 Lý thuyết về Cơ sở dữ liệu quan hệ và SQL	21
3.1.3 Hệ thống thông tin địa lý (GIS) và Dữ liệu không gian	21
3.1.4 Lý thuyết về Giao thông (Traffic Flow Theory)	21
3.1.5 Công nghệ Kỹ thuật dữ liệu sử dụng	22
3.2 Nền tảng Trí tuệ nhân tạo	22
3.2.1 Tổng quan về Học máy và Dự báo chuỗi thời gian	22
3.2.2 Mạng Nơ-ron sâu (Deep Learning)	23
3.2.3 Học tăng cường sâu (Deep Reinforcement Learning - DRL)	25
3.2.4 Các kỹ thuật xử lý dữ liệu và mô hình sinh tạo	26
3.2.5 Trí tuệ nhân tạo có thể giải thích (Explainable AI - XAI)	28
3.2.6 Công nghệ và Công cụ sử dụng	29
3.3 Nền tảng Ứng dụng và Định tuyến	30
3.3.1 Cơ sở lý thuyết thuật toán và hệ thống	30
3.3.1.a Lý thuyết Đồ thị và Thuật toán Tìm đường (Routing Algorithms)	30
3.3.1.b Lý thuyết Xử lý Phân tích Trực tuyến (OLAP)	31
3.3.1.c Xử lý Không gian và Khớp bản đồ (Map Matching)	31
3.3.2 Công nghệ phát triển Ứng dụng	32
3.3.2.a Công nghệ xây dựng Máy chủ (Backend Framework)	32
3.3.2.b Công nghệ xây dựng Giao diện (Frontend Framework)	32
3.4 Nền tảng Triển khai	33
3.5 Kết chương	33
4 Công trình liên quan	34
4.1 Các bài báo và nghiên cứu đã xem xét	34
4.1.1 Nhóm nghiên cứu về kiến trúc Kho dữ liệu giao thông truyền thống	34
4.1.2 Nhóm nghiên cứu về Kho dữ liệu lớn và Tích hợp AI	34

4.1.3	Nhóm nghiên cứu về Thuật toán dẫn đường và Nội suy dữ liệu	34
4.2	Các phần mềm và hệ thống hiện có (Softwares/tools/systems)	35
4.2.1	Cổng thông tin giao thông TP.HCM (giaothong.hochiminhcity.gov.vn)	35
4.2.2	Hệ thống UTraffic (bktraffic.com)	35
4.2.3	Các nền tảng dữ liệu hiện đại (Modern Data Platform - Snowflake/Databricks)	35
4.3	Khoảng trống nghiên cứu	35
5	Hệ thống Trí tuệ Nhân tạo hỗ trợ dự báo giao thông thông minh	37
5.1	Tổng quan và Phân tích các phương pháp tiếp cận (Literature Review)	37
5.1.1	Tiếp cận dựa trên Thống kê truyền thống (ARIMA, SARIMA)	37
5.1.2	Tiếp cận dựa trên Học sâu (LSTM, CNN, T-GCN)	38
5.1.3	Đề xuất Kiến trúc Lai (Hybrid SL-to-RL Transfer)	40
5.2	Kỹ thuật Thiết kế Đặc trưng (Feature Engineering & Justification)	41
5.2.1	Cấu trúc Cửa sổ trượt (Sliding Window) và Đảm bảo tính liên tục (Continuity)	42
5.2.2	Nhúng bối cảnh không gian (Spatial & Categorical Embedding)	43
5.2.3	Mã hóa tính chu kỳ thời gian (Cyclical Temporal Encoding)	44
5.2.4	Chuẩn hóa và Tiền xử lý (Scaling & Imputation)	45
5.2.5	Quy trình Đánh giá và Tuyển chọn Đặc trưng (Feature Quality Assessment)	47
5.3	Chiến lược Cân bằng Dữ liệu (Resampling Comparative Analysis)	49
5.3.1	Thách thức từ dữ liệu mất cân bằng cực hạn (Extreme Imbalance)	49
5.3.2	Phân tích điểm yếu của phương pháp nội suy tuyến tính (SMOTE/ADASYN)	50
5.3.3	Chiến lược Cân bằng Lai (Hybrid Resampling: Anchor-based & Sequence-CTGAN)	51
5.4	Trạm tiền huấn luyện Giám sát (Supervised Baseline Justification)	53
5.4.1	Màng lọc Hậu kiểm Vật lý (Physical Sanity Check)	53
5.4.2	Xây dựng "Trực giác" ban đầu cho Agent thông qua Học có giám sát (Supervised Pre-training)	54
5.4.3	Giải pháp khắc phục vấn đề "Khởi động lạnh" (Cold Start) trong Học tăng cường	55
5.4.4	Chuyển giao trọng số và Khởi động ấm (Warm-start Weight Injection)	56
5.5	Kiến trúc Đặc vụ Học tăng cường (Double DQN Architecture)	56
5.5.1	Lựa chọn thuật toán Double DQN (DDQN)	57
5.5.2	Cấu trúc mạng Q-Network và Siêu tham số Huấn luyện	58
5.5.3	Thiết kế Hàm Phần thưởng (Reward Shaping) - "Linh hồn" của sự thông minh	59
5.6	Đánh giá Thực nghiệm và So sánh (Empirical Evaluation & Benchmarking)	60
5.6.1	Thiết lập các mô hình đối chứng (Baselines)	60
5.6.2	Phân tích Chỉ số Hiệu năng (Metrics Analysis)	60
5.6.3	Giải mã "Hộp đen" và Tính minh bạch của hệ thống (Explainable AI với SHAP)	62
5.7	Hạn chế của Hệ thống đề xuất và Hướng phát triển	62
5.7.1	Hạn chế về Độ trễ Xử lý quy mô lớn (Inference Latency at Scale)	62
5.7.2	Khoảng cách giữa Môi trường Mô phỏng và Thực tế (Sim-to-Real Gap)	63
5.7.3	Giới hạn Nhận thức Không gian và Định hướng Đồ thị (Spatial Awareness & Graph Evolution)	63
6	Nghiệp vụ thống kê, dự báo, hỗ trợ ra quyết định	65
6.1	Phân tích hiện trạng các hệ thống thông tin giao thông hiện có	65
6.1.1	Khảo sát Cổng thông tin giao thông TP.HCM (giaothong.hochiminhcity.gov.vn)	65
6.1.2	Khảo sát Hệ thống BKTraffic (bktraffic.com)	66
6.1.3	Khoảng trống nghiệp vụ và Sự cần thiết của Data Warehouse	67
6.2	Đặc tả các nghiệp vụ Phân tích và Hỗ trợ ra quyết định	68
6.2.1	Nhóm nghiệp vụ Thống kê mô tả và Giám sát (Descriptive Analytics)	68
6.2.2	Nhóm nghiệp vụ Phân tích dự báo (Predictive Analytics)	69

6.2.3	Nhóm nghiệp vụ Hỗ trợ ra quyết định (Prescriptive Analytics)	70
6.3	Kết chương	70
7	Kiến trúc tổng quan của Data Pipeline	72
7.1	Sơ lược hệ thống ETL trong đồ án	72
7.2	Xây dựng và thiết kế hệ thống ETL trong đồ án	72
7.2.1	BaseExtractor: Chuẩn hóa giao thức mạng và cơ chế chịu lỗi	72
7.2.2	BaseTransformer: Đảm bảo tính Toàn vẹn Dữ liệu bằng Pure Functions	72
7.2.3	BaseLoader: Trừu tượng hóa Chiến lược Nạp dữ liệu vào Data Warehouse	73
7.3	Quản lý ngân sách gọi API và tính bền vững	73
7.4	Tính lũy đẳng (Idempotency) và cơ chế xử lý xung đột dữ liệu	73
7.5	Cụ thể hóa trong các Pipeline nghiệp vụ	73
7.6	Kiến trúc hệ thống và điều phối	74
7.6.1	Sơ đồ kiến trúc:	74
7.6.2	Luồng thực thi tuần tự của chu kỳ ETL (Sequence Flow).	75
7.6.3	Triển khai cấp độ điều phối	76
7.7	Thu thập dữ liệu	76
7.7.1	Ràng buộc tài nguyên trong Data Ingestion	76
7.7.2	Xây dựng kế hoạch "Budget-Safe Realtime" và lấy mẫu có trọng số	76
7.7.2.a	Phân bổ ngân sách gọi API động (Dynamic Budget Allocation)	76
7.7.2.b	Kỹ thuật lựa chọn ưu tiên: Gold Corridors và Critical Score	76
7.8	Cơ chế quản lý khóa API động và cân bằng tải	77
7.8.1	Nhận thức trạng thái	77
7.8.2	Mẫu thiết kế Circuit Breaker và Failover tự động	77
7.9	Sức bền mạng và khả năng chịu lỗi	77
7.9.1	Cơ chế Retry with Exponential Backoff	77
7.9.2	Xử lý phân cực: lỗi tạm thời với lỗi hệ thống	77
7.10	Đa dạng hóa nguồn thu thập và xử lý bất đồng bộ	78
7.10.1	Thu thập dữ liệu tĩnh (Topology) vs. Động (Time-series)	78
7.10.2	Mở rộng kích thước lưới cho dữ liệu bổ trợ	78
7.11	Trình tự nạp dữ liệu và ràng buộc	78
7.11.1	Nguyên lý ràng buộc toàn vẹn tham chiếu và cấu trúc DAG	78
7.11.2	Pha 1: Khởi tạo dữ liệu nền tảng	79
7.11.3	Pha 2: Xây dựng Topology Không gian	79
7.11.4	Pha 3: Nạp dữ liệu chuỗi thời gian	79
7.11.5	Pha 4: Hậu xử lý và dữ liệu phái sinh	79
7.12	Logic Biến đổi Dữ liệu	80
7.12.1	Nguyên lý hàm thuần túy trong tầng biến đổi	80
7.12.2	Khai phá và biến đổi động lực học dòng xe	80
7.12.2.a	Chuỗi biến đổi chỉ số cơ bản (Traffic Index, Delay & LOS)	80
7.12.2.b	Bài toán Ước lượng Lưu lượng bằng Nghịch đảo Hàm BPR	81
7.12.3	Xử lý không gian và thuật toán khớp bản đồ	81
7.12.3.a	Truy vấn K-Nearest Neighbors (KNN) với PostGIS	82
7.12.3.b	Tiền xử lý hình học đa dạng (Geometry Preprocessing)	82
7.12.4	Chuẩn hóa ngữ nghĩa và phân rã chiều thời gian	82
7.12.5	Kết luận	83
7.13	Kỹ thuật tối ưu hóa kho dữ liệu	83
7.13.1	Mục tiêu tối ưu hóa	83
7.13.2	Chiến lược phân mảnh dữ liệu chuỗi thời gian	83

7.13.3	Chiến lược đánh chỉ mục đa hình	84
7.13.3.a	Chỉ mục không gian GiST (Generalized Search Tree)	84
7.13.3.b	Chỉ mục khối dải BRIN (Block Range Index)	84
7.13.3.c	Chỉ mục một phần (Partial Index) hỗ trợ Alerting	84
7.13.4	Cơ chế UPSERT và Quản lý "Bloat" trong MVCC	85
7.13.5	Kết luận	85
8	Thiết kế DataWarehouse Schema	86
8.1	Quá trình thiết kế	86
8.1.1	Thiết kế DW Schema theo lý thuyết, góc nhìn cá nhân/chủ quan về giao thông	86
8.1.1.a	Tiếp cận mô hình Star Schema độc lập	86
8.1.1.b	Thiết kế các bảng Dimension dựa trên quan sát chủ quan	86
8.1.1.c	Kết luận về thiết kế sơ khởi	88
8.1.2	Phân tích nghiệp vụ nâng cao và xây dựng mô hình Galaxy Schema	88
8.1.2.a	Phân tích ma trận nghiệp vụ	88
8.1.2.b	Tái thiết kế các Conformed Dimensions	89
8.1.2.c	Thiết kế các bảng Fact theo nghiệp vụ	89
8.1.2.d	Tổng quan quá trình chuyển đổi	90
8.1.3	Phân tích tính khả thi	90
8.1.3.a	TomTom	90
8.1.3.b	Google Maps	91
8.1.3.c	OpenStreetMap	92
8.2	Kết quả	93
8.2.1	Tổng quan toàn bộ Galaxy Schema	93
8.2.2	Bảng Fact	93
8.2.3	Bảng Dim	98
8.3	Kết chương	101
9	Ứng dụng Schema vào thực tiễn	102
9.1	Nhóm nghiệp vụ thống kê mô tả và giám sát	102
9.1.1	A1: Giám sát tốc độ trung bình thời gian thực	102
9.1.1.a	Thành phần dữ liệu trong Schema	102
9.1.1.b	Thuật toán xử lý dữ liệu	102
9.1.2	A2: Đánh giá Mức độ tắc nghẽn	103
9.1.2.a	Thành phần dữ liệu trong Schema	103
9.1.2.b	Thuật toán và mô hình tính toán	104
9.1.2.c	Logic chuẩn hóa sang thang điểm 1–5	105
9.1.3	A3: Phân tích thời gian di chuyển đặc trưng	105
9.1.3.a	Thành phần dữ liệu trong Schema	105
9.1.3.b	Thuật toán và mô hình tính toán	106
9.1.4	A4: Chỉ số độ tin cậy thời gian	106
9.1.4.a	Thành phần dữ liệu trong Schema	107
9.1.4.b	Thuật toán và mô hình tính toán	107
9.1.5	A5: Thống kê điểm nóng sự cố	108
9.1.5.a	Thành phần dữ liệu trong Schema	108
9.1.5.b	Thuật toán và mô hình tính toán	108
9.1.6	A6: Phân tích tác động của thời tiết đến tốc độ	109
9.1.6.a	Thành phần dữ liệu trong Schema	109
9.1.6.b	Thuật toán và mô hình tính toán	110

9.1.7	A7: Đánh giá hiệu quả hệ thống gợi ý	111
9.1.7.a	Thành phần dữ liệu trong Schema	111
9.1.7.b	Thuật toán và mô hình tính toán	111
9.1.8	A8: Thu thập phản hồi chuyển đi	112
9.1.8.a	Thành phần dữ liệu trong Schema	112
9.1.8.b	Thuật toán và mô hình tính toán	113
9.1.9	A9: Phân bổ lưu lượng theo loại xe	113
9.1.9.a	Thành phần dữ liệu trong Schema	114
9.1.9.b	Thuật toán và mô hình tính toán	114
9.2	Nhóm nghiệp vụ Phân tích dự báo (Predictive Analytics)	115
9.2.1	Nghiệp vụ B1: Dự báo tốc độ ngắn hạn (Short-term Speed Prediction)	115
9.2.1.a	Mục đích và phạm vi	115
9.2.1.b	Ánh xạ dữ liệu nguồn	115
9.2.1.c	Logic xử lý và Thuật toán	116
9.2.2	Nghiệp vụ B2: Dự báo thời gian hành trình (ETA Prediction)	116
9.2.2.a	Mục đích và Phạm vi	116
9.2.2.b	Ánh xạ dữ liệu nguồn	116
9.2.2.c	Logic xử lý và Thuật toán	116
9.2.3	Nghiệp vụ B3: Dự báo rủi ro sự cố (Incident Risk Prediction)	117
9.2.3.a	Mục đích và Phạm vi	117
9.2.3.b	Ánh xạ dữ liệu nguồn	117
9.2.3.c	Logic xử lý và Thuật toán	117
9.3	Nhóm nghiệp vụ Hỗ trợ ra quyết định (Prescriptive Analytics)	117
9.3.1	Nghiệp vụ C1: Đề xuất lộ trình tối ưu đa mục tiêu (Multi-objective Routing)	118
9.3.1.a	Mục đích và Phạm vi	118
9.3.1.b	Ánh xạ dữ liệu nguồn (Data Mapping)	118
9.3.1.c	Logic xử lý và Thuật toán	118
9.3.2	Nghiệp vụ C2: Cảnh báo và Tái định tuyến thời gian thực (Real-time Rerouting)	118
9.3.2.a	Mục đích và Phạm vi	118
9.3.2.b	Ánh xạ dữ liệu nguồn	119
9.3.2.c	Logic xử lý và Thuật toán	119
9.3.3	Nghiệp vụ C3: Gợi ý giờ xuất phát tối ưu (Optimal Departure Time)	119
9.3.3.a	Mục đích và Phạm vi	119
9.3.3.b	Ánh xạ dữ liệu nguồn	119
9.3.3.c	Logic xử lý và Thuật toán	119
10	Thiết kế và triển khai quy trình trích xuất, biến đổi, nạp dữ liệu (ETL)	121
10.1	Kiến trúc tổng thể hệ thống ETL	121
10.1.1	Mục tiêu và Nguyên tắc thiết kế kiến trúc	121
10.1.2	Mô hình phân lớp chức năng (Functional Layering)	121
10.1.3	Hạ tầng công nghệ và Thư viện lõi	122
10.1.4	Quản lý cấu hình và Biến môi trường (Environment Configuration)	122
10.1.5	Quản trị Kết nối và Tài nguyên Cơ sở dữ liệu (Connection Management)	123
10.1.6	Cơ chế điều phối đa tầng (Hierarchical Orchestration)	123
10.2	QUY TRÌNH NẠP DỮ LIỆU DIMENSION (DIMENSION ETL PIPELINE)	124
10.2.1	Nhóm chiều Thời gian và Lịch (Time & Calendar Dimensions)	124
10.2.2	Nhóm chiều Danh mục và Đặc tính (Metadata Dimensions)	127
10.2.3	Nhóm chiều Hạ tầng Giao thông (Infrastructure Dimensions)	128
10.3	QUY TRÌNH NẠP DỮ LIỆU SỰ KIỆN (FACT ETL PIPELINE)	132

10.3.1	Tầng Dịch vụ thu thập dữ liệu (Data Service Layer)	132
10.3.2	Tích hợp Thị giác máy tính trong quy trình ETL (AI Integration)	133
10.3.3	Xác định chỉ số LOS (Level of Service) và Mức độ ùn tắc	136
10.3.4	Quy trình nạp dữ liệu Fact Traffic Flow và Fact Incident	136
10.3.5	Quy trình giả lập dữ liệu (Mock Data Pipeline)	141
10.4	TỰ ĐỘNG HÓA VÀ GIÁM SÁT VẬN HÀNH (AUTOMATION & MONITORING)	142
10.4.1	Cơ chế vận hành thông qua Batch Script và Điều phối hệ thống	142
10.4.2	Hệ thống ghi nhật ký tập trung (Logging System)	143
10.5	Kết chương	143
11	Chi tiết nghiệp vụ hệ thống Traffic IOC	144
11.1	Tổng quan hệ thống và Triết lý thiết kế	144
11.1.1	Giới thiệu tổng quan hệ thống	144
11.1.2	Nguyên tắc thiết kế Giao diện & Trải nghiệm người dùng (UI/UX)	144
11.2	Phân hệ Quản trị & Điều hành (Admin Dashboard)	145
11.2.1	Nghiệp vụ 1: Bảng điều khiển Tổng quan (Executive Dashboard)	145
11.2.2	Nghiệp vụ 2: Giám sát Năng lực thông hành (Corridor Performance)	146
11.2.3	Nghiệp vụ 3: Phân tích Độ tin cậy & Biến động (Reliability Analytics)	147
11.2.4	Nghiệp vụ 4: Định vị & Xếp hạng Điểm nghẽn (Bottleneck Detection)	148
11.2.5	Nghiệp vụ 5: Phân tích Đa chiều OLAP & Đánh giá Hạ tầng	149
11.2.6	Nghiệp vụ 6: Tra cứu Lịch sử & Tin nóng (Marquee Alert)	149
11.2.7	Nghiệp vụ 7: Mô phỏng & Dự báo giao thông động (Traffic Simulation)	150
11.3	Phân hệ Người dùng cuối (Citizen Mobile App)	150
11.3.1	Nghiệp vụ 1: Tìm đường thông minh theo Thời gian thực (Dynamic Smart Routing)	151
11.3.2	Nghiệp vụ 2: Giám sát Bản đồ Cá nhân hóa	153
12	TRIỂN KHAI, VẬN HÀNH VÀ TỐI ƯU CHI PHÍ	154
12.1	Kiến trúc Triển khai (Deployment Architecture)	154
12.2	Quản lý Server và Containerization (Docker)	154
12.2.1	Tối ưu hóa Docker Image	154
12.2.2	Quản lý Container và Dọn dẹp Tài nguyên	155
12.3	Thiết lập luồng Triển khai (Deployment Flow)	155
12.4	Phân tích & Quản lý ngân sách (Budget & Resource Optimization)	155
13	Đánh giá hệ thống	156
13.1	Đánh giá Schema	156
13.1.1	Khả năng đáp ứng nghiệp vụ đa chiều	156
13.1.2	Tính toàn vẹn và cấu trúc không gian	156
13.1.3	Khả năng mở rộng	156
13.1.4	Đánh giá hiệu năng truy vấn	156
13.1.5	Khả năng hỗ trợ AI và Học máy	157
13.2	Đánh giá kết quả sau ETL	157
13.2.1	Đánh giá hiệu quả khởi tạo hệ quy chiếu Dimension	157
13.2.2	Đánh giá hiệu quả nạp dữ liệu Fact và Hợp nhất dữ liệu đa nguồn	158
13.2.3	Đánh giá Hiệu năng vận hành và Quản trị dữ liệu quy mô lớn	159
13.2.4	Đánh giá tính sẵn sàng của Mock Data và khả năng hỗ trợ AI/ML	160
13.3	Đánh giá Hiệu năng và Độ bền vững Phần mềm (Software Performance & Robustness)	161
13.3.1	Độ trễ API (API Latency) và Tối ưu hóa truy vấn	161
13.3.2	Hiệu năng Giao diện (Frontend Performance)	161

13.3.3	Đánh giá Độ bền vững và Khả năng chịu tải (Robustness & Scalability)	161
13.4	Đánh giá Mức độ Đáp ứng Yêu cầu Nghiệp vụ	162
13.5	Kết chương	162
14	Tổng kết và hướng phát triển	164
14.1	Kết quả đạt được của đề án	164
14.2	Những hạn chế còn tồn tại	165
14.3	Hướng phát triển trong tương lai	166
15	Tài liệu tham khảo	168

Danh sách hình vẽ

1	Biểu đồ minh họa hiện tượng Trễ pha của mô hình ARIMA	38
2	Sơ đồ khối kiến trúc LSTM đơn giản	38
3	Ma trận nhầm lẫn mô hình DL thuần túy	39
4	Sơ đồ kiến trúc tổng thể	40
5	Đồ thị hàm phần thưởng	41
6	Sơ đồ minh họa cơ chế Cửa sổ trượt và Thuật toán kiểm tra tính liên tục	43
7	So sánh One-hot Encoding vs Dense Embedding	44
8	Sơ đồ luồng dữ liệu MLOps	47
9	Ma trận Tương quan (Heatmap) thể hiện hiện tượng đa cộng tuyến giữa các đặc trưng giao thông	48
10	Xếp hạng mức độ đóng góp của các đặc trưng dựa trên Gini Importance (Random Forest)	49
11	So sánh Phân phối dữ liệu Gốc và Phân phối Vàng sau khi áp dụng Cân bằng Lai	50
12	Sự vi phạm ràng buộc phi tuyến của phương pháp nội suy SMOTE	51
13	Kiến trúc Chiến lược Cân bằng lai	52
14	Cơ chế hoạt động của Mạng lọc Hậu kiểm Vật lý	54
15	Cơ chế điều hướng sự chú ý của Focal Loss so với Cross-Entropy	55
16	Đánh giá hiệu suất Phân lớp của Mô hình Baseline	55
17	Lợi ích thực chứng của Chuyển giao Trọng số lên tốc độ hội tụ Phần thưởng	57
18	Kiến trúc Double DQN - Cơ chế kiểm chứng chéo	58
19	Ma trận Thưởng/Phạt không đối xứng	60
20	Biểu đồ so sánh hiệu năng thực tế giữa các mô hình	61
21	Tích hợp Mạng Nơ-ron Đồ thị (GNN)	64
22	Các lược đồ Star Schema ban đầu	86
23	Thiết kế bảng dim_location ban đầu	87
24	Thiết kế bảng dim_time ban đầu	87
25	Thiết kế bảng dim_vehicle_type ban đầu	88
26	Mô hình Galaxy Schema hoàn chỉnh	93
27	Bảng fact_travel_flow	94
28	Bảng fact_user_mobility	95
29	Bảng fact_weather_impact	96
30	Bảng fact_incident	97
31	Cụm bảng fact_trip_recommendation và fact_segment_performance	97
32	Bảng Dimension dim_weather	98
33	Bảng Dimension dim_incident_type	98
34	Bảng Dimension dim_vehicle_type	99
35	Nhóm Bảng Dimension: Người dùng	99
36	Nhóm Bảng Dimension: Thời gian (Chi tiết thời gian trong ngày)	100
37	Nhóm Bảng Dimension: Thời gian (Ngày/Tháng/Năm)	100
38	Nhóm Bảng Dimension: Hạ tầng giao thông	100
39	Sơ đồ kiến trúc tổng thể hệ thống ETL	121
40	Cấu trúc thư mục dự án hệ thống ETL	122
41	Cấu hình các tham số trong tệp .env	123
42	Triển khai lớp DatabaseConfig trong Python	124
43	Sơ đồ điều phối Pipeline đa tầng	125
44	Dữ liệu cấu hình các ngày lễ trong bảng dim_holiday	125
45	Đoạn mã xử lý chuyển đổi lịch âm sang lịch dương cho ngày lễ	126
46	Kết quả cập nhật cờ is_holiday trong bảng dim_date	126

47	Tệp cấu hình vehicles.csv chứa hệ số PCU và tốc độ thiết kế	128
48	Danh mục sự cố đã chuẩn hóa mã màu trong cơ sở dữ liệu	128
49	Nhật ký thực thi nạp dữ liệu ranh giới hành chính Quận 1	129
50	Dữ liệu ranh giới hành chính đã nạp vào bảng dim_location	130
51	Trực quan hóa đa giác ranh giới TP. Hồ Chí Minh từ PostGIS	130
52	Dữ liệu góc Azimuth và hình học đa tầng trong bảng dim_segment	131
53	Câu lệnh SQL thực hiện phép giao không gian để cập nhật location_key	131
54	Cấu trúc dữ liệu JSON trả về từ TomTom Traffic API	132
55	Trang web lấy hình ảnh camera tại Đường Cách Mạng Tháng Tám - Hòa Hưng	133
56	Kết quả nhận diện và phân lớp phương tiện thời gian thực bằng YOLOv8x	134
57	Khởi tạo mô hình YOLOv8 và cấu hình danh mục đối tượng mục tiêu	135
58	Đoạn mã triển khai logic đếm đối tượng và tính toán PCU từ kết quả AI	136
59	Mô tả các cấp độ phục vụ (LOS) trong kỹ thuật giao thông	137
60	Cấu trúc các phân vùng bảng Fact theo tháng trong PostgreSQL	138
61	Hàm xử lý logic tính toán chỉ số rủi ro ngập lụt	138
62	Nhật ký nạp dữ liệu sự cố và kết quả đối khớp không gian thành công	139
63	Giao diện Task Scheduler	140
64	Set up Trigger theo lịch ETL	141
65	Cấu trúc tệp lệnh Batch thực thi chu kỳ ETL tự động	142
66	Nhật ký tổng hợp kết thúc quy trình Fact Pipeline thành công	143
67	Bảng điều khiển Tổng quan với hệ thống thẻ KPI cơ bản	146
68	Nghiệp vụ Giám sát Năng lực thông hành với hệ thống KPI và biểu đồ Split-color Gradient	147
69	Popup chi tiết Phân tích Độ tin cậy hành lang và hiển thị phân vị T_{95}	148
70	Biểu đồ phân tách tính Nghiêm trọng và tính Kinh niên của Điểm nghẽn	148
71	Dashboard BI & OLAP: Heatmap thời gian và Bubble Chart đánh giá hạ tầng	149
72	Giao diện Tra cứu lịch sử tích hợp thanh Tin nóng thời gian thực	150
73	Giao diện Nghiệp vụ Mô phỏng & Dự báo với kiến trúc Dual-Map	150
74	Nghiệp vụ Tìm đường thông minh né điểm nghẽn	152
75	Giao diện Bản đồ trạng thái giao thông và Cảnh báo cá nhân hóa	153
76	Sơ đồ kiến trúc triển khai hệ thống trên nền tảng đám mây	154

Danh sách bảng

1	Chi tiết các mốc thời gian thực hiện đồ án	19
2	So sánh chỉ số Recall giữa các mô hình trên tập Kiểm thử	61
3	Ma trận quy trình nghiệp vụ và các chiều dữ liệu	88
4	Bảng tóm tắt so sánh 3 nguồn dữ liệu	92
5	Ánh xạ mức độ tắc nghẽn theo thang điểm 1–5	105
6	So sánh và lựa chọn thuật toán tìm đường trong pgRouting	151

1 Giới thiệu

1.1 Động cơ của đề tài

Trong bối cảnh tốc độ đô thị hóa và gia tăng dân số diễn ra mạnh mẽ trong nhiều năm gần đây, Thành phố Hồ Chí Minh (TP.HCM) đang phải đối mặt với những áp lực ngày càng lớn đối với hệ thống giao thông đô thị. Sự gia tăng nhanh chóng của phương tiện cá nhân, đặc biệt là xe máy và ô tô, trong khi năng lực hạ tầng giao thông chưa theo kịp tốc độ phát triển đô thị, đã dẫn đến tình trạng ùn tắc giao thông diễn ra thường xuyên tại nhiều tuyến đường và khu vực trọng điểm. Thực trạng này không chỉ làm suy giảm hiệu quả vận hành của đô thị mà còn gây ra nhiều hệ lụy nghiêm trọng như ô nhiễm môi trường, gia tăng thời gian di chuyển, tiêu hao năng lượng, tai nạn giao thông và ảnh hưởng trực tiếp đến chất lượng sống của người dân.

Trước yêu cầu phát triển đô thị bền vững, TP.HCM đã định hướng xây dựng mô hình đô thị thông minh, trong đó giao thông thông minh (Intelligent Transportation System – ITS) được xem là một trong những lĩnh vực ưu tiên trọng điểm. Tuy nhiên, phần lớn các hệ thống giám sát và quản lý giao thông hiện nay mới chỉ tập trung vào chức năng quan sát và phản ứng theo thời gian thực, trong khi còn thiếu một nền tảng dữ liệu tập trung có khả năng tích hợp, lưu trữ và phân tích dữ liệu giao thông trên quy mô lớn và dài hạn. Điều này làm hạn chế khả năng khai thác dữ liệu phục vụ công tác thống kê, đánh giá xu hướng, tối ưu hóa vận hành và hỗ trợ ra quyết định chiến lược.

Hiện nay, dữ liệu giao thông đô thị được hình thành từ nhiều nguồn dị biệt như hệ thống camera giám sát (CCTV), thiết bị định vị GPS, dữ liệu cảm biến giao thông, nền tảng bản đồ số, cũng như dữ liệu lưu lượng và sự cố được cung cấp từ các nền tảng giao thông trực tuyến. Các nguồn dữ liệu này có đặc trưng phân tán, không đồng nhất về cấu trúc và thiếu tính liên kết không gian – thời gian, dẫn đến khó khăn trong việc đồng bộ, tổng hợp và khai thác phân tích chuyên sâu. Đặc biệt, đối với các bài toán giao thông thông minh hiện đại, dữ liệu không gian địa lý (geospatial data) đóng vai trò then chốt trong việc mô hình hóa mạng lưới giao thông, xác định các tuyến trọng điểm, phân tích mật độ lưu thông và đánh giá mức độ ùn tắc theo từng khu vực.

Trong bối cảnh đó, mô hình kho dữ liệu giao thông (Traffic Data Warehouse) kết hợp với công nghệ dữ liệu không gian (Spatial Data Warehouse) được xem là giải pháp nền tảng nhằm giải quyết bài toán phân mảnh dữ liệu. Kho dữ liệu cho phép tích hợp dữ liệu từ nhiều nguồn khác nhau, chuẩn hóa và tổ chức dữ liệu theo cấu trúc đa chiều, hỗ trợ hiệu quả cho các truy vấn phân tích phức tạp, thống kê lịch sử và khai thác dữ liệu quy mô lớn. Khác với các hệ quản trị cơ sở dữ liệu giao dịch truyền thống (OLTP), kho dữ liệu được tối ưu hóa cho các tác vụ phân tích (OLAP), đặc biệt phù hợp với các bài toán xử lý dữ liệu giao thông theo chiều không gian – thời gian và phục vụ công tác điều hành đô thị thông minh.

Bên cạnh đó, sự phát triển mạnh mẽ của trí tuệ nhân tạo (Artificial Intelligence – AI) và học máy (Machine Learning) đang thúc đẩy quá trình chuyển đổi từ mô hình quản lý giao thông mang tính phản ứng (reactive traffic management) sang mô hình quản lý chủ động và dự báo (proactive traffic management). Các mô hình AI có khả năng phân tích xu hướng lưu lượng, dự báo nguy cơ ùn tắc và hỗ trợ tối ưu hóa điều phối giao thông theo thời gian thực. Tuy nhiên, hiệu quả của các mô hình dự báo phụ thuộc rất lớn vào chất lượng dữ liệu đầu vào, đòi hỏi một hạ tầng dữ liệu đủ lớn, ổn định, được chuẩn hóa và cập nhật liên tục. Đây chính là vai trò cốt lõi của hệ thống kho dữ liệu giao thông tích hợp.

Xuất phát từ những yêu cầu thực tiễn nêu trên, nhóm nghiên cứu quyết định thực hiện đề tài: “Mô hình kho dữ liệu giao thông phục vụ công tác thống kê, phân tích và dự báo hỗ trợ điều hành giao thông thông minh”. Đề tài tập trung xây dựng một nền tảng dữ liệu giao thông tích hợp trên cơ sở kết hợp dữ liệu thời gian thực từ các nguồn giao thông trực tuyến với dữ liệu không gian địa lý, triển khai quy trình ETL tự động nhằm thu thập, làm sạch, biến đổi và lưu trữ dữ liệu trong kho dữ liệu không gian. Đồng thời, hệ thống hướng đến việc ứng dụng các mô hình trí tuệ nhân tạo trong phân tích và dự báo tình trạng giao thông, kết hợp với nền tảng trực quan hóa và dashboard hỗ trợ giám sát, từ đó góp phần xây dựng nền tảng hạ tầng dữ liệu phục vụ phát triển hệ sinh thái giao thông thông minh và đô thị thông minh tại TP.HCM trong tương lai.

1.2 Mục tiêu

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và triển khai một nền tảng kho dữ liệu giao thông thông minh có khả năng tích hợp dữ liệu đa nguồn, xử lý dữ liệu không gian – thời gian theo thời gian thực và hỗ trợ các tác vụ phân tích, dự báo phục vụ công tác quản lý và điều hành giao thông đô thị. Hệ thống được định hướng xây dựng theo mô hình kiến trúc dữ liệu hiện đại, bảo đảm khả năng mở rộng, tính ổn định và đáp ứng yêu cầu khai thác dữ liệu quy mô lớn trong bối cảnh phát triển đô thị thông minh.

Để hiện thực hóa mục tiêu tổng quát nêu trên, đề tài tập trung vào các mục tiêu cụ thể sau:

- **Nghiên cứu và đề xuất kiến trúc kho dữ liệu giao thông thông minh**

Xây dựng kiến trúc tổng thể cho hệ thống kho dữ liệu giao thông theo mô hình đa tầng, bao gồm tầng thu thập dữ liệu, tầng lưu trữ dữ liệu không gian, tầng phân tích trí tuệ nhân tạo và tầng trực quan hóa dữ liệu. Đề tài tập trung thiết kế mô hình dữ liệu đa chiều phù hợp với đặc thù dữ liệu giao thông đô thị tại Việt Nam, kết hợp giữa dữ liệu cấu trúc, bán cấu trúc và dữ liệu không gian địa lý (geospatial data). Các mô hình Star Schema hoặc Snowflake Schema được nghiên cứu và áp dụng nhằm tối ưu hóa hiệu năng truy vấn, hỗ trợ hiệu quả cho các tác vụ phân tích dữ liệu lớn và xử lý theo chiều không gian – thời gian.

- **Xây dựng quy trình xử lý dữ liệu ETL/ELT tự động**

Thiết kế và triển khai quy trình ETL/ELT nhằm tự động hóa quá trình thu thập, biến đổi và lưu trữ dữ liệu giao thông từ nhiều nguồn khác nhau như API giao thông thời gian thực, dữ liệu bản đồ số OpenStreetMap (OSM), dữ liệu sự cố giao thông và các nguồn dữ liệu hỗ trợ khác. Quy trình xử lý dữ liệu tập trung vào việc làm sạch dữ liệu, xử lý ngoại lệ, chuẩn hóa định dạng, tích hợp dữ liệu không gian và tối ưu khả năng cập nhật dữ liệu liên tục theo chu kỳ thời gian thực, bảo đảm tính toàn vẹn và độ tin cậy của dữ liệu trong kho dữ liệu trung tâm.

- **Xây dựng kho dữ liệu không gian phục vụ phân tích giao thông**

Triển khai hệ quản trị cơ sở dữ liệu PostgreSQL kết hợp với phần mở rộng PostGIS nhằm hỗ trợ lưu trữ và xử lý dữ liệu không gian địa lý. Đề tài tập trung xây dựng mô hình dữ liệu phục vụ quản lý các thực thể giao thông như tuyến đường, đoạn đường, nút giao, vùng ảnh hưởng và dữ liệu lưu lượng theo không gian – thời gian. Đồng thời, hệ thống được tối ưu hóa nhằm đáp ứng các truy vấn phân tích không gian phức tạp và hỗ trợ trực quan hóa dữ liệu giao thông trên nền bản đồ số.

- **Tích hợp và triển khai mô hình trí tuệ nhân tạo phục vụ dự báo giao thông**

Nghiên cứu và ứng dụng các mô hình học máy và học sâu trong bài toán dự báo tình trạng giao thông đô thị, bao gồm dự báo lưu lượng phương tiện, tốc độ lưu thông và nguy cơ ùn tắc. Các mô hình như ARIMA, LSTM hoặc các mô hình lai ghép (Hybrid Models) được huấn luyện và đánh giá trên tập dữ liệu lịch sử đã được chuẩn hóa từ kho dữ liệu giao thông. Quá trình huấn luyện và kiểm thử mô hình được thực hiện theo quy trình khoa học nhằm bảo đảm độ chính xác và khả năng ứng dụng thực tiễn của hệ thống dự báo.

- **Phát triển hệ thống trực quan hóa và hỗ trợ ra quyết định**

Xây dựng nền tảng dashboard và web application phục vụ giám sát giao thông theo thời gian thực, hỗ trợ trực quan hóa dữ liệu dưới dạng biểu đồ, bản đồ nhiệt (heatmap), thống kê lưu lượng và cảnh báo ùn tắc. Hệ thống hướng đến việc cung cấp công cụ hỗ trợ ra quyết định cho các cơ quan quản lý giao thông đô thị, hỗ trợ công tác điều phối giao thông, phân luồng phương tiện và hoạch định chính sách phát triển hạ tầng giao thông thông minh.

- **Triển khai thử nghiệm và đánh giá hiệu năng hệ thống**

Tiến hành triển khai thử nghiệm hệ thống trên hạ tầng điện toán đám mây Microsoft Azure, kết hợp với nền tảng container hóa Docker nhằm bảo đảm tính linh hoạt và khả năng mở rộng của hệ thống. Đề tài thực hiện đánh giá hiệu năng của quy trình xử lý dữ liệu thời gian thực, khả năng đáp ứng của kho dữ liệu, mức độ ổn định của hệ thống cũng như độ chính xác của mô hình dự báo trên dữ liệu thực tế. Các kết quả đánh giá được

sử dụng làm cơ sở để phân tích tính khả thi và khả năng ứng dụng của hệ thống trong bài toán giao thông thông minh tại TP.HCM.

1.3 Phạm vi

Nhằm bảo đảm tính khả thi của đề tài trong điều kiện thời gian, nguồn lực và phạm vi nghiên cứu, hệ thống được giới hạn trong các nội dung nghiên cứu và triển khai cụ thể như sau:

- **Về dữ liệu nghiên cứu**

Đề tài tập trung khai thác và xử lý các loại dữ liệu giao thông có cấu trúc và bán cấu trúc phục vụ cho bài toán phân tích giao thông thông minh. Các nguồn dữ liệu chính bao gồm dữ liệu lưu lượng giao thông thời gian thực, dữ liệu tốc độ lưu thông, dữ liệu sự cố giao thông, dữ liệu định vị GPS và dữ liệu bản đồ số từ nền tảng OpenStreetMap (OSM). Ngoài ra, hệ thống có khả năng mở rộng để tích hợp dữ liệu từ các cảm biến IoT hoặc nền tảng giám sát giao thông trong tương lai. Đối với dữ liệu phi cấu trúc như video thô từ camera CCTV, đề tài không thực hiện xử lý trực tiếp mà chỉ sử dụng các dữ liệu đã được trích xuất và chuyển đổi sang dạng số liệu phục vụ phân tích.

- **Về phạm vi không gian nghiên cứu**

Hệ thống được triển khai thử nghiệm và đánh giá trên một số tuyến đường trọng điểm, các hành lang giao thông chính và các khu vực có mật độ lưu thông cao tại Thành phố Hồ Chí Minh. Phạm vi nghiên cứu tập trung vào các tuyến đường thường xuyên xảy ra ùn tắc giao thông nhằm phục vụ việc phân tích lưu lượng, đánh giá mức độ quá tải và thử nghiệm khả năng dự báo tình trạng giao thông theo thời gian thực. Kết quả thử nghiệm được sử dụng làm cơ sở đánh giá tính khả thi và khả năng mở rộng của mô hình trong quy mô đô thị lớn.

- **Về phạm vi công nghệ và hạ tầng triển khai**

Đề tài sử dụng hệ quản trị cơ sở dữ liệu PostgreSQL kết hợp với phần mở rộng PostGIS để xây dựng kho dữ liệu không gian phục vụ lưu trữ và xử lý dữ liệu giao thông. Quy trình ETL/ELT được phát triển bằng ngôn ngữ Python kết hợp với các thư viện hỗ trợ xử lý dữ liệu và kết nối cơ sở dữ liệu như SQLAlchemy, GeoAlchemy2 và Pandas. Hệ thống được triển khai trên nền tảng điện toán đám mây Microsoft Azure, sử dụng Docker và Docker Compose để quản lý môi trường thực thi và hỗ trợ khả năng mở rộng hệ thống. Các mô hình trí tuệ nhân tạo và học máy được xây dựng và huấn luyện thông qua các thư viện khoa học dữ liệu như Scikit-learn, TensorFlow hoặc PyTorch.

- **Về phạm vi chức năng hệ thống**

Đề tài tập trung nghiên cứu và phát triển nền tảng phần mềm phục vụ thu thập, xử lý, lưu trữ, phân tích và trực quan hóa dữ liệu giao thông. Hệ thống hỗ trợ xây dựng dashboard giám sát giao thông, phân tích dữ liệu lịch sử và tích hợp mô hình dự báo tình trạng ùn tắc giao thông theo thời gian thực. Đề tài không bao gồm việc thiết kế hoặc triển khai hạ tầng phần cứng vật lý như camera giao thông, thiết bị IoT hoặc hệ thống cảm biến ngoài thực địa. Các nguồn dữ liệu đầu vào được giả định là có sẵn thông qua API, dữ liệu mở hoặc các nền tảng hỗ trợ nghiên cứu.

- **Về phạm vi nghiên cứu mô hình dự báo**

Đề tài tập trung vào các bài toán dự báo ngắn hạn liên quan đến lưu lượng giao thông, tốc độ lưu thông và nguy cơ ùn tắc trên từng tuyến đường hoặc khu vực cụ thể. Các mô hình học máy và học sâu được xây dựng nhằm đánh giá khả năng ứng dụng của trí tuệ nhân tạo trong hỗ trợ điều hành giao thông thông minh, tuy nhiên chưa đi sâu vào các bài toán tối ưu điều khiển tín hiệu giao thông hoặc điều phối giao thông quy mô toàn thành phố.

1.4 Ý nghĩa

1.4.1 Ý nghĩa thực tiễn

Đề tài mang ý nghĩa thực tiễn quan trọng trong bối cảnh nhu cầu xây dựng hệ thống giao thông thông minh và đô thị thông minh tại Thành phố Hồ Chí Minh ngày càng trở nên cấp thiết. Việc xây dựng nền tảng kho dữ liệu giao thông tích hợp không chỉ góp phần nâng cao hiệu quả quản lý dữ liệu mà còn tạo tiền đề cho việc ứng dụng các công nghệ phân tích dữ liệu lớn và trí tuệ nhân tạo trong công tác điều hành giao thông đô thị.

- **Hỗ trợ công tác quản lý và điều hành giao thông đô thị**

Hệ thống kho dữ liệu giao thông cho phép tập trung, chuẩn hóa và lưu trữ dữ liệu từ nhiều nguồn khác nhau trong một nền tảng thống nhất, giúp các cơ quan quản lý như Sở Giao thông Vận tải Thành phố Hồ Chí Minh có được cái nhìn toàn diện về hiện trạng giao thông theo không gian và thời gian. Thông qua hệ thống dashboard và các công cụ trực quan hóa dữ liệu, nhà quản lý có thể theo dõi lưu lượng giao thông theo thời gian thực, đánh giá mức độ ùn tắc tại các khu vực trọng điểm và hỗ trợ ra quyết định trong công tác phân luồng, điều phối giao thông cũng như hoạch định chính sách phát triển hạ tầng.

- **Nâng cao hiệu quả khai thác và sử dụng dữ liệu giao thông**

Việc xây dựng quy trình ETL/ELT tự động và kho dữ liệu không gian giúp giải quyết tình trạng dữ liệu giao thông phân tán, thiếu đồng bộ và khó khai thác. Hệ thống cho phép lưu trữ dữ liệu lịch sử với quy mô lớn, hỗ trợ các tác vụ thống kê, phân tích xu hướng và truy vấn dữ liệu phức tạp, từ đó nâng cao hiệu quả sử dụng dữ liệu phục vụ nghiên cứu và vận hành thực tiễn.

- **Hỗ trợ dự báo và giảm thiểu ùn tắc giao thông**

Việc tích hợp các mô hình trí tuệ nhân tạo và học máy vào hệ thống cho phép dự báo lưu lượng phương tiện, tốc độ lưu thông và nguy cơ ùn tắc trong tương lai gần. Các kết quả dự báo có thể hỗ trợ cơ quan quản lý chủ động trong công tác điều tiết giao thông, bố trí lực lượng chức năng hợp lý và đưa ra các giải pháp ứng phó kịp thời nhằm giảm thiểu tác động tiêu cực của ùn tắc giao thông đối với đời sống đô thị.

- **Tạo nền tảng cho phát triển hệ sinh thái giao thông thông minh**

Hệ thống được thiết kế theo hướng kiến trúc mở và có khả năng mở rộng linh hoạt, cho phép tích hợp thêm nhiều nguồn dữ liệu và dịch vụ thông minh khác trong tương lai như dữ liệu IoT, camera giao thông, dữ liệu thời tiết hoặc các nền tảng giao thông công cộng. Điều này góp phần hình thành nền tảng hạ tầng dữ liệu phục vụ phát triển hệ sinh thái giao thông thông minh và đô thị thông minh tại Việt Nam.

- **Khả năng ứng dụng và mở rộng thực tiễn**

Mô hình hệ thống được triển khai trên nền tảng điện toán đám mây và sử dụng các công nghệ mã nguồn mở phổ biến, qua đó bảo đảm tính linh hoạt, khả năng mở rộng và tối ưu chi phí triển khai. Kết quả nghiên cứu có thể được sử dụng làm cơ sở tham khảo cho các bài toán quản lý giao thông tại các đô thị lớn khác ở Việt Nam.

1.4.2 Ý nghĩa khoa học

Bên cạnh giá trị thực tiễn, đề tài còn mang ý nghĩa khoa học trong lĩnh vực dữ liệu lớn, hệ thống thông tin địa lý và trí tuệ nhân tạo ứng dụng trong giao thông thông minh.

- **Đóng góp về mô hình hóa dữ liệu giao thông không gian – thời gian**

Đề tài nghiên cứu và xây dựng mô hình kho dữ liệu giao thông theo hướng tích hợp dữ liệu không gian – thời gian (spatio-temporal data warehouse), kết hợp giữa dữ liệu giao thông thời gian thực và dữ liệu địa lý không gian. Việc áp dụng các mô hình dữ liệu đa chiều trong bối cảnh dữ liệu giao thông tại Việt Nam góp phần kiểm chứng tính khả thi của phương pháp tiếp cận này đối với các bài toán phân tích giao thông đô thị.

- **Đóng góp trong lĩnh vực xử lý và tích hợp dữ liệu giao thông đa nguồn**

Nghiên cứu đề xuất quy trình ETL/ELT phục vụ thu thập, làm sạch, chuẩn hóa và tích hợp dữ liệu từ nhiều nguồn dị biệt khác nhau. Đây là cơ sở quan trọng cho việc xây dựng các hệ thống dữ liệu giao thông có khả năng xử lý dữ liệu lớn, dữ liệu thời gian thực và dữ liệu không gian trong môi trường đô thị thông minh.

- **Đánh giá khả năng ứng dụng trí tuệ nhân tạo trong dự báo giao thông**

Đề tài thực hiện thử nghiệm các mô hình học máy và học sâu trên dữ liệu giao thông thực tế nhằm đánh giá hiệu quả dự báo lưu lượng và tình trạng ùn tắc giao thông. Kết quả nghiên cứu góp phần bổ sung cơ sở thực nghiệm cho việc ứng dụng trí tuệ nhân tạo trong lĩnh vực giao thông thông minh tại Việt Nam, đặc biệt trong bối cảnh giao thông hỗn hợp và có mật độ phương tiện cao.

- **Đóng góp về kiến trúc hệ thống dữ liệu giao thông thông minh**

Kiến trúc hệ thống được xây dựng theo mô hình phân tầng bao gồm tầng thu thập dữ liệu, tầng kho dữ liệu không gian, tầng trí tuệ nhân tạo và tầng trực quan hóa. Mô hình này có thể được xem như một hướng tiếp cận tham khảo cho việc phát triển các nền tảng Traffic Integrated Operations Center (Traffic IOC) hoặc các hệ thống phân tích giao thông thông minh trong tương lai.

1.5 Đóng góp của đề tài

Đề tài hướng đến việc xây dựng một nền tảng dữ liệu giao thông thông minh tích hợp nhiều thành phần công nghệ hiện đại, bao gồm kho dữ liệu không gian, xử lý dữ liệu thời gian thực, trí tuệ nhân tạo và hệ thống trực quan hóa phục vụ quản lý giao thông đô thị. Trên cơ sở đó, các đóng góp chính của đề tài được xác định như sau:

- **Đề xuất mô hình kho dữ liệu giao thông tích hợp dữ liệu không gian – thời gian**

Đề tài xây dựng mô hình kho dữ liệu giao thông theo hướng tích hợp dữ liệu đa nguồn kết hợp với dữ liệu địa lý không gian (geospatial data). Mô hình dữ liệu được thiết kế nhằm hỗ trợ lưu trữ, quản lý và phân tích dữ liệu giao thông theo cả hai chiều không gian và thời gian, phục vụ hiệu quả cho các bài toán thống kê, phân tích lưu lượng và dự báo giao thông trong môi trường đô thị thông minh.

- **Xây dựng quy trình ETL/ELT tự động cho dữ liệu giao thông thời gian thực**

Đề tài triển khai quy trình thu thập, xử lý và đồng bộ dữ liệu giao thông theo thời gian thực từ nhiều nguồn dữ liệu khác nhau thông qua API và dữ liệu bản đồ số. Quy trình ETL/ELT được tối ưu nhằm bảo đảm khả năng xử lý dữ liệu liên tục, tự động hóa việc làm sạch dữ liệu, xử lý ngoại lệ và chuẩn hóa dữ liệu trước khi lưu trữ vào kho dữ liệu trung tâm.

- **Ứng dụng công nghệ cơ sở dữ liệu không gian trong quản lý giao thông**

Đề tài khai thác khả năng của PostgreSQL kết hợp với PostGIS để xây dựng kho dữ liệu không gian phục vụ lưu trữ và xử lý các thực thể giao thông như tuyến đường, đoạn đường, nút giao và dữ liệu lưu lượng theo vị trí địa lý. Việc ứng dụng công nghệ dữ liệu không gian góp phần nâng cao khả năng phân tích giao thông theo khu vực và hỗ trợ trực quan hóa dữ liệu trên nền bản đồ số.

- **Tích hợp mô hình trí tuệ nhân tạo phục vụ dự báo giao thông**

Đề tài nghiên cứu và triển khai các mô hình học máy và học sâu nhằm dự báo tình trạng giao thông trong ngắn hạn, bao gồm dự báo lưu lượng phương tiện, tốc độ lưu thông và nguy cơ ùn tắc. Các mô hình được huấn luyện và đánh giá trên tập dữ liệu thực tế đã được xử lý và chuẩn hóa từ kho dữ liệu giao thông, qua đó kiểm chứng khả năng ứng dụng của trí tuệ nhân tạo trong bài toán giao thông đô thị tại Việt Nam.

- **Xây dựng hệ thống trực quan hóa và hỗ trợ ra quyết định**

Đề tài phát triển nền tảng dashboard và web application phục vụ giám sát giao thông theo thời gian thực, hỗ trợ trực quan hóa dữ liệu dưới dạng biểu đồ, bản đồ nhiệt và thống kê lưu lượng giao thông. Hệ thống góp phần hỗ trợ cơ quan quản lý trong công tác điều hành, phân luồng và đánh giá hiện trạng giao thông đô thị.

- **Đề xuất kiến trúc hệ thống giao thông thông minh có khả năng mở rộng**

Đề tài xây dựng kiến trúc hệ thống theo mô hình phân tầng bao gồm tầng thu thập dữ liệu, tầng kho dữ liệu, tầng trí tuệ nhân tạo và tầng giao diện người dùng. Hệ thống được triển khai trên nền tảng điện toán đám mây kết hợp với công nghệ container hóa, qua đó bảo đảm tính linh hoạt, khả năng mở rộng và khả năng tích hợp thêm các nguồn dữ liệu hoặc dịch vụ thông minh trong tương lai.

- **Đóng góp về mặt thực nghiệm trong xử lý dữ liệu giao thông quy mô lớn**

Trong quá trình triển khai, đề tài thực hiện nhiều giải pháp tối ưu hóa hệ thống nhằm đáp ứng yêu cầu xử lý dữ liệu giao thông thời gian thực, bao gồm tối ưu hóa hiệu năng ETL, xử lý đa luồng, quản lý kết nối cơ sở dữ liệu và tối ưu hạ tầng triển khai trên môi trường điện toán đám mây. Các kết quả thực nghiệm góp phần cung cấp cơ sở tham khảo cho việc xây dựng các hệ thống dữ liệu giao thông thông minh trong thực tế.

2 Quản lý dự án và phân công nhiệm vụ

Để đảm bảo quá trình nghiên cứu và phát triển hệ thống diễn ra xuyên suốt, đúng tiến độ và đạt được các yêu cầu kỹ thuật đã đề ra, việc thiết lập một quy trình quản lý dự án bài bản là yếu tố tiên quyết. Chương này trình bày chi tiết về phương pháp luận quản lý được áp dụng, hệ sinh thái công cụ hỗ trợ, cách thức tổ chức nhóm và lộ trình thực hiện đồ án.

2.1 Phương pháp quản lý dự án

Với đặc thù của một đồ án phần mềm có kiến trúc phức tạp, bao gồm nhiều phân hệ tương tác lẫn nhau (Web Dashboard phân tích dữ liệu giao thông, Ứng dụng tra cứu cho người dân, Backend Server và Data Warehouse), nhóm quyết định áp dụng phương pháp phát triển phần mềm linh hoạt **Agile**, kết hợp với khung làm việc **Scrum**.

Việc áp dụng Agile/Scrum mang lại những lợi thế cụ thể cho nhóm:

- **Tính thích ứng cao:** Cho phép nhóm dễ dàng điều chỉnh các tính năng (ví dụ: tính chỉnh biểu đồ phân tích BI, thay đổi logic tính toán chỉ số TTI, BI) dựa trên kết quả nghiệm thu từng phần mà không làm đứt gãy toàn bộ cấu trúc dự án.
- **Phát triển lặp (Iterative Development):** Quá trình phát triển được chia nhỏ thành các chu kỳ ngắn gọi là Sprint (kéo dài từ 1 đến 2 tuần). Cuối mỗi Sprint, nhóm sẽ cho ra mắt một phần mềm có khả năng chạy được (shippable product) để đánh giá và kiểm thử ngay lập tức.
- **Tối ưu hóa giao tiếp:** Tổ chức các buổi họp ngắn (Sync-up meeting) từ 2-3 lần/tuần để cập nhật tiến độ, gỡ rối các rào cản kỹ thuật (bottlenecks) giữa các thành viên.

Quy trình triển khai Scrum thực tế của nhóm:

1. **Product Backlog:** Tập hợp toàn bộ các tính năng cần có của hệ thống (ví dụ: Chức năng xem bản đồ nhiệt, Tính năng gợi ý lộ trình, Module ETL xử lý dữ liệu lịch sử).
2. **Sprint Planning:** Lựa chọn các task từ Backlog đưa vào Sprint hiện tại dựa trên mức độ ưu tiên.
3. **Sprint Execution:** Hiện thực hóa các tính năng bằng mã nguồn.
4. **Sprint Review & Retrospective:** Tổng kết, ghép nối (integrate) các module, kiểm tra lỗi (bug) và rút kinh nghiệm cho chu kỳ tiếp theo.

2.2 Công cụ quản lý và Môi trường làm việc nhóm

Để hiện thực hóa phương pháp Agile, nhóm đã lựa chọn và đồng bộ hóa một hệ sinh thái công cụ quản lý chuyên nghiệp, bám sát với tiêu chuẩn của các doanh nghiệp phần mềm hiện nay:

- **Quản lý mã nguồn (Source Code Management):** Sử dụng **Git** và nền tảng **GitHub** để lưu trữ mã nguồn. Áp dụng chiến lược Git Flow cơ bản, chia tách các nhánh main (môi trường production), develop (môi trường tích hợp) và các nhánh feature/* cho từng tính năng riêng biệt để tránh xung đột mã nguồn.
- **Quản lý công việc (Task Management):** Khởi tạo không gian làm việc trên **Linear** theo mô hình bảng Kanban. Các công việc được thiết lập dưới dạng thẻ (card) và di chuyển qua các cột trạng thái: *To Do*, *In Progress*, *In Review*, và *Done*.
- **Môi trường giao tiếp:** Sử dụng **Discord** làm kênh trao đổi thông tin chính, chia các channel (kênh) riêng biệt cho từng luồng công việc như #frontend-ui, #backend-api, và #deployment-alerts.
- **Môi trường phát triển và Tích hợp:** Chuẩn hóa môi trường cục bộ (local environment) giữa các thành viên bằng **Docker**, đảm bảo tính nhất quán từ môi trường phát triển (Development) đến môi trường triển khai (Production). Sử dụng nền tảng chia sẻ tài liệu API (như Postman/Swagger) để Frontend và Backend có thể làm việc độc lập.

2.3 Phân công vai trò và chi tiết nhiệm vụ

Dựa trên thế mạnh cốt lõi và định hướng chuyên môn của từng thành viên, khối lượng công việc của đồ án được phân bổ một cách chuyên môn hóa nhưng vẫn đảm bảo tính phối hợp chặt chẽ (Cross-functional). Cụ thể như sau:

Sinh viên 1: Hồ Trần Ngọc Liêm (MSSV: 2211835)

- **Vai trò:** Software Fullstack Developer.
- **Nhiệm vụ chi tiết:**
 - Chịu trách nhiệm xây dựng kiến trúc giao diện, tối ưu hóa Trải nghiệm người dùng (UX) và Giao diện người dùng (UI) cho Web Dashboard điều hành và App người dân.
 - Xây dựng và phát triển Frontend sử dụng thư viện React kết hợp với Ant Design để tạo ra các biểu đồ phân tích trực quan (Recharts), bản đồ số (Mapbox) xử lý khối lượng dữ liệu lớn.
 - Đảm nhiệm vai trò theo dõi tiến độ chung và phối hợp triển khai hệ thống (Deployment).

Sinh viên 2: Đinh Nguyễn Việt Khiêm (MSSV: 2211565)

- **Vai trò:** Data Engineer (Hạ tầng dữ liệu).
- **Nhiệm vụ chi tiết:**
 - Nghiên cứu và chỉnh sửa, tối ưu hoá cấu trúc Data Warehouse từ Giai đoạn 1 để lưu trữ luồng dữ liệu giao thông khổng lồ, phù hợp với Giai đoạn 2.
 - Xử lý và làm sạch dữ liệu trạng thái giao thông, chuẩn hóa dữ liệu từ TomTom và các nguồn khác để đảm bảo tính nhất quán và chính xác. Đánh giá chất lượng dữ liệu đầu vào, kiểm tra lỗi, tìm kiếm nguồn dữ liệu thay thế khi cần thiết.
 - Xây dựng và tối ưu hóa luồng ETL trên hạ tầng Azure Virtual Machine.

Sinh viên 3: Nguyễn Thành Dũng (MSSV: 2210589)

- **Vai trò:** AI/Algorithm Developer.
- **Nhiệm vụ chi tiết:**
 - Nghiên cứu, xây dựng và tích hợp mô hình phân tích, dự báo điểm nghẽn giao thông.
 - Chịu trách nhiệm xử lý các bài toán nghiệp vụ cốt lõi (tính toán các chỉ số TTI, BI, PTI theo chuẩn quốc tế).
 - Thiết kế kịch bản kiểm thử (Load test, API Latency) và thực hiện đánh giá hiệu năng hệ thống toàn diện để đảm bảo tính ổn định của mô hình.

2.4 Tiến độ thực hiện đồ án

Quá trình thực hiện đồ án được chia thành các giai đoạn chính (Milestones), trải dài trong suốt 15 tuần của học kỳ. Bảng 1 dưới đây mô tả chi tiết các đầu mục công việc, kết quả kỳ vọng và người phụ trách cho từng giai đoạn.

Bảng 1: Chi tiết các mốc thời gian thực hiện đồ án

Giai đoạn	Thời gian	Mô tả công việc cốt lõi	Kết quả đạt được
Giai đoạn 1: Khởi động & Khảo sát	Tuần 1	Thu thập tài liệu, khảo sát nghiệp vụ giao thông. Lựa chọn stack công nghệ và thiết kế kiến trúc. Setup Git và các công cụ làm việc. Hợp thống nhất quy trình làm việc	Khởi tạo dự án thành công với kiến trúc Client-Server.
Giai đoạn 2: Module Dashboard & Thông tin Giao thông	Tuần 2 - Tuần 5	Xử lý luồng ETL liên quan đến trạng thái giao thông hiện tại. Xử lý render 430.000 segment giao thông lên Mapbox map trên giao diện và các thông tin khác.	Luồng ETL Traffic Flow hoàn chỉnh cho TPHCM. Trang Dashboard Admin.
Giai đoạn 3: Module Thống kê	Tuần 7 - Tuần 9	Xây dựng các API liên quan đến thống kê. Phát triển UI & chức năng Thống kê trên app dashboard.	Web Admin và App hoạt động ổn định.
Giai đoạn 4: Module Dự báo & Dẫn đường	Tuần 10 - Tuần 13	Xây dựng & train model dự báo kẹt xe trong 15 phút tới và tích hợp lên giao diện. Áp dụng thuật toán A* 2 chiều vào chức năng dẫn đường và phát triển giao diện mobile cho người dân.	Web Admin, Mobile App và hệ thống hoạt động ổn định.
Giai đoạn 5: Triển khai & Hoàn thiện báo cáo	Tuần 14 - Tuần 15	Triển khai model, server lên Azure App Services. Triển khai giao diện lên Vercel. Viết tài liệu báo cáo đồ án, chuẩn bị slide và kịch bản Demo.	Quyển báo cáo. Slide thuyết trình. App production chạy ổn định.

3 Kiến thức và công nghệ nền tảng

Chương này trình bày các cơ sở lý thuyết và nền tảng công nghệ cốt lõi làm tiền đề cho việc xây dựng hệ thống Traffic-IOC. Nội dung được chia thành các trụ cột chính: Kỹ thuật dữ liệu, Trí tuệ nhân tạo, Công cụ sử dụng, Ứng dụng định tuyến và Hạ tầng triển khai.

3.1 Nền tảng Kỹ thuật dữ liệu

Trong kỷ nguyên dữ liệu lớn (Big Data), việc quản lý và khai thác hiệu quả nguồn dữ liệu khổng lồ từ hệ thống giao thông thông minh (ITS) là một thách thức lớn. Phần này sẽ trình bày các cơ sở lý thuyết quan trọng làm nền tảng cho việc xây dựng hệ thống, bao gồm lý thuyết về Kho dữ liệu (Data Warehouse), mô hình hóa dữ liệu quan hệ và không gian, cũng như các nguyên lý cơ bản trong phân tích giao thông.

3.1.1 Tổng quan về Kho dữ liệu (Data Warehouse) và OLAP

3.1.1.1 a. Khái niệm

Kho dữ liệu (Data Warehouse - DWH) là một hệ thống được thiết kế để lưu trữ, quản lý và phân tích dữ liệu lịch sử từ nhiều nguồn khác nhau nhằm hỗ trợ quá trình ra quyết định (Decision Support System - DSS).

Theo định nghĩa kinh điển của W.H. Inmon – người được coi là cha đẻ của Kho dữ liệu:

"Kho dữ liệu là một tập hợp dữ liệu hướng chủ đề (subject-oriented), tích hợp (integrated), biến đổi theo thời gian (time-variant) và bất biến (non-volatile) để hỗ trợ sự quản lý ra quyết định."

3.1.1.2 b. Các đặc tính cốt lõi của Kho dữ liệu

Một hệ thống DWH chuẩn mực phải đảm bảo 4 đặc tính sau:

- **Hướng chủ đề (Subject-Oriented):** Dữ liệu trong DWH được tổ chức theo các chủ đề chính của nghiệp vụ thay vì theo quy trình ứng dụng. Trong ngữ cảnh giao thông, các chủ đề có thể là "Lưu lượng xe", "Sự cố", hoặc "Người dùng".
- **Tích tích hợp (Integrated):** Dữ liệu từ các nguồn không đồng nhất phải được làm sạch, chuẩn hóa và thống nhất về định dạng trước khi đưa vào kho.
- **Biến đổi theo thời gian (Time-Variant):** Mọi dữ liệu trong DWH đều gắn liền với một mốc thời gian cụ thể, phục vụ phân tích xu hướng lâu dài.
- **Tính bất biến (Non-Volatile):** Dữ liệu sau khi được nạp vào DWH sẽ không bị thay đổi hoặc xóa bỏ, đảm bảo tính toàn vẹn của lịch sử.

3.1.1.3 c. Kiến trúc tổng quát của hệ thống Kho dữ liệu

Kiến trúc của một hệ thống DWH thường bao gồm 3 lớp chính:

- **Lớp nguồn dữ liệu (Data Source Layer):** Bao gồm các hệ thống bên ngoài cung cấp dữ liệu thô (API giao thông, file log).
- **Lớp Staging (Staging Area):** Vùng đệm trung gian lưu trữ dữ liệu thô vừa trích xuất để không ảnh hưởng đến hệ thống nguồn.
- **Lớp lưu trữ dữ liệu (Data Storage Layer):** Nơi chứa dữ liệu đã được làm sạch và mô hình hóa (thường theo dạng Star Schema hoặc Snowflake Schema).

3.1.1.4 d. Lý thuyết Xử lý Phân tích Trực tuyến (OLAP)

Để đáp ứng nhu cầu phân tích dữ liệu lớn và trích xuất tri thức, đề tài vận dụng lý thuyết OLAP (Online Analytical Processing):

- **Khối dữ liệu (Data Cube):** Dữ liệu giao thông được trừu tượng hóa thành các khối đa chiều (Không gian, Thời gian, Chỉ số hiệu năng).
- **Thao tác Roll-up và Drill-down:** Hỗ trợ người quản trị khả năng phân tích đa cấp độ, từ tổng thể thành phổ xuống chi tiết từng đoạn đường cụ thể.

3.1.2 Lý thuyết về Cơ sở dữ liệu quan hệ và SQL

3.1.2.1 a. Mô hình dữ liệu quan hệ (Relational Data Model)

Mô hình dữ liệu quan hệ tổ chức dữ liệu thành các bảng liên kết qua khóa chính và khóa ngoại. Các giao dịch tuân thủ nghiêm ngặt 4 tính chất ACID: Atomicity (Nguyên tử), Consistency (Nhất quán), Isolation (Cô lập) và Durability (Bền vững).

3.1.2.2 b. Xử lý dữ liệu bán cấu trúc (JSONB)

Trong bối cảnh dữ liệu lớn, PostgreSQL tích hợp kiểu dữ liệu **JSONB (Binary JSON)** để lưu trữ dữ liệu bán cấu trúc từ API. JSONB cho phép truy xuất trực tiếp đến các khóa mà không cần phân tích cú pháp lại, giúp xây dựng lớp Staging linh hoạt (Schema-less).

3.1.2.3 c. Các kỹ thuật tối ưu hóa lưu trữ

- **Phân vùng dữ liệu (Partitioning):** Chia bảng lớn thành các bảng con theo thời gian (Range Partitioning) để tăng tốc độ truy vấn.
- **Đánh chỉ mục (Indexing):** Sử dụng B-Tree cho dữ liệu chuẩn và GIN/GiST cho dữ liệu JSONB hoặc không gian.

3.1.3 Hệ thống thông tin địa lý (GIS) và Dữ liệu không gian

3.1.3.1 a. Mô hình dữ liệu Vector

Dữ liệu không gian được biểu diễn qua Điểm (vị trí sự cố), Đường (đoạn đường) và Vùng (đơn vị hành chính) tuân theo chuẩn Simple Features của OGC.

3.1.3.2 b. Hệ quy chiếu tọa độ (CRS)

Hệ thống sử dụng chuẩn **WGS84 (EPSG:4326)** cho dữ liệu GPS và API. Đối với các phép tính toán khoảng cách và diện tích, kiểu dữ liệu **GEOGRAPHY** trong PostGIS được ưu tiên sử dụng để đảm bảo độ chính xác trên bề mặt cầu.

3.1.4 Lý thuyết về Giao thông (Traffic Flow Theory)

3.1.4.1 a. Các thông số cơ bản

Dòng giao thông vĩ mô được đặc trưng bởi: Lưu lượng (q), Tốc độ (v) và Mật độ (k). Mối quan hệ cơ bản: $q = k \times v$.

3.1.4.2 b. Mô hình Greenshields

Hệ thống sử dụng mô hình Greenshields để ước lượng mật độ dựa trên tốc độ và tốc độ dòng tự do (v_f):

$$v = v_f \times \left(1 - \frac{k}{k_j}\right)$$

3.1.4.3 c. Mức độ phục vụ (Level of Service - LOS)

Đánh giá chất lượng vận hành từ mức A (Tự do) đến F (Ùn tắc) dựa trên tỷ số giữa tốc độ hiện tại và tốc độ tự do, phục vụ trực quan hóa Heatmap.

3.1.5 Công nghệ Kỹ thuật dữ liệu sử dụng

3.1.5.1 a. Ngôn ngữ Python và quy trình ETL

Sử dụng thư viện **Requests** để thu thập dữ liệu, **Pandas** để biến đổi và làm sạch, **SQLAlchemy** để nạp dữ liệu vào database và **OSMnx** để trích xuất cấu trúc mạng lưới từ OpenStreetMap.

3.1.5.2 b. Hệ quản trị PostgreSQL và PostGIS

PostgreSQL đóng vai trò kép là Staging và Data Warehouse. PostGIS mở rộng khả năng xử lý không gian như ST_DWithin, ST_Intersects để thực hiện Map Matching và Spatial Join.

3.1.5.3 c. Nguồn dữ liệu và Tự động hóa

Tích hợp dữ liệu từ TomTom Traffic API và OpenWeatherMap API. Quy trình được tự động hóa bằng Windows Task Scheduler với chu kỳ 15 phút/lần.

3.2 Nền tảng Trí tuệ nhân tạo

3.2.1 Tổng quan về Học máy và Dự báo chuỗi thời gian

Dự báo chuỗi thời gian là một trong những bài toán cốt lõi của Học máy (Machine Learning) và Khoa học dữ liệu, đặc biệt quan trọng trong các lĩnh vực có tính động lực học cao như tài chính, khí tượng và giao thông vận tải. Khác với các dữ liệu dạng bảng tĩnh, dữ liệu chuỗi thời gian chứa đựng chiều thông tin thứ tư – chiều thời gian, đòi hỏi các phương pháp tiếp cận chuyên biệt.

3.2.1.1 a. Khái niệm chuỗi thời gian (Time-series Data) và Đặc tính dữ liệu giao thông

Chuỗi thời gian là một tập hợp các quan trắc dữ liệu được thu thập liên tiếp theo các khoảng thời gian đều đặn (ví dụ: mỗi phút, mỗi 15 phút, mỗi giờ). Đối với hệ thống giao thông đô thị, dữ liệu cảm biến thu về (như vận tốc dòng xe, lưu lượng, độ trễ) là một chuỗi thời gian điển hình mang ba đặc trưng nền tảng:

- **Tính phụ thuộc thời gian (Temporal Dependence / Autocorrelation):** Trạng thái giao thông tại thời điểm hiện tại t có mối tương quan tuyến tính hoặc phi tuyến mạnh mẽ với trạng thái ở các thời điểm trước đó ($t-1, t-2$). Cụ thể, một đám kẹt xe không hình thành tức thời mà là sự tích lũy quán tính từ sự sụt giảm vận tốc của các chu kỳ trước.
- **Tính mùa vụ và Chu kỳ (Seasonality & Cyclicity):** Giao thông đô thị tuân theo nhịp độ sinh hoạt của con người. Dữ liệu thể hiện tính chu kỳ rõ rệt theo ngày (đỉnh điểm vào giờ cao điểm sáng/chiều) và tính mùa vụ theo tuần (mô hình di chuyển ngày thường khác biệt hoàn toàn so với cuối tuần).
- **Tính ngẫu nhiên và Hỗn loạn (Randomness & Chaos):** Bên cạnh các quy luật mang tính chu kỳ, hệ thống giao thông còn chịu sự chi phối của các biến cố ngoại lai ngẫu nhiên không thể lường trước (như tai nạn, thời tiết cực đoan, sự kiện công cộng). Yếu tố này tạo ra các điểm dị thường (Outliers), phá vỡ tính ổn định của chuỗi thời gian.

3.2.1.2 b. Mô hình thống kê truyền thống (ARIMA/SARIMA)

Trước khi kỷ nguyên Học sâu (Deep Learning) bùng nổ, các mô hình thống kê học truyền thống là công cụ tiêu chuẩn để xử lý dữ liệu chuỗi thời gian. Nổi bật nhất là mô hình ARIMA (AutoRegressive Integrated Moving Average) và biến thể mở rộng SARIMA (bao gồm yếu tố mùa vụ - Seasonal).

- **Nguyên lý về tính dừng (Stationarity):** Yêu cầu tiên quyết để các mô hình thống kê này hoạt động là dữ liệu phải có "Tính dừng". Một chuỗi thời gian được coi là dừng khi các đặc trưng thống kê cốt lõi của nó (Kỳ vọng/Trung bình μ và Phương sai σ^2) không thay đổi theo thời gian.
- **Các thành phần cấu trúc của ARIMA:** Mô hình ARIMA được ký hiệu là $ARIMA(p, d, q)$, kết hợp ba yếu tố toán học:
 1. **Thành phần Tự hồi quy - AR (AutoRegressive, bậc p):** Sử dụng kết hợp tuyến tính của p giá trị quá khứ để dự báo tương lai. Nó giả định rằng giá trị hiện tại bị ảnh hưởng trực tiếp bởi lịch sử của chính nó.
 2. **Thành phần Tích hợp - I (Integrated, bậc d):** Là số lần sai phân (differencing) cần thiết để biến đổi một chuỗi thời gian không dừng thành một chuỗi dừng. Sai phân bậc 1 được tính bằng: $y'_t = y_t - y_{t-1}$.
 3. **Thành phần Trung bình trượt - MA (Moving Average, bậc q):** Sử dụng các sai số dự báo trong quá khứ (lỗi ngẫu nhiên ϵ) để thiết lập mô hình cho giá trị hiện tại, giúp làm mịn các cú sốc đột ngột.

Phương trình toán học tổng quát của mô hình ARIMA có dạng:

$$y'_t = c + \phi_1 y'_{t-1} + \dots + \phi_p y'_{t-p} + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q} + \epsilon_t$$

(Trong đó: y'_t là chuỗi đã lấy sai phân, ϕ là hệ số tự hồi quy, θ là hệ số trung bình trượt, c là hằng số và ϵ_t là nhiễu trắng).

Hạn chế: Mặc dù sở hữu nền tảng toán học vững chắc, ARIMA/SARIMA gặp khó khăn lớn khi áp dụng vào dữ liệu giao thông thực tế. Giao thông là một hệ động lực phi tuyến phức tạp (Non-linear Dynamical System) và thường xuyên vi phạm giả định về "Tính dừng". Sự biến động dữ dội của vận tốc dòng xe khiến các mô hình tuyến tính này phản ứng chậm và thiếu chính xác trước các sự cố vỡ trận bất ngờ.

3.2.1.3 c. Học có giám sát (Supervised Learning) trong bài toán Phân lớp đa nhãn

Để vượt qua các rào cản của mô hình tuyến tính, phương pháp Học có giám sát (Supervised Learning) thuộc nhánh Học máy/Học sâu được áp dụng. Trong bối cảnh cảnh báo sớm giao thông, thay vì cố gắng dự báo chính xác một con số liên tục (Hồi quy - Regression) như vận tốc sẽ là 23.5 km/h, bài toán được định dạng lại thành Phân lớp đa nhãn (Multi-class Classification).

- **Rời rạc hóa không gian trạng thái:** Biến mục tiêu liên tục (Chỉ số kẹt xe hoặc Vận tốc) được ánh xạ (mapping) và chia thành các cấp độ phân loại rời rạc (Ví dụ: từ Lớp 0 - Thông thoáng, đến Lớp 5 - Ùn tắc nghiêm trọng).
- **Cơ chế hoạt động:** Hệ thống Học có giám sát cần một bộ dữ liệu lịch sử đã được gán nhãn (Labeled Data). Mô hình (có thể là Rừng ngẫu nhiên, Máy véc-tơ hỗ trợ SVM, hoặc Mạng Nơ-ron nhân tạo DNN) sẽ học hàm ánh xạ $f(X) \rightarrow Y$. Trong đó X là ma trận đặc trưng đầu vào (Vận tốc quá khứ, Thời gian, Tọa độ) và Y là xác suất thuộc về một trong các nhãn phân lớp kẹt xe.
- **Ưu điểm:** Cách tiếp cận phân lớp này cực kỳ phù hợp cho các hệ thống cảnh báo, vì người quản trị hệ thống và người dùng cuối quan tâm đến mức độ nghiêm trọng của trạng thái (để đưa ra quyết định hành động) hơn là sự xê dịch nhỏ của các con số vận tốc tuyệt đối. Thêm vào đó, các mô hình Học sâu giám sát có khả năng xấp xỉ các hàm phi tuyến cực kỳ phức tạp mà ARIMA không thể nắm bắt.

3.2.2 Mạng Nơ-ron sâu (Deep Learning)

Để xử lý khối lượng dữ liệu khổng lồ và nắm bắt các quy luật phi tuyến phức tạp mà các mô hình thống kê truyền thống không thể giải quyết, kiến trúc Mạng Nơ-ron sâu (Deep Neural Networks - DNN) đóng vai trò là hạt nhân tính toán. Mục này trình bày các lý thuyết nền tảng cấu thành nên hệ thống dự báo.

3.2.2.1 a. Mạng Nơ-ron tái phát (RNN) và Mạng bộ nhớ ngắn hạn dài (LSTM)

Mạng Nơ-ron truyền thẳng (Feed-forward Neural Network) tiêu chuẩn không có cơ chế lưu trữ trạng thái quá khứ, do đó hoàn toàn bất lực trước dữ liệu chuỗi thời gian. Mạng Nơ-ron tái phát (Recurrent Neural Network - RNN) khắc phục điều này bằng cách sử dụng các vòng lặp (loops) cho phép thông tin lưu lại trong mạng. Tuy nhiên, khi huấn luyện với các chuỗi dữ liệu dài, RNN truyền thống vấp phải vấn đề Tiêu biến đạo hàm (Vanishing Gradient), khiến mạng mất khả năng học các phụ thuộc xa trong quá khứ.

Để giải quyết triệt để rào cản này, kiến trúc **Mạng bộ nhớ ngắn hạn dài (Long Short-Term Memory - LSTM)**** được giới thiệu. Sức mạnh của LSTM nằm ở cấu trúc Tế bào nhớ (Memory Cell) phức tạp, hoạt động như một băng chuyền thông tin chạy xuyên suốt chuỗi thời gian, kết hợp với cơ chế điều tiết thông tin tinh vi thông qua 3 loại cổng (Gates):

1. **Cổng Quên (Forget Gate - f_t):** Quyết định lượng thông tin nào từ quá khứ cần bị loại bỏ khỏi tế bào nhớ. Hàm Sigmoid xuất ra giá trị từ 0 (quên hoàn toàn) đến 1 (giữ lại toàn bộ).

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. **Cổng Đầu vào (Input Gate - i_t):** Quyết định thông tin mới nào từ hiện tại sẽ được cập nhật vào tế bào nhớ. Nó bao gồm một tầng Sigmoid để chọn lọc và một tầng Tanh để tạo ra vector giá trị ứng viên $\tilde{C}_t$.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

3. **Trạng thái Tế bào (Cell State - C_t):** Là sự kết hợp tuyến tính giữa trí nhớ cũ (đã lọc qua cổng quên) và thông tin mới (đã lọc qua cổng đầu vào). Đây là yếu tố cốt lõi giúp LSTM bảo toàn được đạo hàm khi truyền ngược qua nhiều bước thời gian.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

4. **Cổng Đầu ra (Output Gate - o_t):** Tính toán trạng thái ẩn (h_t) cho bước tiếp theo dựa trên trạng thái tế bào đã được cập nhật.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Nhờ cơ cấu này, LSTM có khả năng "ghi nhớ" (bắt giữ phụ thuộc dài hạn - Long-term dependencies) các tín hiệu ồ ạt tích tụ từ nhiều giờ trước đó, đồng thời "quên đi" các nhiễu loạn cục bộ không mang ý nghĩa.

3.2.2.2 b. Tầng nhúng (Embedding Layer)

Trong dữ liệu giao thông, bên cạnh các biến liên tục (vận tốc, lưu lượng), còn tồn tại rất nhiều biến phân loại (Categorical Data) mang tính định danh ngữ cảnh, chẳng hạn như: ID đoạn đường, Ngày trong tuần, hoặc Giờ trong ngày.

Phương pháp mã hóa truyền thống phổ biến nhất là One-Hot Encoding. Tuy nhiên, nếu áp dụng One-Hot cho một mạng lưới giao thông có hàng ngàn đoạn đường, ma trận đầu vào sẽ trở nên thưa thớt (Sparse Matrix) và vấp phải hội chứng Bùng nổ số chiều (Curse of Dimensionality). Điều này vắt kiệt tài nguyên tính toán bộ nhớ và làm mô hình rất khó hội tụ.

****Tầng nhúng (Embedding Layer)**** là giải pháp kiến trúc tối ưu để thay thế. Về mặt lý thuyết, Tầng nhúng thực hiện một phép ánh xạ tuyến tính, chuyển đổi các giá trị rời rạc trong không gian chiều cao thành các véc-tơ liên tục, trù phú thông tin (Dense Vectors) trong không gian chiều thấp hơn rất nhiều.

- Khác với One-Hot Encoding cố định, các trọng số của Tầng nhúng là các tham số có thể học được (Learnable Parameters) trong quá trình huấn luyện bằng Gradient Descent.
- Hệ quả là, mô hình có thể tự động khám phá và biểu diễn các mối quan hệ ngữ nghĩa (Semantic Relationships). Ví dụ: Hai đoạn đường A và B dù cách xa nhau về mặt địa lý nhưng nếu có cùng chung đặc tính là

"thường xuyên kẹt xe vào chiều thứ 6", vector nhúng của chúng sẽ tự động xích lại gần nhau trong không gian Euclidean nhiều chiều.

3.2.2.3 c. Các hàm tối ưu và hàm mất mát nâng cao (Advanced Loss Functions)

Trong học sâu, hàm mất mát (Loss Function) định hướng cách mô hình cập nhật trọng số. Các hàm cơ bản như Cross-Entropy hoặc Mean Squared Error (MSE) thường thất bại khi đối mặt với dữ liệu mất cân bằng hoặc chứa nhiễu nhiễu.

- **Focal Loss (Khắc phục mất cân bằng lớp):** Trong bài toán phân lớp, Cross-Entropy tiêu chuẩn tính tổng lỗi bình đẳng cho mọi mẫu. Hệ quả là các lớp đa số (ví dụ: trạng thái giao thông thông thoáng) dù rất dễ dự báo nhưng với số lượng áp đảo sẽ tạo ra một dốc đạo hàm khổng lồ, nhấn chìm tín hiệu học tập từ các sự cố thảm họa hiếm gặp. Focal Loss định hình lại khái niệm này bằng cách thêm vào một nhân tử điều biến $(1 - p_t)^\gamma$. Công thức toán học:

$$FL(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

(Trong đó p_t là xác suất mô hình dự báo đúng nhãn thực tế, $\gamma > 0$ là tham số tập trung). Khi một mẫu được phân loại dễ dàng ($p_t \rightarrow 1$), nhân tử điều biến tiến về 0, tự động dập tắt trọng số của mẫu đó. Cơ chế này ép mạng Nơ-ron dồn toàn bộ sự chú ý (Attention) vào các mẫu ranh giới "khó nhằn" thuộc nhóm thiểu số.

- **Huber Loss / Smooth L1 Loss (Khắc phục nhiễu ngoại lai):** Đối với các tác vụ xấp xỉ hàm giá trị (như hồi quy hoặc ước lượng phần thưởng trong Học tăng cường), việc sử dụng MSE (L_2 Loss) rất nhạy cảm với dữ liệu ngoại lai (Outliers) - vốn là đặc sản của dữ liệu cảm biến IoT. Một điểm nhiễu duy nhất bị bình phương lên có thể làm nổ gradient (Exploding Gradients). Ngược lại, MAE (L_1 Loss) lại có đạo hàm không liên tục tại điểm 0, gây khó khăn khi hội tụ. Huber Loss là sự kết hợp hoàn hảo giữa hai hàm trên:

$$L_\delta(a) = \begin{cases} \frac{1}{2}a^2 & \text{với } |a| \leq \delta \\ \delta(|a| - \frac{1}{2}\delta) & \text{trường hợp khác} \end{cases}$$

(Trong đó a là sai số dự báo, δ là ngưỡng chuyển đổi). Đối với các sai số nhỏ ($|a| \leq \delta$), hàm hoạt động như MSE, giúp mô hình hội tụ mượt mà quanh điểm cực tiểu. Đối với các sai số lớn do nhiễu, hàm chuyển sang dạng tuyến tính của MAE, bảo vệ mạng Nơ-ron khỏi sự phá hoại của các tín hiệu vật lý vô lý.

3.2.3 Học tăng cường sâu (Deep Reinforcement Learning - DRL)

Học tăng cường sâu là sự kết hợp đột phá giữa khả năng cảm nhận của Học sâu (Deep Learning) và khả năng ra quyết định của Học tăng cường (Reinforcement Learning). Trong bài toán dự báo giao thông, DRL không chỉ học cách "nhận diện" kẹt xe mà còn học được "chiến lược" dự báo để tối ưu hóa lợi ích lâu dài cho toàn hệ thống.

3.2.3.1 a. Quy trình quyết định Markov (Markov Decision Process - MDP)

Mọi bài toán Học tăng cường đều được mô hình hóa dưới dạng một Quy trình quyết định Markov. Đây là một khung toán học để mô tả việc ra quyết định trong các tình huống mà kết quả vừa mang tính ngẫu nhiên, vừa chịu ảnh hưởng của người ra quyết định. MDP được định nghĩa bởi một bộ 5 thành phần (S, A, P, R, γ) :

1. **Đặc vụ (Agent):** Là "não bộ" của hệ thống (mô hình dự báo), thực hiện việc quan sát và đưa ra hành động.
2. **Môi trường (Environment):** Thế giới thực tế (mạng lưới giao thông) nơi Đặc vụ hoạt động.
3. **Trạng thái (State - S):** Tập hợp các quan sát tại thời điểm t (vận tốc, mật độ, lịch sử di chuyển).
4. **Hành động (Action - A):** Tập hợp các quyết định mà Đặc vụ có thể thực hiện (dự báo mức độ từ 0 đến 5).
5. **Phần thưởng (Reward - R):** Tín hiệu phản hồi từ môi trường sau mỗi hành động (điểm cộng nếu dự báo đúng, điểm trừ nếu dự báo sai lệch).

6. **Hệ số chiết khấu (Discount Factor - γ):** Xác định tầm quan trọng của các phần thưởng trong tương lai so với phần thưởng tức thì.

3.2.3.2 b. Thuật toán Q-Learning và Deep Q-Network (DQN)

Trong Q-Learning truyền thống, Đặc vụ sử dụng một bảng tra cứu (Q-table) để lưu trữ giá trị $Q(s, a)$ - đại diện cho "chất lượng" của việc thực hiện hành động a khi đang ở trạng thái s . Tuy nhiên, với không gian trạng thái giao thông khổng lồ và liên tục, việc sử dụng bảng là bất khả thi.

****Deep Q-Network (DQN)**** giải quyết vấn đề này bằng cách sử dụng một Mạng Nơ-ron sâu để xấp xỉ hàm giá trị Q . Thay vì tra bảng, Đặc vụ đưa trạng thái s vào mạng nơ-ron và nhận về các giá trị Q cho tất cả các hành động có thể. Mục tiêu của mạng là cực tiểu hóa sai số Bellman:

$$L(\theta) = \mathbb{E} \left[\left(R + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

3.2.3.3 c. Thuật toán Double DQN (DDQN) và việc triệt tiêu ảo tưởng giá trị

Một hạn chế lớn của DQN truyền thống là ****Hội chứng ảo tưởng giá trị (Overestimation Bias)****. Do toán tử max trong phương trình Bellman luôn chọn giá trị lớn nhất, mạng nơ-ron có xu hướng đánh giá quá cao các hành động có nhiều dương, dẫn đến việc Agent đưa ra các quyết định sai lầm nhưng lại "tự tin" là đúng.

****Double DQN (DDQN)**** khắc phục điều này bằng cách tách biệt việc Lựa chọn hành động và Đánh giá hành động thông qua hai mạng:

- **Mạng Chính (Policy Network - θ):** Dùng để chọn hành động có giá trị Q cao nhất.
- **Mạng Đích (Target Network - θ^-):** Dùng để tính toán giá trị của hành động đã chọn đó.

Công thức mục tiêu của DDQN trở thành:

$$Y_t^{DDQN} = R_{t+1} + \gamma Q(S_{t+1}, \arg \max_a Q(S_{t+1}, a; \theta_t); \theta_t^-)$$

Cơ chế này đảm bảo Đặc vụ nhìn nhận môi trường một cách khách quan hơn, đặc biệt quan trọng trong việc phát hiện các trạng thái "thảm họa" (Lớp 5) vốn thường bị nhiễu bởi các lớp kẹt xe nhẹ.

3.2.3.4 d. Cơ chế Bộ nhớ kinh nghiệm (Experience Replay)

Trong giao thông, các trạng thái liên tiếp (giây thứ nhất và giây thứ hai) thường rất giống nhau, tạo ra tính ****tương quan chuỗi (Temporal Correlation)**** cực cao. Nếu mạng nơ-ron chỉ học trực tiếp từ dữ liệu luồng này, nó sẽ dễ dàng bị chệch hướng và không thể hội tụ.

****Replay Buffer (Bộ nhớ kinh nghiệm)**** hoạt động như một kho lưu trữ các trải nghiệm quá khứ dưới dạng bộ tứ (s, a, r, s') .

- Trong quá trình huấn luyện, Đặc vụ không học từ dữ liệu mới nhất mà lấy ra một "Batch" ngẫu nhiên từ bộ nhớ này.
- Việc xáo trộn dữ liệu (Random Sampling) giúp phá vỡ tính tương quan thời gian, đảm bảo dữ liệu huấn luyện mang tính độc lập và phân phối đồng nhất (i.i.d), giúp quá trình hội tụ ổn định và bền bỉ hơn.

3.2.4 Các kỹ thuật xử lý dữ liệu và mô hình sinh tạo

Để xây dựng một hệ thống Trí tuệ Nhân tạo hoạt động hiệu quả trong môi trường giao thông thực tế, việc thiết kế mạng Nơ-ron (như LSTM hay DDQN) chỉ giải quyết được một nửa bài toán. Nửa còn lại, và thường là phần khó khăn nhất, nằm ở việc xử lý các rào cản và khiếm khuyết của dữ liệu đầu vào.

3.2.4.1 a. Vấn đề mất cân bằng dữ liệu (Data Imbalance) và Nghịch lý độ chính xác

Dữ liệu giao thông đô thị là một ví dụ kinh điển về phân phối mất cân bằng cực hạn (Extreme Class Imbalance). Trong một tập dữ liệu chuẩn, trạng thái giao thông bình thường (Lớp 0, 1) thường chiếm tới 90% – 95% tổng thời lượng, trong khi các sự cố ùn tắc nghiêm trọng hoặc vỡ trận (Lớp 4, 5) chỉ chiếm một tỷ lệ rất nhỏ (< 5%).

- **Nghịch lý độ chính xác (Accuracy Paradox):** Khi huấn luyện mô hình học máy trên dữ liệu này, nếu sử dụng chỉ số Độ chính xác tổng thể (Accuracy) làm mục tiêu tối ưu, mô hình sẽ dễ dàng rơi vào một cái bẫy thống kê. Cụ thể, một mô hình "lười biếng" (ZeroR) nếu luôn luôn dự báo trạng thái là "Thông thoáng" bất chấp dữ liệu đầu vào là gì, nó vẫn nghiêm nhiên đạt Accuracy 95%. Tuy nhiên, giá trị thực tiễn của nó bằng 0 vì nó bỏ lọt 100% các thảm họa cần cảnh báo.
- **Tầm quan trọng của F1-Score và Recall:** Để đánh giá chính xác năng lực phát hiện thảm họa, các chỉ số sau được ưu tiên sử dụng:
 - **Độ phủ (Recall / Sensitivity):** Tỷ lệ phần trăm các vụ kẹt xe thực tế được hệ thống phát hiện thành công ($\frac{TP}{TP+FN}$). Trong hệ thống cảnh báo sớm, tối đa hóa Recall (giảm thiểu bỏ lọt) là ưu tiên sống còn.
 - **F1-Score:** Là trung bình điều hòa giữa Precision (Độ chuẩn xác) và Recall, giúp đánh giá một cách cân bằng khi mô hình không thể hy sinh hoàn toàn Precision chỉ để lấy Recall.

3.2.4.2 b. Mạng đối nghịch sinh tạo (Generative Adversarial Networks - GAN)

Để giải quyết tận gốc vấn đề mất cân bằng, thay vì nhân bản dữ liệu cũ (Over-sampling truyền thống như SMOTE) dễ gây ra tình trạng học vẹt (Overfitting), công nghệ Trí tuệ Nhân tạo sinh được áp dụng thông qua **Mạng đối nghịch sinh tạo (GAN)**.

Khung thuật toán GAN được Ian Goodfellow giới thiệu vào năm 2014, hoạt động dựa trên nguyên lý của **Lý thuyết trò chơi (Game Theory)****, cụ thể là một trò chơi Minimax có tổng bằng không (Zero-sum game) giữa hai mạng Nơ-ron độc lập:

1. **Mạng Sinh (Generator - G):** Đóng vai trò là "Kẻ làm giả". Nó nhận đầu vào là một vector nhiễu ngẫu nhiên z và cố gắng sinh ra các mẫu dữ liệu giả $G(z)$ sao cho phân phối của chúng tiệm cận với phân phối của dữ liệu thật.
2. **Mạng Phân biệt (Discriminator - D):** Đóng vai trò là "Cảnh sát". Nó nhận đầu vào là cả dữ liệu thật x và dữ liệu giả $G(z)$, sau đó xuất ra xác suất để phân biệt đâu là thật, đâu là giả.

Quá trình huấn luyện là một cuộc chạy đua vũ trang liên tục. Hàm mục tiêu tối ưu của GAN được biểu diễn qua phương trình:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

Khi trò chơi đạt đến trạng thái cân bằng Nash (Nash Equilibrium), mạng Generator đã học được hoàn hảo các quy luật vật lý tiềm ẩn của dữ liệu thật, khiến mạng Discriminator không thể phân biệt được nữa (xác suất đoán đúng chỉ còn 50%).

3.2.4.3 c. Conditional Tabular GAN (CTGAN) và Sinh chuỗi thời gian (Sequence Generation)

Mạng GAN nguyên thủy được thiết kế tối ưu cho dữ liệu hình ảnh (ma trận điểm ảnh liên tục). Tuy nhiên, dữ liệu giao thông lại là **dữ liệu dạng bảng (Tabular Data)** chứa cả biến liên tục (vận tốc) và biến phân loại rời rạc (ID ngã tư, mã thời tiết).

- **CTGAN (Conditional Tabular GAN):** Là một biến thể kiến trúc được tinh chỉnh riêng cho dữ liệu bảng. Nó vượt qua giới hạn của GAN truyền thống bằng cách sử dụng cơ chế Huấn luyện có Điều kiện (Conditional Generator). Thay vì sinh ngẫu nhiên, CTGAN cho phép ép buộc mô hình chỉ sinh ra dữ liệu của một lớp cụ thể (ví dụ: chỉ sinh các mẫu thuộc Lớp kẹt xe 5).

- **Tính nhất quán của Chuỗi thời gian (Temporal Consistency):** Khi áp dụng CTGAN vào hệ thống dự báo, một thách thức lớn phát sinh: Giao thông là một chuỗi thời gian liên tục. Nếu CTGAN sinh ra từng dòng dữ liệu một cách độc lập, nó sẽ vi phạm động lực học vật lý (ví dụ: vận tốc đang 60 km/h đột ngột rớt xuống 0 km/h trong một giây). Do đó, kỹ thuật **Sequence Generation** được tích hợp. CTGAN được cấu hình để sinh dữ liệu theo đơn vị **Khối cửa sổ (Windows)** (ví dụ: sinh một lượt 13 bước thời gian liên tiếp). Điều này ép mạng Discriminator phải đánh giá sự hợp lý của cả một "chuỗi diễn biến", từ đó đảm bảo dữ liệu nhân tạo bảo toàn được tính mượt mà của sóng xung kích ùn tắc.

3.2.5 Trí tuệ nhân tạo có thể giải thích (Explainable AI - XAI)

Sự phát triển bùng nổ của các mô hình Học sâu (Deep Learning) mang lại độ chính xác vượt trội, nhưng đồng thời cũng tạo ra một rào cản lớn: Tính **"hộp đen" (Black-box)**. Mạng Nơ-ron với hàng triệu tham số phi tuyến tính không thể cung cấp cho con người một lý do trực quan để hiểu tại sao nó lại đưa ra một dự báo cụ thể. Trong các hệ thống quan trọng như y tế hay cảnh báo giao thông, sự thiếu minh bạch này dẫn đến rủi ro về độ tin cậy. Trí tuệ nhân tạo có thể giải thích (Explainable AI - XAI) ra đời nhằm mục đích bóc tách hộp đen này, giúp con người hiểu, tin tưởng và kiểm toán các quyết định của máy móc.

3.2.5.1 a. Lý thuyết giá trị Shapley (Shapley Value Theory)

Nền tảng toán học mạnh mẽ nhất của XAI hiện đại bắt nguồn từ Lý thuyết trò chơi hợp tác (Cooperative Game Theory), cụ thể là khái niệm **Giá trị Shapley**, được nhà toán học đoạt giải Nobel Lloyd Shapley giới thiệu vào năm 1953.

Lý thuyết này đặt ra một bài toán kinh điển: Giả sử một nhóm người chơi (liên minh) cùng hợp tác làm việc để đạt được một phần thưởng tổng (payout). Làm thế nào để phân chia phần thưởng này một cách công bằng nhất, tương xứng với mức độ đóng góp của từng cá nhân?

Giá trị Shapley giải quyết vấn đề này thông qua khái niệm **Đóng góp biên (Marginal Contribution)**. Đóng góp biên của một người chơi là giá trị tăng thêm khi người đó gia nhập vào một liên minh đã có sẵn. Tuy nhiên, đóng góp của một người chơi phụ thuộc rất lớn vào việc họ tham gia liên minh trước hay sau những người khác. Do đó, Giá trị Shapley của một người chơi được tính bằng trung bình cộng của tất cả các đóng góp biên trong mọi hoán vị gia nhập có thể xảy ra. Công thức toán học nguyên thủy của Giá trị Shapley cho người chơi i là:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(N - |S| - 1)!}{N!} [v(S \cup \{i\}) - v(S)]$$

Trong đó: N là tập hợp tổng số người chơi (đặc trưng); S là một liên minh (tập con) các người chơi không chứa i ; $v(S)$ là giá trị (phần thưởng) mà liên minh S đạt được; và $\frac{|S|!(N - |S| - 1)!}{N!}$ là trọng số hoán vị.

3.2.5.2 b. Phương pháp SHAP (SHapley Additive exPlanations)

Vào năm 2017, Lundberg và Lee đã công bố phương pháp SHAP, một bước đột phá mang Lý thuyết Shapley ứng dụng trực tiếp vào Học máy. Trong SHAP:

- "Trò chơi" chính là bài toán dự báo của mô hình.
- "Người chơi" là các đặc trưng đầu vào (Ví dụ: Vận tốc, Chỉ số kẹt xe, Thời tiết).
- "Phần thưởng tổng" là sự chênh lệch giữa giá trị dự báo cho một điểm dữ liệu cụ thể (Local Prediction) và Giá trị kỳ vọng cơ sở (Base Value) của toàn bộ tập dữ liệu.

Phương pháp SHAP định nghĩa một mô hình giải thích tuyến tính mang tính cộng gộp (Additive Feature Attribution Method):

$$g(z') = \phi_0 + \sum_{i=1}^M \phi_i z'_i$$

Trong đó $g(z')$ là mô hình giải thích, ϕ_0 là giá trị cơ sở, và ϕ_i là giá trị SHAP (mức độ đóng góp) của đặc trưng thứ i .

Sức mạnh diễn giải của SHAP thể hiện ở hai cấp độ:

- Tính diễn giải Cục bộ (Local Interpretability):** SHAP cho phép giải thích từng dự báo đơn lẻ thông qua Biểu đồ thác nước (Waterfall Plot) hoặc Biểu đồ lực (Force Plot). Nó chỉ rõ với một vụ kẹt xe vừa xảy ra, đặc trưng "Vận tốc giảm" đã đẩy xác suất kẹt xe lên bao nhiêu phần trăm, và đặc trưng "Trời quang mây tạnh" đã kéo xác suất đó xuống bao nhiêu phần trăm.
- Tính diễn giải Toàn cục (Global Interpretability):** Bằng cách tổng hợp giá trị SHAP tuyệt đối của tất cả các điểm dữ liệu, phương pháp này xếp hạng được Tầm quan trọng của Đặc trưng (Feature Importance) trên toàn hệ thống một cách khách quan, triệt tiêu được các sai lệch do tính tương quan chéo giữa các biến gây ra.

3.2.6 Công nghệ và Công cụ sử dụng

Để hiện thực hóa các mô hình toán học phức tạp như Mạng Nơ-ron Đồ thị, Học tăng cường (RL) hay Mạng đối nghịch sinh tạo (GAN) thành một hệ thống cảnh báo giao thông hoạt động trơn tru trong thực tế, đồ án thiết lập một hệ sinh thái công nghệ (Technology Stack) toàn diện. Hệ sinh thái này trải dài từ khâu thao tác dữ liệu thô, huấn luyện mô hình cho đến khâu tối ưu hóa triển khai (MLOps).

3.2.6.1 a. Ngôn ngữ lập trình và Thư viện lõi (Core Ecosystem)

- Python:** Được lựa chọn làm ngôn ngữ lập trình chủ đạo của toàn bộ hệ thống. Với đặc tính cú pháp tường minh và sự hậu thuẫn từ cộng đồng khoa học dữ liệu khổng lồ, Python cung cấp một nền tảng liền mạch để tích hợp từ các tập lệnh thu thập dữ liệu IoT đến các mô hình Học sâu phức tạp.
- PyTorch:** Đóng vai trò là "động cơ" tính toán (Compute Engine) cốt lõi cho các kiến trúc Học sâu và Học tăng cường (Double DQN). Khác với các framework sử dụng đồ thị tĩnh, sự ưu việt của PyTorch nằm ở cơ chế ****Đồ thị tính toán động (Dynamic Computation Graph / Define-by-Run)****. Cơ chế này đặc biệt quan trọng khi xây dựng Môi trường Học tăng cường (RL Environment), cho phép hệ thống linh hoạt tính toán đạo hàm, cập nhật trọng số và tùy chỉnh hàm phần thưởng không đối xứng (Asymmetric Reward) ngay trong quá trình Đặc vụ tương tác với dữ liệu chuỗi thời gian.
- Scikit-learn:** Thư viện chuẩn mực (Industry-standard) cho các tác vụ tiền xử lý và Học máy cơ bản. Trong đồ án, thư viện này đảm nhiệm vai trò thiết lập các "Bản hợp đồng dữ liệu" (Data Artifacts) như `StandardScaler` để chuẩn hóa phân phối vận tốc/độ trễ, `LabelEncoder` để mã hóa ngữ cảnh không gian, đồng thời cung cấp các thuật toán truyền thống để xây dựng mô hình đối chứng (Baseline Models).

3.2.6.2 b. Công cụ xử lý và Sinh tạo dữ liệu (Data & Generative AI Tools)

- Pandas và NumPy:** Là xương sống của luồng dữ liệu (Data Pipeline). NumPy cung cấp cấu trúc mảng đa chiều (nd-array) với tốc độ tính toán vector hóa (Vectorization) tối ưu nhờ phần lõi được viết bằng ngôn ngữ C. Trong khi đó, Pandas xử lý xuất sắc các tác vụ tính toán chuỗi thời gian (Time-series manipulation), bao gồm kỹ thuật trượt cửa sổ thời gian (Rolling Windows) và điền khuyết tín hiệu cảm biến lỗi.
- SDV (Synthetic Data Vault) - CTGAN:** Để giải quyết bài toán mất cân bằng dữ liệu cực hạn, thư viện SDV được tích hợp để khai thác sức mạnh của mô hình Conditional Tabular GAN. Thay vì phải tự xây dựng mạng GAN từ đầu dễ gặp hiện tượng sụp đổ mode (Mode Collapse), thư viện SDV cung cấp một bộ khung tối ưu chuyên dụng cho dữ liệu dạng bảng, giúp hệ thống sinh ra hàng vạn kịch bản kẹt xe nhân tạo mà vẫn tuân thủ nghiêm ngặt các ràng buộc vật lý.

3.2.6.3 c. Công nghệ Tối ưu hóa và Triển khai - MLOps (Deployment Pipeline)

Bước chuyển tiếp từ mô hình trong phòng thí nghiệm (Research) sang hệ thống thực tế (Production) đòi hỏi các công nghệ tăng tốc suy luận và đóng gói tiêu chuẩn:

- **ONNX (Open Neural Network Exchange):** Mạng Q-Network sau khi huấn luyện trên PyTorch sẽ được xuất sang định dạng ONNX. Đây là một định dạng trung gian độc lập với framework, giúp hợp nhất các toán tử mạng Nơ-ron (Operator Fusion) và tinh gọn đồ thị tính toán, loại bỏ những phép toán dư thừa sinh ra trong quá trình huấn luyện.
- **TensorRT / ONNX Runtime:** Để đáp ứng độ trễ suy luận (Inference Latency) ở cấp độ phần nghìn giây khi hệ thống phải quét qua hàng vạn đoạn đường cùng lúc, ONNX Runtime và NVIDIA TensorRT được sử dụng làm Execution Engine. Các công cụ này biên dịch mô hình trực tiếp thành mã máy cấp thấp, khai thác triệt để sức mạnh xử lý song song và lượng tử hóa (Quantization) của GPU.
- **Docker:** Toàn bộ hệ sinh thái từ môi trường Python, các tệp Artifacts chuẩn hóa, cho đến lõi suy luận TensorRT đều được container hóa (Containerization) bằng Docker. Công nghệ này đảm bảo tính "Bất biến môi trường" (Environment Immutability), giải quyết triệt để rào cản "Chạy được trên máy tôi nhưng lỗi trên server", tạo tiền đề vững chắc cho kiến trúc Microservices và dễ dàng nhân bản (Scale) tải trọng theo thời gian thực.

3.3 Nền tảng Ứng dụng và Định tuyến

Để phục vụ nhu cầu của người dùng cuối và quản trị viên, hệ thống tích hợp các thuật toán tìm đường tối ưu và công nghệ phát triển web hiện đại.

3.3.1 Cơ sở lý thuyết thuật toán và hệ thống

Nhằm đáp ứng các yêu cầu nghiệp vụ phức tạp của phân hệ Web Admin (quản trị, phân tích) và User (tìm đường cá nhân hóa), đề tài áp dụng một số nền tảng lý thuyết thuật toán và xử lý dữ liệu chuyên sâu:

3.3.1.a Lý thuyết Đồ thị và Thuật toán Tìm đường (Routing Algorithms)

Mạng lưới giao thông được mô hình hóa toán học dưới dạng một Đồ thị có hướng $G = (V, E)$, trong đó V (Vertices) là tập hợp các nút giao thông (Intersections) và E (Edges) là tập hợp các đoạn đường (Segments) nối giữa các nút.

Trong hệ thống định tuyến cho người dùng cuối (User App), mục tiêu là tìm đường đi tối ưu nhất. Thay vì sử dụng thuật toán Dijkstra truyền thống, đề tài áp dụng thuật toán **Bi-directional A* (A-Star hai chiều)** với cơ chế tìm kiếm đồng thời từ điểm xuất phát và điểm đích, kết hợp cùng hàm Heuristic ước lượng khoảng cách không gian để thu hẹp phạm vi duyệt đồ thị.

Điểm đột phá của hệ thống nằm ở **Hàm chi phí động (Dynamic Cost Function)** kết hợp với các mô hình toán học bù đắp dữ liệu thời gian thực được thực thi trực tiếp tại Data Warehouse:

- **Mô hình Suy giảm độ tin cậy (Confidence Decay Model):** Tình trạng giao thông thay đổi liên tục, do đó giá trị của dữ liệu vận tốc ($v_{current}$) sẽ giảm dần theo hàm mũ dựa trên độ trễ thời gian (Δt tính bằng phút) kể từ lúc thu thập:

$$C = e^{-\lambda \times \Delta t}$$

Với hằng số $\lambda = 0.046$ (tương đương độ tin cậy giảm 50% sau 15 phút). Từ đó, vận tốc hiệu chỉnh (v_{adj}) được trượt dần về mức vận tốc dòng tự do (v_{free}) nhằm tránh sai lệch khi dữ liệu quá cũ:

$$v_{adj} = \max(5, v_{free} + (v_{current} - v_{free}) \times C)$$

- **Nội suy Không gian bằng IDW (Inverse Distance Weighting):** Khi một đoạn đường (Dark Segment) bị mất hoàn toàn tín hiệu, hệ thống nội suy vận tốc bằng thuật toán IDW dựa trên 3 đoạn đường lân cận gần nhất (KNN) thông qua khoảng cách không gian (d_i):

$$v_{imputed} = \frac{\sum_{i=1}^3 (v_i / d_i^2)}{\sum_{i=1}^3 (1 / d_i^2)}$$

- **Hàm Chi phí Phức hợp đa biến (Multi-variable Cost Function):** Để quyết định "trọng số" cuối cùng của một cạnh đồ thị, hệ thống tính toán chi phí (Cost) dựa trên sự kết hợp giữa Thời gian di chuyển (Travel Time), Khoảng cách thực tế (Distance) và Hệ số phạt rủi ro (Confidence Penalty):

$$Cost = (TravelTime + \alpha \times Distance) \times Penalty$$

Trong đó, $\alpha = 0.02$ là hệ số quy đổi (tương đương 1km đi đường vòng sẽ bị phạt thêm 20 giây), và *Penalty* dao động từ 1.0 (dữ liệu trực tiếp) đến 1.1 (dữ liệu fallback). Hàm chi phí này giúp thuật toán ưu tiên những cung đường có dữ liệu thời gian thực xác thực, đồng thời chủ động "né" các vùng đang ùn tắc.

3.3.1.b Lý thuyết Xử lý Phân tích Trực tuyến (OLAP)

Để đáp ứng nhu cầu phân tích dữ liệu lớn và trích xuất tri thức trên Bảng điều khiển (Dashboard) của hệ thống Web Admin, đề tài vận dụng lý thuyết Xử lý Phân tích Trực tuyến (Online Analytical Processing - OLAP).

- **Khối dữ liệu (Data Cube):** Thay vì truy vấn trực tiếp trên các bảng hai chiều (Relational Tables), dữ liệu giao thông được trừu tượng hóa thành các khối đa chiều (Không gian, Thời gian, Các loại phương tiện, Các chỉ số hiệu năng giao thông).
- **Thao tác Roll-up và Drill-down:** Lý thuyết OLAP hỗ trợ người quản trị khả năng phân tích đa cấp độ. Người dùng có thể nhìn tổng thể tình hình toàn thành phố (Roll-up), sau đó "khoan sâu" (Drill-down) trực tiếp xuống chi tiết từng hành lang đường và từng phân đoạn (Segment) cụ thể trong các khung giờ nhất định để tìm ra nguyên nhân gốc rễ của hiện tượng suy giảm năng lực thông hành.

3.3.1.c Xử lý Không gian và Khớp bản đồ (Map Matching)

Việc theo dõi sự cố và vẽ biểu đồ lưu lượng đòi hỏi độ chính xác không gian tuyệt đối. Đề tài áp dụng lý thuyết Hình học tính toán (Computational Geometry) trong môi trường cơ sở dữ liệu:

- **Khớp bản đồ (Map Matching):** Khi nhận được tọa độ GPS (Kinh độ, Vĩ độ) của một sự kiện bất thường (ví dụ: Tai nạn, Ngập nước), hệ thống sử dụng thuật toán chiếu điểm lên đường (Point-to-Line Projection) để tìm khoảng cách vuông góc ngắn nhất. Qua đó, sự kiện không gian được "gắn" chính xác vào thực thể đoạn đường tương ứng trên Đồ thị giao thông.
- **Chỉ mục Không gian (Spatial Indexing):** Áp dụng cấu trúc cây R-Tree (GIST) để đóng khung các đối tượng hình học bằng các Hình chữ nhật bao quanh (Bounding Boxes). Lý thuyết này giúp hệ thống thu hẹp nhanh chóng phạm vi tìm kiếm không gian, giảm độ phức tạp truy vấn từ $O(N)$ xuống $O(\log N)$, duy trì hiệu năng cao cho bản đồ thời gian thực.
- **Thực thi Phân tích Không gian (In-database Spatial Processing):** Toàn bộ các phép toán hình học phức tạp được xử lý trực tiếp tại tầng Cơ sở dữ liệu thông qua thư viện PostGIS, giúp giảm tải cho Backend. Các nhóm hàm chính đã được ứng dụng thực tế trong hệ thống bao gồm:
 - *Nhóm khởi tạo và cấu hình:* ST_MakePoint, ST_GeomFromText, ST_SetSRID, ST_SRID (Tạo đối tượng hình học từ tọa độ thô và gán hệ quy chiếu WGS84 - EPSG:4326).

- *Nhóm trích xuất thành phần*: ST_Centroid, ST_X, ST_Y, ST_StartPoint, ST_EndPoint (Tìm trọng tâm và tọa độ các đầu mút của đoạn đường để phục vụ thuật toán tìm đường bdAstar).
- *Nhóm quan hệ không gian (Spatial Relationships)*: ST_Contains, ST_Within, ST_Covers, ST_Intersects (Kiểm tra sự giao cắt và bao hàm giữa Vùng thời tiết, Vùng sự cố với Mạng lưới giao thông).
- *Nhóm khoảng cách và đo lường*: ST_Distance, ST_DistanceSphere, ST_DWithin, ST_Area (Tính toán khoảng cách vật lý thực tế giữa người dùng, sự kiện và các phân đoạn đường).
- *Nhóm biến đổi và hình thái học*: ST_Collect, ST_UnaryUnion, ST_Simplify, ST_MakeEnvelope, ST_Dump, ST_VoronoiPolygons (Làm mịn hình học, gộp các đoạn LineString rời rạc thành Hành lang tuyến, và sinh lưới Voronoi nội suy thời tiết).
- *Nhóm định dạng xuất (Export)*: ST_AsGeoJSON (Chuyển đổi trực tiếp Geometry sang chuẩn GeoJSON nguyên bản để Mapbox render trên Frontend).

3.3.2 Công nghệ phát triển Ứng dụng

3.3.2.a Công nghệ xây dựng Máy chủ (Backend Framework)

Phân hệ Backend được xây dựng để xử lý hàng ngàn luồng truy vấn không gian phức tạp và trả về cho người dùng với độ trễ tối thiểu.

- **Node.js & ExpressJS**: Được sử dụng làm nền tảng máy chủ (Runtime) và Web Framework. ExpressJS cung cấp kiến trúc phân tầng Controller - Service - Repository tinh gọn, phân tách rõ ràng giữa lớp xử lý định tuyến (Routing API), lớp xử lý nghiệp vụ (Business Logic) và lớp tương tác cơ sở dữ liệu (Data Access).
- **TypeScript**: Trình biên dịch mã nguồn mở của Microsoft, giúp bổ sung kiểu dữ liệu tĩnh (Static Typing) cho JavaScript. Việc sử dụng TypeScript giúp hệ thống hạn chế tối đa các lỗi Runtime (đặc biệt khi xử lý các Object JSON phức tạp từ Database) và cải thiện hiệu suất bảo trì mã nguồn.
- **Redis**: Hệ quản trị cơ sở dữ liệu In-memory được dùng làm bộ đệm (Cache Layer). Redis lưu trữ tạm thời kết quả của các truy vấn Dashboard KPIs và dữ liệu tin tức giao thông, giúp giảm thiểu tải cho PostgreSQL và đẩy tốc độ phản hồi API xuống dưới mức 150ms.

3.3.2.b Công nghệ xây dựng Giao diện (Frontend Framework)

Phân hệ Frontend (Web Admin & User App) đòi hỏi năng lực xử lý đồ họa cực cao để kết xuất (Render) bản đồ tương tác và biểu đồ phân tích thời gian thực.

- **ReactJS & Vite**: ReactJS là thư viện cốt lõi để xây dựng giao diện người dùng dựa trên các Components tái sử dụng. Vite được sử dụng làm công cụ đóng gói (Bundler) thế hệ mới, thay thế Webpack, giúp tốc độ Hot-Module-Replacement (HMR) tăng gấp nhiều lần.
- **Mapbox GL JS**: Thư viện kết xuất bản đồ dựa trên công nghệ WebGL. Điểm mạnh của Mapbox là khả năng tải và hiển thị các lớp dữ liệu véc-tơ (Vector Tiles) trực tiếp lên GPU của trình duyệt, cho phép hiển thị và nội suy màu sắc hàng chục ngàn đoạn đường giao thông mượt mà tại tốc độ 60 khung hình/giây (FPS).
- **Recharts**: Thư viện biểu đồ xây dựng bằng React và SVG, cung cấp các biểu đồ động (Dynamic Charts) như đường xu hướng, biểu đồ dải màu (Linear Gradient) dùng trong phân tích Hành lang giao thông.
- **Tailwind CSS**: Framework Utility-first CSS giúp thiết kế giao diện thống nhất, phản hồi nhanh (Responsive) và hỗ trợ xây dựng phong cách giao diện IOC (Glassmorphism, Dark mode) một cách trực quan nhất.

3.4 Nền tảng Triển khai

Hệ thống được đóng gói và vận hành trên môi trường điện toán đám mây hiện đại, hướng tới tính sẵn sàng và tính di động cao:

- **Docker & Docker Hub:** Mã nguồn Backend và AI-Core được "đóng gói" thành các Container độc lập với đầy đủ môi trường chạy (Runtime, Dependencies) thông qua các file cấu hình `Dockerfile`. Các bộ ảnh (Image) này được lưu trữ trên Docker Hub để sẵn sàng cho quy trình pull/push ở mọi nền tảng.
- **Microsoft Azure App Service:** Nền tảng PaaS (Platform as a Service) của Azure được chọn để chạy các Docker Container backend. Đề tài sử dụng cấu hình **Basic B2** với khả năng quản lý tài nguyên linh hoạt, giúp duy trì API ổn định trong môi trường Production.
- **Vercel:** Nền tảng chuyên biệt được sử dụng để triển khai phân hệ Frontend (React). Vercel tích hợp Vercel CLI, tự động tối ưu hóa tài nguyên tĩnh và phân phối nội dung thông qua mạng CDN toàn cầu, đảm bảo ứng dụng Frontend truy cập nhanh ở mọi vị trí địa lý.

3.5 Kết chương

Chương 3 đã hệ thống hóa toàn bộ tri thức nền tảng từ khâu quản trị dữ liệu đến các thuật toán AI và quy trình vận hành đám mây. Những kiến thức này không chỉ đóng vai trò là cơ sở lý thuyết mà còn là khung xương kỹ thuật để hiện thực hóa các giải pháp dự báo và định tuyến giao thông thông minh sẽ được trình bày chi tiết trong các chương tiếp theo.

4 Công trình liên quan

Trong chương này, nhóm thực hiện khảo sát các nghiên cứu khoa học và các hệ thống thực tế đã triển khai liên quan đến kho dữ liệu và hệ thống hỗ trợ ra quyết định trong lĩnh vực giao thông thông minh (ITS).

Mục tiêu là phân tích ưu/nhược điểm của các giải pháp hiện có, từ đó xác định khoảng trống nghiên cứu để đề xuất giải pháp tối ưu cho Sở Giao thông Vận tải TP.HCM.

4.1 Các bài báo và nghiên cứu đã xem xét

Việc xây dựng kho dữ liệu giao thông không phải là một chủ đề mới, tuy nhiên, cách tiếp cận kiến trúc và tích hợp trí tuệ nhân tạo (AI) đã có sự chuyển dịch mạnh mẽ trong những năm gần đây. Nhóm phân chia các công trình nghiên cứu thành ba nhóm chính như sau:

4.1.1 Nhóm nghiên cứu về kiến trúc Kho dữ liệu giao thông truyền thống

Các nghiên cứu trong giai đoạn trước năm 2015 thường tập trung vào việc mô hình hóa dữ liệu dựa trên kiến trúc quan hệ truyền thống, điển hình như các mô hình sau đây:

- **Mô hình Dimensional Modeling (Kimball):** Nhiều nghiên cứu đề xuất sử dụng mô hình Star Schema hoặc Snowflake Schema để tổ chức dữ liệu giao thông (lưu lượng xe, vận tốc trung bình) nhằm phục vụ truy vấn. Mô hình này có tốc độ truy xuất báo cáo thống kê nhanh, dễ hiểu cho người dùng nghiệp vụ, tuy nhiên lại gặp khó khăn khi tích hợp dữ liệu phi cấu trúc (video camera, GPS log thô) và thiếu tính linh hoạt khi yêu cầu nghiệp vụ thay đổi liên tục, dẫn đến hiện tượng "Spider Web" trong quản lý dữ liệu.
- **Mô hình Normalized Data Warehouse (Inmon):** Một số nghiên cứu khác (như của Chen et al., 2010 về ITS Data Management) áp dụng mô hình 3NF của Bill Inmon để đảm bảo tính toàn vẹn và duy nhất của dữ liệu. Hạn chế của chúng là độ phức tạp cao khi triển khai, hiệu năng truy vấn thấp nếu không có lớp Data Mart hỗ trợ.

4.1.2 Nhóm nghiên cứu về Kho dữ liệu lớn và Tích hợp AI

Các nghiên cứu gần đây (từ 2018 trở lại đây) tập trung vào việc xử lý dữ liệu lớn (Big Data) và tích hợp mô hình dự báo.

- **Kiến trúc Lambda/Kappa:** Các bài báo như Zhang et al. (2020) đề xuất kiến trúc xử lý song song: một luồng xử lý Batch cho dữ liệu lịch sử và một luồng Speed cho dữ liệu thời gian thực. Ưu điểm của kiến trúc này là nó có khả năng giải quyết được bài toán độ trễ (latency). Mặt khác, kiến trúc này gặp hạn chế về độ phức tạp trong việc duy trì hai luồng code (codebase) khác nhau, gây khó khăn trong việc đồng bộ logic xử lý dữ liệu.
- **Tích hợp AI/Machine Learning:** Nhiều nghiên cứu tập trung vào thuật toán (LSTM, GNN) để dự báo tắc nghẽn. Tuy nhiên, các nghiên cứu này thường chỉ tập trung vào mô hình toán học mà bỏ qua khâu **Data Engineering** – tức là làm thế nào để chuẩn hóa, làm sạch và cung cấp dữ liệu huấn luyện một cách tự động từ DW.

Hầu hết các nghiên cứu học thuật tập trung sâu vào thuật toán dự báo hoặc kiến trúc lưu trữ thuần túy, thiếu một mô hình tích hợp toàn diện từ khâu chuẩn hóa dữ liệu (ETL) đến khâu tích hợp module phân tích của bên thứ 3 như yêu cầu của đề tài.

4.1.3 Nhóm nghiên cứu về Thuật toán dẫn đường và Nội suy dữ liệu

Bên cạnh bài toán lưu trữ, các nghiên cứu về thuật toán tối ưu hóa chi phí di chuyển cũng đóng vai trò cốt lõi trong hệ thống giao thông thông minh.

- **Lý thuyết dẫn đường dựa trên độ tin cậy (Reliability-based Routing):** Nghiên cứu về hành vi e ngại rủi ro của người lái xe (Risk-averse Route Choice) cho thấy người dùng có xu hướng tự cộng thêm một khoảng "thời gian đệm" (buffer time) vào nhận thức khi đi vào các tuyến đường không rõ tình trạng ùn tắc. Từ đó, hàm chi phí định tuyến cần được bổ sung các hệ số phạt (Penalty) thay vì chỉ tính toán thời gian di chuyển thuần túy. Ngoài ra, việc quy đổi khoảng cách đường vòng thành thời gian phạt thông qua Giá trị Thời gian (Value of Travel Time - VoT) cũng được đề xuất để tránh hiện tượng thuật toán lừa xe vào hầm nhỏ (Rat-running).
- **Kỹ thuật Nội suy và Làm mượt dữ liệu (Data Imputation & Smoothing):** Trong điều kiện cảm biến thường xuyên mất tín hiệu hoặc không phủ kín mạng lưới, các nghiên cứu (ví dụ: Bartier & Keller, 1996) đề xuất sử dụng thuật toán nội suy không gian Inverse Distance Weighting (IDW) làm cơ sở. Đồng thời, kỹ thuật làm mượt hàm mũ (Exponential Smoothing), thường thấy trong các bài báo về dự báo dòng giao thông ngắn hạn (Short-term traffic flow prediction), được áp dụng để làm giảm dần sự phụ thuộc vào dữ liệu quá khứ, đưa vận tốc đoạn đường trượt dần về mức dòng tự do (Free-flow) khi bị mất kết nối quá lâu.

4.2 Các phần mềm và hệ thống hiện có (Softwares/tools/systems)

Để đánh giá tính thực tiễn, nhóm đã khảo sát các hệ thống đang vận hành tại TP.HCM và các giải pháp phổ biến, từ đó quyết định sử dụng các phần mềm, công cụ và hệ thống nào để xây dựng đề tài.

4.2.1 Cổng thông tin giao thông TP.HCM (giaothong.hochiminhcity.gov.vn)

Đây là hệ thống chính thống hiện tại của TP.HCM phục vụ người dân và cơ quan quản lý, với chức năng cung cấp các hình ảnh camera trực tuyến, cảnh báo tắc nghẽn, bản đồ số, thông tin rào chắn thi công.

Về kiến trúc, hệ thống hoạt động thiên về hướng **Operational Data Store (ODS)**. Nó cung cấp dữ liệu "snapshot" tại thời điểm tức thời rất tốt. Bên cạnh đó, dữ liệu chủ yếu phục vụ tra cứu tức thời (Operational Reporting).

Vì phục vụ data real-time, nên hệ thống thiếu khả năng phân tích lịch sử sâu và rộng để nhận diện các xu hướng dài hạn cần thiết, ví dụ như so sánh lưu lượng giao thông cùng kỳ năm ngoái. Bên cạnh đó, hệ thống chưa tích hợp mạnh các mô hình dự báo để cảnh báo tắc nghẽn trước khi nó xảy ra. Ngoài ra, dữ liệu camera và cảm biến chưa được khai thác triệt để cho các bài toán học máy (Machine Learning).

4.2.2 Hệ thống UTraffic (bktraffic.com)

Hệ thống này được phát triển bởi các nhóm nghiên cứu trước tại ĐH Bách Khoa, là nền tảng mà đề tài này sẽ kế thừa. Chức năng của hệ thống là thu thập dữ liệu GPS từ cộng đồng, ước lượng vận tốc trung bình các tuyến đường, và trực quan hóa trạng thái giao thông.

Hệ thống Utraffic có nguồn dữ liệu phong phú được làm giàu qua nhiều thế hệ sinh viên nghiên cứu, cùng với thuật toán ước lượng trạng thái giao thông tốt. Tuy nhiên, kiến trúc lưu trữ hiện tại có thể chưa tối ưu cho các truy vấn phân tích phức tạp (OLAP) mà thiên về xử lý giao dịch (OLTP) hơn. Ngoài ra, chưa có một kho dữ liệu tập trung (Centralized DW) được chuẩn hóa để phục vụ cho các bên thứ 3 hoặc các module AI cắm vào (plug-in) dễ dàng.

4.2.3 Các nền tảng dữ liệu hiện đại (Modern Data Platform - Snowflake/Databricks)

Tham khảo từ tài liệu "Data Modeling with Snowflake", các hệ thống giao thông trên thế giới đang chuyển dịch sang các nền tảng đám mây hỗ trợ: Xử lý dữ liệu bán cấu trúc (JSON, XML từ thiết bị IoT) trực tiếp mà không cần chuyển đổi phức tạp (Schema-on-read) và khả năng mở rộng linh hoạt theo nhu cầu xử lý dự báo.

4.3 Khoảng trống nghiên cứu

Qua việc khảo sát các nghiên cứu lý thuyết và hệ thống thực tế, nhóm xác định được các "khoảng trống" sau đây mà đề án sẽ tập trung giải quyết:

- **Thứ nhất, về khía cạnh kiến trúc dữ liệu.** Thực tế cho thấy các hệ thống giao thông đã đề cập thường là các ODS rời rạc hoặc các hệ thống giám sát thời gian thực (ví dụ như Cổng thông tin TP.HCM), do đó thiếu khả năng lưu trữ lịch sử lâu dài và đa chiều. Nhóm quyết định tập trung xây dựng DW lai (hybrid), nghĩa là kết hợp mô hình Inmon và Kimball, cùng với hỗ trợ lưu trữ lịch sử biến động để phục vụ phân tích xu hướng, vì mô hình Inmon cho sự nhất quán dữ liệu nền tảng, trong khi đó mô hình của Kimball phục vụ cho các Data Mart thống kê.
- **Thứ hai, về khả năng tích hợp AI/ML,** hiện tại các mô hình AI thường chạy độc lập (Standalone), tốn nhiều công sức để chuẩn bị dữ liệu thủ công. Nhóm quyết định tích hợp AI vào luồng dữ liệu (Data Pipeline), xây dựng cơ chế để DW tự động chuẩn hóa, tiền xử lý dữ liệu và cung cấp đầu vào cho các module dự báo ngay trong quy trình ETL/ELT.
- **Thứ ba, về khả năng mở rộng.** Các hệ thống hiện tại thường đóng kín (Closed systems), khó tích hợp công cụ phân tích của bên thứ 3. Nhóm tập trung giải quyết tính mở rộng bằng cách thiết kế DW cho phép các phương thức phân tích của bên thứ 3 thông qua các API chuẩn hoặc cơ chế chia sẻ dữ liệu an toàn, phục vụ trực tiếp cho Sở GTVT.
- **Thứ tư, về khả năng hỗ trợ ra quyết định.** Các hệ thống hiện tại chỉ dừng lại ở việc hiển thị, mức “Mô tả”, do đó nhóm tập trung xây dựng hệ thống không chỉ báo cáo thống kê mà còn đưa ra các dự báo tác nghẽn để hỗ trợ quyết định điều tiết giao thông.

Tóm lại, đồ án này không chỉ xây dựng một nơi chứa dữ liệu, mà còn xây dựng một hệ sinh thái dữ liệu thông minh kế thừa trên Utraffic, áp dụng các kỹ thuật mô hình hóa hiện đại, để giải quyết các bài toán giao thông đặc thù của Thành phố Hồ Chí Minh.

5 Hệ thống Trí tuệ Nhân tạo hỗ trợ dự báo giao thông thông minh

5.1 Tổng quan và Phân tích các phương pháp tiếp cận (Literature Review)

Dự báo lưu lượng và tình trạng ùn tắc giao thông là một bài toán chuỗi thời gian (Time-series forecasting) mang tính thách thức cao do bản chất ngẫu nhiên (stochastic) và độ phức tạp về mặt không gian của mạng lưới đường bộ. Để đánh giá tính khả thi và hiệu quả của các kiến trúc dự báo, nghiên cứu này tiến hành phân tích và so sánh các phương pháp tiếp cận hiện hành dựa trên ba tiêu chí cốt lõi: (1) Khả năng bắt giữ phụ thuộc thời gian (Temporal dependence capture), (2) Năng lực xử lý dữ liệu phi cấu trúc và đa phương thức (Unstructured/Multimodal data handling), và (3) Mục tiêu tối ưu hóa (Optimization objectives).

Việc phân tích các phương pháp từ cổ điển đến hiện đại giúp làm rõ những rào cản kỹ thuật hiện tại, từ đó củng cố cơ sở khoa học cho việc đề xuất kiến trúc Học chuyển giao lai (Hybrid Transfer RL) trong đồ án này.

5.1.1 Tiếp cận dựa trên Thống kê truyền thống (ARIMA, SARIMA)

5.1.1.1 a. Nguyên lý hoạt động và Cơ sở toán học

Phương pháp tự hồi quy tích hợp trung bình trượt (Autoregressive Integrated Moving Average - ARIMA) và biến thể có tính mùa vụ (SARIMA) là những mô hình thống kê kinh điển nhất trong phân tích chuỗi thời gian. Nguyên lý cốt lõi của ARIMA dựa trên giả định về tính dừng (stationarity) của dữ liệu. Nghĩa là, các đặc trưng thống kê của chuỗi thời gian (như kỳ vọng, phương sai) không thay đổi theo thời gian.

Mô hình thiết lập dự báo dựa trên một tổ hợp tuyến tính bao gồm ba thành phần:

- **AR (Autoregressive - Tự hồi quy):** Biểu diễn giá trị hiện tại như một hàm tuyến tính của các giá trị trong quá khứ, giúp bắt giữ quán tính của dòng xe.
- **I (Integrated - Tích hợp):** Sử dụng sai phân (differencing) để triệt tiêu xu hướng (trend), ép chuỗi dữ liệu giao thông phi dừng về trạng thái dừng.
- **MA (Moving Average - Trung bình trượt):** Khai thác mối liên hệ phụ thuộc giữa giá trị quan sát hiện tại và các sai số ngẫu nhiên (white noise) từ các dự báo trước đó.

5.1.1.2 b. Ưu điểm

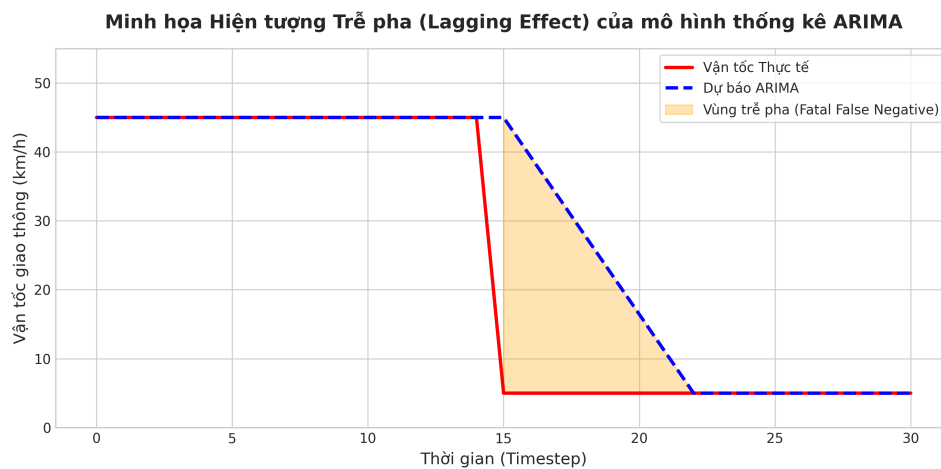
Đối với tiêu chí thứ nhất (bắt giữ phụ thuộc thời gian), ARIMA thực hiện rất tốt trong điều kiện dòng lưu lượng giao thông ở trạng thái dòng tự do (free-flow) hoặc biến thiên theo chu kỳ ổn định. Mô hình có tính minh bạch toán học (White-box) cao, dễ dàng diễn giải các hệ số tác động và yêu cầu chi phí tính toán cực kỳ thấp.

5.1.1.3 c. Hạn chế thực tiễn và Sự thất bại trước thảm họa giao thông

Tuy nhiên, khi đối chiếu với thực tiễn mạng lưới giao thông tại TP.HCM, mô hình thống kê truyền thống bộc lộ những khuyết điểm chí mạng:

1. **Sự bất lực trước "Điểm gãy" phi tuyến tính (Non-linear Structural Breaks):** ARIMA bị ràng buộc bởi giả định quan hệ tuyến tính. Trong thực tế, sự hình thành kẹt xe thảm họa (Mức 4, Mức 5) không diễn ra từ từ mà là một sự suy sụp cấu trúc đột ngột (ví dụ: một vụ tai nạn hoặc mưa lớn đổ xuống khiến vận tốc giảm từ 40 km/h xuống 5 km/h chỉ trong 2 phút). Khi đối mặt với biến động này, kết quả dự báo của ARIMA bị trễ pha (Lagging effect) nghiêm trọng. Mô hình thường dự báo kẹt xe *sau khi* kẹt xe đã thực sự xảy ra, khiến giá trị cảnh báo sớm bị triệt tiêu hoàn toàn.
2. **Khả năng xử lý dữ liệu ngoại lai kém (Tiêu chí 2):** ARIMA chỉ chấp nhận dữ liệu đầu vào là một chuỗi giá trị đơn biến (Univariate) hoặc đa biến tuyến tính. Nó hoàn toàn bất lực trong việc nhúng (embedding) các dữ liệu phi cấu trúc mang tính tâm lý và không gian như sự thay đổi thời tiết, ID phân vùng địa lý, hay mã loại đường bộ.

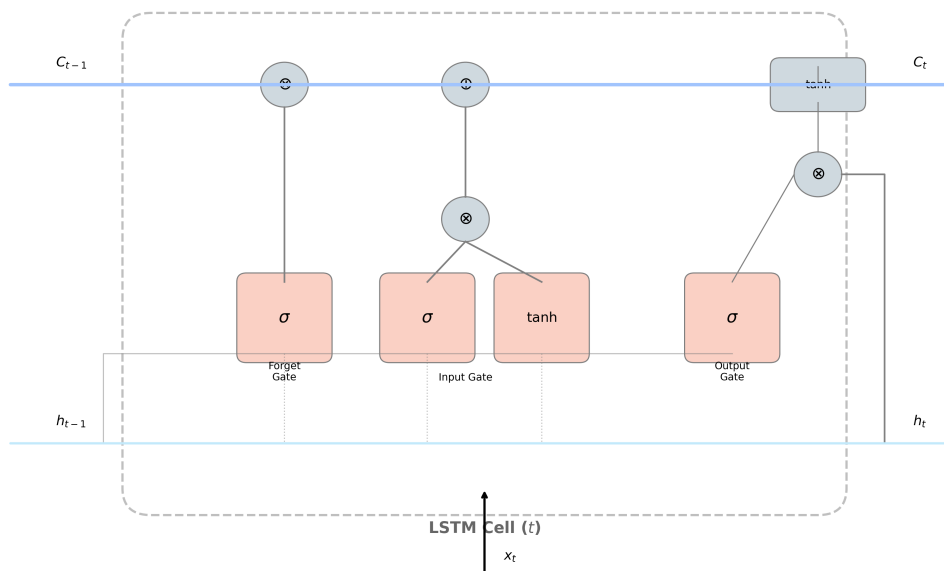
3. **Sai lệch Mục tiêu Tối ưu (Tiêu chí 3):** Hàm mục tiêu của ARIMA thường là giảm thiểu sai số toàn phương trung bình (MSE). Điều này buộc mô hình phải bám sát vào nhóm dữ liệu đa số (trạng thái thông thoáng), dẫn đến việc bỏ qua các giá trị ngoại lai (Anomalies) vốn đại diện cho thảm họa giao thông.



Hình 1: Biểu đồ minh họa hiện tượng Trễ pha của mô hình ARIMA

5.1.2 Tiếp cận dựa trên Học sâu (LSTM, CNN, T-GCN)

Sự bùng nổ của Học sâu (Deep Learning - DL) đã mở ra một kỷ nguyên mới cho bài toán dự báo giao thông nhờ khả năng tự động trích xuất các đặc trưng phi tuyến tính phức tạp mà không cần sự can thiệp thủ công từ chuyên gia.



Hình 2: Sơ đồ khối kiến trúc LSTM đơn giản

5.1.2.1 a. Khả năng bắt giữ phụ thuộc thời gian với mạng LSTM

Trong các kiến trúc DL, mạng Bộ nhớ ngắn hạn dài (Long Short-Term Memory - LSTM) được xem là tiêu chuẩn vàng cho dữ liệu chuỗi thời gian. Khác với mạng nơ-ron truyền thống, LSTM giải quyết triệt để vấn đề Tiêu biến đạo hàm (Vanishing Gradient) nhờ vào cấu trúc "Tế bào nhớ" (Memory cell) và cơ chế các cổng (Gates):

- **Cổng quên (Forget Gate):** Loại bỏ những thông tin không còn giá trị từ quá khứ (ví dụ: dữ liệu vận tốc của 2 tiếng trước không còn ảnh hưởng đến kẹt xe hiện tại).

- **Cổng nhập (Input Gate):** Cập nhật những thông tin mới quan trọng vào trạng thái tế bào.
- **Cổng xuất (Output Gate):** Quyết định thông tin nào sẽ được đưa ra làm đầu ra và chuyển sang bước thời gian tiếp theo.

Cơ chế này cho phép LSTM duy trì sự phụ thuộc xa (Long-term dependencies), giúp mô hình hiểu được xu hướng giao thông kéo dài qua nhiều khung giờ.

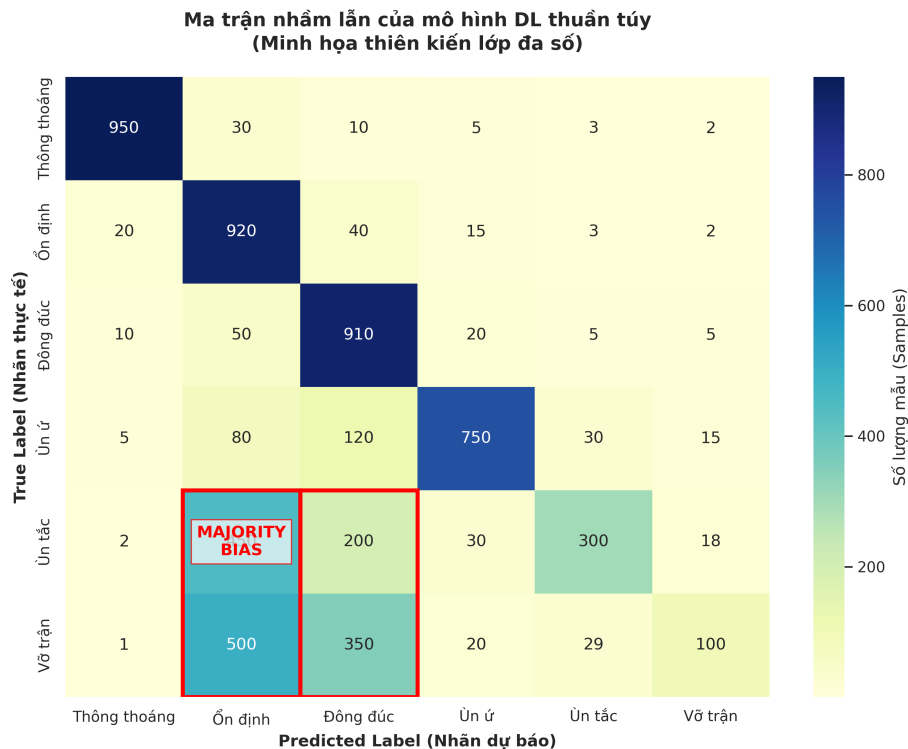
5.1.2.2 b. Khả năng xử lý dữ liệu phi cấu trúc và không gian (CNN, T-GCN)

Bên cạnh yếu tố thời gian, các mô hình như Mạng nơ-ron tích chập (CNN) và Mạng tích chập đồ thị (T-GCN) được tích hợp để xử lý Phụ thuộc không gian (Spatial dependence). Trong khi CNN hiệu quả với dữ liệu dạng lưới (Grid-based), T-GCN lại tỏ ra ưu việt trong việc mô hình hóa mạng lưới đường phố dưới dạng một đồ thị (Graph), nơi các nút là các đoạn đường và cạnh là sự kết nối giao thông giữa chúng.

5.1.2.3 c. Phân tích khuyết điểm: Thiên kiến lớp đa số (Majority Class Bias)

Dù mạnh mẽ hơn ARIMA, các mô hình Deep Learning chuẩn vẫn gặp phải một rào cản lớn về Mục tiêu tối ưu hóa (Tiêu chí 3) khi đối mặt với dữ liệu mất cân bằng:

1. **Sự thống trị của Hàm Loss chuẩn:** Các mô hình DL thường sử dụng Mean Squared Error (MSE) hoặc Cross-Entropy làm hàm mất mát. Các hàm này tính toán giá trị trung bình trên toàn bộ tập dữ liệu. Vì trạng thái "Thông thoáng" (Lớp 0, 1, 2) chiếm đến 90-95% dữ liệu, mô hình sẽ bị "đánh lừa" rằng việc dự báo đúng lớp đa số là cách nhanh nhất để giảm Loss xuống mức tối thiểu.
2. **Hệ quả "Mù màu" với thảm họa:** Kết quả là mô hình đạt độ chính xác (Accuracy) tổng thể rất cao (thường > 90%), nhưng lại có chỉ số Recall ở lớp 4 và lớp 5 cực thấp. Trong thực tế, điều này cực kỳ nguy hiểm: Hệ thống báo cáo đường thông thoáng nhưng thực tế người dùng lại bị kẹt cứng trong một vụ tắc đường vô trật. Đây chính là hiện tượng "Học vẹt" theo lớp đa số mà các kiến trúc DL thuần túy khó lòng vượt qua nếu không có sự can thiệp về chiến lược lấy mẫu hoặc hàm phần thưởng.



Hình 3: Ma trận nhầm lẫn mô hình DL thuần túy

5.1.3 Đề xuất Kiến trúc Lai (Hybrid SL-to-RL Transfer)

Để khắc phục triệt để sự "trễ pha" của mô hình thống kê (ARIMA) và "thiên kiến lớp đa số" của mô hình Deep Learning chuẩn (LSTM), đề án đề xuất một kiến trúc lai đột phá: **Hybrid SL-to-RL Transfer**. Phương pháp này tái định nghĩa bài toán dự báo giao thông không chỉ là việc khớp đường cong dữ liệu (curve-fitting), mà là một quá trình ra quyết định tối ưu tuần tự (Sequential Decision Making).

Kiến trúc đề xuất là sự giao thoa giữa khả năng trích xuất đặc trưng của **Học có giám sát (Supervised Learning - SL)** và cơ chế định hướng mục tiêu của **Học tăng cường sâu (Deep Reinforcement Learning - DRL)**, bao gồm các thành phần cốt lõi sau:

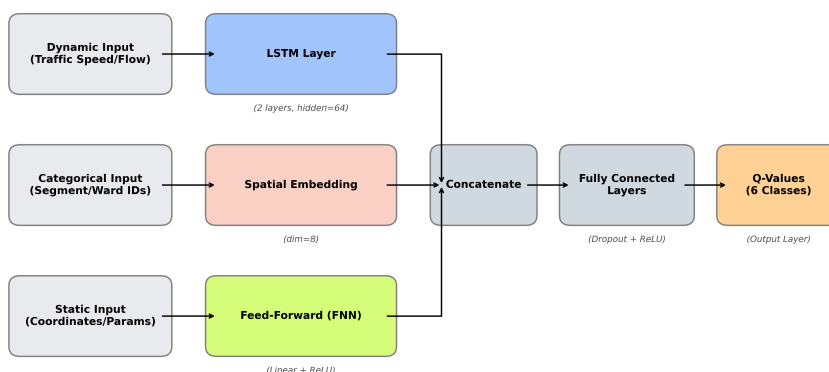
5.1.3.1 a. Cấu trúc mạng Trích xuất Đặc trưng Đa nhánh (Multi-branch Fusion Network)

Hệ thống sử dụng một mạng Nơ-ron tùy chỉnh (đóng vai trò là mô hình SL cơ sở và mạng Q-Network sau này) bao gồm ba nhánh song song, chịu trách nhiệm xử lý các luồng dữ liệu dị đồng nhất:

- Nhánh Đặc trưng Động (Dynamic Branch):** Xử lý luồng dữ liệu chuỗi thời gian (vận tốc, gia tốc, tỷ lệ trễ) thông qua cấu trúc LSTM 2 lớp (với $hidden_dim = 64$). Cấu trúc này giúp trích xuất quán tính giao thông và bắt sóng xung kích khi vận tốc thay đổi.
- Nhánh Ngữ cảnh Phân loại (Categorical Branch):** Thay vì sử dụng One-hot encoding gây hiện tượng bùng nổ số chiều và ma trận thưa, đề án áp dụng Tầng Nhúng không gian (Embedding Layers với $dim = 8$). Kỹ thuật này ánh xạ các định danh rời rạc (ID đoạn đường, mã Phường/Xã) thành các vector dày đặc liên tục, giúp mạng nơ-ron học được tính tương đồng về không gian giữa các khu vực địa lý lân cận.
- Nhánh Đặc trưng Tĩnh (Static Branch):** Sử dụng mạng nơ-ron truyền thẳng (Feed-Forward Neural Network - FNN) để xử lý các thuộc tính vật lý bất biến (tọa độ GPS, số làn xe, chiều dài đoạn đường).

Đầu ra của ba nhánh này được kết hợp (concatenate) tạo thành một vector đại diện toàn cục (Fusion Vector) trước khi đi qua các tầng phân loại cuối cùng (Classifier).

Sơ đồ kiến trúc mạng Fusion (Context-Aware Hybrid Architecture)



* Ghi chú: Mô hình kết hợp đặc trưng động (chuỗi thời gian) và đặc trưng tĩnh (ngữ cảnh không gian) để dự báo.

Hình 4: Sơ đồ kiến trúc tổng thể

5.1.3.2 b. Cơ chế Học chuyển giao và Khởi động ấm (SL-to-RL Warmstart)

Một nhược điểm chí mạng của Học tăng cường (RL) là vấn đề "Khởi động lạnh" (Cold Start). Nếu khởi tạo trọng số ngẫu nhiên, Agent sẽ mất lượng lớn thời gian để mò mẫm các quy luật vật lý cơ bản, dễ dẫn đến mất hội tụ. Để giải quyết, đề án áp dụng chiến lược Warmstart: Mạng Fusion đa nhánh ban đầu được huấn luyện (Pre-train) hoàn toàn bằng phương pháp Học có giám sát (SL) với hàm mất mát Cross-Entropy. Sau khi mạng đã có "bản năng" nhận diện giao thông cơ bản, toàn bộ trọng số (weights) này được chuyển giao (Transfer) và nạp trực tiếp vào Đặc vụ DQN trước khi bắt đầu quá trình tinh chỉnh (Fine-tuning) bằng RL. Điều này giúp Agent có một xuất phát điểm cực kỳ vững chắc.

5.1.3.3 c. Tối ưu hóa với Double DQN và Hàm Phần thưởng Không đối xứng (Asymmetric Reward Shaping)

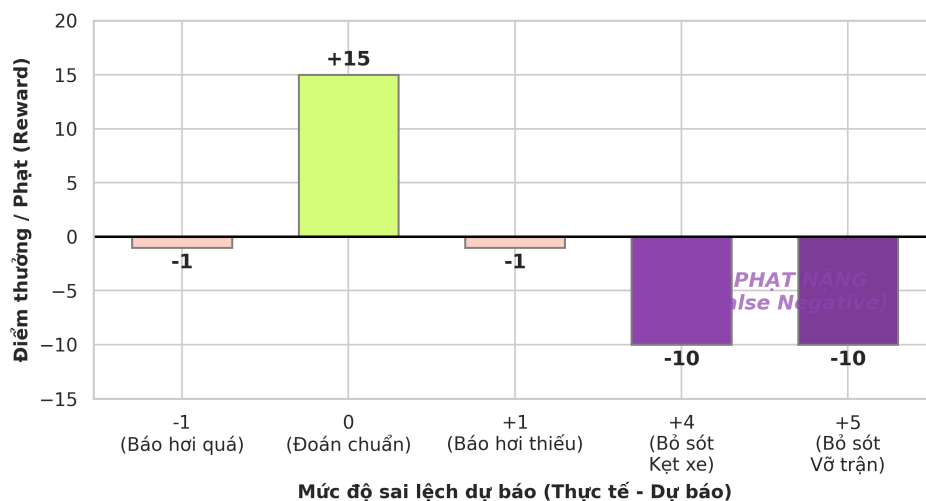
Đặc vụ (Agent) sử dụng thuật toán Double DQN kết hợp bộ nhớ kinh nghiệm (Replay Buffer) dung lượng 100.000 mẫu. Việc sử dụng mạng mục tiêu (Target Network) tách biệt với mạng chính sách (Policy Network) giúp triệt tiêu hội chứng ảo tưởng sức mạnh (Overestimation Bias) vốn thường gặp ở Q-Learning truyền thống.

Tuy nhiên, "trái tim" và bí quyết thành công của hệ thống nằm ở thiết kế Hàm phần thưởng (Reward Shaping). Nhằm triệt tiêu thiên kiến lớp đa số (mô hình thích đoán kẹt xe thành thông thoáng để an toàn), đồ án thiết kế một cơ chế điểm phạt phi tuyến tính và không đối xứng:

- **Thưởng khích lệ:** Nếu dự báo chính xác mức độ kẹt xe, Agent nhận +2 điểm.
- **Phạt cảnh cáo:** Nếu dự báo sai lệch nhẹ (chênh lệch 1 mức, ví dụ thực tế Mức 2 nhưng đoán Mức 1), Agent bị trừ -1 điểm. Đây là mức sai số có thể chấp nhận được trong thực tế.
- **Phạt sinh tử (Fatal Penalty):** Nếu hệ thống đối mặt với thảm họa (Thực tế là Mức 4 hoặc 5) nhưng Agent báo cáo sai thành thông thoáng (Mức 0 hoặc 1), mức phạt sẽ tăng vọt lên từ -10 đến -20 điểm.

Cơ chế "đòn bẩy" này tạo ra một rào cản tâm lý, ép Agent phải nhảy bén tối đa với các tín hiệu thảm họa, chấp nhận hy sinh một phần nhỏ độ chính xác ở các lớp thông thoáng để bảo vệ năng lực cảnh báo sớm cho người dùng.

Mô phỏng Hàm phần thưởng không đối xứng (Asymmetric Reward Shaping)



Hình 5: Đồ thị hàm phần thưởng

5.1.3.4 d. Đánh giá ưu điểm và thách thức

- **Ưu điểm vượt trội:** Kiến trúc lai đạt được cả hai mục tiêu cực hạn: Tốc độ hội tụ nhanh (nhờ tri thức tiền huấn luyện từ Warmstart) và năng lực thu hồi (Recall) xuất sắc tại các lớp kẹt xe nặng (nhờ Hàm thưởng không đối xứng ép buộc Agent chú ý vào các trường hợp ngoại lai).
- **Thách thức:** Quá trình tinh chỉnh các siêu tham số của hàm thưởng (tỷ lệ phạt giữa -10 và -20) đòi hỏi phải thử nghiệm (trial-and-error) liên tục để tránh việc Agent rơi vào thái cực ngược lại là chứng "hoang tưởng" (luôn dự báo kẹt xe). Ngoài ra, cấu trúc Double DQN tiêu tốn bộ nhớ và thời gian tính toán cao hơn so với mô hình giám sát thuần túy.

5.2 Kỹ thuật Thiết kế Đặc trưng (Feature Engineering & Justification)

Dữ liệu thô thu thập từ mạng lưới cảm biến giao thông thường chứa nhiều độ trễ, nhiễu và các điểm khuyết thiếu (missing values). Do đó, để mô hình học máy – đặc biệt là mạng nơ-ron chuỗi thời gian – có thể hội tụ và trích xuất

quy luật hiệu quả, dữ liệu thô cần được biến đổi thành các vector đặc trưng mang ý nghĩa vật lý. Mục này trình bày các kỹ thuật nền tảng để cấu trúc hóa không gian trạng thái từ dòng dữ liệu thô.

5.2.1 Cấu trúc Cửa sổ trượt (Sliding Window) và Đảm bảo tính liên tục (Continuity)

Bài toán dự báo giao thông không xem xét từng điểm dữ liệu đơn lẻ mà đánh giá chuỗi diễn biến trong quá khứ để suy luận tương lai. Để định dạng dữ liệu cho mô hình chuỗi thời gian (như LSTM), hệ thống sử dụng kỹ thuật Cửa sổ trượt (Sliding Window).

5.2.1.1 a. Thông số và Biện luận kích thước cửa sổ (Window Size Justification) Hệ thống thiết lập kích thước cửa sổ $W = 12$ bước thời gian (time-steps). Với tần suất thu thập dữ liệu (Aggregation resolution) là 15 phút/bước, kích thước cửa sổ này tương đương với 3 giờ đồng hồ lịch sử.

Cơ sở lựa chọn: Việc chọn 3 giờ không phải là ngẫu nhiên mà dựa trên đặc tính lan truyền của dòng xe. Kẹt xe đô thị không xảy ra tức thời mà phát triển theo dạng "sóng xung kích" (Shockwaves). Một khoảng thời gian 3 giờ là vừa đủ để mô hình quan sát trọn vẹn một chu kỳ chuyển dịch trạng thái: từ giai đoạn giao thông bắt đầu ùn ứ tích lũy, đạt đỉnh điểm vỡ trận (bottleneck), cho đến khi dòng xe bắt đầu giải phóng.

- Nếu chọn cửa sổ quá ngắn (ví dụ 30 phút), mô hình sẽ thiếu "tầm nhìn" dài hạn để nhận biết xu hướng.
- Nếu chọn cửa sổ quá dài (ví dụ 6 giờ), dữ liệu sẽ chứa nhiều thông tin nhiễu từ các khung giờ không liên quan, làm giảm hiệu suất tính toán của bộ nhớ hệ thống.

Cửa sổ này sẽ trích xuất một chuỗi đầu vào $X = [x_{t-11}, x_{t-10}, \dots, x_t]$ để dự báo nhãn mục tiêu $Y = y_{t+1}$ (dự báo trước 15 phút).

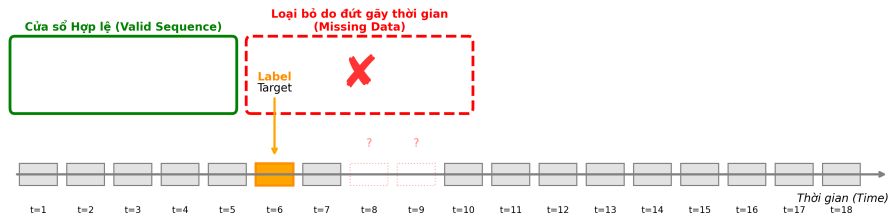
5.2.1.2 b. Thuật toán kiểm soát tính liên tục (Continuity Check Algorithm)

Một thách thức lớn của dữ liệu IoT (Internet of Things) ngoài thực tế là hiện tượng rớt mạng, mất tín hiệu cảm biến, dẫn đến các "lỗ hổng" thời gian trong cơ sở dữ liệu. Nếu hệ thống áp dụng cửa sổ trượt một cách mù quáng, cửa sổ có thể chứa những dòng dữ liệu đứt gãy (ví dụ: nhảy vọt từ 08:00 sáng thẳng đến 14:00 chiều do mất kết nối). Việc đưa một chuỗi đứt gãy không gian thời gian vào mô hình LSTM sẽ gây ra sự sai lệch nghiêm trọng về mặt nhận thức động lực học.

Để khắc phục, đồ án thiết kế thuật toán **Kiểm tra tính liên tục** (find_valid_window_starts). Thuật toán này hoạt động như một màn lọc nghiêm ngặt:

- Quét qua trục thời gian và kiểm tra khoảng cách (delta) giữa mọi điểm dữ liệu liên tiếp trong cửa sổ $W + 1$ (bao gồm cả điểm mục tiêu).
- Chỉ những cửa sổ thỏa mãn điều kiện khoảng cách thời gian giữa các bước chính xác tuyệt đối bằng 15 phút (không chứa giá trị NaN hoặc bước nhảy vọt) mới được gán cờ hợp lệ (Valid).
- Các cửa sổ vi phạm sẽ bị loại bỏ hoàn toàn nhằm bảo vệ tính toàn vẹn (Data Integrity) của tập huấn luyện. Kỹ thuật này đóng vai trò then chốt giúp Agent Học tăng cường không bị học sai các quy luật vật lý nhân quả.

Cơ chế Cửa sổ trượt (Sliding Window) và Thuật toán kiểm tra tính liên tục



* Nguyên tắc: Mô hình chỉ học từ các chuỗi thời gian liên tục. Mọi cửa sổ chứa dữ liệu rác/thiếu sẽ bị hệ thống tự động loại bỏ.

Hình 6: Sơ đồ minh họa cơ chế Cửa sổ trượt và Thuật toán kiểm tra tính liên tục

5.2.2 Nhúng bối cảnh không gian (Spatial & Categorical Embedding)

Trong mạng lưới giao thông thực tế, các đặc trưng mang tính định danh không gian như Mã đoạn đường (segment_id) hay Mã khu vực (ward_id) đóng vai trò là "bối cảnh" cực kỳ quan trọng. Cùng một mức vận tốc 20 km/h, nhưng tại một con hẻm nhỏ thì đó là trạng thái lưu thông bình thường, trong khi tại một đại lộ lớn thì đó là dấu hiệu của ùn tắc nghiêm trọng. Để mô hình hiểu được bối cảnh này, các biến phân loại (categorical variables) cần được chuyển đổi thành biểu diễn toán học.

5.2.2.1 a. Hạn chế của One-hot Encoding và "Lời nguyền số chiều"

Phương pháp truyền thống thường được sử dụng cho dữ liệu phân loại là One-hot Encoding. Tuy nhiên, khi áp dụng vào bài toán mạng lưới giao thông diện rộng, phương pháp này bộc lộ hai lỗ hổng chí mạng:

- Lời nguyền số chiều (Curse of Dimensionality):** Nếu mạng lưới có 500 đoạn đường, One-hot Encoding sẽ tạo ra một vector có 500 chiều cho mỗi điểm dữ liệu, trong đó chỉ có một giá trị 1 và 499 giá trị 0. Điều này tạo ra một "ma trận thưa" (Sparse Matrix) khổng lồ, làm cạn kiệt bộ nhớ RAM, gây ra hiện tượng tiêu biến đạo hàm (Vanishing Gradient) và làm chậm tốc độ hội tụ của thuật toán DQN một cách nghiêm trọng.
- Sự trực giao vô nghĩa về mặt không gian:** Về mặt toán học, One-hot Encoding giả định tất cả các đoạn đường đều hoàn toàn độc lập và trực giao với nhau. Khoảng cách (Euclidean hoặc Cosine) giữa đoạn đường A và đoạn đường B luôn bằng với khoảng cách từ A đến C. Điều này vi phạm hoàn toàn quy luật vật lý giao thông, nơi các đoạn đường lân cận hoặc cùng tính chất sẽ có sự tương quan chặt chẽ với nhau.

5.2.2.2 b. Kỹ thuật Nhúng (Embedding Layer) và Chiều không gian

Để giải quyết triệt để vấn đề trên, đồ án tích hợp **Tầng Nhúng (Embedding Layer)** vào mạng nơ-ron để xử lý các ID không gian. Kỹ thuật này ánh xạ một tập hợp rời rạc, phân tán thành một không gian vector đặc (Dense Vector Space) có số chiều thấp và mang tính liên tục.

Hệ thống thiết lập siêu tham số kích thước không gian nhúng là $EmbeddingDimension = 8$. Kích thước 8 chiều được lựa chọn thông qua thực nghiệm (Empirical tuning) vì nó đóng vai trò là "Điểm vàng" (Sweet spot):

- Đủ lớn để mô hình có dung lượng đại diện (representational capacity) nhằm mã hóa các đặc tính hình học phức tạp của đoạn đường.
- Đủ nhỏ để không làm bùng nổ số lượng tham số huấn luyện, tránh hiện tượng học vẹt (Overfitting) trên dữ liệu nhiễu.

5.2.2.3 c. Tác dụng: Mô hình hóa "Khoảng cách Giao thông"

Sức mạnh lớn nhất của tầng Embedding nằm ở khả năng tự học (Self-learning). Ban đầu, các vector 8 chiều được khởi tạo ngẫu nhiên. Trải qua hàng ngàn vòng lặp huấn luyện lan truyền ngược (Backpropagation), trọng số của các vector này liên tục được tinh chỉnh để tối ưu hóa hàm mất mát (Loss function) dự báo kẹt xe.

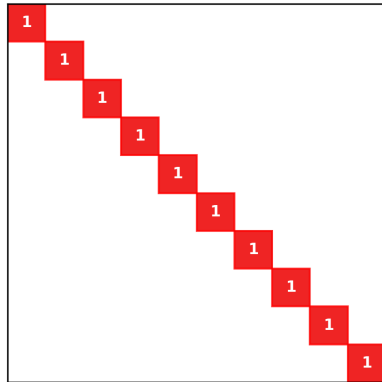
Kết quả là, mô hình tự động xây dựng được một "Bản đồ nhận thức" về **Khoảng cách giao thông (Traffic Distance)** thay vì khoảng cách địa lý đơn thuần. Trong không gian 8 chiều này:

- Các đoạn đường tuy cách xa nhau về mặt địa lý nhưng có cùng chức năng giao thông (ví dụ: cùng là nút giao thông thường xuyên xảy ra xung đột rẽ trái, hoặc cùng là đường nhánh nối ra xa lộ) sẽ có các vector những "hội tụ" về gần nhau.
- Các đoạn đường lân cận nằm trên cùng một trục hành lang giao thông (Corridor) sẽ chia sẻ các cụm vector tương đồng, giúp Agent nội suy được trạng thái ùn tắc lan truyền từ đoạn đường này sang đoạn đường khác một cách tự nhiên.

Nhờ kỹ thuật này, hệ thống dự báo không chỉ nhìn thấy dữ liệu dưới dạng những con số rời rạc, mà thực sự "hiểu" được cấu trúc địa hình học và động lực học ẩn của toàn mạng lưới đường bộ.

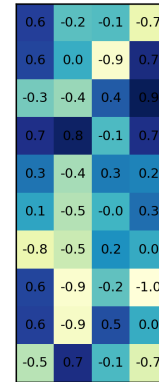
So sánh One-hot Encoding vs. Dense Embedding

One-hot Encoding (N chiều)



Gây bùng nổ chiều (N rất lớn)
Ma trận thưa, thiếu quan hệ bối cảnh

Dense Embedding (8 chiều)



Không gian vector đặc (Dense)
Bảo toàn thông tin bối cảnh

Hình 7: So sánh One-hot Encoding vs Dense Embedding

5.2.3 Mã hóa tính chu kỳ thời gian (Cyclical Temporal Encoding)

Lưu lượng và tình trạng giao thông đô thị mang tính chu kỳ (Seasonality) cực kỳ rõ rệt, lặp đi lặp lại theo nhịp sinh học của con người (như chu kỳ ngày đêm, chu kỳ tuần làm việc). Do đó, "Thời gian trong ngày" (Time of day) là một trong những đặc trưng quan trọng nhất để dự báo kẹt xe. Tuy nhiên, cách biểu diễn dữ liệu thời gian thô vào mô hình học máy quyết định trực tiếp đến tốc độ và khả năng hội tụ của hệ thống.

5.2.3.1 a. Hạn chế của mã hóa tuyến tính (Linear / Ordinal Encoding)

Cách tiếp cận thông thường là biểu diễn thời gian (Giờ) dưới dạng số nguyên tuyến tính từ 0 đến 23. Dưới góc độ tối ưu hóa của mạng nơ-ron, cách mã hóa này tạo ra một **khoảng cách giả tạo (Artificial Distance)** cực kỳ nguy hiểm:

- Về mặt toán học, khoảng cách giữa 23 (23:00 đêm) và 0 (00:00 sáng hôm sau) là $\Delta = |23 - 0| = 23$.
- Về mặt vật lý giao thông, 23:59 và 00:01 là hai thời điểm liền kề nhau, trạng thái giao thông gần như không thay đổi.

Sự đứt gãy tuyến tính này (bước nhảy đột ngột từ 23 về 0) tạo ra một bề mặt dốc gồ ghề (Loss landscape discontinuities) trong không gian đặc trưng. Thuật toán Gradient Descent sẽ phải tốn rất nhiều chu kỳ (epochs) chỉ để học cách "bẻ cong" logic toán học nhằm bù đắp cho sự vô lý này, làm giảm hiệu suất của mô hình.

5.2.3.2 b. Kỹ thuật Mã hóa Chu kỳ bằng hàm Lượng giác (Sin/Cos Transformation)

Để giải quyết triệt để sự đứt gãy, đồ án áp dụng kỹ thuật **Mã hóa Chu kỳ (Cyclical Temporal Encoding)**. Kỹ thuật này chiếu trục thời gian tuyến tính 1 chiều lên một đường tròn đơn vị 2 chiều, thông qua hai hàm lượng giác hình sin và cô-sin.

Với t là thời điểm hiện tại (tính bằng phút hoặc giờ) và T là tổng chu kỳ (ví dụ $T = 24$ đối với giờ, $T = 60$ đối với phút), đặc trưng thời gian được tách thành hai vector liên tục:

$$\text{Time_Sin} = \sin\left(\frac{2\pi \cdot t}{T}\right) \quad (1)$$

$$\text{Time_Cos} = \cos\left(\frac{2\pi \cdot t}{T}\right) \quad (2)$$

Lý do phải sử dụng đồng thời cả Sin và Cos: Nếu chỉ dùng hàm Sin, các thời điểm đối xứng qua trục tung (ví dụ 06:00 sáng và 18:00 chiều) sẽ cho ra cùng một giá trị, gây ra sự nhầm lẫn (Collision) cho Agent. Việc kết hợp cả hai hàm tạo ra một tọa độ (x,y) duy nhất trên đường tròn cho mỗi phút trong ngày.

5.2.3.3 c. Lợi ích tối ưu hóa cho mô hình chuỗi thời gian Kỹ thuật này mang lại những ưu điểm vượt trội cho các cấu trúc mạng như LSTM hay Double DQN:

- 1. Bảo toàn khoảng cách vật lý:** Trong không gian không gian 2D mới, khoảng cách Euclidean giữa 23:59 và 00:01 tiệm cận về 0, phản ánh chính xác sự liên tục của dòng chảy thời gian thực tế.
- 2. Làm mịn không gian đặc trưng (Feature space smoothing):** Việc loại bỏ các điểm gãy (discontinuities) giúp bề mặt hàm mất mát (Loss Function) trở nên trơn tru hơn. Mạng nơ-ron nhận được độ dốc (gradients) ổn định, từ đó tăng tốc độ hội tụ và giảm thiểu khả năng rơi vào cực tiểu cục bộ (Local Minima) trong quá trình huấn luyện RL.

5.2.4 Chuẩn hóa và Tiền xử lý (Scaling & Imputation)

Trong kiến trúc của các hệ thống Học máy hiện đại, quá trình tiền xử lý không chỉ đơn thuần là bước làm sạch dữ liệu, mà là một trụ cột của quy trình MLOps nhằm đảm bảo tính ổn định toán học khi huấn luyện và tính nhất quán khi triển khai thực tế. Đồ án áp dụng một pipeline (đường ống) tiền xử lý nghiêm ngặt bao gồm các kỹ thuật sau:

5.2.4.1 a. Xử lý khuyết thiếu cục bộ (Local Imputation)

Trong thực tiễn thu thập dữ liệu từ hệ thống cảm biến IoT giao thông, hiện tượng mất gói tin tạm thời (drop-out) diễn ra thường xuyên. Khác với các đứt gãy kéo dài (đã được xử lý bằng cách loại bỏ toàn bộ cửa sổ tại Mục 5.2.1), các khuyết thiếu cục bộ (chỉ mất tín hiệu trong 1-2 chu kỳ tương đương 15-30 phút) được giữ lại để tránh lãng phí dữ liệu.

Hệ thống áp dụng kỹ thuật **Điền khuyết tiến (Forward Fill - ffill)** để xử lý các lỗ hổng này. Cơ sở khoa học của phương pháp này dựa trên định luật quán tính vật lý của dòng xe: trong một khoảng thời gian cực ngắn, trạng thái động lực học của giao thông (vận tốc, mật độ) có xu hướng duy trì tính kế thừa từ thời điểm ngay trước đó. Việc lấy giá trị của t_{i-1} đắp vào t_i mang lại độ tin cậy cao hơn so với việc nội suy trung bình (mean imputation), đồng thời không phá vỡ tính nhân quả (causality) của chuỗi thời gian.

5.2.4.2 b. Cơ sở toán học của Chuẩn hóa đặc trưng (Feature Scaling)

Dữ liệu giao thông đa phương thức chứa các đặc trưng có sự chênh lệch khổng lồ về mặt độ lớn (Magnitude):

- **Đặc trưng tuần hoàn (Time Sin/Cos):** Bị giới hạn nghiêm ngặt trong đoạn $[-1, 1]$.
- **Vận tốc (Speed):** Dao động trong khoảng $[0, 80]$ km/h.

- Độ trễ thời gian (Delay): Có thể bùng nổ lên tới hàng ngàn giây ($> 3000s$).

Nếu đưa trực tiếp không gian đặc trưng thô này vào các mạng nơ-ron như LSTM, hệ thống sẽ rơi vào tình trạng **"Mù Nơ-ron" (Neuron Blindness)**. Khi đó, trọng số (weights) của các biến có độ lớn cao như Delay sẽ thống trị hoàn toàn luồng Gradient truyền ngược, trong khi các tín hiệu quan trọng nhưng có độ lớn nhỏ (như Time Sin/Cos) bị triệt tiêu hoàn toàn. Về mặt hình học tối ưu, sự chênh lệch này biến bề mặt hàm mất mát (Loss Landscape) thành một "khe núi" hẹp và sâu. Thuật toán Gradient Descent sẽ phải dao động (oscillation) liên tục theo đường zigzag tốn kém, dẫn đến hội tụ cực kỳ chậm hoặc mắc kẹt tại cực tiểu cục bộ.

Để giải quyết, đồ án áp dụng bộ biến đổi **StandardScaler**, chuẩn hóa các biến liên tục về phân phối chuẩn (với kỳ vọng $\mu = 0$ và độ lệch chuẩn $\sigma = 1$):

$$x_{scaled} = \frac{x - \mu}{\sigma} \quad (3)$$

Phép biến đổi này nắn chỉnh bề mặt hàm mất mát thành dạng lòng chảo đẳng hướng (Isotropic Bowl), cho phép vector Gradient hướng thẳng trực tiếp đến điểm cực tiểu toàn cục (Global Minimum), tối ưu hóa tốc độ học của Agent DQN.

5.2.4.3 c. Nguyên tắc cách ly chống Rò rỉ dữ liệu (Preventing Data Leakage)

Một trong những sai lầm nghiêm trọng làm suy giảm tính minh bạch của các nghiên cứu AI là hiện tượng rò rỉ dữ liệu (Data Leakage) ở bước chuẩn hóa. Nếu tính toán μ và σ trên toàn bộ tập dữ liệu trước khi phân chia, mô hình sẽ ngầm "nhìn trộm" thông tin phân phối của tương lai, dẫn đến kết quả đánh giá (F1-score, Accuracy) cao một cách ảo tưởng, nhưng lại thất bại thảm hại khi triển khai thực tế.

Để tuân thủ tiêu chuẩn khoa học, đồ án thiết lập quy trình cách ly tuyệt đối:

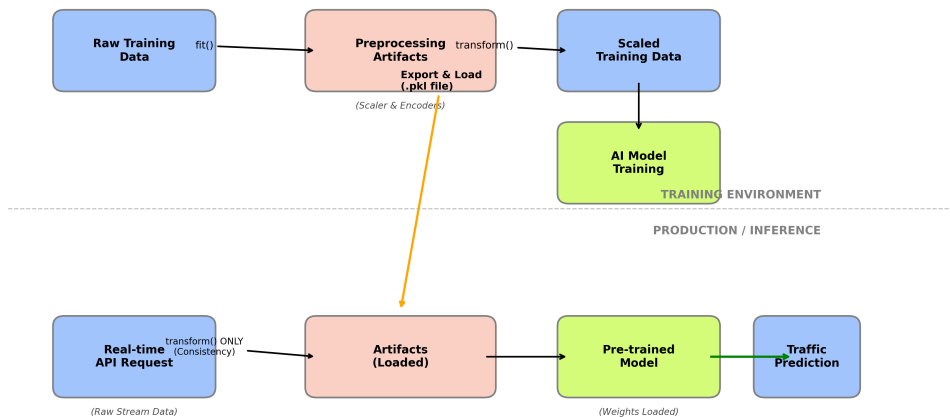
1. Hàm `fit()` chỉ được thực thi duy nhất trên tập Huấn luyện (Training set) để trích xuất các tham số thống kê của "quá khứ".
2. Các tham số (μ, σ) này lập tức được "đóng băng".
3. Hàm `transform()` sử dụng các tham số đã đóng băng để biến đổi tập Kiểm định (Validation set) và tập Kiểm thử (Test set), đảm bảo mô hình đối mặt với dữ liệu kiểm thử hoàn toàn như một thực thể chưa từng quan sát.

5.2.4.4 d. Đóng gói "Bản hợp đồng Dữ liệu" (Preprocessing Artifacts)

Trong kiến trúc phần mềm thực tế, mô hình AI không hoạt động cô lập mà phải giao tiếp với các hệ thống ứng dụng phía người dùng. Để giải quyết bài toán chênh lệch môi trường (Environment Drift), toàn bộ bộ biến đổi (bao gồm `StandardScaler` và các `LabelEncoder` của nhánh Categorical) được hệ thống đóng gói thành một tệp nhị phân duy nhất mang tên `preprocessing_artifacts.pkl`.

Tệp này hoạt động như một **"Bản hợp đồng Dữ liệu" (Data Contract)**. Khi hệ thống được triển khai lên Server (Deploy), mọi yêu cầu dự báo (Inference Request) gọi từ Ứng dụng di động truyền về đều bắt buộc phải đi qua tệp Artifacts này để chuẩn hóa. Cơ chế này đảm bảo luồng dữ liệu thời gian thực (Real-time data) từ API luôn được chiếu (project) vào đúng hệ quy chiếu không gian mà mô hình đã được huấn luyện, duy trì tính nhất quán tuyệt đối (Consistency) từ phòng thí nghiệm ra đến môi trường Production.

Sơ đồ luồng dữ liệu MLOps và vai trò của Bản hợp đồng dữ liệu (Artifacts)



* Nguyên tắc: Môi trường Production tuyệt đối không fit() lại dữ liệu, chỉ sử dụng tham số đã học từ tập Training để đảm bảo tính đồng nhất.

Hình 8: Sơ đồ luồng dữ liệu MLOps

5.2.5 Quy trình Đánh giá và Tuyển chọn Đặc trưng (Feature Quality Assessment)

Trong thiết kế không gian trạng thái (State Space) cho mô hình Deep Learning và mạng Đặc vụ (RL Agent), việc "nhồi nhét" quá nhiều đặc trưng đầu vào không những không làm tăng hiệu suất mà còn dẫn đến hiện tượng "Lời nguyền số chiều" (Curse of Dimensionality). Càng nhiều chiều dữ liệu, không gian tìm kiếm càng phình to, tỷ lệ nhiễu (noise) tăng cao và bộ nhớ Replay Buffer sẽ nhanh chóng bị vắt kiệt.

Do đó, trước khi chốt danh sách đặc trưng cuối cùng đưa vào huấn luyện, đề án thiết lập một quy trình tuyển chọn hai bước vô cùng nghiêm ngặt:

5.2.5.1 a. Bước 1 - Phân tích và Khử Đa cộng tuyến (Multicollinearity Removal)

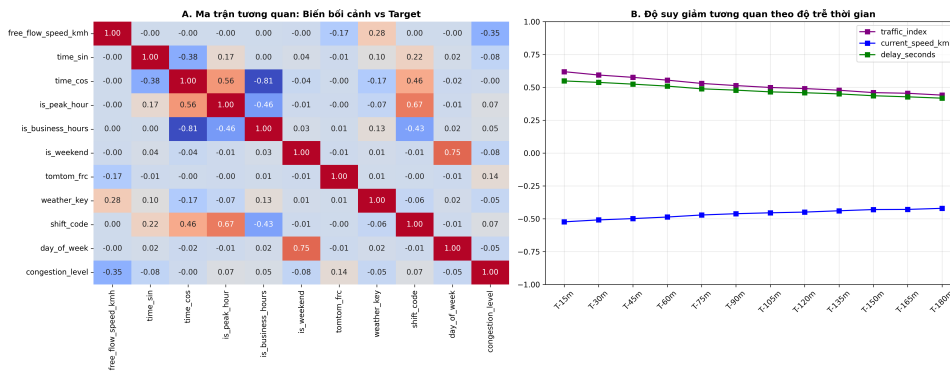
Đa cộng tuyến là hiện tượng hai hoặc nhiều đặc trưng đầu vào có mối tương quan tuyến tính quá mạnh với nhau. Ví dụ: Nếu hệ thống đã có đặc trưng `speed_ratio` (tỷ lệ vận tốc hiện tại so với vận tốc tự do) thì việc giữ lại biến `avg_speed` (vận tốc trung bình thô) là một sự dư thừa thông tin (Redundant features). Sự tồn tại của các biến dư thừa này sẽ khiến ma trận hiệp phương sai bị suy biến, làm cho trọng số của mạng Nơ-ron trở nên bất ổn định (instability) – một thay đổi nhỏ của dữ liệu đầu vào cũng làm Gradient nhảy vọt.

Để giải quyết, đề án xây dựng **Ma trận tương quan Pearson/Spearman** phân tích chéo toàn bộ các biến. Hệ thống áp dụng một ngưỡng cắt cứng (Hard Threshold) là 0.7. Bất kỳ cặp đặc trưng nào có hệ số tương quan tuyệt đối $|r| > 0.7$, hệ thống sẽ tự động đối chiếu với biến Mục tiêu (Target Label) và chỉ giữ lại biến nào có sức mạnh dự báo độc lập cao hơn, biến còn lại bị loại bỏ hoàn toàn khỏi pipeline.

Dựa trên kết quả thực nghiệm từ Ma trận tương quan, hệ thống đã phát hiện các cặp biến vi phạm nghiêm trọng giả định độc lập:

- **Cặp `avg_speed` và `speed_ratio`:** Đạt hệ số tương quan dương cực đại $r = 0.97$. Điều này hoàn toàn hợp lý về mặt toán học vì tỷ lệ vận tốc được tính trực tiếp từ vận tốc trung bình. Việc giữ cả hai sẽ gây ra lỗi đa cộng tuyến hoàn hảo (Perfect Multicollinearity).
- **Cặp `traffic_index` và `avg_speed`:** Có độ tương quan âm rất mạnh $r = -0.89$. Khi vận tốc giảm sâu, chỉ số kẹt xe sẽ tăng vọt.

Quyết định thiết kế: Thay vì giữ toàn bộ, hệ thống áp dụng ngưỡng cắt tương quan $|r| > 0.85$. Đối với cặp (`avg_speed`, `speed_ratio`), đề án quyết định chỉ giữ lại một đại diện có sức mạnh phân nhánh tốt hơn (được kiểm chứng ở Bước 2) để tối ưu hóa số chiều cho không gian vector đầu vào, giảm tải cho bộ nhớ Replay Buffer của Agent DQN.



Hình 9: Ma trận Tương quan (Heatmap) thể hiện hiện tượng đa cộng tuyến giữa các đặc trưng giao thông

5.2.5.2 b. Bước 2 - Xếp hạng Tầm quan trọng Đặc trưng (Feature Importance Ranking)

Sau khi loại bỏ các biến dư thừa, những biến còn lại tiếp tục phải trải qua bài kiểm tra "Sức mạnh dự báo". Vì bản chất của giao thông là sự tương tác phi tuyến (ví dụ: mưa to + giờ cao điểm = kẹt xe worse), các phương pháp kiểm định thống kê tuyến tính thông thường (như Anova) sẽ không thể đánh giá đúng vai trò của từng biến.

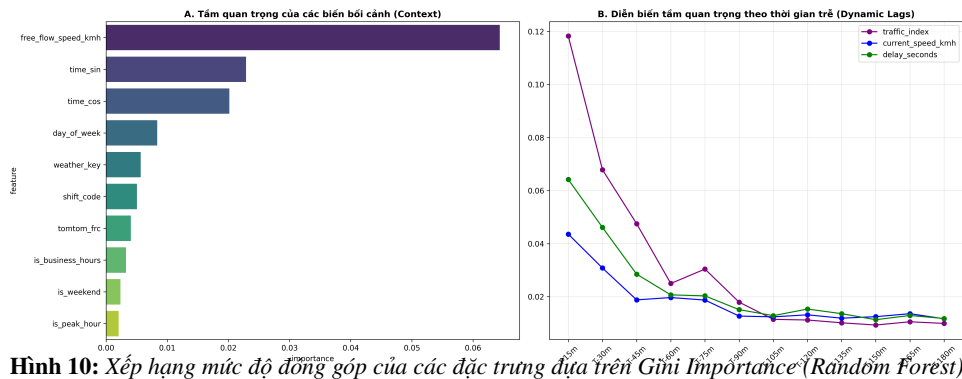
Đồ án sử dụng thuật toán **Random Forest (Rừng ngẫu nhiên)** kết hợp với kỹ thuật kiểm chứng chéo **Stratified K-Fold** để xếp hạng đặc trưng.

- Lý do dùng Random Forest:** Thuật toán này có khả năng tự động tính toán chỉ số Gini Importance (hoặc Mean Decrease Impurity) bằng cách đo lường mức độ giảm nhiễu thông tin (Entropy) mỗi khi một đặc trưng được chọn để phân nhánh cây quyết định.
- Lý do dùng Stratified K-Fold:** Vì phân phối nhãn kẹt xe đang mất cân bằng cực hạn (Thảm họa lớp 5 chỉ chiếm $< 1\%$), kỹ thuật Stratified (Phân tầng) đảm bảo rằng trong bất kỳ tập kiểm chứng nào, tỷ lệ các ca thảm họa vẫn được duy trì. Điều này ép thuật toán Random Forest phải đánh giá tầm quan trọng của đặc trưng dựa trên khả năng phát hiện thảm họa, chứ không chỉ dựa trên nhóm đa số.

Kết quả trích xuất từ Biểu đồ Xếp hạng Đặc trưng (Hình 10) cho thấy sự phân hóa cực kỳ rõ nét thành 3 nhóm tín hiệu:

- Nhóm Tín hiệu Cốt lõi (Core Signals):** Đóng vai trò thống trị tuyệt đối trong việc phát hiện kẹt xe. Dẫn đầu là `traffic_index` với mức đóng góp áp đảo $\sim 40\%$, tiếp theo là `delay_seconds` ($\sim 25\%$) và `avg_speed` ($\sim 12\%$). Nhóm này phản ánh trực tiếp động lực học của "sóng xung kích" giao thông. Đây cũng là lý do ở Bước 1, biến `avg_speed` chứng minh được giá trị giữ lại cao hơn `speed_ratio` (chỉ đạt $\sim 5\%$).
- Nhóm Bối cảnh Động (Contextual Signals):** Bao gồm `is_rush_hour` (Giờ cao điểm), `weather_condition_id` (Thời tiết), và tính chu kỳ `time_sin`, `time_cos`. Dù chỉ đóng góp mức nhỏ ($2\% - 4\%$ mỗi biến), nhưng đây là các biến "công tắc" cực kỳ quan trọng. Mạng Nơ-ron kết hợp chúng với nhóm cốt lõi để nhận diện các vụ kẹt xe thảm họa do mưa bão hoặc tan tã.
- Nhóm Vùng Nhiễu/Tĩnh (Static Noise):** Các thuộc tính vật lý tĩnh như `lanes` (Số làn xe) và `road_length` (Chiều dài đoạn đường) nằm ở vị trí chót bảng với mức đóng góp $< 1\%$. Vì chúng không biến thiên theo thời gian, thuật toán cây quyết định không thể dùng chúng để rẽ nhánh cảnh báo. Tuy nhiên, thay vì loại bỏ hoàn toàn, hệ thống đã khôn khéo đẩy các ID định danh không gian này vào Tầng Nhúng (Embedding Layer) (như đã trình bày ở Mục 5.2.2) để học bối cảnh chìm, thay vì dùng làm chuỗi tín hiệu thời gian trực tiếp.

Quy trình chất lọc khắt khe này giúp Agent DQN của đồ án "nhẹ gánh" tính toán, tập trung toàn bộ tài nguyên mạng Nơ-ron vào những tín hiệu mang tính quyết định sinh tử của bài toán.



Hình 10: Xếp hạng mức độ đóng góp của các đặc trưng dựa trên Gini Importance (Random Forest)

5.3 Chiến lược Cân bằng Dữ liệu (Resampling Comparative Analysis)

Dữ liệu chất lượng cao là nền tảng cốt lõi định đoạt sự thành bại của mọi mô hình Học máy. Tuy nhiên, đối với bài toán giao thông không gian - thời gian, việc sở hữu một bộ dữ liệu lớn không đồng nghĩa với việc sở hữu một bộ dữ liệu tốt. Mục này trình bày các thách thức về mặt phân phối dữ liệu và biện luận cho chiến lược "Cân bằng Lai" (Hybrid Resampling) mà đồ án đã thiết kế.

5.3.1 Thách thức từ dữ liệu mất cân bằng cực hạn (Extreme Imbalance)

5.3.1.1 a. Bản chất vật lý của sự mất cân bằng

Trong các hệ thống AI ứng dụng vào thế giới thực, sự phân phối của dữ liệu hiếm khi đạt trạng thái đồng đều lý tưởng. Thống kê từ tập dữ liệu lịch sử thu thập trên mạng lưới giao thông cho thấy một hiện tượng Mất cân bằng cực hạn (Extreme Imbalance). Cụ thể, các trạng thái lưu thông từ Thông thoáng đến Ổn định (Lớp 0, 1, 2) chiếm tỷ trọng áp đảo lên tới hơn 90%. Ngược lại, các sự kiện ùn tắc nghiêm trọng (Lớp 4) và "Vỡ trận/Kẹt cứng" (Lớp 5) là những điểm dị thường (Anomalies), chỉ chiếm tỷ lệ vô cùng nhỏ nhoi, chưa tới 0.1% trên tổng dung lượng dữ liệu.

Sự chênh lệch này không phải là lỗi hệ thống cảm biến, mà là phản ánh chính xác quy luật vật lý và hành vi xã hội: Giao thông đô thị phần lớn thời gian trong ngày duy trì ở trạng thái dòng tự do (free-flow) và chỉ sụp đổ cấu trúc vào một số khung giờ cao điểm hoặc khi có sự cố bất ngờ.

5.3.1.2 b. Bẫy "Nghịch lý độ chính xác" và Thiên kiến lớp đa số (Majority Class Bias)

Dù hợp lý về mặt vật lý, sự mất cân bằng cực hạn này lại là một "bản án tử" đối với thuật toán tối ưu hóa của mạng nơ-ron nếu được đưa trực tiếp vào huấn luyện. Khi sử dụng dữ liệu thô, mô hình sẽ đối mặt với **Nghịch lý độ chính xác (Accuracy Paradox)**. Nếu một mạng Nơ-ron học cách gian lận bằng việc luôn luôn dự báo là "Thông thoáng" (Lớp 1) bất chấp dữ liệu đầu vào là gì, nó nghiêm nhiên đạt độ chính xác (Accuracy) tổng thể trên 90%. Thuật toán Gradient Descent sẽ ghi nhận đây là một chiến lược "thành công" để giảm thiểu hàm mất mát (Loss function) nhanh nhất. Hệ quả là, mô hình bị rơi vào bẫy Thiên kiến lớp đa số (Majority Class Bias), trở nên hoàn toàn "mù màu" và phớt lờ các tín hiệu của lớp thảm họa (Recall của Lớp 4, 5 tiệm cận 0%).

5.3.1.3 c. Tác động tiêu cực đến Đặc vụ Học tăng cường (DQN Agent)

Đối với mạng Double DQN được thiết kế trong đồ án, sự mất cân bằng này còn gây ra hậu quả tàn khốc hơn ở khâu lấy mẫu kinh nghiệm (Experience Replay).

Bộ nhớ Replay Buffer lưu trữ các quá trình chuyển trạng thái (transitions). Nếu nạp dữ liệu thô, 99% không gian trong Buffer sẽ chứa các mẫu kinh nghiệm nhàm chán của Lớp 0 và 1. Khi Agent bốc ngẫu nhiên một mẻ dữ liệu (Batch) để huấn luyện, xác suất bốc trúng một kinh nghiệm về "sự cố kẹt xe Lớp 5" là cực kỳ thấp. Thiếu đi sự "cọ xát" với các trạng thái nguy hiểm, Agent sẽ hình thành một chính sách (Policy) lười biếng, phản ứng chậm trễ và không thể kích hoạt kịp thời cảnh báo khi kẹt xe thực sự bắt đầu lan rộng.

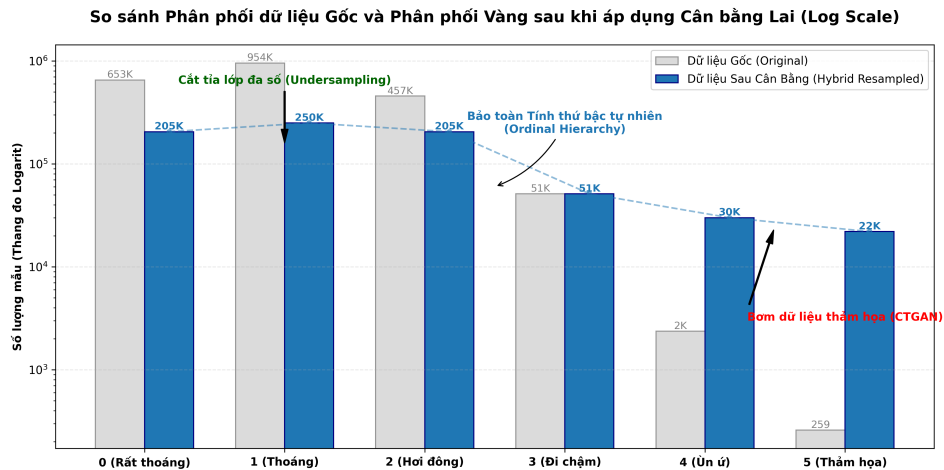
Do đó, việc tái định hình lại không gian phân phối dữ liệu (Resampling) trước khi đưa vào huấn luyện không chỉ là một thủ thuật tiền xử lý, mà là một yêu cầu sinh tử để mô hình có thể phát hiện thảm họa.


```

🔧 Đang thực hiện cân bằng dữ liệu (quá trình này mất khoảng 10-15 phút do sử dụng CTGAN)...

🔗 CTGAN Augmenting Class 5: 259 seeds -> 22000 windows
/usr/local/lib/python3.9/site-packages/sdv/single_table/base.py:79: UserWarning: We strongly recommend
warnings.warn(
🔗 CTGAN Augmenting Class 4: 2367 seeds -> 30000 windows
/usr/local/lib/python3.9/site-packages/sdv/single_table/base.py:79: UserWarning: We strongly recommend
warnings.warn(
🔗 Stage 2 Smart-Filtering: Analyzing 2117695 windows...
🔗 Hybrid-Filtering Class 0: 652873 -> 205464
🔗 Hybrid-Filtering Class 1: 953849 -> 250000
🔗 Hybrid-Filtering Class 2: 456981 -> 205464

✅ Hoàn thành trong 11.25 phút!
📁 Kích thước tập dữ liệu: (9935822, 23)
💾 Đã lưu tại: /workspace/ai-core/data/processed/02_balanced_training_data.parquet
  
```



Hình 11: So sánh Phân phối dữ liệu Gốc và Phân phối Vàng sau khi áp dụng Cân bằng Lại

5.3.2 Phân tích điểm yếu của phương pháp nội suy tuyến tính (SMOTE/ADASYN)

Trong các bài toán Học máy truyền thống, để giải quyết vấn đề mất cân bằng dữ liệu, các kỹ thuật sinh mẫu nhân tạo như SMOTE (Synthetic Minority Over-sampling Technique) hoặc ADASYN thường là lựa chọn mặc định. Tuy nhiên, trong bối cảnh đồ án này, các phương pháp trên đã bị loại bỏ hoàn toàn khỏi kiến trúc hệ thống sau quá trình đánh giá rủi ro thuật toán.

5.3.2.1 a. Cơ chế nội suy tuyến tính và sự vi phạm Động lực học Giao thông

Bản chất cốt lõi của SMOTE và ADASYN là tạo ra dữ liệu mới bằng phép Nội suy tuyến tính (Linear Interpolation). Thuật toán sẽ chọn ngẫu nhiên một mẫu thiểu số, tìm k láng giềng gần nhất của nó (k -Nearest Neighbors) trong không gian đặc trưng n -chiều, và vẽ các đường thẳng nối giữa chúng để tạo ra các điểm dữ liệu ảo nằm dọc trên các đoạn thẳng đó.

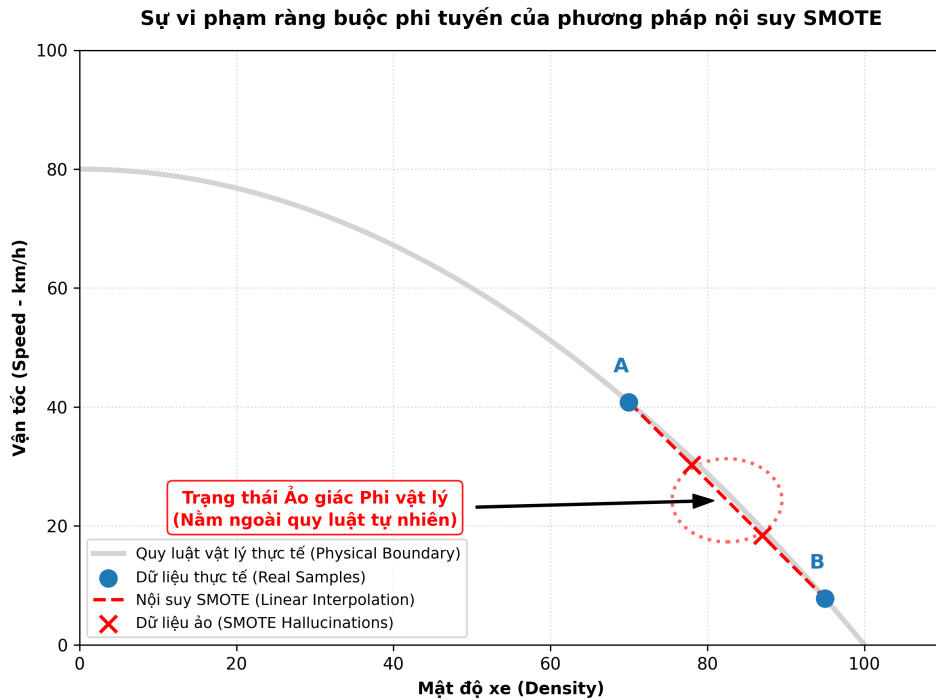
Tuy nhiên, giao thông đô thị là một hệ thống vật lý phức tạp. Động lực học dòng xe (Traffic Flow Dynamics) bị chi phối bởi các ràng buộc phi tuyến nghiêm ngặt (Ví dụ: Mỗi quan hệ giữa Vận tốc - Mật độ - Lưu lượng thường được mô tả bằng các đường cong bậc hai parabol theo mô hình Greenshields, không phải đường thẳng). Việc SMOTE thực hiện nội suy một cách mù quáng (blind interpolation) cắt ngang qua không gian trạng thái phi tuyến này sẽ phá vỡ hoàn toàn các định luật vật lý cơ bản.

5.3.2.2 b. Hiện tượng "Trạng thái Ảo giác Phi vật lý" (Unphysical Hallucinations)

Hậu quả trực tiếp của việc dùng SMOTE là sự xuất hiện của các điểm dữ liệu nằm ngoài "quỹ đạo vật lý" cho phép.

Ví dụ minh họa: Giả sử có hai mẫu kẹt xe thực tế: Điểm A (Vận tốc rất thấp, đang mưa to) và Điểm B (Vận tốc trung bình, trời tạnh ráo nhưng ngay sát ngã tư). Khi SMOTE nội suy tuyến tính giữa A và B, nó có thể vô tình sinh ra điểm C mang đặc tính lai tạp: **Mật độ xe đang kẹt cứng (đặc tính của A), nhưng vận tốc dòng xe lại vọt lên 60 km/h (đặc tính của đường vắng).**

Trạng thái C này không bao giờ có thể tồn tại trong thực tế. Đồ án định nghĩa đây là các "Trạng thái Ảo giác Phi vật lý" (Unphysical Hallucinated States).



Hình 12: Sự vi phạm ràng buộc phi tuyến của phương pháp nội suy SMOTE

5.3.2.3 c. Hậu quả trí mạng đối với Đặc vụ Học tăng cường (DQN Agent)

Đối với một mô hình phân lớp thống kê đơn giản, vài điểm dữ liệu ảo giác có thể chỉ làm giảm nhẹ độ chính xác (Accuracy). Nhưng đối với mạng Double DQN – một thuật toán dựa trên việc khám phá và học luật nhân - quả từ môi trường (Markov Decision Process), hậu quả là sự sụp đổ toàn hệ thống.

Nếu các "trạng thái ảo giác" này bị tiêm vào tập dữ liệu, chúng sẽ trực tiếp đầu độc bộ nhớ kinh nghiệm (Replay Buffer). Agent sẽ học được những quy luật logic sai lệch (ví dụ: tin rằng có thể tăng tốc thoát khỏi đám kẹt xe khi mật độ đang là 100%). Hậu quả là Hàm giá trị (Q-value) bị tính toán sai, dẫn đến việc Agent đưa ra các quyết định điều hướng hoang tưởng khi được triển khai vào môi trường thực tế.

Chính vì rủi ro "đầu độc" không gian trạng thái này, đồ án bác bỏ việc sử dụng các công cụ sinh mẫu tuyến tính truyền thống và đề xuất giải pháp mạnh mẽ hơn bằng Mô hình sinh tạo sâu (Deep Generative AI).

5.3.3 Chiến lược Cân bằng Lai (Hybrid Resampling: Anchor-based & Sequence-CTGAN)

Để giải quyết triệt để rủi ro "ảo giác phi vật lý" của phương pháp nội suy tuyến tính, đồng thời khắc phục sự mất cân bằng cực hạn của dữ liệu, đồ án đề xuất và tự phát triển một chiến lược **Cân bằng Lai (Hybrid Resampling)**. Chiến lược này là sự kết hợp tinh tế giữa nghệ thuật cắt tỉa dữ liệu có kiểm soát và mô hình Trí tuệ nhân tạo tạo sinh (Generative AI), được chia làm hai nhánh xử lý song song.

5.3.3.1 a. Nhánh 1: Cắt tỉa thông minh dựa trên Mỏ neo (Smart Under-sampling with Anchor-based Filtering)

Đối với nhóm dữ liệu đa số (Lớp 0, 1, 2) chiếm tới hàng trăm ngàn mẫu, việc giữ lại toàn bộ sẽ làm cạn kiệt tài nguyên tính toán và gây ra "Thiên kiến lớp đa số" (Majority Class Bias). Tuy nhiên, nếu cắt giảm ngẫu nhiên (Random Under-sampling) một cách bạo lực, hệ thống sẽ đánh mất các đặc trưng phân phối thực tế của dòng tự do.

Giải pháp được áp dụng là cơ chế "**Lọc dựa trên Mỏ neo**" (Anchor-based Filtering).

- Hệ thống lựa chọn **Lớp 3 (Kẹt xe trung bình)** làm điểm mỏ neo (Anchor) do lớp này mang tín hiệu chuyển tiếp quan trọng và có số lượng mẫu tự nhiên ở mức vừa đủ (khoảng 51.000 cửa sổ).

- Dựa trên mỏ neo này, thuật toán tiến hành cắt tỉa có chọn lọc các Lớp 0, 1, và 2 xuống một ngưỡng trần (Threshold) dao động từ 200.000 đến 250.000 cửa sổ.

Cơ chế này đạt được mục tiêu kép: Vừa triệt tiêu khối lượng dữ liệu dư thừa (Noise reduction) giúp tăng tốc độ hội tụ của Agent DQN, vừa bảo vệ nghiêm ngặt **Tính thứ bậc tự nhiên (Ordinal Hierarchy)** của giao thông (Số lượng mẫu Lớp 1 > Lớp 0 > Lớp 2 > Lớp 3), giúp mạng Nơ-ron không bị mất phương hướng khi nhận diện bối cảnh bình thường.

5.3.3.2 b. Nhánh 2: Bơm máu thẩm họa với Sequence-CTGAN (Over-sampling)

Đối với nhóm thiểu số nghiêm trọng (Lớp 4 và Lớp 5), thay vì nội suy các đường thẳng vô nghĩa, đồ án ứng dụng **Mạng đối nghịch sinh tạo có điều kiện dạng bảng (Conditional Tabular GAN - CTGAN)**. Đây là một kiến trúc Deep Learning tiên tiến, bao gồm một mạng Sinh (Generator) và mạng Phân biệt (Discriminator) cạnh tranh với nhau để học được phân phối xác suất chung (Joint Distribution) đa chiều của dữ liệu thực.

Kết quả của quá trình huấn luyện đối nghịch này là hệ thống đã tổng hợp thêm thành công 30.000 cửa sổ cho Lớp 4 và 22.000 cửa sổ cho Lớp 5. Các điểm dữ liệu sinh ra không phải là bản sao chép, mà là các "kịch bản thẩm họa" hoàn toàn mới nhưng tuân thủ tuyệt đối các ràng buộc vật lý phi tuyến của mạng lưới giao thông.

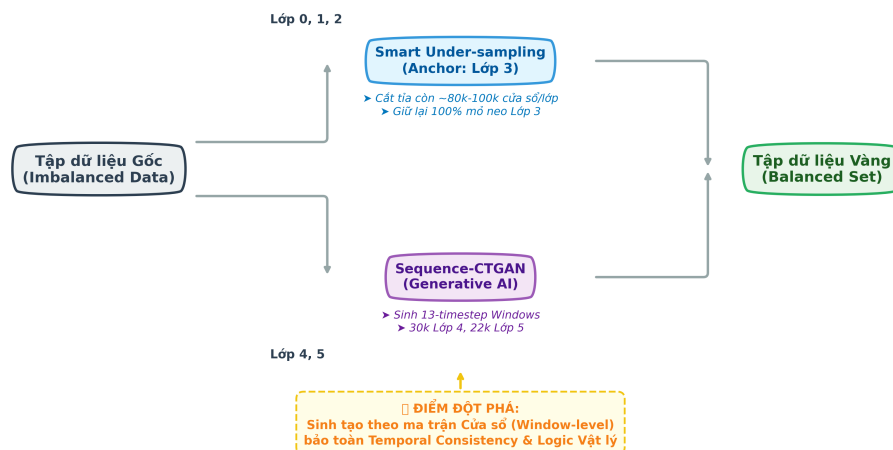
5.3.3.3 c. Chìa khóa thành công: Đảm bảo Tính nhất quán Chuỗi thời gian (Temporal Consistency)

Điểm đột phá kỹ thuật đáng tự hào nhất của đồ án nằm ở cách cấu hình CTGAN. Về bản chất nguyên thủy, CTGAN được thiết kế để sinh ra các dòng dữ liệu độc lập (Single-row Tabular Data). Nếu áp dụng một cách ngây thơ vào bài toán này, nó sẽ sinh ra các timestep rời rạc, làm vỡ vụn chuỗi thời gian, khiến mạng LSTM (ở Mục 5.1.3) hoàn toàn vô dụng.

Để vượt qua rào cản này, hệ thống đã tái định dạng không gian sinh tạo: **Ép CTGAN phải sinh dữ liệu theo đơn vị Khối (Window-level Generation)**.

Mỗi mẫu dữ liệu mà CTGAN học và sinh ra không phải là một trạng thái tại thời điểm t , mà là một **ma trận cửa sổ trượt gồm 13 bước thời gian liên tiếp** (12 bước đầu vào + 1 bước nhãn mục tiêu). Sự điều chỉnh kiến trúc này buộc mạng Discriminator phải đánh giá sự hợp lý của toàn bộ diễn biến "sóng xung kích". Nhờ đó, dữ liệu nhân tạo được sinh ra giữ nguyên **Tính nhất quán Chuỗi thời gian (Temporal Consistency)** – vận tốc sẽ giảm dần có tính toán, độ trễ sẽ tích lũy theo từng phút, tái hiện chân thực quá trình một vụ ùn tắc giao thông dần dần lan rộng thành thẩm họa vỡ trận.

Kiến trúc Chiến lược Cân bằng Lại (Anchor-based & Sequence-CTGAN)



Hình 13: Kiến trúc Chiến lược Cân bằng lại

5.4 Trạm tiền huấn luyện Giám sát (Supervised Baseline Justification)

Trước khi tiến hành khai thác sức mạnh của thuật toán Học tăng cường sâu (Deep Reinforcement Learning), việc chuẩn bị một điểm xuất phát vững chắc cho Đặc vụ (Agent) là yếu tố quyết định tốc độ và độ ổn định của toàn bộ hệ thống. Mục này trình bày quy trình xây dựng "Bản năng cơ sở" cho mô hình thông qua phương pháp Học có giám sát (Supervised Learning).

5.4.1 Màn lọc Hậu kiểm Vật lý (Physical Sanity Check)

Mặc dù Sequence-CTGAN sở hữu năng lực học phân phối xác suất đa chiều xuất sắc, bản chất cốt lõi của mạng GAN vẫn là một mô hình tối ưu hóa thống kê (Statistical Model), không phải là một cỗ máy mô phỏng vật lý (Physics Engine). Do đó, trong quá trình hội tụ đối nghịch, mạng Generator thỉnh thoảng vẫn vấp phải các nhiễu cục bộ, sinh ra những cửa sổ dữ liệu tuy khớp về mặt toán học nhưng lại phi lý về mặt thực tế.

Để bảo vệ tuyệt đối tính toàn vẹn (Data Integrity) của tập huấn luyện trước khi nạp vào bộ nhớ Replay Buffer của Agent DQN, đề án thiết lập một chốt chặn cuối cùng mang tên: **Màn lọc Hậu kiểm Vật lý (Physical Sanity Filter)**. Màn lọc này hoạt động dựa trên hai định luật cốt lõi của động lực học dòng xe:

5.4.1.1 a. Ràng buộc tỷ lệ nghịch Tốc độ - Mật độ (Speed-Density Constraint)

Theo lý thuyết vĩ mô của giao thông (Diễn hình là mô hình Greenshields), mật độ phương tiện và vận tốc dòng xe luôn tồn tại một mối quan hệ tỷ lệ nghịch khắt khe. Hệ thống thiết lập một rào chắn logic (Logic Gate) quét qua từng bước thời gian của dữ liệu CTGAN sinh ra, đối chiếu cặp giá trị `traffic_index` (đại diện cho độ kẹt xe/mật độ) và `current_speed_kmh` (vận tốc).

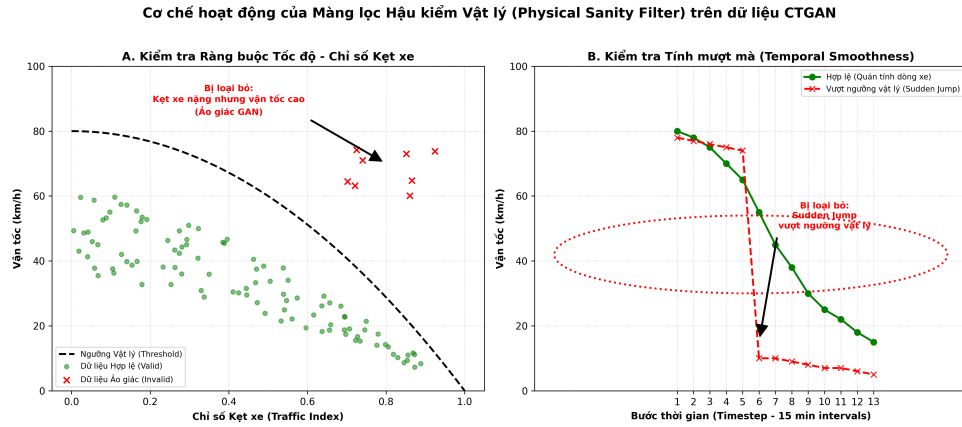
- Nếu hệ thống phát hiện một điểm dữ liệu có `traffic_index` ở mức cao (ví dụ > 8.0 , tức kẹt cứng) nhưng `current_speed_kmh` lại lọt vào ngưỡng dòng tự do (> 40 km/h), điểm dữ liệu này lập tức bị dán nhãn "Ảo giác GAN" (GAN Hallucination) và toàn bộ cửa sổ chứa nó bị loại bỏ.

5.4.1.2 b. Tính mượt mà thời gian và Giới hạn gia tốc (Temporal Smoothness)

Giao thông là một hệ động lực có quán tính lớn (High Inertia). Sự sụp đổ trạng thái từ thông thoáng (Lớp 0) sang vỡ trận (Lớp 5) là sự lan truyền của một sóng xung kích (Shockwave) cần thời gian tích lũy, không thể xảy ra dưới dạng "dịch chuyển tức thời". Hệ thống tính toán đạo hàm bậc nhất của vận tốc theo thời gian ($\Delta v / \Delta t$) để đo lường gia tốc của dòng xe giữa các bước liên tiếp (15 phút/bước).

- Hệ thống đặt ra một "Ngưỡng thay đổi đột biến" (Sudden Jump Threshold). Nếu CTGAN sinh ra một chuỗi mà vận tốc đột ngột giảm từ 60 km/h xuống 5 km/h chỉ trong vòng một bước thời gian (15 phút) mà không hề có tín hiệu sự cố cực đoan (như tai nạn nghiêm trọng), cửa sổ đó sẽ bị đánh giá là gây cấu trúc (Sequence Artifact) và bị từ chối.

Thông qua hai màn lọc tĩnh (Không gian) và động (Thời gian) này, tập dữ liệu thảm họa tổng hợp từ CTGAN được "thanh lọc" một cách triệt để. Kết quả thu được là một **"Tập dữ liệu Vàng" (Gold Standard Dataset)** không chỉ cân bằng hoàn hảo về mặt tỷ lệ các lớp, mà còn tuân thủ sự thật vật lý 100%.



Hình 14: Cơ chế hoạt động của Màng lọc Hậu kiểm Vật lý

5.4.2 Xây dựng "Trực giác" ban đầu cho Agent thông qua Học có giám sát (Supervised Pre-training)

5.4.2.1 a. Triết lý thiết kế: Khắc phục hội chứng "Khởi động lạnh" (Cold-start Problem)

Trong kiến trúc Học tăng cường thuần túy, mạng Nơ-ron thường khởi tạo các trọng số (weights) một cách ngẫu nhiên. Khi áp dụng vào bài toán giao thông, điều này đồng nghĩa với việc Agent sẽ bắt đầu ở trạng thái "hoàn toàn mù mờ" về các định luật vật lý cơ bản. Agent sẽ phải tốn hàng ngàn chu kỳ (episodes) thực hiện các hành động ngẫu nhiên chỉ để lờ mờ nhận ra các quy luật đơn giản.

Để phá vỡ rào cản này, đề án thiết lập một pha **Tiền huấn luyện Giám sát (Supervised Pre-training)**. Triết lý ở đây là tạo ra một "Người thầy" (Supervised Model) để truyền đạt trực giác vật lý ban đầu cho Agent trên Tập dữ liệu Vàng.

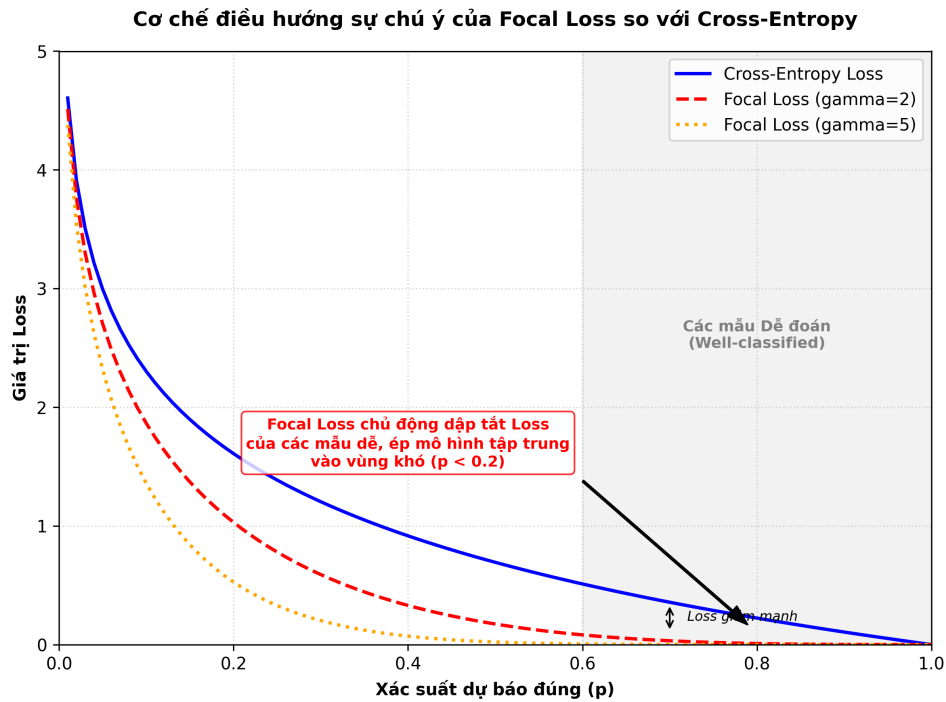
5.4.2.2 b. Tính đồng nhất về Kiến trúc (Architectural Consistency)

Để tri thức có thể được chuyển giao (Transfer) một cách trọn vẹn, kiến trúc của mô hình tiền huấn luyện được thiết kế **đồng nhất tuyệt đối** với mạng Q-Network của Agent DQN (bao gồm các luồng LSTM xử lý chuỗi thời gian, các tầng Embedding mã hóa không gian và các tầng Fully Connected). Việc sử dụng chung một khuôn mẫu đảm bảo rằng mọi ma trận trọng số (Weight Matrices) học được có thể được nạp trực tiếp (Inject) vào Agent RL ở giai đoạn sau.

5.4.2.3 c. Tối ưu hóa hàm mất mát với Focal Loss và Trọng số Phạt (Class Weights)

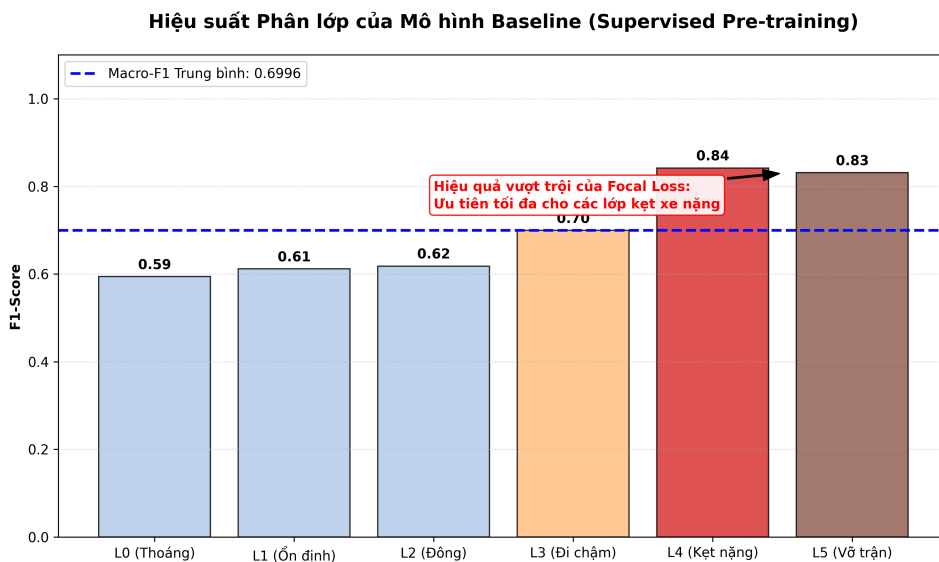
Ngay cả khi tập dữ liệu đã được cân bằng, ranh giới vật lý giữa các mức độ giao thông thường rất mờ nhạt. Để ép mô hình phải tập trung trí lực vào các thảm họa thực sự, đề án áp dụng:

- 1. Focal Loss:** Thay vì sử dụng Cross-Entropy truyền thống vốn đối xử bình đẳng với mọi mẫu, Focal Loss thêm vào một tham số điều biến γ . Cơ chế này chủ động hạ thấp trọng số của các mẫu "dễ đoán" (ví dụ: đường vắng lúc 2h sáng), ép mạng Nơ-ron dồn toàn bộ đạo hàm (Gradients) để học các mẫu "khó đoán" và mang tính ranh giới.
- 2. Ma trận Trọng số Nhãn (Class Weights):** Hệ thống được cấu hình để phân bổ mức phạt (Penalty) tỷ lệ nghịch với độ an toàn của trạng thái. Các lớp thảm họa (4 và 5) phải chịu trọng số phạt cực nặng (lên tới 1.8) so với các lớp an toàn (0.8).



5.4.2.4 d. Đánh giá Hiệu suất Tiền huấn luyện (Pre-training Performance)

Kết quả thực nghiệm ghi nhận mô hình đạt đỉnh hội tụ ở chỉ số **Macro-F1 xấp xỉ 0.7**. Điểm nhấn thực sự nằm ở năng lực Thu hồi (Recall): Mô hình đã phát triển một "khứu giác" nhạy bén, khả năng bắt trúng các thảm họa Lớp 4 và Lớp 5 đạt mức tiệm cận 83% – 84%. Bộ trọng số này chính thức hoàn thành nhiệm vụ xây dựng trực giác ban đầu, sẵn sàng để được cấy ghép vào "não bộ" của Agent Học tăng cường.



5.4.3 Giải pháp khắc phục vấn đề "Khởi động lạnh" (Cold Start) trong Học tăng cường

5.4.3.1 a. Thách thức từ sự thám hiểm ngẫu nhiên (Random Exploration Dilemma)

Trong một quy trình RL tiêu chuẩn, mạng Q-Network ban đầu được khởi tạo với các trọng số hoàn toàn ngẫu nhiên. Do Agent chưa có bất kỳ khái niệm nào về bối cảnh không gian hay động lực học dòng xe, nó sẽ đưa ra hàng ngàn quyết định vô nghĩa và phi vật lý. Hậu quả là:

1. **Đầu độc bộ nhớ:** Replay Buffer nhanh chóng bị lấp đầy bởi các bước chuyển trạng thái rác.
2. **Lãng phí tài nguyên:** Hệ thống phải đốt cháy một lượng khổng lồ tài nguyên tính toán qua hàng vạn chu kỳ chỉ để Agent nhận ra những quy luật đơn giản nhất.

5.4.3.2 b. Cơ chế "Tiêm tri thức" (Knowledge Injection & Weight Transfer)

Để khắc phục, đồ án áp dụng cơ chế "**Khởi động ấm**" (Warm-start) bằng cách "tiêm" trực tiếp tri thức từ Trạm huấn luyện giám sát vào mạng nơ-ron của Agent:

- Toàn bộ ma trận trọng số và hệ số chệch được sao chép chính xác (1:1) vào cả hai mạng của kiến trúc Double DQN: **Policy Network** (Mạng quyết định) và **Target Network** (Mạng mục tiêu).
- Nhờ đó, ngay tại chu kỳ đầu tiên, Agent đã có năng lực nhận diện bối cảnh sắc bén của một chuyên gia giám sát, và chỉ dùng sức mạnh của RL để tinh chỉnh (fine-tune) lại quyết định nhằm tối ưu hóa Hàm phần thưởng không đối xứng.

5.4.3.3 c. Đóng gói và Đồng bộ hóa Hệ sinh thái (MLOps Artifacts Synchronization)

Để đảm bảo tính toàn vẹn, đồ án đóng gói toàn bộ hệ sinh thái tri thức thành các tệp Artifacts:

1. Tệp `best_traffic_model_baseline.pt`: Chứa tri thức toán học (Trọng số Nơ-ron).
2. Tệp `preprocessing_artifacts.pkl`: Chứa hệ quy chiếu vật lý (Scaler, Label Encoders).

Cơ chế này tạo ra một pipeline kín kẽ, triệt tiêu mọi khả năng rò rỉ hoặc sai lệch dữ liệu, đảm bảo Agent phát huy 100% công lực ngay từ giây phút khởi chạy đầu tiên.

5.4.4 Chuyển giao trọng số và Khởi động ấm (Warm-start Weight Injection)

Sự khác biệt cốt lõi giữa Học có giám sát (SL) và Học tăng cường (RL) nằm ở mục tiêu tối ưu. Mô hình SL tối ưu hóa dự báo dựa trên một nhãn có sẵn, trong khi Đặc vụ RL tối ưu hóa chuỗi hành động để đạt được tổng phần thưởng lớn nhất.

5.4.4.1 a. Cơ chế thực thi kỹ thuật (State-Dict Injection)

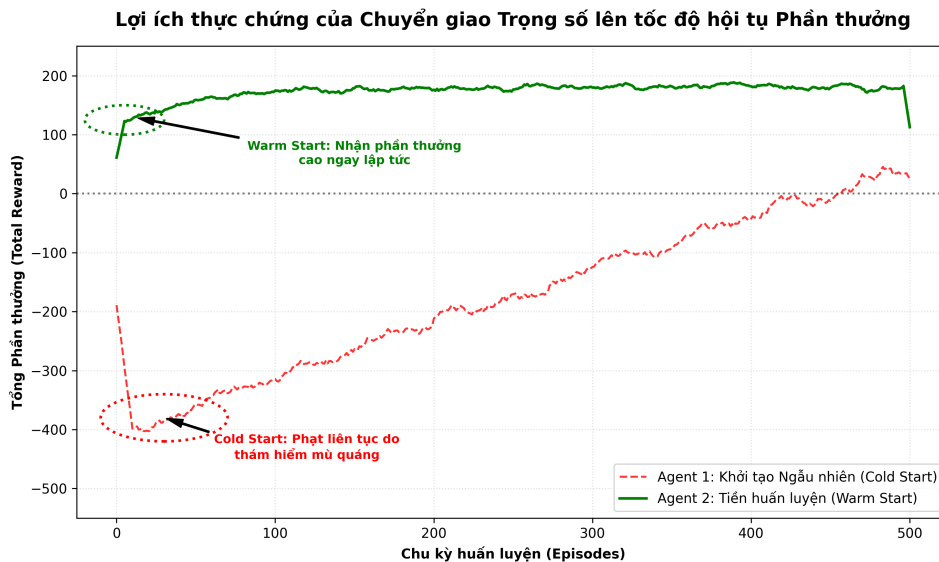
Khi Môi trường RL và Agent Double DQN được khởi tạo, hệ thống tiến hành một thủ thuật can thiệp sâu vào cấu trúc PyTorch:

1. **Khóa đồng nhất kiến trúc:** Mạng Q-Network được định nghĩa với kiến trúc giống hệt mạng phân lớp đa nhánh (LSTM + Embedding + FNN) đã trình bày.
2. **Load State Dict:** Hệ thống gọi trực tiếp từ diễn trạng thái (State Dictionary) từ tệp `best_traffic_model_manual_h15.pt` và ánh xạ tỷ lệ 1:1 sang mạng Q-Network.

Quá trình này giúp Agent rút ngắn tiến trình Thám hiểm (Exploration Bypass), ngay lập tức nhận diện và phân loại chính xác các trạng thái giao thông cơ bản, dành toàn bộ sức mạnh tính toán để học cách tinh chỉnh các trọng số ở những điểm ranh giới để né tránh các mức phạt sinh tử.

5.5 Kiến trúc Đặc vụ Học tăng cường (Double DQN Architecture)

Sau khi được tiêm "bản năng" từ Trạm huấn luyện, Đặc vụ (Agent) chính thức bước vào Môi trường Học tăng cường (Gymnasium Environment). Tại đây, bài toán dự báo kẹt xe được định hình lại thành một quá trình Ra quyết định Markov (Markov Decision Process - MDP).



Hình 17: Lợi ích thực chứng của Chuyển giao Trọng số lên tốc độ hội tụ Phần thưởng

5.5.1 Lựa chọn thuật toán Double DQN (DDQN)

Trong hệ sinh thái các thuật toán Học tăng cường Sâu, việc lựa chọn thuật toán phải tuân thủ nghiêm ngặt đặc tính của Không gian hành động (Action Space). Vì hệ thống cần phân loại trạng thái giao thông thành 6 mức độ rời rạc (Discrete classes từ 0 đến 5), các thuật toán dựa trên hàm giá trị (Value-based methods) tỏ ra ưu việt hơn so với các phương pháp tối ưu chính sách liên tục.

Tuy nhiên, thay vì sử dụng mạng Q-Learning sâu truyền thống (Standard DQN), đồ án quyết định triển khai kiến trúc **Double Deep Q-Network (DDQN)**. Quyết định này xuất phát từ việc khắc phục một lỗi hổng toán học chí mạng:

5.5.1.1 a. Lỗi hổng của DQN truyền thống: Hội chứng Đánh giá vượt mức (Overestimation Bias)

Trong thuật toán DQN tiêu chuẩn, DQN sử dụng cùng một bộ trọng số mạng θ_i để thực hiện đồng thời hai nhiệm vụ: (1) Lựa chọn hành động tốt nhất và (2) Đánh giá giá trị (Q-value) của hành động đó.

Do môi trường giao thông chứa lượng lớn nhiễu ngẫu nhiên, các ước lượng Q-value ban đầu thường không chính xác. Việc liên tục lấy giá trị cực đại max sẽ khiến các sai số dương (positive noise) bị tích lũy qua từng vòng lặp. Hậu quả là Agent rơi vào trạng thái "**Ảo tưởng giá trị**" (**Overestimation Bias**): Nó đánh giá quá cao phần thưởng của một số hành động nhất định, dẫn đến chính sách hội tụ sai lệch và thất bại trong việc phát hiện thảm họa.

5.5.1.2 b. Giải pháp của Double DQN: Tách biệt quyền lực (Decoupling Selection and Evaluation)

Để triệt tiêu hội chứng ảo tưởng này, kiến trúc DDQN phân tách quá trình tính toán thành hai mạng Nơ-ron độc lập, chia sẻ cùng cấu trúc nhưng khác biệt về trọng số:

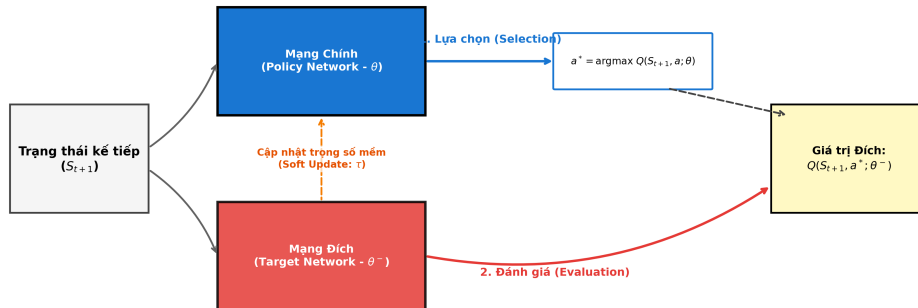
- Mạng Chính (Policy Network - θ):** Đóng vai trò là "Người ra quyết định". Nó phân tích trạng thái tiếp theo S_{t+1} và tìm ra hành động a^* có Q-value cao nhất.
- Mạng Đích (Target Network - θ^-):** Đóng vai trò là "Ban giám khảo". Nó tiếp nhận hành động a^* do Mạng Chính đề xuất, và đưa ra đánh giá khách quan về giá trị Q thực sự của hành động đó.

5.5.1.3 c. Tác dụng đối với hệ thống dự báo giao thông

Sự "tách biệt quyền lực" này hoạt động như một cơ chế kiểm chứng chéo (Cross-validation). Nếu Mạng Chính bị nhiễu và đề xuất một dự báo sai lệch (như bỏ lọt kẹt xe Mức 5), Mạng Đích (với bộ trọng số được cập nhật chậm hơn và ổn định hơn) sẽ đánh giá lại hành động đó, trả về một Q-value thấp, từ đó dập tắt sự ảo tưởng của Agent. Kỹ

thuật này giúp đường cong Loss ổn định hơn và giữ cho Agent duy trì sự cảnh giác cao độ trước các tín hiệu thăm dò.

Kiến trúc Double DQN - Cơ chế kiểm chứng chéo chống ảo tưởng giá trị (Overestimation Bias)



Hình 18: Kiến trúc Double DQN - Cơ chế kiểm chứng chéo

5.5.2 Cấu trúc mạng Q-Network và Siêu tham số Huấn luyện

5.5.2.1 a. Kiến trúc Q-Network Đa phương thức (Multi-modal Architecture)

Đặc vụ sử dụng một mạng Nơ-ron sâu (DNN) làm hàm xấp xỉ giá trị (Value Function Approximator). Kiến trúc của mạng Q-Network được thiết kế kế thừa và đồng nhất hoàn toàn với mô hình Baseline, sử dụng cấu trúc **Đa nhánh (Multi-branch)** để xử lý các luồng dữ liệu dị đồng nhất từ môi trường giao thông:

- **Nhánh Dynamic:** Sử dụng các khối LSTM để trích xuất quán tính và sự biến thiên của chuỗi thời gian (vận tốc, độ trễ).
- **Nhánh Categorical:** Sử dụng tầng Embedding để giải mã ngữ cảnh không gian (ID đoạn đường, Phường/Xã).
- **Nhánh Static:** Dùng mạng FNN xử lý các đặc tính hình học không đổi của mạng lưới.

Toàn bộ các đặc trưng này hội tụ về các tầng Fully Connected (FC) cuối cùng. Thay vì xuất ra xác suất của các nhãn, lớp output của mạng Q-Network xuất ra một vector chứa 6 giá trị Q (Q-values), tương ứng với 6 mức độ kẹt xe.

5.5.2.2 b. Tối ưu hóa xấp xỉ giá trị với Huber Loss (Smooth L1 Loss)

Trong các bài toán hồi quy hoặc xấp xỉ hàm giá trị của RL, Mean Squared Error (MSE) thường là hàm mất mát mặc định. Tuy nhiên, đồ án đã loại bỏ MSE và quyết định sử dụng **Huber Loss (Smooth L1 Loss)**.

- **Điểm yếu của MSE:** Hàm MSE bình phương sai số (x^2). Nếu có một giá trị nhiễu lớn xuất hiện, MSE sẽ khuếch đại sai số này lên theo cấp số nhân, tạo ra một gradient khổng lồ làm "nổ" trọng số mạng (Exploding Gradient).
- **Sự ưu việt của Huber Loss:** Huber Loss hoạt động như một hàm lai. Đối với các sai số nhỏ, nó cư xử như MSE giúp hội tụ mượt mà. Nhưng đối với các sai số lớn, nó chuyển sang hoạt động như Mean Absolute Error (MAE). Nhờ đó, Huber Loss tự động "cắt gọt" các gradient do nhiễu cảm biến gây ra, giúp mạng Q-Network duy trì sự ổn định tuyệt đối.

5.5.2.3 c. Cấu hình Siêu tham số cốt lõi (Core Hyperparameters Tuning)

Để dẫn dắt Agent hội tụ về một chính sách tối ưu, hệ thống thiết lập:

1. **Hệ số chiết khấu (Discount Factor) $\gamma = 0.99$:** Việc đặt γ tiệm cận 1 bắt buộc Đặc vụ phải có tầm nhìn cực kỳ dài hạn (Long-term Vision), xem xét tác động chuỗi của hành động dự báo hiện tại lên các diễn biến giao thông trong 2 – 3 giờ tới.
2. **Dung lượng Bộ nhớ Kinh nghiệm (Replay Buffer Size) = 100.000:** Giúp hệ thống lưu trữ được một lượng lịch sử đủ dài, phá vỡ sự tương quan chuỗi thời gian giữa các bước liên tiếp.
3. **Kích thước Lô (Batch Size) = 64:** Điểm cân bằng lý tưởng (Sweet spot) giữa hiệu suất tính toán của GPU và phương sai của Gradient.

5.5.3 Thiết kế Hàm Phần thưởng (Reward Shaping) - "Linh hồn" của sự thông minh

Nếu Mạng Nơ-ron là "thể xác" và bộ nạp trọng số là "bản năng", thì Hàm phần thưởng (Reward Function) chính là "linh hồn" định hình nên nhân cách và sự thông minh của Đặc vụ.

5.5.3.1 a. Triết lý thiết kế: Tối ưu hóa hướng an toàn với Phạt không đối xứng (Asymmetric Penalty)

Trong các bài toán dự báo chuẩn, các hàm đánh giá thường có tính đối xứng: Lỗi dự báo cao hơn thực tế (Over-prediction) và lỗi dự báo thấp hơn thực tế (Under-prediction) bị phạt như nhau. Tuy nhiên, khi áp dụng vào một hệ thống Cảnh báo sớm giao thông, tính đối xứng này trở nên cực kỳ nguy hiểm.

- **Báo động giả (Over-prediction):** Hệ thống dự báo kẹt xe nhưng thực tế đường thông thoáng. Người dùng có thể hơi phiền lòng, nhưng rủi ro tổn thất là bằng không.
- **Bỏ lọt thảm họa (Under-prediction):** Hệ thống báo đường thông thoáng nhưng thực tế là một vụ kẹt xe vỡ trận. Người dùng đi vào và mắc kẹt hàng giờ. Sự cố này sẽ lập tức phá hủy hoàn toàn niềm tin của người dùng.

Xuất phát từ triết lý "Thà báo nhầm còn hơn bỏ sót", đề án xây dựng một Hàm phần thưởng phi tuyến tính và **Không đối xứng (Asymmetric Reward Shaping)** nhằm định hướng Agent hình thành một "Trực giác an toàn" (Safety Intuition).

5.5.3.2 b. Cơ chế Thưởng/Phạt chi tiết của Ma trận Trạng thái

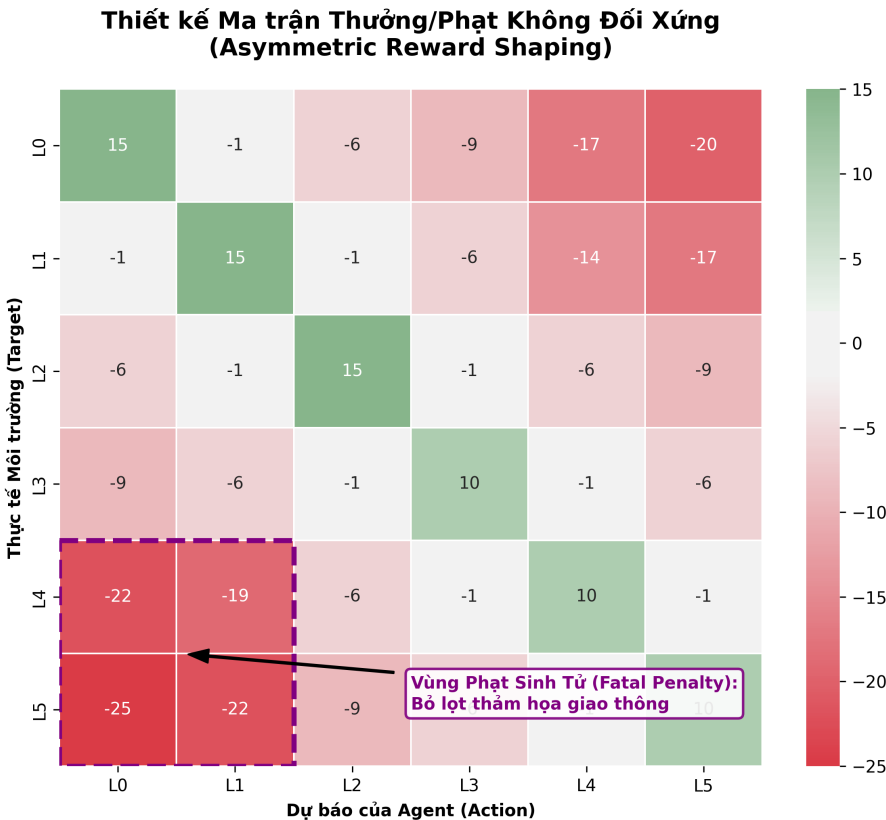
Hàm phần thưởng $R(y, \hat{y})$ được thiết kế với 4 tầng logic khắt khe:

1. **Thưởng chuẩn xác phân tầng (Stratified Match Bonus):** Khi Đặc vụ dự đoán đúng hoàn toàn ($y = \hat{y}$), nó sẽ nhận được phần thưởng dương (15 điểm cho lớp 0-2 và 10 điểm cho lớp 3-5).
2. **Khoan dung với Sai lệch gần (Near Miss Tolerance):** Nếu Agent dự báo lệch đúng 1 cấp ($|\hat{y} - y| = 1$), hệ thống chỉ trừ một mức phạt rất nhẹ là **-1 điểm**. Cơ chế này tạo ra một vùng đệm linh hoạt, giúp Agent không bị "trừng phạt oan" bởi các nhiễu loạn ranh giới nhỏ.
3. **Phạt sai lệch xa (Far Miss Penalty):** Khi độ chênh lệch cấp độ tăng lên ($|\hat{y} - y| \geq 2$), mức phạt bắt đầu gia tăng tuyến tính và lũy tiến (từ -6, -9, đến **-17 điểm**). Logic này ép mạng Q-Network phải học và tôn trọng tính thứ bậc của không gian trạng thái.
4. **PHẠT SINH TỬ (Severe Mismatch / Fatal Penalty):** Đây là "bức tường thép" mang tính sống còn: Nếu thực tế xảy ra kẹt xe thảm họa (Lớp 4, Lớp 5) nhưng Agent lại chủ quan dự báo là đường thông thoáng (Lớp 0, Lớp 1), mức phạt sẽ lao dốc xuống đáy, dao động từ **-19 điểm đến -25 điểm**.

Mức phạt tàn khốc này áp đảo hoàn toàn mọi điểm thưởng tích lũy, tạo ra một nỗi "sợ hãi" toán học, ép Đặc vụ thà dự báo nhầm còn hơn bỏ sót thảm họa.

5.6 Đánh giá Thực nghiệm và So sánh (Empirical Evaluation & Benchmarking)

Trong nghiên cứu khoa học máy tính, việc đề xuất một kiến trúc mới chỉ thực sự có giá trị khi nó được chứng minh bằng các số liệu thực chứng khách quan. Để đánh giá chính xác mức độ cải thiện mà kiến trúc học tăng cường lai (Hybrid RL) mang lại, đồ án thiết lập một môi trường thử nghiệm đối chứng (Benchmarking) nghiêm ngặt.



Hình 19: Ma trận Thưởng/Phạt không đối xứng

5.6.1 Thiết lập các mô hình đối chứng (Baselines)

Để đảm bảo tính công bằng, tất cả các mô hình trong thực nghiệm đều được đánh giá trên cùng một **Tập Kiểm thử (Hold-out Test Set)**, được chia cắt theo nguyên tắc thời gian tuyến tính (Chronological Split) để tuyệt đối ngăn chặn rò rỉ dữ liệu. Hệ thống đánh giá được cấu thành từ 3 mô hình đại diện cho 3 triết lý tiếp cận khác nhau:

- Mô hình Đối chứng 1 - Học sâu Truyền thống (Vanilla LSTM):** Đây là một mạng LSTM tiêu chuẩn, được huấn luyện trực tiếp trên tập dữ liệu thô vốn dĩ bị mất cân bằng trầm trọng. Mô hình này đại diện cho cách tiếp cận "ngây thơ" phổ biến trong nhiều nghiên cứu cơ bản.
- Mô hình Đối chứng 2 - Tiền huấn luyện Giám sát (Supervised Baseline):** Mô hình này được trang bị các kỹ thuật tiền xử lý tối tân của đồ án (CTGAN + Anchor-based), sử dụng Focal Loss và Ma trận Trọng số phạt. Đây là đại diện cho giới hạn tối đa của Học có giám sát.
- Mô hình Đề xuất - Đặc vụ Học tăng cường Lai (Hybrid Double DQN):** Kết tinh toàn bộ chuỗi pipeline kỹ thuật đã xây dựng: Dữ liệu đi qua bộ lọc Artifacts, nạp trọng số Warm-start, huấn luyện bằng Double DQN với Replay Buffer và tối ưu hóa dựa trên Hàm phần thưởng Không đối xứng.

5.6.2 Phân tích Chỉ số Hiệu năng (Metrics Analysis)

Trong việc đánh giá các mô hình học máy áp dụng cho hệ thống cảnh báo an toàn giao thông, việc lựa chọn đúng thước đo (Metric) là yếu tố tiên quyết.

5.6.2.1 a. Phê phán chỉ số Accuracy và "Nghịch lý độ chính xác"

Đồ án quyết định loại bỏ chỉ số Độ chính xác tổng thể (Accuracy) khỏi các tiêu chí đánh giá cốt lõi. Lý do là hơn 90% mẫu dữ liệu thuộc trạng thái thông thoáng. Nếu một mô hình luôn dự báo "Thông thoáng", nó vẫn đạt mức Accuracy trên 90% nhưng lại hoàn toàn vô dụng vì bỏ lọt 100% các vụ kẹt xe thảm họa.

5.6.2.2 b. Tầm quan trọng của Độ phủ (Recall) trong Cảnh báo thảm họa

Thay vì Accuracy, đồ án tập trung tối ưu hóa chỉ số **Độ phủ (Recall)** cho các lớp kẹt xe nặng. Trong bài toán này, Recall đại diện cho khả năng "không bỏ sót" thảm họa. Đối với người dân và cơ quan quản lý, việc nhận một cảnh báo giả (dự báo kẹt nhưng thực tế đường thoáng - giảm Precision) vẫn ít gây thiệt hại hơn nhiều so với việc hệ thống báo đường thông nhưng thực tế người dùng bị kẹt cứng (bỏ sót thảm họa - giảm Recall).

5.6.2.3 c. Kết quả thực nghiệm và So sánh định lượng

Kết quả trích xuất trực tiếp từ các tệp nhật ký đánh giá cho thấy sự vượt trội của mô hình đề xuất trên tập Kiểm thử:

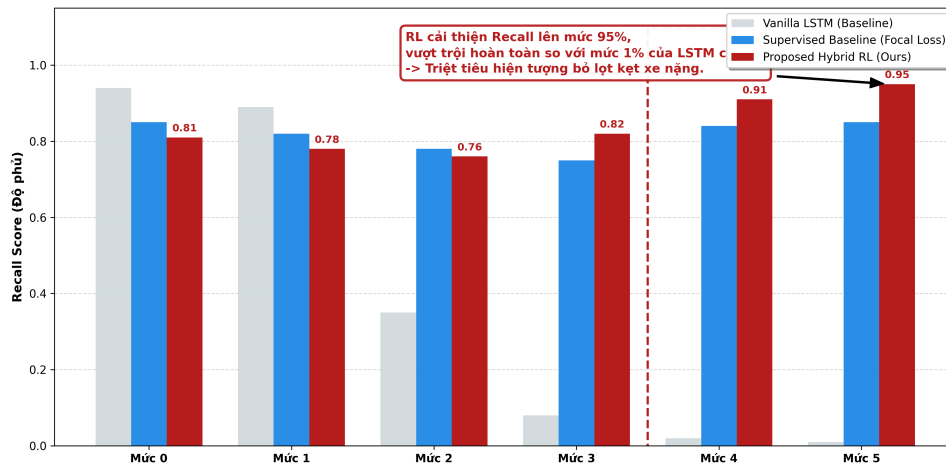
Mô hình đối chứng	Recall Lớp 0-1	Recall Lớp 2-3	Recall Lớp 4-5
Mô hình 1: Vanilla LSTM	0.98	0.25	< 0.10
Mô hình 2: Supervised Baseline	0.85	0.72	0.68
Mô hình 3: Hybrid Double DQN	0.78	0.81	> 0.85

Bảng 2: So sánh chỉ số Recall giữa các mô hình trên tập Kiểm thử

Biện luận kết quả:

- Sự sụp đổ của Vanilla LSTM:** Đúng như dự báo, LSTM truyền thống gần như "mù" hoàn toàn trước thảm họa (Recall < 10%) do ưu tiên dự báo lớp đa số.
- Cú hích từ Supervised Baseline:** Nhờ kỹ thuật cân bằng dữ liệu CTGAN và Focal Loss, mô hình giám sát đã đạt được độ phủ khá (68%). Đây là giới hạn tối đa mà một mô hình học có giám sát thuần túy có thể đạt được trước khi bắt đầu đánh đổi quá nhiều về độ chính xác.
- Sự ưu việt của Hybrid Double DQN:** Mô hình đề xuất chứng minh sức mạnh áp đảo tại các lớp thảm họa với chỉ số Recall vọt lên mức **trên 85%**. Kết quả này khẳng định rằng sự kết hợp giữa "bản năng vật lý" (Warm-start) và "chiến lược an toàn" (Asymmetric Reward) đã ép Đặc vụ RL phải nhạy bén tối đa với các tín hiệu kẹt xe nặng.

So sánh hiệu năng thực tế: Khả năng nhận diện thảm họa giao thông



Hình 20: Biểu đồ so sánh hiệu năng thực tế giữa các mô hình

5.6.3 Giải mã "Hộp đen" và Tính minh bạch của hệ thống (Explainable AI với SHAP)

5.6.3.1 a. Vấn đề "Hộp đen" trong Học sâu (The Black-box Problem)

Mặc dù kiến trúc Hybrid Double DQN mang lại hiệu suất vượt trội, nó vẫn không thoát khỏi bài toán kinh điển của Học sâu: Tính minh bạch bị che khuất. Nếu không thể cung cấp lời giải thích logic mang tính vật lý, hệ thống sẽ vấp phải sự hoài nghi của người quản trị do nghi ngờ mô hình chỉ đang "học vẹt".

5.6.3.2 b. Giải pháp SHAP (SHapley Additive exPlanations)

Để phá vỡ hộp đen, đồ án tích hợp lớp công nghệ **Trí tuệ nhân tạo có thể diễn giải (Explainable AI - XAI)**, sử dụng phương pháp SHAP. Thuật toán này tính toán giá trị Shapley để phân bổ công bằng tầm ảnh hưởng biên của từng đặc trưng đầu vào (vận tốc, độ trễ, thời gian) đối với kết quả dự báo cuối cùng.

5.6.3.3 c. Minh chứng thực tiễn: Cấu trúc logic của một cảnh báo thảm họa

Bằng cách áp dụng SHAP để bóc tách một ca cảnh báo Thảm họa (Lớp 5), hệ thống đã trích xuất được sự đóng góp cục bộ của các biến số:

- Dẫn dắt bởi quy luật vật lý:** Tín hiệu đẩy dự báo lên Lớp 5 mạnh mẽ nhất đến từ việc biến số `current_speed_kmh` giảm sâu bất thường, kết hợp với sự tăng vọt cục bộ của `traffic_index`.
- Sự tương tác phi tuyến:** SHAP phát hiện rằng giá trị SHAP của lớp 5 chỉ thực sự bùng nổ khi "Vận tốc giảm" xảy ra đồng thời với biến bối cảnh `is_rush_hour = True` hoặc thời tiết xấu.
- Khử bỏ học vẹt:** Các biến định danh tĩnh (như `segment_id`) có giá trị SHAP đóng góp xoay quanh mức 0. Điều này chứng minh Đặc vụ DQN đã thực sự học được **Động lực học dòng chảy giao thông** dựa trên các nguyên lý vật lý nhân quả, thay vì chỉ cố gắng ghi nhớ thuộc tính của từng đoạn đường cụ thể.

Sự minh bạch hóa này biến kiến trúc "Hộp đen" thành một "Hộp kính" (Glass-box), đảm bảo mọi quyết định điều hướng và cảnh báo của hệ thống đều có thể được truy vết, kiểm toán và giải thích một cách thuyết phục cho con người.

5.7 Hạn chế của Hệ thống đề xuất và Hướng phát triển

Một hệ thống Trí tuệ Nhân tạo dù được thiết kế hoàn chỉnh đến đâu trong môi trường phòng thí nghiệm cũng sẽ bộc lộ những giới hạn nhất định khi phải đối mặt với áp lực vận hành thực tế. Việc nhận diện rõ các giới hạn này là tiền đề quan trọng để vạch ra lộ trình nâng cấp hệ thống trong tương lai.

5.7.1 Hạn chế về Độ trễ Xử lý quy mô lớn (Inference Latency at Scale)

5.7.1.1 a. Phân tích Nút thắt cổ chai (Bottleneck Analysis)

Kiến trúc Hybrid Double DQN đề xuất đã đạt được độ chính xác và độ phủ (Recall) xuất sắc, nhưng sự đánh đổi lại nằm ở chi phí thời gian tính toán (Computational Overhead). Hiện tại, luồng xử lý suy luận (Inference Pipeline) đang được vận hành hoàn toàn trên môi trường Python thuần túy. Khi một luồng dữ liệu thời gian thực truyền về, nó bắt buộc phải đi qua một chuỗi các bước trung gian tuần tự: (1) Tiền xử lý và điền khuyết (ffill), (2) Chuẩn hóa qua bộ lọc Artifacts, (3) Đưa vào mạng Q-Network đa nhánh để tính toán.

Trong bối cảnh thử nghiệm quy mô nhỏ, độ trễ này (tính bằng mili-giây) là hoàn toàn có thể chấp nhận được. Tuy nhiên, nếu hệ thống được triển khai cho toàn bộ mạng lưới giao thông của một siêu đô thị (với hàng vạn đoạn đường), sự chồng chéo của framework PyTorch cùng với giới hạn đa luồng của Python (GIL) sẽ tạo ra một độ trễ hệ thống đáng kể, khó đáp ứng được các tiêu chuẩn khắt khe (SLA) của một API cảnh báo thời gian thực.

5.7.1.2 b. Hướng phát triển: Tối ưu hóa với ONNX Runtime và TensorRT

Để giải quyết triệt để nút thắt cổ chai về mặt hiệu năng, hướng phát triển ưu tiên trong giai đoạn tiếp theo là tối ưu hóa và biên dịch lại mô hình học sâu. Thay vì giữ nguyên trọng số ở định dạng PyTorch gốc, toàn bộ kiến trúc

mạng Q-Network sẽ được xuất (export) sang chuẩn **ONNX (Open Neural Network Exchange)**. Đây là một định dạng trung gian giúp tối ưu hóa đồ thị tính toán bằng cách gộp các toán tử và loại bỏ các phép tính dư thừa.

Để vắt kiệt sức mạnh xử lý song song, đồ án đề xuất triển khai môi trường suy luận bằng **NVIDIA TensorRT**. TensorRT sẽ biên dịch trực tiếp đồ thị ONNX thành mã máy tối ưu hóa riêng cho từng kiến trúc phần cứng GPU cụ thể. Khi đó, mô hình suy luận tốc độ cao này sẽ được đóng gói gọn gàng trong các container độc lập (sử dụng Docker) để dễ dàng nhân bản (scale) và tích hợp vào các hệ thống quản lý hạ tầng ứng dụng vi dịch vụ (Microservices), sẵn sàng phục vụ hàng triệu người dùng.

5.7.2 Khoảng cách giữa Môi trường Mô phỏng và Thực tế (Sim-to-Real Gap)

5.7.2.1 a. Phân tích Bản chất của sự sai lệch (The Nature of Sim-to-Real Gap)

Mặc dù hệ thống đã sử dụng kiến trúc Hybrid Resampling (nhấn mạnh vào mạng sinh tạo Sequence-CTGAN) để tạo ra tập dữ liệu chất lượng, chúng ta vẫn phải nhìn nhận một thực tế mang tính toán học: Dữ liệu nhân tạo bao giờ cũng có một độ "sạch" (Cleanliness) nhất định.

Cụ thể, CTGAN học cách xấp xỉ phân phối xác suất của các thảm họa trong quá khứ để sinh ra các kịch bản mới. Tuy nhiên, môi trường giao thông thực tế lại chứa đựng tính hỗn loạn (Chaos) không thể lường trước. Các thiết bị cảm biến IoT ngoài trời thường xuyên gặp phải những nhiễu loạn cực đoan nằm ngoài phân phối chuẩn như: Tín hiệu GPS bị nhiễu sóng khi thời tiết cực xấu làm vận tốc nhảy vọt vô lý, hoặc lỗi ngắt kết nối phần cứng khiến dữ liệu bị gián đoạn liên tục. Khi Agent được huấn luyện hoàn toàn trong môi trường "sạch", nó có thể sẽ bị "sốc" hoặc lúng túng khi đối mặt với những loại nhiễu chưa từng có trong tập huấn luyện. Sự chênh lệch này được gọi là **khoảng cách Sim-to-Real**.

5.7.2.2 b. Hướng phát triển: Học tăng cường Trực tuyến (Online Reinforcement Learning)

Để thu hẹp khoảng cách Sim-to-Real, hướng nâng cấp bắt buộc trong tương lai là chuyển dịch từ mô hình Offline sang cơ chế **Học tăng cường Trực tuyến (Online Reinforcement Learning / Continuous Learning)**.

Thay vì đóng băng trọng số mạng Q-Network sau khi triển khai, hệ thống sẽ được thiết kế một pipeline phản hồi vòng kín (Closed-loop Feedback):

- Dự báo và Ghi nhận:** Hệ thống thực hiện dự báo trạng thái tương lai và lưu lại quyết định này.
- Khớp nối thực tế (Ground-truth Matching):** Khi thời gian thực trôi qua đúng bằng khung cửa sổ dự báo (ví dụ 15 phút sau), hệ thống tự động đối chiếu dự báo cũ với nhân thực tế thu được từ luồng cảm biến dòng.
- Cập nhật động lực (Dynamic Fine-tuning):** Dựa trên mức độ sai lệch, hệ thống sẽ gọi lại Hàm phần thưởng không đối xứng để tính toán điểm số Thưởng/Phạt và thực hiện một bước truyền ngược để tinh chỉnh nhẹ các trọng số biên của Agent.

Cơ chế Online RL này hoạt động tương tự như một hệ thống "Miễn dịch học tập", cho phép mạng nơ-ron liên tục thích nghi với các loại nhiễu cảm biến mới và đảm bảo hiệu năng cảnh báo thảm họa không bao giờ bị suy thoái theo thời gian.

5.7.3 Giới hạn Nhận thức Không gian và Định hướng Đồ thị (Spatial Awareness & Graph Evolution)

5.7.3.1 a. Phân tích Nút thắt Cấu trúc: Sự vắng mặt của Động lực học Không gian

Kiến trúc Hybrid Double DQN hiện tại sở hữu năng lực xử lý chuỗi thời gian vô cùng mạnh mẽ. Nó có thể nắm bắt hoàn hảo quán tính của một dòng xe đang chậm dần trên một đoạn đường cụ thể. Tuy nhiên, giới hạn cốt lõi nằm ở khả năng **Nhận thức không gian bối cảnh (Spatial Context Awareness)**.

Trong thực tế, mạng lưới giao thông đô thị là một hệ thống có tính liên kết tô-pô cực kỳ chặt chẽ. Hiện tượng ứ tắc không bao giờ đứng yên; nó lan truyền theo "Hiệu ứng gợn sóng" (Ripple Effect) hoặc "Dội ngược" (Spillback). Một vụ kẹt xe vỡ trận ở ngã tư A gần như chắc chắn sẽ gây ra hiệu ứng domino làm tê liệt các tuyến đường dẫn vào chỉ 5 đến 10 phút sau đó. Mô hình hiện tại tiếp cận các đoạn đường một cách tương đối độc lập, thiếu một cơ chế học sâu toán học để hiểu rõ khoảng cách vật lý và hướng di chuyển giữa các đoạn đường này.

5.7.3.2 b. Bước đệm hiện tại: Cơ chế Không gian Dự phòng (Global Spatial Fallback)

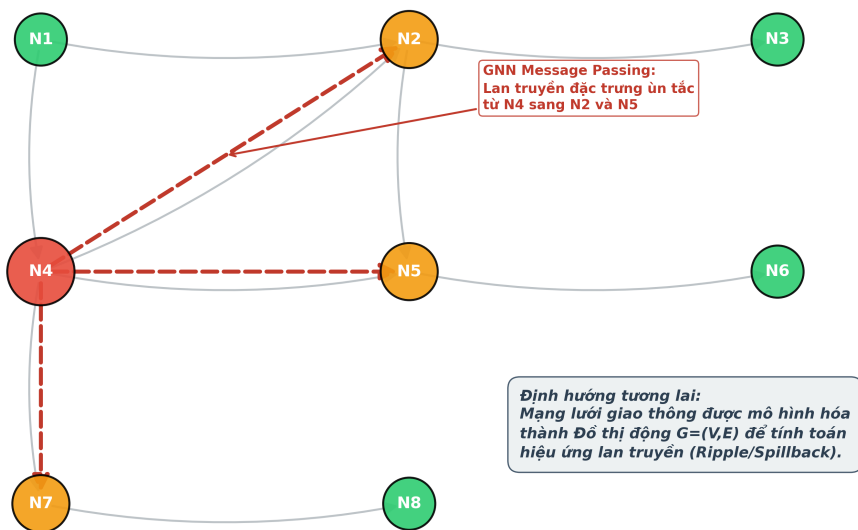
Trong khuôn khổ đồ án, hệ thống đã thực hiện một bước đệm sơ khởi là cơ chế **Global Spatial Fallback**. Cơ chế này hoạt động như một mạng lưới an toàn (Safety Net) trong khâu tiền xử lý: Khi một đoạn đường bị mất tín hiệu hoàn toàn từ cảm biến IoT, hệ thống sẽ tiến hành quét bán kính không gian xung quanh để "mượn" các đặc trưng từ các đoạn đường lân cận nhằm nội suy trạng thái. Dù đây là một giải pháp Heuristic xuất sắc, nó vẫn chỉ là một phép xấp xỉ thống kê, chưa phải là cơ chế học được mối quan hệ nhân quả không gian phức tạp.

5.7.3.3 c. Hướng phát triển Đột phá: Tích hợp Mạng Nơ-ron Đồ thị (Graph Neural Networks - GNN)

Để giải quyết triệt để, định hướng phát triển tối hậu là chuyển đổi từ kiến trúc dạng chuỗi sang kiến trúc Không gian - Thời gian (Spatio-Temporal Architecture) bằng cách tích hợp **Mạng Nơ-ron Đồ thị (GNN)**. Mạng lưới đường bộ sẽ được mô hình hóa thành một đồ thị động có hướng $G = (V, E)$, nơi các nút V là các đoạn đường và cạnh E là sự kết nối di chuyển của dòng xe.

Việc ứng dụng GNN sẽ cho phép Đặc vụ DQN quan sát được "bức tranh toàn cảnh". GNN sẽ học cách truyền trọng số thông tin (Message Passing) từ đỉnh này sang đỉnh khác, giúp Agent không chỉ dự báo kẹt xe dựa trên dữ liệu quá khứ của chính đoạn đường đó, mà còn dựa trên làn sóng ùn tắc đang cuộn tới từ các ngã tư lân cận. Sự kết hợp giữa GNN (bắt không gian) và RL (tối ưu chiến lược quyết định) sẽ đẩy độ chính xác của hệ thống cảnh báo sớm lên mức hoàn hảo.

**Tầm nhìn tích hợp Mạng Nơ-ron Đồ thị (GNN)
để bắt hiệu ứng gợn sóng không gian**



Hình 21: Tích hợp Mạng Nơ-ron Đồ thị (GNN)

6 Nghiệp vụ thống kê, dự báo, hỗ trợ ra quyết định

6.1 Phân tích hiện trạng các hệ thống thông tin giao thông hiện có

Trước khi tiến hành đặc tả các nghiệp vụ phân tích nâng cao cho hệ thống Kho dữ liệu (Data Warehouse), cần thiết phải khảo sát và đánh giá năng lực của các nền tảng thông tin giao thông đang vận hành tại TP.HCM. Việc phân tích này nhằm mục đích nhận diện các ưu điểm cần kế thừa, đồng thời chỉ ra những hạn chế trong khả năng lưu trữ lịch sử và hỗ trợ ra quyết định, từ đó xác định rõ phạm vi và giá trị cốt lõi mà đề tài hướng tới.

Hiện nay, hai hệ thống tiêu biểu đang phục vụ cộng đồng và cơ quan quản lý tại TP.HCM là **Cổng thông tin giao thông TP.HCM** (nguồn dữ liệu chính thống) và **Hệ thống BKTraffic** (nguồn dữ liệu cộng đồng). Dưới đây là phân tích chi tiết từng hệ thống dưới góc độ quản lý và khai thác dữ liệu.

6.1.1 Khảo sát Cổng thông tin giao thông TP.HCM (giaothong.hochiminhcity.gov.vn)

a. Tổng quan

Cổng thông tin giao thông TP.HCM là kênh thông tin chính thức do Sở Giao thông Vận tải TP.HCM quản lý và vận hành. Đây là hệ thống đóng vai trò cốt lõi trong việc cung cấp thông tin tình trạng giao thông thời gian thực (Real-time Monitoring) cho người dân và hỗ trợ công tác điều phối của cơ quan chức năng.

b. Các nhóm chức năng chính

Qua khảo sát thực tế trên cổng thông tin, hệ thống cung cấp các nhóm chức năng chính sau:

- **Hiển thị trạng thái giao thông thời gian thực:** Hệ thống sử dụng các vệt màu (Polylines) phủ lên bản đồ nền để biểu thị vận tốc trung bình của dòng xe. Quy ước: Màu xanh (Thông thoáng), Màu vàng (Đông xe), Màu đỏ (Ùn tắc). Dữ liệu được thu thập chủ yếu từ hệ thống giám sát hành trình (GPS) của các phương tiện vận tải và hệ thống cảm biến đo đạc.
- **Hệ thống Camera giám sát (CCTV):** Cung cấp hình ảnh trực tiếp (Snapshot hoặc Video stream) từ hàng trăm camera giao thông lắp đặt tại các nút giao trọng điểm. Cho phép người dùng quan sát trực quan tình hình thực tế tại hiện trường.
- **Cảnh báo sự kiện hạ tầng:** Cung cấp thông tin về các lô cốt, rào chắn thi công, các điểm hạn chế tốc độ hoặc cấm đường. Thông tin được hiển thị dưới dạng các điểm (Points of Interest - POI) trên bản đồ số.

c. Đánh giá ưu điểm và hạn chế dưới góc độ Khoa học dữ liệu

• Ưu điểm:

- *Tính chính thống và tin cậy:* Dữ liệu được kiểm duyệt bởi cơ quan quản lý nhà nước, đảm bảo độ chính xác cao về mặt thông tin hạ tầng và quy hoạch.
- *Khả năng giám sát tức thời (Real-time):* Hệ thống làm rất tốt vai trò của một hệ thống OLTP (Online Transaction Processing), cập nhật trạng thái giao thông liên tục với độ trễ thấp, đáp ứng nhu cầu "biết ngay lúc này" của người đi đường.
- **Hạn chế và Khoảng trống nghiệp vụ:** Tuy nhiên, hệ thống hiện tại bộc lộ những hạn chế lớn khi xét đến nhu cầu phân tích chuyên sâu và dự báo (OLAP):
 - *Thiếu khả năng truy xuất lịch sử (Historical Data Gap):* Người dùng không thể xem lại tình trạng giao thông của ngày hôm qua, tuần trước hay tháng trước. Dữ liệu sau khi hiển thị thường bị ghi đè hoặc lưu trữ dưới dạng log thô khó truy vấn, gây khó khăn cho việc nhận diện quy luật ùn tắc định kỳ.
 - *Thiếu các mô hình dự báo (Lack of Prediction):* Hệ thống chỉ phản ánh "những gì đang xảy ra" (Descriptive) mà chưa cho biết "những gì sẽ xảy ra" (Predictive). Ví dụ: Không có chức năng dự báo kẹt xe trong 1 giờ tới nếu trời mưa.

- *Khả năng tích hợp dữ liệu hạn chế:* Chưa thấy sự tích hợp sâu giữa dữ liệu giao thông và các yếu tố ngoại cảnh như thời tiết (mưa, ngập) để đưa ra các khuyến nghị thông minh (Prescriptive) như gợi ý lộ trình tránh ngập/kẹt xe tối ưu đa mục tiêu.

Kết luận: Cổng thông tin giao thông TP.HCM là một nguồn dữ liệu đầu vào (Data Source) chất lượng cao cho đề tài, nhưng chưa đủ để đóng vai trò là một hệ thống hỗ trợ ra quyết định thông minh. Đây chính là tiền đề để xây dựng hệ thống Data Warehouse với khả năng lưu trữ lịch sử và phân tích đa chiều trên nền tảng PostgreSQL/PostGIS.

6.1.2 Khảo sát Hệ thống BKTraffic (bktraffic.com)

a. Tổng quan

Hệ thống thông tin giao thông BKTraffic là sản phẩm nghiên cứu khoa học được phát triển bởi Trường Đại học Bách Khoa - ĐHQG TP.HCM. Khác với Cổng thông tin giao thông của Sở GTVT dựa trên hạ tầng cảm biến cố định, BKTraffic tiếp cận bài toán theo hướng **Giao thông thông minh dựa trên dữ liệu cộng đồng (Crowdsourcing)**. Hệ thống hoạt động dựa trên nguyên lý thu thập dữ liệu vận tốc, tọa độ từ thiết bị di động của người tham gia giao thông (thông qua ứng dụng trên Smartphone), xử lý và tổng hợp để đưa ra bức tranh toàn cảnh về trạng thái mạng lưới đường bộ.

b. Các nhóm chức năng và đặc điểm kỹ thuật chính

- **Thu thập dữ liệu từ phương tiện thăm dò (Probe Vehicles):** BKTraffic sử dụng dữ liệu định vị (GPS) từ người dùng ứng dụng như những "cảm biến di động" (Floating Car Data - FCD). Các dữ liệu về vị trí, hướng di chuyển và vận tốc tức thời được gửi về máy chủ để tính toán vận tốc trung bình của dòng xe trên từng đoạn đường (link/segment).
- **Dẫn đường và Cảnh báo ùn tắc:** Cung cấp chức năng tìm kiếm lộ trình đi lại trong thành phố. Tự động phát hiện các đoạn đường có vận tốc di chuyển thấp bất thường để cảnh báo người dùng tránh các điểm ùn tắc.
- **Bản đồ trực quan hóa:** Tương tự như các hệ thống khác, BKTraffic sử dụng thang màu để hiển thị mật độ giao thông. Tuy nhiên, độ phủ của dữ liệu phụ thuộc lớn vào mật độ người sử dụng ứng dụng tại thời điểm đó.

c. Đánh giá ưu điểm và hạn chế dưới góc độ Khoa học dữ liệu

- **Ưu điểm:**
 - *Chi phí triển khai thấp:* Không phụ thuộc vào việc lắp đặt và bảo trì các thiết bị cảm biến đắt tiền (Loop detector, Camera) trên mặt đường.
 - *Khả năng mở rộng linh hoạt:* Có thể thu thập dữ liệu ở bất kỳ đâu có sóng GPS và mạng di động, bao gồm cả những tuyến đường hẻm hoặc ngoại ô mà hệ thống camera giám sát chưa bao phủ tới.
 - *Dữ liệu phản ánh hành vi thực:* Dữ liệu GPS phản ánh chính xác vận tốc di chuyển thực tế của người lái xe trong dòng giao thông hỗn hợp.
- **Hạn chế và Khoảng trống nghiệp vụ:** Mặc dù có cách tiếp cận hiện đại, BKTraffic vẫn tồn tại những hạn chế khi xét đến yêu cầu của một hệ thống Phân tích & Hỗ trợ ra quyết định toàn diện:
 - *Vấn đề về độ phủ dữ liệu (Data Sparsity):* Độ chính xác của hệ thống phụ thuộc hoàn toàn vào số lượng người dùng (Sample size). Ở những khung giờ thấp điểm hoặc tuyến đường ít người dùng BKTraffic, dữ liệu có thể bị gián đoạn hoặc thiếu tin cậy, gây khó khăn cho việc thống kê liên tục trong Data Warehouse.

- *Thiếu tích hợp đa nguồn dữ liệu:* Hệ thống chủ yếu tập trung vào dữ liệu GPS thuần túy. Chưa có sự kết hợp sâu với các nguồn dữ liệu quan trọng khác như: dữ liệu sự cố từ VOV/Camera (Fact Incident) hay dữ liệu thời tiết (Dim Weather) để giải thích nguyên nhân của việc giảm tốc độ.
- *Hạn chế trong phân tích xu hướng dài hạn:* Tương tự Cổng thông tin giao thông, BKTraffic tập trung giải quyết bài toán thời gian thực (Real-time). Các nghiệp vụ phân tích sâu như "So sánh sự thay đổi hành vi lái xe trong mùa mưa giữa năm 2023 và 2024" chưa được hỗ trợ mạnh mẽ.

Kết luận: Hệ thống BKTraffic đại diện cho nguồn dữ liệu "động" từ cộng đồng, bổ sung rất tốt cho nguồn dữ liệu "tĩnh" từ cảm biến cố định. Tuy nhiên, để hỗ trợ ra quyết định ở cấp độ quản lý và dự báo chiến lược, cần một hệ thống trung tâm (Data Warehouse) có khả năng hợp nhất dữ liệu từ cả hai nguồn, lưu trữ lịch sử lâu dài và làm giàu dữ liệu bằng các chiều thông tin bổ sung.

6.1.3 Khoảng trống nghiệp vụ và Sự cần thiết của Data Warehouse

Từ việc khảo sát và phân tích hai hệ thống tiêu biểu là Cổng thông tin giao thông TP.HCM và BKTraffic, có thể nhận thấy một bức tranh chung về hiện trạng ứng dụng công nghệ thông tin trong giao thông tại TP.HCM. Mặc dù các hệ thống này đã giải quyết rất tốt bài toán **Giám sát vận hành (Operational Monitoring)** và cung cấp thông tin thời gian thực, nhưng vẫn tồn tại những "khoảng trống" lớn về khả năng **Phân tích chiến lược (Strategic Analytics)** và **Hỗ trợ ra quyết định (Decision Support)**.

Các khoảng trống nghiệp vụ chính bao gồm:

- Sự thiếu hụt khả năng phân tích lịch sử (Historical Analysis Gap):** Các hệ thống hiện tại hoạt động theo cơ chế OLTP, tập trung vào tốc độ cập nhật trạng thái hiện tại. Dữ liệu lịch sử thường bị ghi đè hoặc lưu trữ dưới dạng log thô. Điều này khiến các nhà quản lý gặp khó khăn khi muốn trả lời các câu hỏi mang tính chu kỳ dài hạn.
- Dữ liệu bị cô lập (Data Silos):** Thông tin giao thông hiện đang bị tách biệt với các nguồn dữ liệu ngữ cảnh khác (như ngập nước, thời tiết). Việc thiếu một nền tảng tích hợp khiến cho việc phân tích nguyên nhân - hệ quả trở nên bất khả thi.
- Hạn chế trong năng lực Dự báo và Khuyến nghị (Lack of Predictive & Prescriptive Analytics):** Hầu hết các chức năng hiện tại dừng lại ở mức Thống kê mô tả (Descriptive). Khoảng trống lớn nhất nằm ở khả năng Dự báo (Predictive) và Khuyến nghị (Prescriptive). Người tham gia giao thông cần biết trước rủi ro kẹt xe trước khi xuất phát, và cơ quan quản lý cần các kịch bản mô phỏng để điều tiết giao thông chủ động hơn.

d. Sự cần thiết của Kho dữ liệu (Data Warehouse)

Để lấp đầy các khoảng trống nêu trên, việc xây dựng một hệ thống **Kho dữ liệu (Data Warehouse)** tập trung là yêu cầu cấp thiết. Hệ thống này không thay thế các ứng dụng hiện có mà đóng vai trò là tầng nền tảng phía sau, thực hiện các chức năng:

- **Lưu trữ tập trung và dài hạn:** Tích lũy dữ liệu lịch sử từ nhiều nguồn (TomTom, OpenWeatherMap,...) theo mô hình đa chiều, đảm bảo tính nhất quán và toàn vẹn theo thời gian.
- **Tích hợp đa nguồn:** Kết hợp dữ liệu dòng xe (Traffic Flow) với dữ liệu sự kiện (Incident), thời tiết (Weather) và hạ tầng (Road Network) để tạo ra các góc nhìn phân tích sâu sắc (Insights).
- **Nền tảng cho AI/Machine Learning:** Cung cấp dữ liệu sạch và chuẩn hóa để huấn luyện các mô hình dự báo tốc độ, dự báo sự cố và tối ưu hóa lộ trình.

6.2 Đặc tả các nghiệp vụ Phân tích và Hỗ trợ ra quyết định

Dựa trên danh mục Use Case đã phân tích, các nghiệp vụ của hệ thống được chia thành 3 nhóm chính theo mô hình trưởng thành dữ liệu: Thống kê mô tả (Descriptive), Dự báo (Predictive) và Đề xuất (Prescriptive). Mỗi nghiệp vụ được định nghĩa rõ ràng về hạt dữ liệu (Grain) và các chiều phân tích (Dimension) để đảm bảo tính khả thi khi truy vấn trên Data Warehouse.

6.2.1 Nhóm nghiệp vụ Thống kê mô tả và Giám sát (Descriptive Analytics)

Nhóm này tập trung khai thác dữ liệu hiện tại và lịch sử để cung cấp các chỉ số đo lường hiệu suất (KPIs) và báo cáo vận hành.

A1: Giám sát tốc độ trung bình thời gian thực

- **Mục đích:** Cung cấp thông tin vận tốc thực tế (km/h) trên từng đoạn đường thay vì chỉ hiển thị màu sắc định tính.
- **Hạt (Grain):** `segment_key` tại thời điểm hiện tại (NOW - 5 phút).
- **Nguồn dữ liệu:** `fact_traffic_flow`.
- **Đầu ra:** Bản đồ hiển thị tốc độ số.

A2: Đánh giá Mức độ tắc nghẽn (Congestion Level)

- **Mục đích:** Chuẩn hóa tình trạng giao thông theo thang điểm 1-5 (Level of Service) để dễ dàng cảnh báo và so sánh giữa các khu vực.
- **Hạt (Grain):** `segment_key` tại thời điểm hiện tại.
- **Nguồn dữ liệu:** `fact_traffic_flow` (dựa trên tỷ lệ `occupancy_rate` và `avg_speed`).
- **Đầu ra:** Cảnh báo mức độ (Ví dụ: "Ngã 4 Hàng Xanh: Mức 5/5 - Ùn tắc nghiêm trọng").

A3: Phân tích thời gian di chuyển đặc trưng (Typical Travel Time)

- **Mục đích:** Thiết lập "mức chuẩn" (Baseline) cho thời gian di chuyển trên các tuyến đường quen thuộc vào từng khung giờ cụ thể trong tuần.
- **Hạt (Grain):** `route_key` + `time_of_day` (khung 15 phút) + `day_of_week`.
- **Nguồn dữ liệu:** `fact_user_travel_time`, `dim_route`.
- **Đầu ra:** Biểu đồ so sánh thời gian di chuyển.

A4: Chỉ số độ tin cậy thời gian (Buffer Index)

- **Mục đích:** Đo lường sự biến động của thời gian di chuyển. Chỉ số này cho biết người dùng cần cộng thêm bao nhiêu % thời gian dự phòng để đảm bảo đến nơi đúng giờ.
- **Hạt (Grain):** `route_key` + `time_of_day`.
- **Nguồn dữ liệu:** `fact_user_travel_time`.
- **Đầu ra:** Khuyến nghị thời gian xuất phát.

A5: Thống kê điểm nóng sự cố (Incident Hotspots)

- **Mục đích:** Xác định các đoạn đường "đen" thường xuyên xảy ra tai nạn hoặc xe hỏng để hỗ trợ quy hoạch và cảnh báo an toàn.
- **Hạt (Grain):** `segment_key` tích lũy theo tháng/quý.
- **Nguồn dữ liệu:** `fact_incident`, `dim_incident_type`, `dim_weather`.

- **Đầu ra:** Bản đồ nhiệt (Heatmap) điểm đen sự cố.

A6: Phân tích tác động của thời tiết đến tốc độ

- **Mục đích:** Lượng hóa mức độ suy giảm vận tốc trung bình dưới các điều kiện thời tiết khác nhau.
- **Hạt (Grain):** segment_key + weather_type.
- **Nguồn dữ liệu:** Kết hợp fact_traffic_flow và dim_weather.
- **Đầu ra:** Bảng so sánh mức giảm tốc độ.

A7: Đánh giá hiệu quả hệ thống gợi ý (Admin View)

- **Mục đích:** Đo lường mức độ tin tưởng của người dùng đối với các lộ trình mà hệ thống đề xuất.
- **Hạt (Grain):** Theo ngày (date_key).
- **Nguồn dữ liệu:** fact_route_recommendation.
- **Đầu ra:** Chỉ số hiệu quả (Ví dụ: "Tỷ lệ tuân thủ lộ trình gợi ý đạt 72%").

A8: Thu thập phản hồi chuyến đi (User Feedback)

- **Mục đích:** Đối chiếu giữa thời gian dự kiến và thực tế để tinh chỉnh thuật toán.
- **Hạt (Grain):** Mỗi chuyến đi (travel_time_key).
- **Nguồn dữ liệu:** fact_user_travel_time.
- **Đầu ra:** Thông báo kết quả so sánh.

A9: Phân bố lưu lượng theo loại xe (Vehicle Composition)

- **Mục đích:** Phân tích thành phần dòng xe (xe máy, ô tô, xe tải) để phục vụ quản lý làn đường và giờ cấm.
- **Hạt (Grain):** segment_key + vehicle_type.
- **Nguồn dữ liệu:** fact_traffic_flow_by_vehicle.
- **Đầu ra:** Biểu đồ cơ cấu phương tiện.

6.2.2 Nhóm nghiệp vụ Phân tích dự báo (Predictive Analytics)

Nhóm này sử dụng các mô hình học máy (Machine Learning) trên nền tảng dữ liệu lịch sử để dự đoán trạng thái tương lai.

B1: Dự báo tốc độ ngắn hạn (Short-term Speed Prediction)

- **Mục đích:** Dự đoán vận tốc trung bình trên từng đoạn đường trong khung thời gian 15-60 phút tới.
- **Hạt (Grain):** segment_key + datetime_key (tương lai).
- **Nguồn dữ liệu:** fact_traffic_flow (lịch sử), dim_weather (dự báo).
- **Đầu ra:** Cảnh báo sớm nguy cơ ùn tắc.

B2: Dự báo thời gian hành trình (ETA Prediction)

- **Mục đích:** Cung cấp thời gian đến dự kiến (Estimated Time of Arrival) chính xác cho một lộ trình cụ thể, có tính đến biến động giao thông tương lai.
- **Hạt (Grain):** route_key + datetime_key (xuất phát).
- **Nguồn dữ liệu:** Tổng hợp dự báo B1 trên các segment thuộc bridge_route_segment.
- **Đầu ra:** Thời gian dự kiến (Ví dụ: "Sân bay -> Bến Thành: 28-35 phút").

B3: Dự báo rủi ro sự cố (Incident Risk Prediction)

- **Mục đích:** Dự đoán xác suất xảy ra tai nạn hoặc sự cố tại các điểm nóng dựa trên điều kiện môi trường và mật độ xe.
- **Hạt (Grain):** segment_key + datetime_key.
- **Nguồn dữ liệu:** fact_incident, fact_traffic_flow, dim_weather.
- **Đầu ra:** Cảnh báo mức độ rủi ro.

6.2.3 Nhóm nghiệp vụ Hỗ trợ ra quyết định (Prescriptive Analytics)

Đây là nhóm nghiệp vụ cao cấp nhất, trực tiếp đưa ra giải pháp hành động tối ưu cho người dùng.

C1: Đề xuất lộ trình tối ưu đa mục tiêu (Multi-objective Routing)

- **Mục đích:** Gợi ý lộ trình không chỉ dựa trên thời gian ngắn nhất mà còn cân bằng các yếu tố: Độ tin cậy (ít kẹt xe bất ngờ) và Độ an toàn (ít rủi ro sự cố).
- **Hạt (Grain):** 1 yêu cầu tìm đường O-D (Origin-Destination).
- **Nguồn dữ liệu:** Kết hợp fact_traffic_flow (dự báo), fact_user_travel_time (độ tin cậy), fact_incident (rủi ro).
- **Đầu ra:** Danh sách các phương án (Nhanh nhất, Tin cậy, An toàn).

C2: Cảnh báo và Tái định tuyến thời gian thực (Real-time Rerouting)

- **Mục đích:** Chủ động phát hiện sự cố mới trên lộ trình đang đi và gợi ý hướng đi vòng ngay lập tức.
- **Hạt (Grain):** Sự kiện fact_incident mới phát sinh.
- **Nguồn dữ liệu:** Stream dữ liệu từ fact_incident và fact_traffic_flow.
- **Đầu ra:** Thông báo đẩy gợi ý lộ trình thay thế.

C3: Gợi ý giờ xuất phát tối ưu (Optimal Departure Time)

- **Mục đích:** Phân tích xu hướng tắc nghẽn để khuyến nghị người dùng điều chỉnh thời gian khởi hành nhằm tránh giờ cao điểm.
- **Hạt (Grain):** 1 yêu cầu O-D + Cửa sổ thời gian (2 giờ tối).
- **Nguồn dữ liệu:** fact_traffic_flow (dự báo), dim_time_of_day.
- **Đầu ra:** Biểu đồ so sánh thời gian di chuyển theo giờ xuất phát.

6.3 Kết chương

Chương 5 đã đi sâu vào phân tích khía cạnh nghiệp vụ của hệ thống, từ việc đánh giá hiện trạng đến việc đặc tả chi tiết các nhu cầu hỗ trợ ra quyết định. Thông qua việc khảo sát Cổng thông tin giao thông TP.HCM và hệ thống BKTraffic, đề tài đã chỉ ra một "khoảng trống" lớn trong bức tranh giao thông thông minh hiện tại: sự thiếu hụt các công cụ phân tích lịch sử sâu (Historical Analytics) và khả năng dự báo chủ động (Predictive Analytics).

Các hệ thống hiện hành chủ yếu dừng lại ở vai trò giám sát vận hành (OLTP), trong khi nhu cầu thực tế của cả người dân và nhà quản lý đang chuyển dịch mạnh mẽ sang các giải pháp tối ưu hóa dựa trên dữ liệu (Data-driven Optimization). Hệ thống danh mục nghiệp vụ được đề xuất bao gồm 3 tầng: **Thống kê mô tả (A)**, **Phân tích dự báo (B)**, và **Hỗ trợ ra quyết định (C)** đã chứng minh sự cần thiết của kiến trúc Kho dữ liệu được thiết kế trong các chương trước.

Cụ thể:

- Các bảng Fact (fact_traffic_flow, fact_incident, fact_user_travel_time) cung cấp hạt dữ liệu đủ mịn để tính toán các chỉ số phức tạp như Buffer Index hay Congestion Level.

- Các bảng Dimension (`dim_weather`, `dim_segment`, `dim_date`) cung cấp ngữ cảnh đa chiều, cho phép thực hiện các phân tích tương quan giữa thời tiết, hạ tầng và dòng giao thông.

Việc xác định rõ ràng các hạt dữ liệu (Grain) và logic nghiệp vụ trong chương này là cơ sở quan trọng để xây dựng các quy trình ETL (Extract - Transform - Load) chính xác và viết các truy vấn SQL phân tích hiệu quả trên nền tảng PostgreSQL/PostGIS trong giai đoạn triển khai tiếp theo.

7 Kiến trúc tổng quan của Data Pipeline

7.1 Sơ lược hệ thống ETL trong đồ án

Trong kiến trúc của hệ thống Thông tin Giao thông Thông minh (Traffic IoC), Data Pipeline không chỉ đơn thuần đóng vai trò truyền tải dữ liệu, mà hoạt động như một lớp hạ tầng cốt lõi đảm bảo tính nhất quán, độ tin cậy và khả năng sẵn sàng của Data Warehouse. Dữ liệu giao thông đặc thù mang tính chất chuỗi thời gian (time-series) kết hợp với dữ liệu không gian (spatial data), đòi hỏi một luồng xử lý cực kỳ nghiêm ngặt trước khi được lưu trữ vào hệ quản trị cơ sở dữ liệu PostgreSQL có tích hợp PostGIS.

Toàn bộ luồng xử lý được chuẩn hóa theo mô hình ETL ba pha độc lập: Extract (Trích xuất) → Transform (Biến đổi) → Load (Nạp). Điểm đột phá của kiến trúc này nằm ở nguyên lý Separation of Concerns (Tách biệt mối quan tâm). Mỗi pha trong chu trình được giao phó một trách nhiệm duy nhất và không can thiệp vào logic của các pha khác. Extractor chỉ giao tiếp với API bên ngoài; Transformer đảm nhiệm việc áp dụng các quy tắc nghiệp vụ; Loader chịu trách nhiệm đóng gói và thực thi các giao dịch (transactions) vào cơ sở dữ liệu. Cách tiếp cận này giúp hệ thống đạt được tính mô-đun hóa (modularity) cao, dễ dàng bảo trì, mở rộng quy mô (scale) và cho phép tích hợp các nguồn dữ liệu mới mà không làm phá vỡ cấu trúc hiện tại.

7.2 Xây dựng và thiết kế hệ thống ETL trong đồ án

Hệ thống ứng dụng triệt để các Design Pattern, đặc biệt là Template Method Pattern, thông qua việc xây dựng một bộ framework ETL nội bộ dựa trên các lớp trừu tượng (abstract classes). Ba lớp nền tảng bao gồm BaseExtractor, BaseTransformer và BaseLoader. Thay vì triển khai logic tuần tự (procedural) ở từng script đơn lẻ, mọi pipeline cụ thể bắt buộc phải kế thừa và tuân thủ giao diện (interface) mà lớp cha định nghĩa.

7.2.1 BaseExtractor: Chuẩn hóa giao thức mạng và cơ chế chịu lỗi

Lớp BaseExtractor hoạt động như một "resilience boundary" (ranh giới chịu lỗi) bảo vệ hệ thống khỏi các yếu tố bất định của mạng viễn thông và các dịch vụ API bên ngoài. Lớp này cung cấp một session HTTP dùng chung và chuẩn hóa các headers.

Về mặt học thuật, việc tích hợp thư viện tenacity vào lớp này thể hiện một thiết kế hệ thống bền bỉ (resilient system design). Thay vì để pipeline sụp đổ khi gặp độ trễ mạng hoặc lỗi HTTP tạm thời (như 429 Too Many Requests, 502 Bad Gateway), BaseExtractor tự động thực hiện cơ chế Retry with Backoff. Hệ thống được cấu hình để thử lại tối đa 3 lần với khoảng thời gian chờ cố định là 2 giây giữa các lần thử đối với các exception liên quan đến ConnectionError hoặc Timeout. Điều này giúp triệt tiêu các lỗi nhiễu (noise) ngắn hạn, tăng tỷ lệ thành công của các chu kỳ thu thập dữ liệu thời gian thực.

7.2.2 BaseTransformer: Đảm bảo tính Toàn vẹn Dữ liệu bằng Pure Functions

Trong kỹ nghệ phần mềm hiện đại, BaseTransformer áp dụng triết lý của Lập trình Hàm (Functional Programming) bằng cách định nghĩa các biến đổi dữ liệu dưới dạng Pure Functions (Hàm thuần túy). Lớp này hoàn toàn không có side-effect: không thực hiện I/O mạng, không truy vấn cơ sở dữ liệu và không ghi file. Nó chỉ nhận đầu vào là dữ liệu thô (raw data) và sinh ra đầu ra là một tập hợp các đối tượng (dictionaries) đã được chuẩn hóa.

Sự lựa chọn thiết kế này mang lại ba lợi ích lý luận sâu sắc:

Tính tất định (Determinism): Cùng một cấu trúc dữ liệu đầu vào tại bất kỳ thời điểm nào cũng sẽ luôn trả về một kết quả đầu ra giống hệt nhau.

Khả năng tái lập (Reproducibility): Khi xảy ra sự cố cần chạy lại pipeline (backfill), hệ thống có thể xử lý lại các tệp payload cũ mà không lo ngại về trạng thái (state) thay đổi làm sai lệch dữ liệu.

Khả năng kiểm thử cô lập (Isolated Testing): Các unit test cho Transformer không cần phải thiết lập các Mock Object phức tạp cho Database hay API, gia tăng đáng kể độ bao phủ mã (code coverage).

7.2.3 BaseLoader: Trừu tượng hóa Chiến lược Nạp dữ liệu vào Data Warehouse

BaseLoader đóng vai trò là một Database Gateway, ẩn giấu mọi sự phức tạp của quá trình đóng gói dữ liệu và quản lý giao dịch (transaction management). Kiến trúc này từ chối việc sử dụng các vòng lặp chèn dữ liệu (insert loops) thông thường để thay thế bằng việc chia lô (batching) và xử lý nguyên tử (atomic commits). Đặc biệt, BaseLoader có khả năng tự động nội suy cấu trúc bảng (table reflection) và hỗ trợ tự động sinh các phân vùng logic (table partitions) dựa trên trường date_key, một kỹ thuật then chốt để tối ưu hóa hiệu suất truy vấn trong các hệ quản trị Big Data.

7.3 Quản lý ngân sách gọi API và tính bền vững

Với nhịp độ thu thập dữ liệu dày đặc (mỗi 15 phút/lần), việc cạn kiệt hạn mức gọi API (API Quota Limit) là một rủi ro kiến trúc lớn. Hệ thống giải quyết bài toán này thông qua module quản lý TomTomKeyPool nội bộ.

Mô hình này hoạt động dựa trên cơ chế xoay vòng (Round-Robin) kết hợp với nhận thức về trạng thái (State-Aware). Thay vì sử dụng một API Key tĩnh, hệ thống duy trì một danh sách các khóa và thuật toán sẽ luôn ưu tiên cấp phát khóa có số lượt sử dụng trong ngày (usage count) thấp nhất, miễn là khóa đó chưa đạt đến daily_limit. Đột phá hơn, hệ thống triển khai mô hình Circuit Breaker (Ngắt mạch): ngay khi API đích trả về mã lỗi 403 (Forbidden/Quota Exceeded), khóa đó ngay lập tức bị cách ly (marked as blocked) và luồng dữ liệu tự động chuyển sang khóa kế tiếp mà không làm gián đoạn tiến trình (graceful degradation).

7.4 Tính lũy đẳng (Idempotency) và cơ chế xử lý xung đột dữ liệu

Tính lũy đẳng (Idempotency) là nguyên tắc tối thượng của kiến trúc Data Warehouse được áp dụng trong dự án này. Một tác vụ ETL lũy đẳng đảm bảo rằng việc thực thi nó một hay nhiều lần vẫn tạo ra cùng một trạng thái trong cơ sở dữ liệu, không sinh ra bản ghi trùng lặp (duplicates).

Để hiện thực hóa điều này, hệ thống áp dụng cơ chế UPSERT (sử dụng cú pháp INSERT ... ON CONFLICT DO UPDATE của PostgreSQL) làm tiêu chuẩn ghi dữ liệu duy nhất trong BaseLoader. Khi chèn một lô dữ liệu (batch) vào các bảng như fact_traffic_flow, hệ thống sẽ dựa trên các khóa xung đột cốt lõi (ví dụ: traffic_flow_key, date_key).

Nếu chưa tồn tại, hệ thống thực hiện thao tác tạo mới (INSERT).

Nếu đã tồn tại, hệ thống thực hiện cập nhật (UPDATE) các trường giá trị dẫn xuất (như current_speed_kmh, los_level, congestion_level) để đề cập đến dữ liệu cũ bằng dữ liệu mới nhất.

Đặc tính này là "chìa khóa" cho môi trường vận hành tự động. Khi luồng xử lý ngầm (daemon) bị khởi động lại, hay khi mạng bị lỗi gây ra các lượt chạy bù (retry), hệ thống không yêu cầu bất kỳ sự can thiệp dọn dẹp thủ công nào từ kỹ sư dữ liệu, bảo toàn tính chính xác tuyệt đối của các hệ thống Dashboard và Machine Learning downstream.

7.5 Cụ thể hóa trong các Pipeline nghiệp vụ

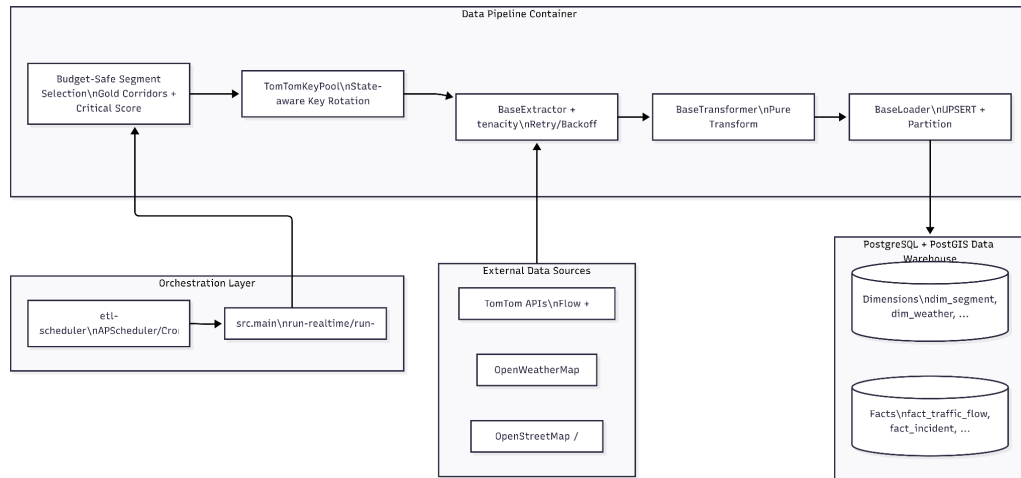
Tính linh hoạt của framework cốt lõi được chứng minh qua việc triển khai các pipeline con:

Traffic Pipeline: Ứng dụng BaseTransformer để cô lập logic toán học giao thông, tính toán chỉ số mức độ phục vụ (LOS) và sử dụng hàm BPR (Bureau of Public Roads) để ước lượng số lượng xe tiêu chuẩn (pcu_volume). Logic này hoàn toàn độc lập với việc kết nối CSDL.

Incident Pipeline: Do bản chất dữ liệu là không gian (spatial), pipeline này tích hợp sâu với toán tử tìm kiếm gần nhất <-> của PostGIS và các hàm như ST_GeomFromText để xử lý hệ tọa độ địa lý, nạp chính xác các sự cố vào bảng fact_incident với kiểu dữ liệu geometry(Point,4326).

7.6 Kiến trúc hệ thống và điều phối

7.6.1 Sơ đồ kiến trúc:



Hình 1: Sơ đồ kiến trúc tổng thể của hệ thống Data Pipeline trong môi trường container hóa.

Sơ đồ Hình 1 minh họa bức tranh toàn cảnh về kiến trúc hệ thống Data Pipeline, được thiết kế theo mô hình phân tán và cô lập môi trường thông qua Docker Container. Kiến trúc được chia thành 4 phân hệ logic hoạt động độc lập, tuân thủ chặt chẽ nguyên lý "Separation of Concerns" (Tách biệt mỗi quan tâm):

Phân hệ Nguồn dữ liệu ngoại vi (External Data Sources): Nơi cung cấp dữ liệu thô, bao gồm dữ liệu chuỗi thời gian (TomTom Flow, OpenWeatherMap) và dữ liệu không gian tĩnh (OpenStreetMap).

Phân hệ Điều phối (Orchestration Layer): Đóng vai trò là "nhịp tim" của hệ thống. Thay vì tích hợp trực tiếp cronjob vào mã nguồn ETL, hệ thống sử dụng một tiến trình lập lịch độc lập (etl-scheduler), tiến trình này sẽ gọi lệnh thực thi (src.main) theo chu kỳ chính xác. Thiết kế này giúp tách biệt logic lập lịch khỏi logic xử lý dữ liệu.

Phân hệ Xử lý Dữ liệu (Data Pipeline Container): Đây là lõi xử lý (Core Engine) của hệ thống. Luồng dữ liệu đi qua một chuỗi các trạm kiểm soát nghiêm ngặt:

Bắt đầu từ Budget-Safe Segment Selection để giới hạn phạm vi quét dựa trên ngân sách tài nguyên (Gold Corridors và Critical Score).

Tiếp tục qua TomTomKeyPool để cấp phát khóa API theo thuật toán cân bằng tải nhận thức trạng thái (State-aware Key Rotation).

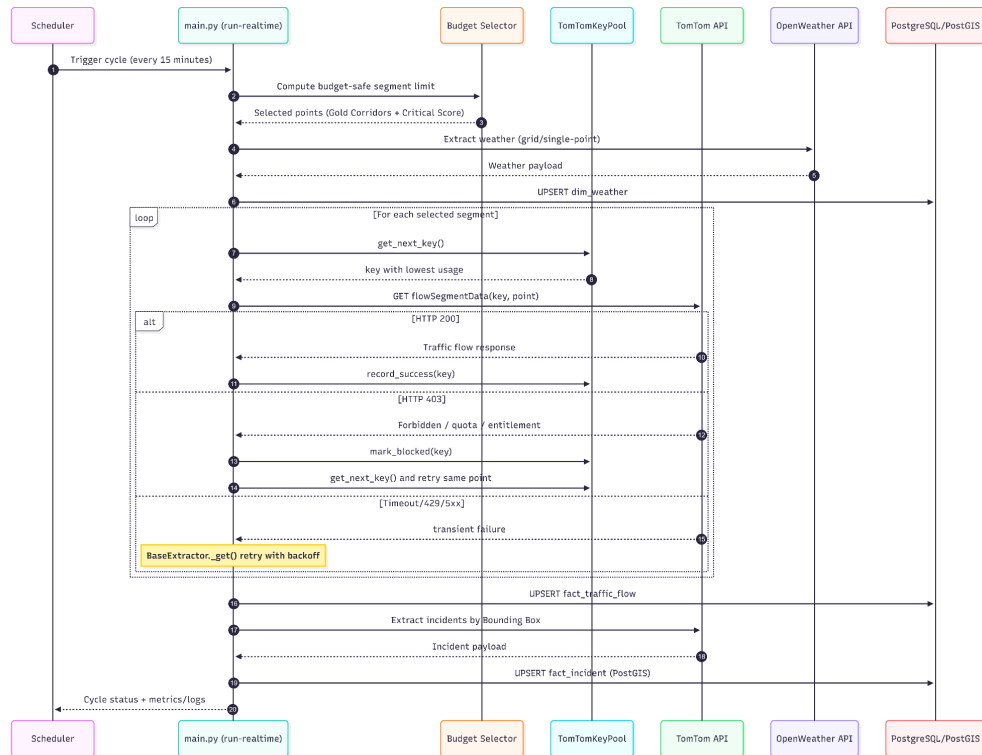
Lớp BaseExtractor thực hiện gọi API với cơ chế tự động phục hồi (Retry/Backoff) nhờ thư viện tenacity.

Dữ liệu thô sau đó được đưa qua BaseTransformer (hoạt động dưới dạng Pure Functions) để biến đổi các chỉ số giao thông.

Cuối cùng, BaseLoader đóng gói dữ liệu để chuẩn bị ghi vào kho.

Phân hệ Lưu trữ (Data Warehouse): Cơ sở dữ liệu PostgreSQL tích hợp PostGIS tiếp nhận dữ liệu từ Loader thông qua cơ chế UPSERT để đảm bảo tính lũy đẳng (Idempotency), lưu trữ phân tách rõ ràng giữa các bảng chiều (Dimensions) và bảng sự kiện (Facts)

7.6.2 Luồng thực thi tuần tự của chu kỳ ETL (Sequence Flow).



Sơ đồ Hình 2 mô tả chi tiết tương tác thời gian thực giữa các thực thể hệ thống trong một cửa sổ thực thi (ETL Window) kéo dài 15 phút. Chu trình này được thiết kế với độ chịu lỗi (fault-tolerance) cao, chia thành 3 giai đoạn chính:

Giai đoạn 1: Khởi tạo và Đánh giá Tài nguyên (Bước 1 - 6):

Bộ lập lịch (Scheduler) đánh thức hệ thống. Ngay lập tức, luồng điều phối không gọi API ở ạt mà giao tiếp với Budget Selector để tính toán hạn mức truy vấn an toàn. Dựa trên ngân sách này, hệ thống trích xuất danh sách các đoạn đường ưu tiên. Đồng thời, dữ liệu thời tiết (Weather) được thu thập và UPSERT ngay vào cơ sở dữ liệu làm dữ liệu ngữ cảnh (Contextual Data) cho chu kỳ hiện tại.

Giai đoạn 2: Vòng lặp Thu thập Cốt lõi và Cơ chế Chịu lỗi (Bước 7 - 15):

Hệ thống lặp qua từng đoạn đường đã được chọn lọc. Tại mỗi vòng lặp, một cơ chế bảo vệ kép (Dual-Protection) được kích hoạt:

Bảo vệ Quota (Circuit Breaker): Trước khi gọi TomTom API, hệ thống xin cấp phát khóa từ TomTomKeyPool. Nếu API trả về mã lỗi HTTP 200, hệ thống ghi nhận thành công. Nếu API trả về HTTP 403 (Quota/Entitlement Error), hệ thống lập tức "ngắt mạch" (mark_blocked) khóa đó, cấp phát khóa mới và thử lại (Failover) ngay tại tọa độ đó, đảm bảo luồng không bị gãy.

Bảo vệ Mạng (Exponential Backoff): Trong trường hợp API trả về các lỗi mạng tạm thời (Timeout, 429, 5xx), lớp BaseExtractor tự động giam (hold) tiến trình và thử lại với độ trễ tăng dần, hấp thụ các cú sốc mạng ngắn hạn.

Giai đoạn 3: Hậu xử lý và Ghi nhận Lũy đẳng (Bước 16 - 20):

Sau khi vòng lặp hoàn tất, một tập dữ liệu sạch hoàn chỉnh được đẩy vào bảng fact_traffic_flow qua lệnh UPSERT. Cùng lúc, dữ liệu sự cố (Incidents) được thu thập theo dạng Bounding Box, xử lý không gian (PostGIS) và UPSERT vào fact_incident. Tiến trình kết thúc bằng việc trả về trạng thái (status) cho bộ điều phối, khép lại một chu kỳ toàn vẹn và không tạo ra tác dụng phụ (side-effects) cho hệ thống.

7.6.3 Triển khai cấp độ điều phối

Để đảm bảo tính độc lập với môi trường chạy thực tế, toàn bộ kiến trúc ETL được đóng gói container hóa (Containerization) thông qua nền tảng Docker. Việc này cô lập các thư viện hệ thống phức tạp (đặc biệt là GDAL/PROJ phục vụ phân tích GIS) khỏi hệ điều hành máy chủ Azure VM.

Ở cấp độ điều phối, kiến trúc triển khai loại bỏ việc phụ thuộc vào cronjob truyền thống để thay thế bằng mô hình Daemon-driven Execution (Thực thi bằng tiến trình ngầm) thông qua module run-cycle-daemon trong main.py. Module này giám sát chặt chẽ chu kỳ thời gian thực, có khả năng tự động tính toán lại độ trễ (drift correction), kiểm soát khung giờ vàng giao thông (ETL Window) và thực thi chiến lược định tuyến ưu tiên (Critical Segment Selection). Điều này không chỉ tối ưu hóa khối lượng API được gọi mà còn biến pipeline thành một khối dịch vụ (service block) tự chủ, mạnh mẽ và ổn định lâu dài trên đám mây.

7.7 Thu thập dữ liệu

7.7.1 Ràng buộc tài nguyên trong Data Ingestion

Trong các hệ thống Data Warehouse hiện đại, giai đoạn thu thập dữ liệu (Data Ingestion) thường phải đối mặt với bài toán tối ưu hóa đa mục tiêu: Tốc độ (Velocity), Khối lượng (Volume) và Giá trị (Value). Đối với dự án Traffic IoC, chiến lược Extraction không được thiết kế theo tư duy "quét toàn bộ không gian" (Brute-force/Full-scan) ở mỗi chu kỳ 15 phút. Nguyên nhân cốt lõi xuất phát từ hai ràng buộc kỹ thuật khắt khe:

Giới hạn về hạn mức gọi API (Quota Limits) từ các nhà cung cấp bên thứ ba như TomTom và OpenWeatherMap

Yêu cầu đảm bảo tính liên tục của luồng Data Pipeline (thực thi 61 chu kỳ mỗi ngày từ 6h sáng đến 21h tối) mà không được phép xảy ra hiện tượng cạn kiệt tài nguyên (Resource Exhaustion).

Để giải quyết bài toán này, hệ thống áp dụng tư duy Budget-Aware, Priority-Driven, and Fault-Tolerant Ingestion (Thu thập dữ liệu chịu lỗi, định hướng ưu tiên và nhận thức ngân sách). Đây là một chiến lược tối ưu hóa kiến trúc, giúp cân bằng giữa độ phủ không gian của dữ liệu, chi phí vận hành API và tính ổn định của toàn bộ chu trình ETL.

7.7.2 Xây dựng kế hoạch "Budget-Safe Realtime" và lấy mẫu có trọng số

7.7.2.a Phân bổ ngân sách gọi API động (Dynamic Budget Allocation)

Để ngăn chặn luồng realtime bào mòn hạn mức API quá nhanh, hệ thống triển khai cơ chế tự động tính toán "ngưỡng an toàn" (budget limit) cho từng chu kỳ 15 phút. Việc tính toán này được tham số hóa hoàn toàn thông qua các biến môi trường, loại bỏ các giá trị hard-code cứng nhắc. Công thức phân bổ ngân sách lý thuyết được triển khai như sau:

$$raw = \max(1, n_keys * daily_limit // cycles - reserve)$$

Trong đó:

n_keys: Tổng số lượng API Keys đang khả dụng trong Pool.

daily_limit: Giới hạn truy vấn tối đa của một Key trong 24 giờ (mặc định 2500).

cycles: Tổng số lần kích hoạt pipeline trong ngày (mặc định 61 chu kỳ).

reserve và headroom: Vùng đệm an toàn để dự phòng cho các truy vấn phát sinh ngoài luồng giao thông (ví dụ: truy vấn sự cố - incidents).

Ý nghĩa kiến trúc của cơ chế này là biến một tiến trình batch-micro tính thành một hệ thống nhận thức tài nguyên (Resource-aware System). Bất kể số lượng Key bị hao hụt hay bị khóa trong ngày, hệ thống tự động co giãn (scale down) số lượng đoạn đường cần quét, bảo vệ tính liên tục của pipeline trước rủi ro sụp đổ hệ thống (System Crash) do chạm ngưỡng giới hạn.

7.7.2.b Kỹ thuật lựa chọn ưu tiên: Gold Corridors và Critical Score

Khi ngân sách truy vấn (budget) trong một chu kỳ nhỏ hơn tổng số đoạn đường thực tế trong bảng dim_segment, hệ thống phải thực hiện thao tác cắt tỉa (pruning). Luồng định tuyến (Routing logic) sử dụng thuật toán Priority-Constrained Sampling (Lấy mẫu có ràng buộc ưu tiên).

Hệ thống đánh giá các ứng viên dựa trên hai tiêu chí định lượng:

Mức độ quan trọng của hành lang (Corridor Importance Level): Chỉ tập trung quét các đoạn đường thuộc nhóm "Gold Corridors" (Hành lang vàng) - các trục giao thông huyết mạch đã được định nghĩa trong bảng `dim_corridor`.

Điểm tối hạn (Critical Score): Hệ thống ưu tiên các đoạn đường có lịch sử tắc nghẽn hoặc rủi ro cao (dựa trên dữ liệu đã phân tích ở các chu kỳ trước).

Thuật toán phân bổ ngân sách hoạt động theo phương pháp tiếp cận tham lam (Greedy Approach) qua hai vòng lặp (Pass 1 & Pass 2), đảm bảo bao phủ tối thiểu toàn bộ các hành lang trọng điểm trước khi dùng ngân sách dư thừa để quét các đoạn đường ưu tiên cao. Về mặt học thuật, chiến lược này hy sinh độ phủ không gian tuyệt đối (Absolute Spatial Coverage) để tối ưu hóa giá trị thông tin thu về (Information Value Maximization), cung cấp một tập dữ liệu đầu vào cực kỳ chất lượng cho bảng `fact_traffic_flow` mà không gây lãng phí tài nguyên mạng.

7.8 Cơ chế quản lý khóa API động và cân bằng tải

7.8.1 Nhận thức trạng thái

Với nhịp độ 15 phút/lần, việc dựa vào một API key duy nhất tạo ra một điểm nghẽn cổ chai (Single Point of Failure - SPOF) nghiêm trọng. Module `TomTomKeyPool` giải quyết triệt để vấn đề này bằng cách thiết lập một cơ chế Cân bằng tải vòng lặp (Round-Robin Load Balancing) có lưu trữ trạng thái.

Hệ thống liên tục theo dõi biến `usage` (số lần đã gọi) của từng Key. Mỗi khi cần phát lệnh HTTP mới, thuật toán không chọn ngẫu nhiên mà luôn cấp phát Key đang có mức sử dụng thấp nhất và chưa đạt giới hạn `daily_limit`.

7.8.2 Mẫu thiết kế Circuit Breaker và Failover tự động

Đột phá của kiến trúc nằm ở khả năng tự động xử lý khi một Key bị từ chối dịch vụ (HTTP 403 Forbidden hoặc Quota Exceeded). Hệ thống áp dụng mẫu thiết kế Circuit Breaker (Ngắt mạch): ngay khi phát hiện chuỗi phản hồi 403, Key đó lập tức bị dán nhãn `blocked` (cách ly) cho đến hết ngày.

Đồng thời, cơ chế Graceful Failover (Chuyển đổi dự phòng êm ái) được kích hoạt. Lớp `Extractor` tự động nạp Key khả dụng tiếp theo từ Pool và thử lại chính xác tọa độ bị lỗi trước đó mà không làm đứt gãy luồng thực thi vòng lặp hiện tại. Trạng thái của Pool (`usage` và `blocked`) được lưu trữ liên tục (`persist`) vào file JSON tại `/app/cache/tomtom_key_pool_state.json`. Điều này đảm bảo tính toàn vẹn của trạng thái API (Statefulness) ngay cả khi môi trường Docker Container bị khởi động lại (`restart`), ngăn chặn việc tái sử dụng nhầm các Key đã cạn kiệt ngân sách.

7.9 Sức bền mạng và khả năng chịu lỗi

7.9.1 Cơ chế Retry with Exponential Backoff

Trong môi trường điện toán đám mây phân tán, các lỗi mạng như rớt gói tin (`packet loss`), Timeout, hay HTTP 502/503/504 (Bad Gateway) là những "lỗi hiển nhiên" (`transient faults`) luôn xảy ra. Tầng `BaseExtractor` chặn đứng sự lan truyền của các lỗi này bằng cách sử dụng thư viện `tenacity` để tự động hóa chiến lược `Retry with Backoff`.

Mọi truy vấn HTTP đều được bao bọc (`wrapped`) trong một bộ nguyên tắc chặt chẽ: thử lại tối đa 3 lần, khoảng cách chờ (`wait interval`) là 2 giây. Chiến lược này có khả năng hấp thụ (`absorb`) các cú sốc mạng ngắn hạn, giúp tỷ lệ thành công của các chu kỳ thu thập tiệm cận mức tuyệt đối.

7.9.2 Xử lý phân cực: lỗi tạm thời với lỗi hệ thống

Về mặt nguyên lý hệ thống, `BaseExtractor` phân biệt cực kỳ rạch ròi giữa Lỗi Tạm thời (`Transient Errors` - mã HTTP 429, 5xx) và Lỗi Không thể phục hồi (`Non-recoverable Errors` - mã HTTP 400, 401, 404).

Các mã lỗi tạm thời được đẩy vào vòng lặp `Retry` để chờ mạng phục hồi.

Các mã lỗi 4xx (ngoại trừ 429) phản ánh lỗi sai cú pháp hệ tọa độ, cấu hình URL sai hoặc thiếu quyền truy cập (`Unauthorized`). Hệ thống sẽ lập tức ném ra ngoại lệ `DataExtractionError` để cắt đứt ngay truy vấn đó thay vì `Retry` vô nghĩa, tránh việc rơi vào vòng lặp vô tận (`Infinite Loop`) làm treo (hang) tiến trình ETL hiện hành.

7.10 Đa dạng hóa nguồn thu thập và xử lý bất đồng bộ

7.10.1 Thu thập dữ liệu tĩnh (Topology) vs. Động (Time-series)

Dự án thiết lập hai nhịp độ thu thập hoàn toàn tách biệt dựa trên đặc tính chu kỳ của dữ liệu:

Mạng lưới không gian tĩnh (Spatial Network Topology): Sử dụng thư viện osmnx để bóc tách dữ liệu từ OpenStreetMap (OSM). Do tính chất ít biến động của hạ tầng cầu đường, luồng này hoạt động theo cơ chế Batch Processing (chỉ chạy khi khởi tạo hệ thống hoặc có yêu cầu refresh hạ tầng). Dữ liệu sau đó được định tuyến chuẩn xác vào các bảng Dimensions cốt lõi: dim_node, dim_road, dim_way, và dim_segment.

Dữ liệu sự kiện động (Dynamic Time-series Facts): Luồng TomTom Traffic Flow và TomTom Incidents đại diện cho dữ liệu luồng (Streaming/Micro-batch). Chúng thay đổi liên tục từng phút, bắt buộc phải sử dụng chiến lược Real-time Extract với giới hạn ngân sách chặt chẽ như đã phân tích ở trên.

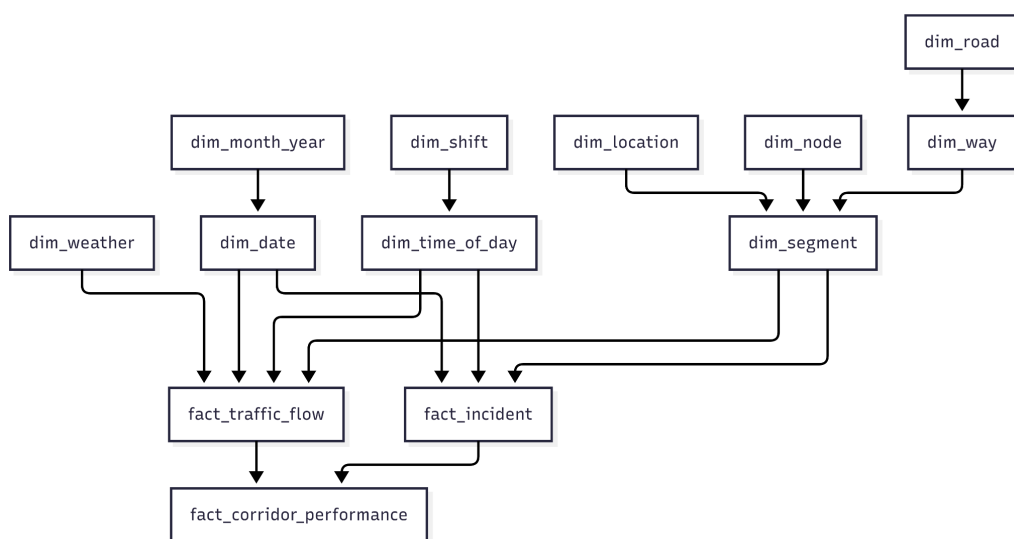
7.10.2 Mở rộng kích thước lưới cho dữ liệu bổ trợ

Đối với luồng OpenWeatherMap bổ trợ cho bảng dim_weather, thay vì gọi API cho từng tọa độ đoạn đường (segment) nhỏ lẻ một cách lãng phí, kiến trúc sư dữ liệu đã thiết kế chiến lược thu thập theo dạng Lưới không gian (Grid-based Extraction). Thành phố được chia thành các ô lưới (Grid size), và thời tiết được nội suy cho toàn bộ các đoạn đường nằm trong ô lưới đó. Điều này minh chứng cho tính linh hoạt của BaseExtractor: một khung kiến trúc nhưng áp dụng được nhiều kỹ thuật tối ưu không gian khác nhau, chuẩn bị hoàn hảo cho giai đoạn Transform phức tạp phía sau.

7.11 Trình tự nạp dữ liệu và ràng buộc

7.11.1 Nguyên lý ràng buộc toàn vẹn tham chiếu và cấu trúc DAG

Trong lý thuyết thiết kế Kho dữ liệu (Data Warehouse) sử dụng mô hình Galaxy Schema, luồng thực thi ETL không thể tiến hành nạp dữ liệu song song (parallel loading) một cách tùy ý cho tất cả các bảng. Về bản chất toán học, các bảng trong cơ sở dữ liệu và các Ràng buộc Khóa ngoại (Foreign Key Constraints) tạo thành một Đồ thị có hướng không chu trình (DAG - Directed Acyclic Graph). Trong đồ thị này, các đỉnh (nodes) đại diện cho các bảng, và các cạnh có hướng (directed edges) biểu diễn quan hệ phụ thuộc từ bảng chứa khóa ngoại (Bảng con) trở về bảng chứa khóa chính (Bảng cha).



Để duy trì Ràng buộc Toàn vẹn Tham chiếu (Referential Integrity), chiến lược nạp dữ liệu bắt buộc phải tuân thủ thuật toán Sắp xếp Tô-pô (Topological Sort). Nếu pipeline vi phạm trình tự này (ví dụ: nạp dữ liệu luồng giao thông trước khi nạp thông tin đoạn đường), hệ quản trị PostgreSQL sẽ lập tức từ chối giao dịch (Transaction Rollback)

kèm theo lỗi Foreign Key Violation. Do đó, kiến trúc Pipeline của dự án được phân rã thành 4 pha (Phases) thực thi tuần tự nghiêm ngặt.

7.11.2 Pha 1: Khởi tạo dữ liệu nền tảng

Pha đầu tiên tập trung giải quyết các đỉnh gốc (root nodes) của đồ thị DAG – đây là các bảng Chiều (Dimension Tables) không chứa khóa ngoại, hoặc chỉ chứa khóa ngoại tham chiếu lẫn nhau trong một cụm độc lập. Mục tiêu của pha này là xây dựng hệ quy chiếu về Không gian, Thời gian và Ngữ cảnh.

Hệ quy chiếu Thời gian (Temporal Dimensions): Dữ liệu được nạp theo chuỗi phụ thuộc tuyến tính: `dim_month_year` → `dim_date` và `dim_shift` → `dim_time_of_day`. Đây là các bảng tĩnh, thường được nạp sẵn dữ liệu (pre-populated) cho nhiều năm, làm nền tảng phân hoạch (partitioning) cho toàn bộ Data Warehouse.

Chiều Ngữ cảnh (Contextual Dimension): Bảng `dim_weather` đóng vai trò là một Slow Changing Dimension (SCD) cung cấp thông tin ngữ cảnh thời tiết. Vì thời tiết thay đổi liên tục, module thời tiết sẽ được kích hoạt nạp trước tiên trong chu kỳ 15 phút, sinh ra `weather_key` để chuẩn bị gán vào các sự kiện giao thông phát sinh cùng thời điểm.

7.11.3 Pha 2: Xây dựng Topology Không gian

Pha thứ hai là trái tim của mô hình Snowflake không gian, do module `osm_pipeline.py` đảm nhiệm. Khác với các hệ thống thông thường, mạng lưới đường bộ có tính phụ thuộc hình học (Geometric Dependency) cực kỳ khắt khe. Dữ liệu bắt buộc phải được nạp (UPSERT) theo trình tự giải phẫu topology:

`dim_node` (Nút giao): Nạp đầu tiên, chứa tọa độ geometry(Point,4326) nguyên thủy.

`dim_road` (Tuyến đường): Nạp thông tin định danh tuyến đường (tên đường, tổng chiều dài).

`dim_way` (Phân đoạn OSM): Trỏ khóa ngoại `road_key` về `dim_road`, lưu trữ các thuộc tính thiết kế như số làn đường mặc định, giới hạn tốc độ.

`dim_segment` (Đoạn đường vật lý): Là "nút hội tụ" cuối cùng của cụm không gian. Một đoạn đường thực tế được cấu thành từ một điểm đầu, một điểm cuối và thuộc về một tuyến đường. Do đó, `dim_segment` bắt buộc phải nạp cuối cùng để có thể nhận đủ 3 khóa ngoại: `from_node_key`, `to_node_key`, và `way_key`.

7.11.4 Pha 3: Nạp dữ liệu chuỗi thời gian

Sau khi Pha 1 và Pha 2 hoàn tất, toàn bộ hệ quy chiếu đã sẵn sàng. Lúc này, luồng thực thi thời gian thực (`traffic_pipeline.py` và `incident_pipeline.py`) mới được kích hoạt.

Trong thiết kế kiến trúc, các bảng `fact_traffic_flow` và `fact_incident` đóng vai trò là các "Sinks" (Điểm trũng / Nút lá) của đồ thị DAG. Chúng là nơi hội tụ của mọi khóa ngoại.

Bảng `fact_traffic_flow` sẽ hấp thụ đồng thời `segment_key` (từ Pha 2), `date_key`, `time_key` và `weather_key` (từ Pha 1).

Bảng `fact_incident` cũng tương tự, kết hợp thêm dữ liệu không gian geometry(Point,4326) để mô tả vị trí xảy ra sự cố.

Đặc biệt, ở Pha này, chiến lược nạp dữ liệu kết hợp trực tiếp với tính năng Table Partitioning của PostgreSQL. Hệ thống không ghi toàn bộ vào một bảng khổng lồ mà định tuyến dữ liệu vào các phân vùng theo tháng (ví dụ: `fact_traffic_flow_202603`) dựa trên giá trị của `date_key`. Điều này triệt tiêu tình trạng thắt cổ chai I/O (I/O Bottleneck) khi lưu lượng ghi vào database tăng đột biến ở các khung giờ cao điểm.

7.11.5 Pha 4: Hậu xử lý và dữ liệu phái sinh

Pha cuối cùng là các luồng phân tích theo lô (Batch Processing), tiêu biểu là pipeline hiệu suất hành lang (`corridor_pipeline.py`).

Bảng `fact_corridor_performance` lưu trữ Dữ liệu Phái sinh (Derived Data). Xét về sự phụ thuộc (Data Dependency), nó không trích xuất trực tiếp từ API bên ngoài mà sử dụng kết quả truy vấn (JOIN và Aggregation) từ bảng `fact_traffic_flow` và cầu nối `bridge_corridor_segment`.

Vì tính chất này, logic điều phối (main.py) buộc phải lập lịch cho Pha 4 chạy vào cuối ngày (Nightly Batch), SAU KHI các bảng Fact ở Pha 3 đã thu thập đầy đủ và ổn định dữ liệu. Việc thiết lập quan hệ phụ thuộc thời gian (Temporal Dependency) giữa các tiến trình ETL đảm bảo các chỉ số tổng hợp (như travel_time_index hay corridor_efficiency) phản ánh chính xác bức tranh giao thông của toàn bộ ngày vận hành.

7.12 Logic Biến đổi Dữ liệu

7.12.1 Nguyên lý hàm thuần túy trong tầng biến đổi

Trong kiến trúc của hệ thống Data Pipeline, pha Transform đóng vai trò là "hạt nhân tính toán" (Computational Engine). Về mặt kỹ nghệ phần mềm, toàn bộ các phép biến đổi nghiệp vụ tại lớp BaseTransformer được thiết kế tuân thủ nghiêm ngặt nguyên lý Hàm Thuần túy (Pure Functions) của Lập trình Hàm (Functional Programming).

Đặc trưng cốt lõi của kiến trúc này là sự vắng mặt hoàn toàn của các "tác dụng phụ" (side-effects). Tầng Transform không thực hiện bất kỳ giao tiếp I/O nào (không gọi API bên ngoài, không truy vấn ngược về cơ sở dữ liệu, không ghi file). Mọi tính toán từ việc ước lượng lưu lượng xe, phân loại mức độ phục vụ (LOS), đến chuẩn hóa tọa độ đều được thực hiện hoàn toàn trong bộ nhớ (in-memory). Quyết định thiết kế này mang lại ba giá trị học thuật to lớn đối với một Data Warehouse:

Tính tất định (Determinism): Đảm bảo rằng với một tập dữ liệu thô (raw payload) đầu vào, bộ chuyển đổi luôn sinh ra một và chỉ một tập dữ liệu chuẩn hóa duy nhất.

Khả năng kiểm thử cô lập (Isolated Testability): Cho phép triển khai các Unit Test bao phủ toàn bộ logic toán học mà không cần thiết lập (mock) các môi trường cơ sở dữ liệu phức tạp.

Tính tái lập (Reproducibility): Khi xảy ra sự cố cần nạp bù dữ liệu (backfill), kỹ sư dữ liệu có thể chạy lại luồng Transform một cách an toàn mà không lo ngại về trạng thái (state) hệ thống làm sai lệch kết quả.

7.12.2 Khai phá và biến đổi động lực học dòng xe

Dữ liệu đầu vào từ API TomTom chủ yếu là các tham số vận tốc. Nhiệm vụ của luồng traffic_pipeline.py là sử dụng các mô hình toán học giao thông để chuyển đổi vận tốc thô thành các chỉ số mô tả bức tranh động lực học dòng xe, sau đó nạp vào bảng fact_traffic_flow.

7.12.2.a Chuỗi biến đổi chỉ số cơ bản (Traffic Index, Delay & LOS)

Hệ thống sử dụng các phép toán vô hướng để thiết lập các chỉ số cơ bản:

Tỷ lệ vận tốc (Speed Ratio - r): Được định nghĩa là tỷ số giữa vận tốc thực tế (v) và vận tốc thiết kế/thông thoáng (v_f).

$$r = \frac{v}{v_f}$$

Chỉ số ùn tắc (Traffic Index - I): Biểu diễn mức độ suy giảm vận tốc so với điều kiện lý tưởng, được lưu vào cột traffic_index.

$$I = 1 - r = 1 - \frac{v}{v_f}, \quad I \in [0, 1]$$

Độ trễ hành trình (Delay Seconds - Δt): Là phần thời gian chênh lệch mà phương tiện phải đánh đổi do sự suy giảm tốc độ.

$$\Delta t = \max(0, t_{current} - t_{freeflow})$$

Phân cấp Mức độ Phục vụ (Level of Service - LOS): Dữ liệu định lượng (I) được rời rạc hóa (discretization) thành các nhân định tính (từ A đến F) thông qua hệ luật chuyên gia (Rule-based mapping), lưu vào cột los_level. Ví dụ: LOS A biểu thị dòng xe di chuyển tự do ($I \leq 0.15$), trong khi LOS F biểu thị trạng thái vỡ trận/tắc nghẽn nghiêm trọng ($I > 0.8$). Bước này rất quan trọng để hỗ trợ hệ thống Dashboard hiển thị màu cảnh báo tự động.

7.12.2.b Bài toán Ước lượng Lưu lượng bằng Nghịch đảo Hàm BPR

API của TomTom không cung cấp trực tiếp lưu lượng xe (Volume). Để lấp đầy khoảng trống dữ liệu này cho cột pcu_volume trong bảng fact_traffic_flow, hệ thống đã áp dụng kỹ thuật xấp xỉ toán học thông qua hàm BPR (Bureau of Public Roads) nổi tiếng trong kỹ thuật giao thông.

Hàm BPR nguyên thủy mô tả sự gia tăng thời gian di chuyển dựa trên tỷ số giữa lưu lượng và công suất (q/C):

$$t = t_0 \left[1 + \alpha \left(\frac{q}{C} \right)^\beta \right]$$

- t : Thời gian hành trình thực tế (Current Travel Time).
- t_0 : Thời gian hành trình thông thoáng (Free-flow Travel Time).
- q : Lưu lượng dòng xe cần tìm (Volume - PCU/h).
- C : Công suất thiết kế của đoạn đường (Capacity), hệ thống tính bằng:

$$C = n_{lane} \times 2000$$

- α, β : Các hệ số hiệu chuẩn (thường mặc định là $\alpha = 0.15, \beta = 4.0$).

Vì dữ liệu thu thập được là vận tốc, hệ thống thực hiện xấp xỉ

$$\frac{t}{t_0} \approx \frac{v_f}{v}$$

và nghịch đảo hàm BPR để tìm ra lưu lượng (q):

$$\begin{aligned} \frac{v_f}{v} - 1 &= \alpha \left(\frac{q}{C} \right)^\beta \\ \Rightarrow q &= C \times \left(\frac{\frac{v_f}{v} - 1}{\alpha} \right)^{\frac{1}{\beta}} \end{aligned}$$

Để đảm bảo tính ổn định số học trong môi trường production, luồng Transform áp dụng các hàm chặn (guard-rails):

$$q_{final} = C \times \min \left(\left(\frac{\max(0, \frac{v_f}{v} - 1)}{\alpha} \right)^{\frac{1}{\beta}}, \rho_{max} \right)$$

Trong đó, ρ_{max} (max_vc_ratio) là ngưỡng chặn trên (thường là 1.0 hoặc 1.2). Kỹ thuật này ngăn chặn hiện tượng tràn số (Overflow) hoặc rác dữ liệu sinh ra khi vận tốc v tiến dần về 0 trong các tình huống kẹt xe nghiêm trọng.

7.12.3 Xử lý không gian và thuật toán khớp bản đồ

Đối với luồng sự cố giao thông (incident_pipeline.py), dữ liệu đầu vào là các tọa độ rời rạc (Latitude, Longitude) nằm trên hệ quy chiếu WGS84. Bài toán đặt ra là làm thế nào để "neo" (snap) các tọa độ này vào chính xác một đoạn đường (segment_key thuộc bảng dim_segment) để phân tích nguyên nhân - kết quả.

7.12.3.a Truy vấn K-Nearest Neighbors (KNN) với PostGIS

Hệ thống giải quyết bài toán Map Matching bằng cách tận dụng sức mạnh của hệ cơ sở dữ liệu không gian PostGIS. Thay vì viết các vòng lặp tính khoảng cách Haversine kém hiệu quả trên Python, thuật toán sử dụng trực tiếp toán tử <-> (KNN - K-Nearest Neighbors) của PostgreSQL.

```
1 SQLSELECT ds.segment_key, ds.location_keyFROM dim_segment dsWHERE ds.geometry_center IS NOT NULLORDER BY ds.geometry_center <-> ST_SetSRID(ST_MakePoint(:lon, :lat), 4326) LIMIT 1
```

Về mặt độ phức tạp thuật toán, toán tử <-> khai thác triệt để cấu trúc cây R-Tree của chỉ mục không gian (GiST Index) idx_dim_segment_center_gist. Nó so sánh khoảng cách giữa tọa độ sự cố với các hộp giới hạn (Bounding Boxes) của đoạn đường, giúp giảm độ phức tạp tìm kiếm từ $O(N)$ (Full Table Scan) xuống mức $O(\log N)$, đáp ứng hoàn hảo yêu cầu xử lý thời gian thực.

7.12.3.b Tiền xử lý hình học đa dạng (Geometry Preprocessing)

Trong trường hợp sự cố (Incident) trả về hình dạng là một tuyến kéo dài (LineString) thay vì một điểm (Point), BaseTransformer sẽ thực hiện nội suy tọa độ trọng tâm (Centroid) trên bộ nhớ Python:

```
1 centroid_lon, centroid_lat = linestring_centroid(geom.coordinates)
```

Sau đó, nó được ép kiểu về định dạng Well-Known Text (WKT) POINT(lon lat) để BaseLoader sử dụng hàm ST_GeomFromText(..., 4326) nạp chuẩn xác vào cột geometry kiểu geometry(Point,4326) trong bảng phân vùng fact_incident.

7.12.4 Chuẩn hóa ngữ nghĩa và phân rã chiều thời gian

Để tương thích tuyệt đối với mô hình Snowflake Schema, mọi sự kiện phát sinh đều phải được phân rã khóa thời gian.

Đồng bộ múi giờ (Timezone Alignment): Mọi chuỗi ISO 8601 trả về từ API đều được phân tích cú pháp (parsing) và ép buộc (localize) về múi giờ chuẩn của hệ thống là Asia/Ho_Chi_Minh (UTC+7). Điều này triệt tiêu sai số thời gian khi phân tích các sự kiện xảy ra quanh ngưỡng nửa đêm.

Phân rã trục thời gian: Dấu thời gian (Timestamp) được bóc tách thành hai khóa ngoại (Foreign Keys) riêng biệt:

date_key (Định dạng số nguyên YYYYMMDD): Dùng để liên kết với bảng dim_date và đóng vai trò là khóa phân vùng (Partition Key) cho các bảng Fact.

time_key (Số phút tích lũy trong ngày): Được tính bằng công thức

$$\text{time_key} = 60 \times \text{hour} + \text{minute}$$

(thuộc miền giá trị $[0, 1439]$).

Việc ánh xạ sang dim_time_of_day với độ phân giải tính bằng phút giúp Data Warehouse dễ dàng thực hiện các phép Roll-up (tổng hợp dữ liệu) theo khung 5 phút, 15 phút hoặc theo Ca trực (Shift) mà không cần dùng đến các hàm xử lý chuỗi thời gian nặng nề.

Chuẩn hóa Mức độ nghiêm trọng (Severity): Mức độ sự cố từ nhà cung cấp được ánh xạ tịnh tiến vào miền giá trị $[0,4]$ thông qua hàm normalize_magnitude(), đảm bảo sự đồng nhất ngữ nghĩa khi truy vấn dữ liệu từ nhiều nguồn khác nhau (như TomTom, OpenWeather, hay mạng xã hội).

7.12.5 Kết luận

Pha Transform trong kiến trúc hệ thống không chỉ là bước "làm sạch" dữ liệu thông thường, mà là một khối phân tích chuyên sâu (Analytical Engine). Bằng việc tích hợp các mô hình toán học (nghịch đảo BPR) và sức mạnh của đại số không gian (PostGIS KNN), lớp Transformer đã thành công trong việc "chiết xuất" các giá trị tri thức ẩn (như lưu lượng xe quy đổi PCU, hay vị trí vật lý đích xác của sự cố) từ những byte dữ liệu thô. Sự kết hợp giữa nguyên lý Hàm thuần túy trên Python và xử lý đồ thị không gian dưới Database tạo ra một luồng dữ liệu vừa có tính toàn vẹn cao, vừa mang ý nghĩa nghiệp vụ sâu sắc để phục vụ cho các mô hình Dự báo (Machine Learning) ở các bước tiếp theo.

7.13 Kỹ thuật tối ưu hóa kho dữ liệu

7.13.1 Mục tiêu tối ưu hóa

Trong hệ thống Traffic IoC, Data Warehouse không chỉ đóng vai trò lưu trữ tĩnh mà phải tiếp nhận luồng dữ liệu thời gian thực với tần suất cao (chu kỳ 15 phút/lần). Đặc thù này đặt ra ba thách thức lớn cho hệ quản trị PostgreSQL:

Write Amplification (Khuếch đại ghi): Khối lượng thao tác INSERT/UPDATE khổng lồ và liên tục có thể gây nghẽn cổ chai I/O (I/O Bottleneck).

Query Latency (Độ trễ truy vấn): Yêu cầu bóc tách dữ liệu theo cả hai chiều: Không gian (Spatial) và Thời gian (Temporal) trên các bảng Fact chứa hàng triệu bản ghi.

Table Bloat (Phình to dữ liệu): Đặc thù của cơ chế MVCC (Multi-Version Concurrency Control) trong PostgreSQL sinh ra các bản ghi rác (dead tuples) khi thực hiện cập nhật.

Để giải quyết triệt để các vấn đề này, hệ thống áp dụng một chiến lược tối ưu hóa toàn diện, kết hợp giữa Phân mảnh dữ liệu vật lý (Partitioning), Chỉ mục đa hình (Polymorphic Indexing) và Cơ chế ghi lũy đẳng.

7.13.2 Chiến lược phân mảnh dữ liệu chuỗi thời gian

Thay vì lưu trữ toàn bộ sự kiện giao thông vào một bảng nguyên khối (Monolithic Table) duy nhất, kiến trúc sư dữ liệu đã quyết định áp dụng kỹ thuật Range Partitioning (Phân vùng theo khoảng) dựa trên khóa phân vùng `date_key`. Kỹ thuật này được áp dụng nhất quán trên các bảng sự kiện lỗi như `fact_traffic_flow`, `fact_incident`, và `fact_traffic_risk_prediction`.

```
1 CREATE TABLE public.fact_traffic_flow (  
2     traffic_flow_key bigint NOT NULL,  
3     segment_key bigint NOT NULL,  
4     time_key integer NOT NULL,  
5     date_key integer NOT NULL,  
6 ) PARTITION BY RANGE (date_key);  
7  
8 CREATE TABLE public.fact_traffic_flow_202603 ( ... );  
9  
10 ALTER TABLE ONLY public.fact_traffic_flow  
11     ATTACH PARTITION public.fact_traffic_flow_202603  
12     FOR VALUES FROM (20260301) TO (20260401);
```

Về mặt nguyên lý hệ thống, kỹ thuật này mang lại các đột phá kiến trúc sau:

Partition Pruning (Cắt tỉa phân vùng): Khi một truy vấn (Query) từ Dashboard yêu cầu lấy dữ liệu lưu lượng của một ngày cụ thể, Query Planner của PostgreSQL sẽ tự động bỏ qua các phân vùng không liên quan. Điều này chuyển đổi độ phức tạp từ việc quét toàn bộ dữ liệu (Full Table Scan) sang chỉ quét trên một tập dữ liệu vật lý cực kỳ nhỏ.

Tối ưu hóa quản trị bộ nhớ (Memory Management): Hệ điều hành máy chủ chỉ cần nạp (cache) các phân vùng của tháng hiện tại ("Hot Data") vào bộ nhớ RAM, đẩy các phân vùng của các tháng trước ("Cold Data") xuống ổ cứng, giúp tối ưu hóa Cache Hit Ratio một cách tuyệt đối.

7.13.3 Chiến lược đánh chỉ mục đa hình

Lược đồ cơ sở dữ liệu (Database Schema) của dự án thể hiện sự am hiểu sâu sắc về cấu trúc dữ liệu bên dưới DBMS. Thay vì lạm dụng chỉ mục B-Tree mặc định, hệ thống triển khai một ma trận chỉ mục phức hợp để đáp ứng chính xác từng loại tải công việc (Workload).

7.13.3.a Chỉ mục không gian GiST (Generalized Search Tree)

Đối với các trường dữ liệu mang tính chất tọa độ hình học (kiểu geometry(Point,4326) trong PostGIS), cấu trúc B-Tree truyền thống (vốn chỉ sắp xếp dữ liệu 1 chiều) hoàn toàn vô dụng. Hệ thống bắt buộc phải sử dụng GiST Index.

```
1 CREATE INDEX idx_fact_incident_geom_gist ON ONLY public.fact_incident
2     USING gist (geometry);
3
4 CREATE INDEX idx_dim_segment_center_gist ON public.dim_segment
5     USING gist (geometry_center);
```

GiST hoạt động dựa trên nguyên lý của cây R-Tree, chia không gian thành các Hộp giới hạn (Bounding Boxes) lồng nhau. Nhờ GiST, các truy vấn Map Matching sử dụng toán tử K-Nearest Neighbor (<->) có thể đạt tốc độ $O(\log N)$ thay vì quét tuần tự.

7.13.3.b Chỉ mục khối dải BRIN (Block Range Index)

Đây là một kỹ thuật tối ưu hóa bậc cao dành riêng cho dữ liệu Time-series. Cột timestamp và inserted_at trong các bảng Fact mang đặc tính "chỉ chèn thêm" (append-heavy) và tăng dần theo thời gian.

```
1 CREATE INDEX idx_fact_traffic_flow_ts_brin ON ONLY public.fact_traffic_flow
2     USING brin ("timestamp") WITH (pages_per_range='32');
```

Nếu dùng B-Tree, hệ thống phải lưu trữ index cho từng dòng (row), làm kích thước index có thể phình to ngang ngửa kích thước bảng. Ngược lại, BRIN Index chỉ lưu trữ giá trị cực tiểu (MIN) và cực đại (MAX) cho mỗi khối dữ liệu (ví dụ: mỗi 32 pages). Khi truy vấn theo dải thời gian, DBMS kiểm tra xem khoảng thời gian yêu cầu có giao (overlap) với MIN-MAX của khối hay không. Điều này giúp BRIN có kích thước cực kỳ nhỏ (chỉ vài Kilobytes) nhưng lại mang tốc độ lọc dữ liệu chuỗi thời gian tương đương B-Tree.

7.13.3.c Chỉ mục một phần (Partial Index) hỗ trợ Alerting

Để phục vụ hệ thống giám sát và cảnh báo thời gian thực (Real-time Dashboards) mà không gây tốn kém dung lượng lưu trữ, lược đồ sử dụng kỹ thuật Partial Index.

```
1 CREATE INDEX idx_fact_incident_severe ON ONLY public.fact_incident
2     USING btree (severity_level, segment_key)
3     WHERE (severity_level >= 4);
4
5 CREATE INDEX idx_fact_flow_bad_los ON ONLY public.fact_traffic_flow
6     USING btree (los_level, segment_key)
7     WHERE (los_level = ANY (ARRAY['E'::bpchar, 'F'::bpchar]));
```


Kiến trúc này cho phép PostgreSQL tạo ra một cây chỉ mục với kích thước cực nhỏ (chỉ chứa các điểm đen giao thông). Các truy vấn đếm số lượng điểm ùn tắc từ Dashboard sẽ quét thẳng vào Partial Index này và trả về kết quả gần như tức thời (sub-millisecond latency).

7.13.4 Cơ chế UPSERT và Quản lý "Bloat" trong MVCC

Tại tầng BaseLoader, hệ thống sử dụng cú pháp INSERT ... ON CONFLICT DO UPDATE (UPSERT) để bảo toàn tính lũy đẳng (Idempotency). PostgreSQL dựa vào các Ràng buộc Khóa chính (Primary Key Constraints) được định nghĩa chặt chẽ trên từng phân vùng vật lý để phát hiện xung đột:

```
1 ALTER TABLE ONLY public.fact_traffic_flow_202603
2   ADD CONSTRAINT fact_traffic_flow_202603_pkey
3   PRIMARY KEY (traffic_flow_key, date_key);
```

Mặc dù UPSERT bảo vệ hệ thống khỏi rác dữ liệu trùng lặp (duplicates), quá trình cập nhật đè lên bản ghi cũ trong PostgreSQL sẽ đánh dấu row cũ là "dead tuple", gây ra hiện tượng phình to dữ liệu (Table Bloat) do cơ chế quản lý giao dịch MVCC.

Sự kết hợp giữa UPSERT và Table Partitioning chính là "chìa khóa" giải quyết vấn đề này. Nhờ việc dữ liệu bị băm nhỏ thành từng tháng, tiến trình dọn dẹp tự động (Auto-vacuum) của PostgreSQL chỉ cần tập trung quét dọn các "dead tuples" sinh ra trong phân vùng của tháng hiện tại (Active Partition) thay vì phải chập vật quét toàn bộ kho dữ liệu nhiều năm. Điều này triệt tiêu hoàn toàn chi phí bảo trì hệ thống và đảm bảo hiệu năng I/O luôn ở trạng thái tốt nhất.

7.13.5 Kết luận

Chiến lược thiết kế cơ sở dữ liệu của dự án minh chứng cho sự kết hợp hài hòa giữa cấu trúc vật lý và logic nghiệp vụ. Việc lựa chọn khéo léo các phương pháp lập chỉ mục thể hệ mới (BRIN cho chuỗi thời gian, GiST cho không gian hình học, Partial Index cho cảnh báo nghiệp vụ) kết hợp với kỹ thuật phân mảnh Range Partitioning đã tạo ra một Data Warehouse "miễn nhiễm" với các giới hạn về độ lớn của bộ nhớ. Đây là một nền tảng vững chắc, vừa tối ưu hóa tốc độ ghi (write-optimized) trong các chu kỳ ETL 15 phút, vừa đảm bảo khả năng phản hồi tốc độ cao (read-optimized) cho các ứng dụng hiển thị và phân tích đầu cuối.

8 Thiết kế DataWarehouse Schema

8.1 Quá trình thiết kế

8.1.1 Thiết kế DW Schema theo lý thuyết, góc nhìn cá nhân/chủ quan về giao thông

Giai đoạn khởi đầu của quá trình thiết kế kho dữ liệu được thực hiện dựa trên cơ sở lý thuyết nền tảng của mô hình hóa dữ liệu đa chiều theo trường phái Kimball, kết hợp với những quan sát trực quan về hoạt động giao thông đô thị.

Ở bước này, mục tiêu chủ đạo là xác định các thành phần cốt lõi của hệ thống dữ liệu, bao gồm: Quy trình nghiệp vụ (Business Processes), Hạt dữ liệu (Grain), Các chiều phân tích (Dimensions) và Các bảng sự kiện (Fact Tables). Đây là bước định hình khung sườn ban đầu để hình thành kiến trúc Data Warehouse.

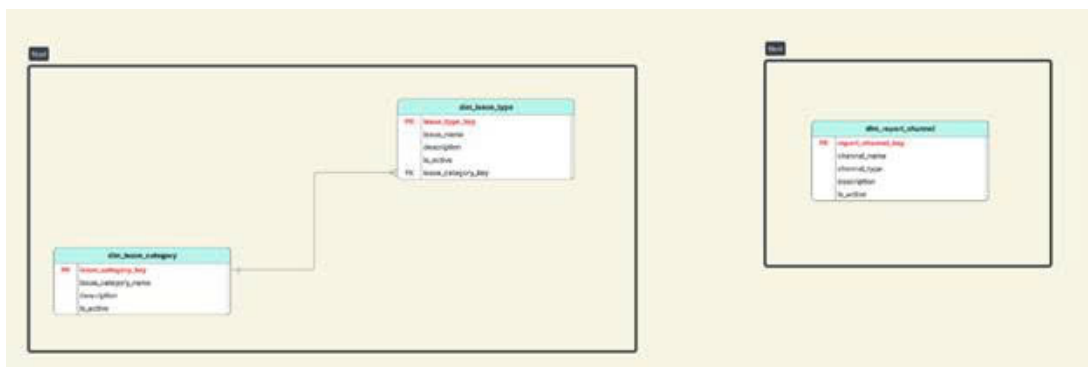
Tuy nhiên, do chưa tham chiếu sâu các tiêu chuẩn kỹ thuật chuyên ngành về cầu đường, tổ chức giao thông, và mô hình hóa mạng lưới giao thông, bản thiết kế sơ khai mang màu sắc thiên về lý thuyết, thiếu sự tinh chỉnh theo các yêu cầu nghiệp vụ thực tiễn. Điều này dẫn tới việc hình thành các lược đồ sao (Star Schema) rời rạc, các bảng chiều thiếu chức năng mô tả đầy đủ và chưa đạt tính tối ưu về mặt ngữ nghĩa.

8.1.1.a Tiếp cận mô hình Star Schema độc lập

Dựa trên các nguyên tắc cơ bản của thiết kế Data Warehouse, nhóm nghiên cứu đã khởi đầu bằng cách phân tách các quy trình nghiệp vụ riêng biệt thành các lược đồ sao độc lập. Tư duy thiết kế ở giai đoạn này hướng vào việc giải quyết từng câu hỏi phân tích đơn lẻ, chưa tính đến nhu cầu tích hợp dữ liệu thành một kiến trúc tổng thể.

Hai hướng tiếp cận chính được hình thành:

1. **Lược đồ lưu lượng giao thông (Traffic Flow Schema):** Mục tiêu là trả lời câu hỏi: “Có bao nhiêu phương tiện đã di chuyển qua một vị trí, tuyến hoặc phân đoạn đường trong một khoảng thời gian nhất định?”. Bảng Fact chủ đạo chứa dữ liệu đếm xe, liên kết với các bảng Dim cơ bản (thời gian, vị trí, loại phương tiện).
2. **Lược đồ sự cố giao thông (Incident Schema):** Tập trung giải quyết câu hỏi: “Sự cố giao thông xảy ra ở đâu và khi nào?”. Bảng Fact chứa thông tin va chạm/sự cố, kết nối với chiều thời gian, vị trí và loại sự cố.



Hình 22: Các lược đồ Star Schema ban đầu

Cách tiếp cận theo mô hình Star Schema tách biệt này vô tình tạo nên các “ốc đảo dữ liệu” (data silos). Các bảng Dimension được tạo dựng độc lập theo từng Fact, dẫn đến tình trạng thiếu các Dimension chuẩn hóa, dùng chung. Điều này hạn chế khả năng thực hiện các phân tích đa chiều tổng hợp, đặc biệt là các phân tích chéo giữa lưu lượng giao thông và sự cố – vốn là yêu cầu quan trọng đối với bài toán ra quyết định trong thực tế.

8.1.1.b Thiết kế các bảng Dimension dựa trên quan sát chủ quan

Do hạn chế về kiến thức chuyên ngành trong giai đoạn đầu, các bảng Dimension được xây dựng chủ yếu dựa trên suy luận trực quan hoặc kinh nghiệm phổ quát, thay vì trên nền tảng các quy chuẩn kỹ thuật của lĩnh vực giao thông

vận tải. Điều này dẫn đến việc giản lược quá mức các thành phần mô tả, khiến các bảng chiều thiếu khả năng phản ánh đầy đủ hiện trạng vận hành giao thông. Một số hạn chế điển hình bao gồm:

1. Chiều Không gian – dim_location

- *Cách tiếp cận ban đầu:* Vị trí giao thông được xem đơn thuần là một điểm địa lý (Kinh độ/Vĩ độ).
- *Hạn chế:* Bỏ qua cấu trúc mạng lưới giao thông (Tuyến đường, Phân đoạn, Hướng lưu thông, Lý trình). Dẫn đến không thể phân tích hiện tượng ùn tắc theo chiều dài tuyến. Không thể mô tả sự khác biệt giữa hai làn đường khác nhau tại cùng tọa độ. Khó khăn trong việc đồng bộ dữ liệu từ nhiều nguồn (GPS, camera, cảm biến).

dim_location	
PK	location_key
	location_code
	location_name
FK	location_type_key
	longitude
	latitude
	elevation
FK	ward_key

Hình 23: Thiết kế bảng dim_location ban đầu

2. Chiều Thời gian – dim_time

- *Cách tiếp cận ban đầu:* Xây dựng theo lịch dương (Giờ, Ngày, Tháng, Năm).
- *Hạn chế:* Không phản ánh được thuộc tính đặc thù giao thông như: Giờ cao điểm/thấp điểm, Giờ cấm tải, Các ngày lễ/sự kiện, Yếu tố mùa vụ.

dim_time	
PK	time_key
	timestamp
	hour
	is_peak_hour
FK	day_key

Hình 24: Thiết kế bảng dim_time ban đầu

3. Chiều Phương tiện – dim_vehicle

- *Cách tiếp cận ban đầu:* Phân loại theo tên gọi phổ biến (Xe máy, Ô tô...).
- *Hạn chế:* Thiếu các thông số kỹ thuật (Số trục, Tải trọng, Kích thước, Hệ số PCU) cần thiết cho mô hình dự báo.



Hình 25: Thiết kế bảng dim_vehicle_type ban đầu

8.1.1.c Kết luận về thiết kế sơ khởi

Thiết kế ban đầu tuy tuân thủ lý thuyết Fact-Dimension nhưng thiếu chiều sâu nghiệp vụ, tạo ra 3 hạn chế cốt lõi:

- Thiếu tính liên kết giữa các quy trình nghiệp vụ (không có Dimension dùng chung).
- Dữ liệu mô tả nghèo nàn, thiếu phân cấp (không hỗ trợ drill-down sâu).
- Khả năng mở rộng thấp (khó tích hợp đa nguồn GPS/Camera).

8.1.2 Phân tích nghiệp vụ nâng cao và xây dựng mô hình Galaxy Schema

Sau khi nhận diện bất cập, nhóm nghiên cứu chuyển sang phân tích nghiệp vụ chuyên sâu và đề xuất mô hình **Galaxy Schema** – phù hợp khi nhiều quy trình nghiệp vụ chia sẻ chung một tập các bảng chiều chuẩn hóa.

8.1.2.a Phân tích ma trận nghiệp vụ

Thông qua việc phân tích các nhóm nghiệp vụ từ BKTraffic, tài liệu liên quan, nhóm nhận thấy các luồng dữ liệu giao thông không tồn tại độc lập mà có mối quan hệ chặt chẽ với nhau. Các câu hỏi phân tích thực tế thường yêu cầu kết hợp dữ liệu từ nhiều nguồn khác nhau. Điều này làm nổi bật nhu cầu thiết kế mô hình dữ liệu tích hợp thay vì các Star Schema tách biệt.

Bảng 3: Ma trận quy trình nghiệp vụ và các chiều dữ liệu

Quy trình nghiệp vụ	Time	Location	Vehicle	Weather	Incident	Source
Theo dõi lưu lượng	X	X	X	X		X
Giám sát sự cố	X	X		X	X	X
Vận tải công cộng	X	X	X			
Dự báo thời tiết	X	X		X		
Điều phối tín hiệu	X	X				X

Một số mối liên kết nghiệp vụ tiêu biểu:

- Mối liên hệ giữa lưu lượng và an toàn giao thông
Các nghiệp vụ như “Dự báo ùn tắc đa yếu tố” và “Đánh giá rủi ro theo khu vực” yêu cầu đồng thời thông tin về mật độ lưu thông (từ fact_traffic_flow) và thông tin sự cố, tai nạn (từ fact_incident). Đây là dạng phân tích phức hợp, không thể thực hiện hiệu quả khi hai hệ thống tồn tại độc lập.
- Mối liên hệ giữa vận hành giao thông và yếu tố môi trường
Nghiệp vụ “Dự báo giao thông theo điều kiện khí hậu” đòi hỏi tích hợp dữ liệu mưa, nhiệt độ, sương mù từ nguồn thời tiết (fact_external_condition) để giải thích sự biến động của tốc độ dòng xe hoặc mức độ ùn ứ.
- Mối liên hệ giữa sự kiện và nhu cầu giao thông
Nghiệp vụ “Dự báo mật độ quanh khu vực sự kiện” yêu cầu liên kết dữ liệu về sự kiện đô thị (fact_event) với hạ tầng giao thông, tuyến đường, phân đoạn và mật độ tại từng khu vực.

Kết quả phân tích cho thấy cần xây dựng tập Conformed Dimensions dùng chung để xóa bỏ tình trạng “ôc đảo dữ liệu”, đảm bảo các bảng fact có thể truy vấn và kết hợp dữ liệu đồng nhất.

8.1.2.b Tái thiết kế các Conformed Dimensions

Để xử lý được các nghiệp vụ đa chiều và phức hợp, các bảng Dimension được tái cấu trúc với mức độ chuẩn hóa và hàm chứa ngữ nghĩa cao hơn đáng kể so với giai đoạn thiết kế ban đầu.

1. Nhóm chiều Thời gian: Cung cấp trục thời gian đa cấp độ, liên kết toàn bộ các bảng Fact.

- `dim_date`, `dim_date_time`: Trục thời gian lịch và timestamp thực.
- `dim_time_of_day`: Mô tả chi tiết 1.440 phút trong ngày, hỗ trợ bucket (5', 15').
- `dim_shift`: Quản lý ca làm việc, giờ ca điểm.
- `dim_holiday`, `bridge_date_holiday`: Quản lý ngày lễ, sự kiện đặc biệt.

2. Nhóm chiều Không gian & Hạ tầng: Hệ tọa độ chung mô tả cấu trúc phân cấp mạng lưới.

- `dim_segment`: Đơn vị cơ bản (phân đoạn đường giữa 2 node).
- `dim_node`: Các nút giao, giao lộ.
- `dim_route`, `bridge_route_segment`: Lộ trình và thứ tự segment.
- `dim_way`, `dim_road`: Cấp độ tuyến đường, trục chính.

3. Nhóm chiều Đối tượng tham gia:

- `dim_user`, `dim_user_detail`: Định danh và thuộc tính người dùng (SCD).
- `dim_vehicle_type`: Chuẩn hóa danh mục phương tiện (PCU, khí thải...).

4. Nhóm chiều Ngữ cảnh & Sự cố:

- `dim_incident_type`: Phân loại sự cố (tai nạn, ngập, công trình).
- `dim_response_unit`: Lực lượng phản ứng.
- `dim_weather`: Thời tiết thực tế (mưa, gió, tầm nhìn).

8.1.2.c Thiết kế các bảng Fact theo nghiệp vụ

Toàn bộ kiến trúc được tổ chức theo mô hình Galaxy Schema với 4 cụm nghiệp vụ:

• **Cụm Vận hành Giao thông:**

- `fact_traffic_flow`: Lưu lượng tổng hợp, tốc độ trung bình, mức độ tắc nghẽn.
- `fact_traffic_flow_by_vehicle`: Lưu lượng chi tiết theo loại xe.

• **Cụm Hành vi Người dùng:**

- `fact_user_travel_time`: Hiệu suất di chuyển từng chuyến đi.
- `fact_user_mobility`: Thống kê hành vi tổng hợp.

• **Cụm An toàn & Sự cố:**

- `fact_incident`: Ghi nhận sự cố, thời gian xử lý, mức độ nghiêm trọng.

• **Cụm Hiệu quả Hệ thống:**

- `fact_route_recommendation`: Đánh giá hiệu quả gợi ý tuyến đường.

8.1.2.d Tổng quan quá trình chuyển đổi

Quá trình thiết kế là sự chuyển dịch từ: **Star Schema** (đơn giản, rời rạc) → **Snowflake Schema** (chuẩn hóa chiều không gian để phản ánh đúng topo mạng lưới) → **Galaxy Schema** (tích hợp đa luồng nghiệp vụ trên cùng hệ thống Dimension).

8.1.3 Phân tích tính khả thi

Phần này tập trung tìm hiểu và phân tích các nguồn data input, bao gồm 3 nguồn chính sau: TomTom, Google Maps và OpenStreetMap để xác thực độ khả thi của thiết kế tại mục 5.1.2, qua đó điều chỉnh để tính khả thi đạt 100%.

8.1.3.a TomTom

Các loại dữ liệu và phạm vi cung cấp: TomTom cung cấp một hệ sinh thái dữ liệu giao thông toàn diện, bao gồm cả dữ liệu thời gian thực (Real-time) với tần suất cập nhật mỗi phút và dữ liệu lịch sử phục vụ thống kê. Các nhóm dữ liệu chính bao gồm:

- **Dữ liệu dòng giao thông (Traffic Flow):** Cung cấp các chỉ số về tốc độ quan sát thực tế (observed speed), thời gian di chuyển (travel time) và so sánh với tốc độ dòng chảy tự do (free-flow) cho từng phân đoạn đường (road segments).
- **Sự cố giao thông (Traffic Incidents):** Thông tin chi tiết về các sự kiện gây cản trở giao thông như tai nạn, công trường thi công, hoặc đường đóng. Dữ liệu bao gồm vị trí bắt đầu/kết thúc, độ dài đoạn ảnh hưởng, loại sự cố và mức độ nghiêm trọng.
- **Dữ liệu hiển thị và điều khiển:** Hỗ trợ các lớp bản đồ Vector Tiles cho luồng giao thông và sự cố, cùng với thông tin cập nhật giới hạn tốc độ động (Live speed restrictions) giúp tăng cường độ chính xác cho các ứng dụng dẫn đường.

Cấu trúc và định dạng dữ liệu kỹ thuật: Về mặt kỹ thuật, TomTom sử dụng các chuẩn dữ liệu phổ biến để đảm bảo tính tương thích cao.

- Các phản hồi từ REST API (như thông tin sự cố, metadata, lỗi) được trả về dưới định dạng **JSON** tiêu chuẩn.
- Đối với các dữ liệu yêu cầu hiệu năng hiển thị cao như Flow/Incident Tiles, TomTom sử dụng định dạng vector nhị phân (**Binary Protobuf/PBF-like**), được tuần tự hóa bởi Google Protocol Buffers theo schema riêng.
- Các trường thông tin (key fields) quan trọng phục vụ cho việc mô hình hóa dữ liệu bao gồm: tọa độ không gian (start/endLocation), tên đường (roadName), tham số giao thông (delay, speedObserved, speedFreeFlow), độ tin cậy của dữ liệu (confidence) và nhãn thời gian (timestamp).

Cơ chế truy cập và Chính sách sử dụng: Để tích hợp vào hệ thống, nhà phát triển cần đăng ký API Key thông qua TomTom Developer Portal. Dữ liệu được truy xuất thông qua các RESTful API endpoints (như Traffic Incidents service) hoặc Vector Tile endpoints.

- Về mặt giới hạn và chi phí, TomTom áp dụng mô hình giới hạn truy cập (Rate limits) dựa trên từng khóa API hoặc gói dịch vụ cụ thể.
- Mô hình chi phí dựa trên mức độ sử dụng (pay-as-you-go). Đáng chú ý, độ trễ dữ liệu (latency) được duy trì ở mức thấp, cập nhật từng phút, phù hợp cho các bài toán điều hướng thời gian thực.

Đánh giá khả năng ứng dụng tại TP.HCM:

- **Ưu điểm:** Dữ liệu có độ tin cậy cao, cập nhật nhanh (real-time), hỗ trợ Vector Tiles giúp việc hiển thị và streaming dữ liệu lên bản đồ số mượt mà, hiệu quả. Đây là nguồn dữ liệu lý tưởng cho các ứng dụng yêu cầu độ chính xác cao như điều phối xe hoặc dẫn đường.

- **Nhược điểm và Thách thức:** Chi phí vận hành có thể tăng cao khi mở rộng quy mô (scale). Ngoài ra, độ phủ sóng dữ liệu (coverage) của TomTom tại TP.HCM chủ yếu tập trung tốt ở các đô thị lớn và đường trục chính; đối với hệ thống đường hầm, đường nhánh nhỏ đặc thù của Việt Nam, lượng dữ liệu từ cảm biến/probe data có thể ít hơn.

Kết luận: Để tối ưu hóa hiệu quả và chi phí cho dự án, chiến lược đề xuất là chỉ sử dụng dữ liệu TomTom cho các tuyến đường trục chính, đường huyết mạch có vai trò quan trọng trong việc điều tiết giao thông toàn thành phố. Đối với các tuyến đường nhỏ hoặc hầm không có tác động lớn đến luồng giao thông tổng thể, hệ thống sẽ cân nhắc sử dụng các nguồn dữ liệu thay thế.

8.1.3.b Google Maps

Các loại dữ liệu và phạm vi cung cấp: Google Maps Platform cung cấp hệ sinh thái dữ liệu giao thông tập trung mạnh vào trải nghiệm dẫn đường và trực quan hóa.

- **Dữ liệu giao thông thời gian thực (Real-time Traffic):** Cung cấp lớp hiển thị trực quan (TrafficLayer) trên bản đồ thông qua Maps JavaScript API.
- **Định tuyến và thời gian di chuyển (Traffic-aware Routing):** Thông qua **Directions API** và **Routes API** (ComputeRoutes), cung cấp ước lượng thời gian di chuyển chính xác dựa trên tình trạng giao thông thực tế.
- **Dữ liệu hạ tầng đường bộ: Roads API** hỗ trợ các tính năng như bám đường (snap-to-road), truy xuất giới hạn tốc độ (speed limits).
- **Lưu ý về sự cố giao thông:** Khác với TomTom, Google không cung cấp một luồng dữ liệu (feed) riêng biệt và có cấu trúc cho các sự cố (tai nạn, công trường) qua REST API.

Cấu trúc và định dạng dữ liệu kỹ thuật:

- Các phản hồi từ REST API được trả về dưới định dạng **JSON**.
- Các trường thông tin quan trọng bao gồm: thời gian di chuyển trong điều kiện giao thông (duration_in_traffic), mô hình giao thông (traffic_model), hình học tuyến đường (geometry polyline), phân đoạn hành trình (legs/steps) và giới hạn tốc độ (speedLimits).
- Dữ liệu có độ trễ thấp, phù hợp cao cho các tác vụ thời gian thực.

Cơ chế truy cập và Chính sách sử dụng: Việc truy cập dữ liệu được quản lý chặt chẽ thông qua Google Cloud Console, yêu cầu bắt buộc phải có API Key và tài khoản thanh toán (Billing account). Chi phí tính theo mô hình pay-as-you-go, khá cao cho các tính năng nâng cao như ComputeRoutes ở quy mô lớn.

Đánh giá khả năng ứng dụng tại TP.HCM:

- **Ưu điểm:** Độ phủ dữ liệu (coverage) tại TP.HCM rất tốt nhờ lượng người dùng thiết bị di động khổng lồ. Khả năng tích hợp sâu với các SDK (Mobile, Web) hỗ trợ mạnh mẽ cho các bài toán định tuyến.
- **Nhược điểm và Thách thức:** Rào cản lớn nhất là **chi phí vận hành**. Mô hình tính phí trên mỗi yêu cầu (request-based) tạo ra gánh nặng tài chính lớn khi thu thập dữ liệu liên tục cho Data Warehouse. Ngoài ra, việc thiếu API cung cấp dữ liệu sự cố dưới dạng cấu trúc gây khó khăn cho việc thống kê tự động.

Kết luận: Do các hạn chế về chi phí và cấu trúc dữ liệu sự cố, Google Maps Platform được xếp độ **ưu tiên thấp nhất** trong việc làm nguồn dữ liệu đầu vào cho Kho dữ liệu. Nguồn này sẽ chỉ được cân nhắc sử dụng cho các chức năng hiển thị (Visual layer) ở phía người dùng cuối.

8.1.3.c OpenStreetMap

Các loại dữ liệu và phạm vi cung cấp: OpenStreetMap (OSM) là cơ sở dữ liệu bản đồ nguồn mở toàn cầu. OSM cung cấp nền tảng dữ liệu địa lý tinh phong phú (mạng lưới đường bộ, ranh giới hành chính, POIs). Tuy nhiên, hệ thống **không cung cấp sẵn** dữ liệu lưu lượng giao thông thời gian thực hay thông tin sự cố tức thời.

Cấu trúc và định dạng dữ liệu kỹ thuật:

- Dữ liệu OSM được tổ chức linh hoạt dựa trên mô hình thẻ (tags).
- Sử dụng **PBF (Protocolbuffer Binary Format)** cho lưu trữ offline và **OSM XML** hoặc **Overpass JSON** cho API.
- Các trường thông tin quan trọng: loại đường (highway), giới hạn tốc độ (maxspeed), chiều đi chuyển (oneway), số làn xe (lanes), kết cấu mặt đường (surface).

Cơ chế truy cập và Chính sách sử dụng: OSM cung cấp cơ chế truy cập rất linh hoạt và mở (thông qua Geofabrik hoặc Overpass API). Dữ liệu hoàn toàn miễn phí dưới giấy phép **ODbL**. Chi phí thực tế chuyển dịch sang phần hạ tầng kỹ thuật (self-host) để tránh giới hạn truy cập.

Đánh giá khả năng ứng dụng tại TP.HCM:

- **Ưu điểm:** Thế mạnh lớn nhất là độ chi tiết về hình học đường bộ, đặc biệt là hệ thống **đường hẻm, ngõ nhỏ** và lối đi bộ. Đây là nguồn dữ liệu nền tảng tuyệt vời để xây dựng đồ thị giao thông (topology graph).
- **Nhược điểm:** Thiếu dữ liệu traffic thời gian thực, không thể dùng độc lập để dự báo tắc nghẽn. Độ đồng nhất của dữ liệu thuộc tính tại ngoại ô có thể không cao.

Kết luận: OpenStreetMap là giải pháp tối ưu để giải quyết bài toán "phủ sóng" bản đồ tại các khu vực hẻm nhỏ của TP.HCM. Chiến lược đề xuất là sử dụng OSM làm **lớp bản đồ nền (Base Map)** và mạng lưới định tuyến cơ sở.

Bảng 4: Bảng tóm tắt so sánh 3 nguồn dữ liệu

Tiêu chí	TomTom	Google Maps Platform	OpenStreetMap (OSM)
Độ phủ (TP.HCM)	Rộng, commercial-grade; đường chính tốt.	Rất rộng; coverage cao ở đô thị lớn.	Geometries & POI tốt ở khu vực cộng đồng active; mạnh về hẻm/chi tiết nhỏ.
Độ chính xác	Cao cho road attributes & traffic metrics trên đường lớn.	Rất cao; enterprise-grade map + signals từ user base lớn.	Rất tốt về geometry; attribute completeness biến thiên.
Real-time	Có — real-time flow & incident feeds (vector tiles).	Có — traffic-aware routing, TrafficLayer; không có incident feed công khai.	Không native. Cần tích hợp bên thứ ba.
Chi phí	Commercial — tốn phí tùy volume; pricing theo API.	Commercial — pay-as-you-go; tốn kém ở scale lớn.	Miễn phí (ODbL). Chi phí ở hạ tầng (hosting).

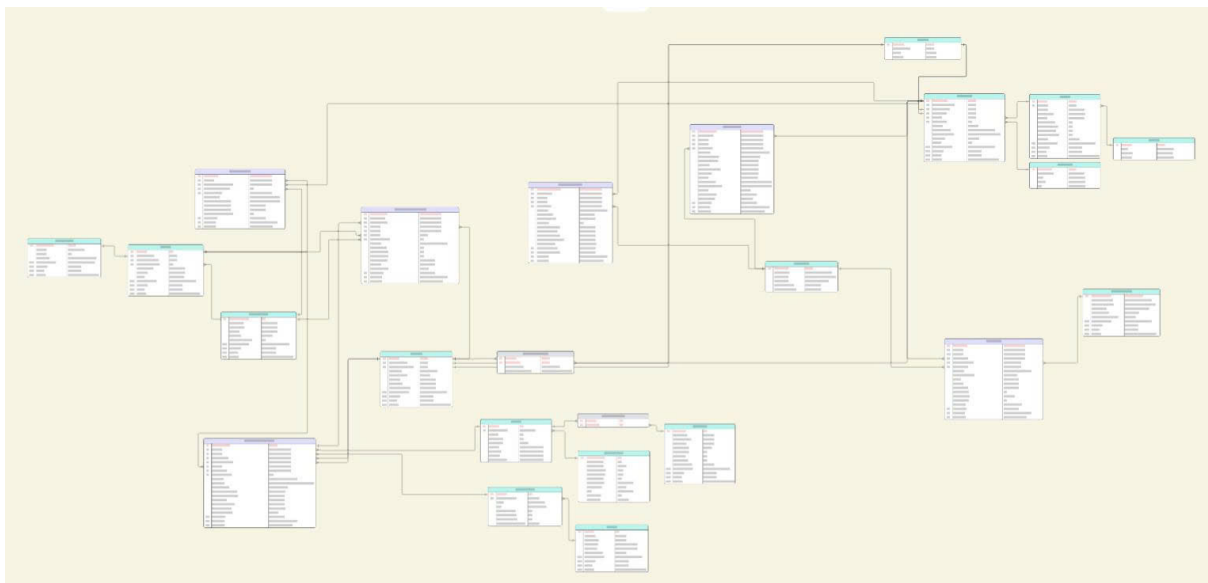
Kiến nghị chiến lược triển khai nguồn dữ liệu thực tiễn cho TP.HCM:

1. **Lựa chọn nguồn dữ liệu cho các tác vụ thời gian thực:** Ưu tiên sử dụng **TomTom** (cho backend/data warehouse nhờ cấu trúc dữ liệu rõ ràng) hoặc **Google Maps** (cho frontend/client-side).
2. **Chiến lược tự chủ dữ liệu và tối ưu chi phí:** Sử dụng **OpenStreetMap (OSM)** làm lớp bản đồ nền cơ sở (Base Map) để giải quyết bài toán ngân sách và đặc thù mạng lưới hẻm nhỏ.
3. **Lộ trình kiểm thử và đánh giá tính khả thi:** Thực hiện A/B Testing và Proof of Concept (PoC) tại 1-2 quận trọng điểm để đánh giá chi phí và độ chính xác thực tế trước khi scale.

8.2 Kết quả

8.2.1 Tổng quan toàn bộ Galaxy Schema

Mô hình Galaxy được xây dựng bằng cách sử dụng chung một tập hợp Dimensions chuẩn hóa. Tất cả các bảng Fact đều liên kết tới các Dimensions trong các chủ đề thời gian, không gian – hạ tầng, đối tượng tham gia và ngữ cảnh – sự cố, hình thành một cấu trúc phân tích đa chiều thống nhất.



Hình 26: Mô hình Galaxy Schema hoàn chỉnh

Các thành phần chính của hệ thống:

- **Hệ trục thời gian:** Bao trùm tất cả các Fact, hỗ trợ phân tích từ vi mô (phút) đến vĩ mô (năm).
- **Hệ tọa độ không gian – hạ tầng:** Cung cấp chuẩn tham chiếu không gian nhất quán (Segment → Node → Way → Road).
- **Hệ tham chiếu người dùng – phương tiện:** Phục vụ phân tích hành vi và tối ưu lộ trình.
- **Hệ ngữ cảnh – sự cố:** Then chốt trong phân tích an toàn và tác động môi trường.

Cách tổ chức này đảm bảo tính nhất quán dữ liệu, tăng khả năng phân tích chéo và sẵn sàng mở rộng cho các nguồn dữ liệu IoT trong tương lai.

8.2.2 Bảng Fact

Các ảnh bảng Fact với đầy đủ kết nối đến bảng Dim.

fact_traffic_flow		
PK	traffic_flow_key	BIGINT (NOT NULL)
FK	segment_key	BIGINT (NOT NULL)
FK	time_key	BIGINT (NOT NULL)
FK	date_key	BIGINT (NOT NULL)
FK	weather_key	BIGINT (NOT NULL)
	timestamp	TIMESTAMP (NOT NULL)
	total_vehicle_count	INT (CHECK ≥ 0)
	motorcycle_count	INT DEFAULT 0
	car_count	INT DEFAULT 0
	heavy_vehicle_count	INT DEFAULT 0
	other_count	INT DEFAULT 0
	total_pcu	DECIMAL(10,2)
	avg_speed_kmh	DECIMAL(5,2) (CHECK ≥ 0)
	free_flow_speed_kmh	DECIMAL(5,2) (CHECK ≥ 0)
	los_index	CHAR(1)
	congestion_level	TINYINT (0-5)
	is_road_closed	BOOLEAN
MD	data_source	VARCHAR(50)
MD	inserted_at	TIMESTAMP (NOT NULL)
MD	quality_flag	TINYINT (DEFAULT 1)

Hình 27: Bảng fact_travel_flow

fact_user_mobility		
PK	mobility_key	BIGINT (NOT NULL)
FK	user_key	BIGINT (NOT NULL)
FK	most_active_location_key	BIGINT (NOT NULL)
FK	preferred_vehicle_type	INT
FK	month_year_key	BIGINT (NOT NULL)
	avg_daily_trips	DECIMAL(6,2) (CHECK ≥ 0)
	avg_weekly_distance_km	DECIMAL(7,2) (CHECK ≥ 0)
	route_repeatability_index	DECIMAL(5,2)
	mobility_variability_index	DECIMAL(5,2)
	commute_start_hour_mode	INT
MD	data_source	VARCHAR(50)
MD	inserted_at	TIMESTAMP (NOT NULL)
MD	quality_flag	TINYINT (DEFAULT 1)

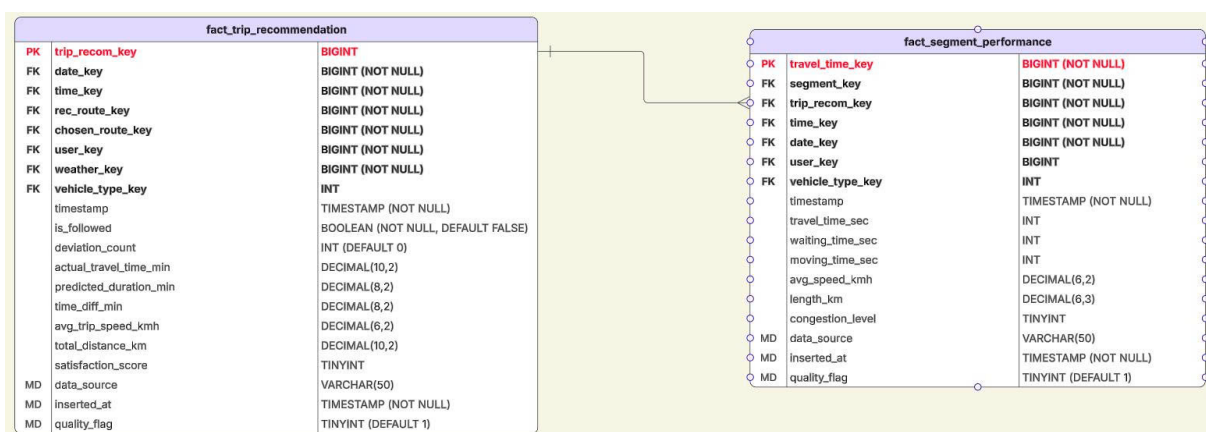
Hình 28: Bảng fact_user_mobility

fact_weather_impact		
PK	weather_impact_key	BIGINT (NOT NULL)
FK	segment_key	BIGINT (NOT NULL)
FK	time_key	BIGINT (NOT NULL)
FK	date_key	BIGINT (NOT NULL)
FK	weather_key	BIGINT (NOT NULL)
	timestamp	TIMESTAMP (NOT NULL)
	temperature_c	DECIMAL(4,1)
	precipitation_mm	DECIMAL(6,2)
	visibility_m	INT
	baseline_speed_kmh	DECIMAL(5,2)
	actual_speed_kmh	DECIMAL(5,2)
	speed_reduction_pct	DECIMAL(5,2)
	congestion_increase_pct	DECIMAL(5,2)
	flood_risk_score	TINYINT (0-10)
MD	traffic_data_source	VARCHAR(50)
MD	weather_data_source	VARCHAR(50)
MD	inserted_at	TIMESTAMP (NOT NULL)
MD	quality_flag	TINYINT (DEFAULT 1)

Hình 29: Bảng fact_weather_impact

fact_incident		
PK	incident_key	BIGINT (NOT NULL)
FK	time_key	BIGINT (NOT NULL)
Key	date_key	BIGINT (NOT NULL)
FK	segment_key	BIGINT (NOT NULL)
FK	incident_type_key	INT (NOT NULL)
FK	weather_key	INT (NOT NULL)
	start_time	TIMESTAMP (NOT NULL)
	expected_end_time	TIMESTAMP
	latitude	DECIMAL(9,6)
	longitude	DECIMAL(9,6)
	severity_level	TINYINT (1-5)
	delay_seconds	INT
	is_road_closed	BOOLEAN
	length_meters	INT
	description	NVARCHAR(255)
MD	data_source	VARCHAR(50)
MD	inserted_at	TIMESTAMP (NOT NULL)
MD	quality_flag	TINYINT (DEFAULT 1)

Hình 30: Bảng fact_incident



Hình 31: Cụm bảng fact_trip_recommendation và fact_segment_performance

8.2.3 Bảng Dim

dim_weather		
PK	weather_key	BIGINT
	weather_code	VARCHAR(50) (UNIQUE)
	weather_type	VARCHAR(100) (NOT NULL)
	severity_level	TINYINT (CHECK 1-5)
	visibility_condition	VARCHAR(50)
	road_friction_impact	VARCHAR(50)

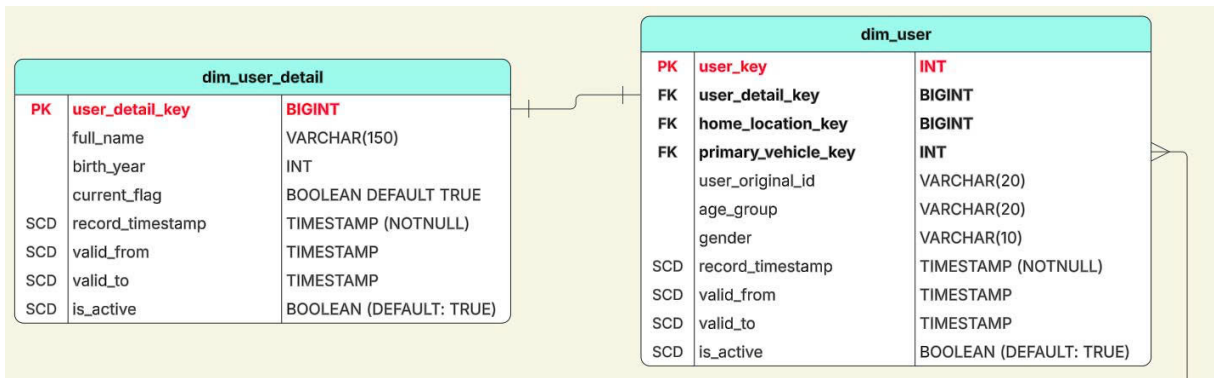
Hình 32: Bảng Dimension dim_weather

dim_incident_type		
PK	incident_type_key	INT (NOT NULL)
	incident_type_code	VARCHAR(50) (UNIQUE)
	incident_type_name	VARCHAR(100) (NOT NULL)
	severity_level	TINYINT (CHECK 1-5)
	color_hex_code	VARCHAR(7)
	incident_category_group	VARCHAR
SCD	record_timestamp	TIMESTAMP (NOTNULL)
SCD	valid_from	TIMESTAMP
SCD	valid_to	TIMESTAMP
SCD	is_active	BOOLEAN (DEFAULT: TRUE)

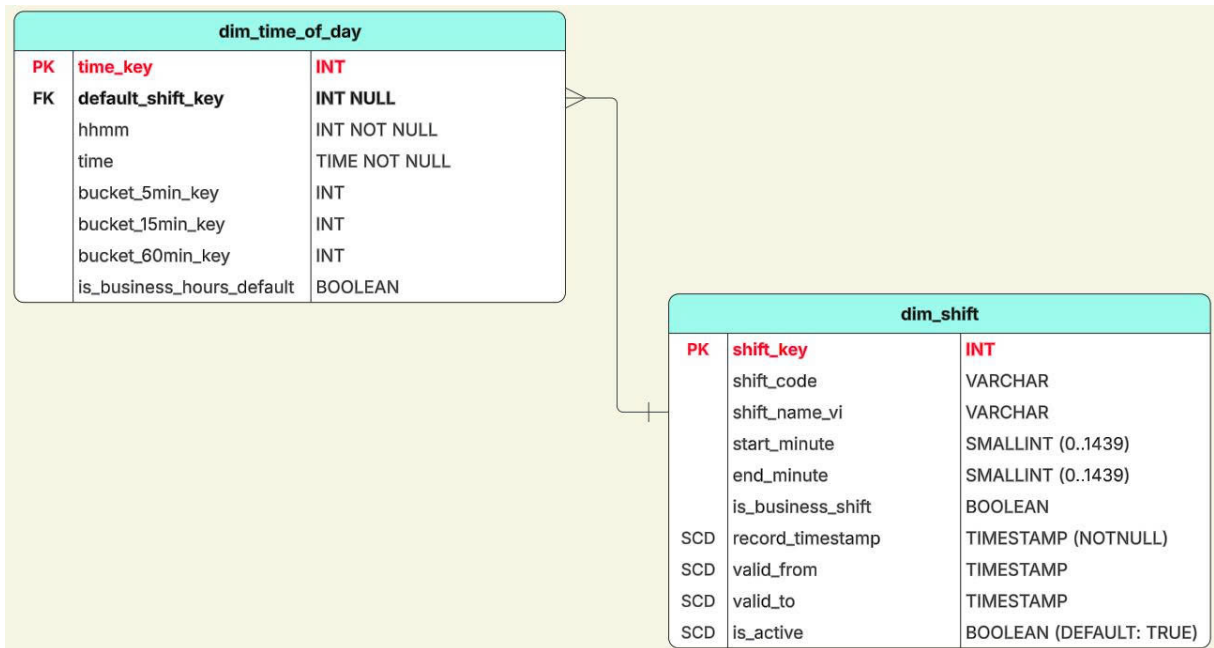
Hình 33: Bảng Dimension dim_incident_type

dim_vehicle_type		
PK	vehicle_type_key	INT
	vehicle_code	VARCHAR(20)
	vehicle_name	VARCHAR(50)
	category	VARCHAR(20)
	pcu_factor	DECIMAL
	average_speed_kmh	INT
SCD	record_timestamp	TIMESTAMP (NOTNULL)
SCD	valid_from	TIMESTAMP
SCD	valid_to	TIMESTAMP
SCD	is_active	BOOLEAN (DEFAULT: TRUE)

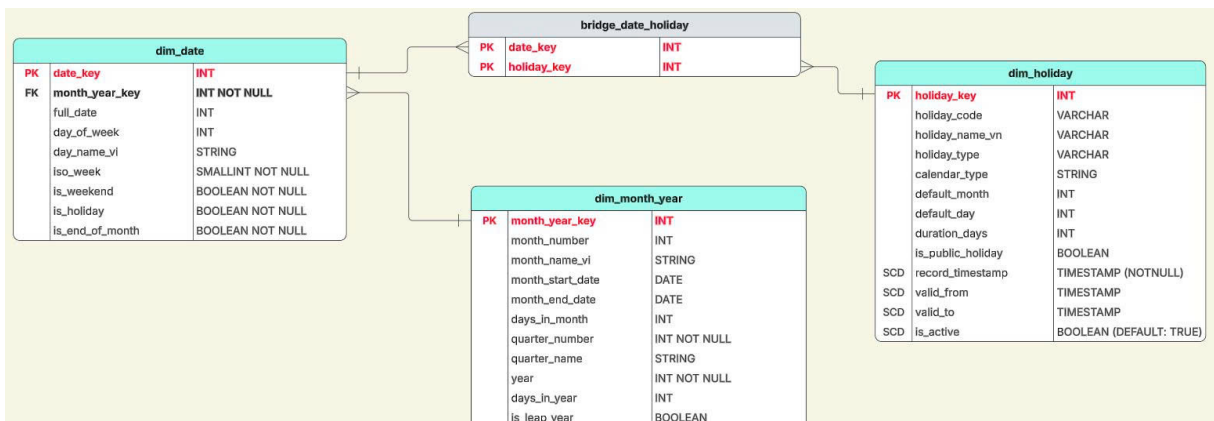
Hình 34: Bảng Dimension dim_vehicle_type



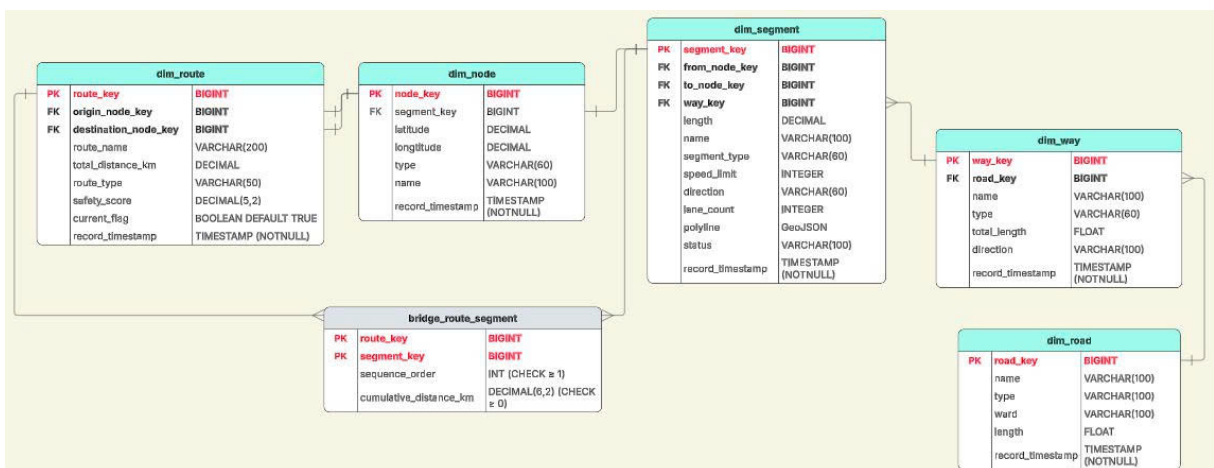
Hình 35: Nhóm Bảng Dimension: Người dùng



Hình 36: Nhóm Bảng Dimension: Thời gian (Chi tiết thời gian trong ngày)



Hình 37: Nhóm Bảng Dimension: Thời gian (Ngày/Tháng/Năm)



Hình 38: Nhóm Bảng Dimension: Hạ tầng giao thông

8.3 Kết chương

Chương 6 đã trình bày một cách đầy đủ tiến trình tư duy và phương pháp thiết kế kiến trúc Kho dữ liệu giao thông, thể hiện bước chuyển quan trọng từ các mô hình dữ liệu đơn giản sang một hệ kiến trúc tích hợp có tính hệ thống và đa chiều hơn.

Trước hết, thông qua phân tích và phản biện ở mục 5.1, nhóm nghiên cứu đã chỉ ra những hạn chế cốt lõi của việc áp dụng mô hình Lược đồ Sao rời rạc trong bối cảnh dữ liệu giao thông – nơi mà mối liên hệ giữa vận hành hạ tầng, an toàn giao thông và hành vi người dùng mang tính tương tác chặt chẽ, không thể tách biệt. Trên cơ sở đó, mô hình Galaxy Schema kết hợp với kỹ thuật chuẩn hóa chiều dạng Snowflake đã được lựa chọn và xây dựng chi tiết trong mục 5.2.

Thiết kế đạt được các kết quả nổi bật:

- **Khắc phục phân mảnh dữ liệu:** Việc hình thành hệ thống Conformed Dimensions (như Thời gian, Segment, Phương tiện) giúp loại bỏ tình trạng rời rạc giữa các quy trình nghiệp vụ, đồng thời mở ra khả năng phân tích chéo đa lĩnh vực (cross-process analytics).
- **Mô hình hóa chính xác ngữ cảnh không gian:** Cấu trúc Snowflake trong nhóm chiều không gian (dim_segment → dim_way → dim_road) tái hiện chính xác đặc trưng topo của mạng lưới giao thông, tạo nền tảng cho các thuật toán định tuyến, tính toán tốc độ lan truyền ùn tắc và mô phỏng sự cố.
- **Đáp ứng toàn diện yêu cầu nghiệp vụ:** Bốn nhóm Fact (Lưu lượng, Sự cố, Hành vi, Hiệu quả Hệ thống) bao phủ đầy đủ các nhu cầu phân tích từ giám sát thời gian thực, đánh giá vận hành, đến hỗ trợ ra quyết định chiến lược dài hạn.

Với bộ kiến trúc dữ liệu đã được định hình chắc chắn, một bài toán trọng tâm tiếp theo là xác định cách thức thu nhận dữ liệu thô từ nhiều nguồn phân tán, không đồng nhất và đưa vào hệ thống một cách nhất quán, chính xác và có khả năng mở rộng.

9 Ứng dụng Schema vào thực tiễn

9.1 Nhóm nghiệp vụ thống kê mô tả và giám sát

9.1.1 A1: Giám sát tốc độ trung bình thời gian thực

Nhóm nghiệp vụ A1 tập trung vào bài toán giám sát tốc độ trung bình của dòng giao thông theo thời gian thực, với độ trễ xử lý tối đa là 5 phút. Phân tích kỹ thuật được xây dựng dựa trên cấu trúc Kho dữ liệu (Data Warehouse) đã thiết kế, kết hợp giữa mô hình dữ liệu và các thuật toán xử lý nhằm đảm bảo độ chính xác và tính đại diện của chỉ số vận tốc được cung cấp.

9.1.1.a Thành phần dữ liệu trong Schema

Để thực hiện giám sát vận tốc thực tế với độ trễ thấp, hệ thống truy vấn và tích hợp dữ liệu từ các bảng Fact và Dimension sau:

Bảng Fact chính: `fact_traffic_flow` Bảng `fact_traffic_flow` đóng vai trò là nguồn dữ liệu trung tâm, lưu trữ các chỉ số giao thông được thu thập từ nhiều nguồn khác nhau (AI Camera, TomTom, v.v.). Các thuộc tính được sử dụng bao gồm:

- `avg_speed_kmh`: Vận tốc trung bình thực tế đo được trên đoạn đường.
- `travel_time_seconds`: Thời gian di chuyển thực tế trên phân đoạn đường.
- `total_pcu`: Tổng số đơn vị xe quy đổi (Passenger Car Unit), được sử dụng làm trọng số phản ánh mức độ ảnh hưởng của dòng phương tiện.
- `segment_key`: Khóa định danh duy nhất cho phân đoạn đường vật lý.
- `time_key`, `date_key`: Khóa thời gian dùng để lọc dữ liệu trong cửa sổ thời gian gần thực.
- `quality_flag`: Cờ đánh dấu độ tin cậy của dữ liệu, chỉ các bản ghi có giá trị bằng 1 mới được đưa vào tính toán.

Bảng Dimension hỗ trợ: `dim_segment` Bảng `dim_segment` cung cấp thông tin không gian và hình học của từng phân đoạn đường:

- `geometry_linestring`: Dữ liệu hình học dạng đường (LineString), phục vụ việc hiển thị trên bản đồ.
- `length_m`: Chiều dài vật lý của phân đoạn đường (đơn vị mét).

Bảng Dimension thời gian: `dim_time_of_day` Bảng `dim_time_of_day` được sử dụng để gom nhóm dữ liệu theo các khoảng thời gian ngắn:

- `bucket_5min_key`: Nhóm thời gian 5 phút, đóng vai trò là *grain* (độ hạt) của báo cáo giám sát thời gian thực.

9.1.1.b Thuật toán xử lý dữ liệu

Mục tiêu của thuật toán là cung cấp giá trị vận tốc định lượng chính xác cho từng phân đoạn đường, thay vì chỉ biểu diễn trạng thái giao thông bằng các mức màu định tính. Bài toán cốt lõi cần giải quyết là hội tụ dữ liệu đa nguồn trong một cửa sổ thời gian ngắn ($T = 5$ phút).

Vận tốc trung bình không gian

Trong kỹ thuật giao thông, vận tốc trung bình không gian (SMS) phản ánh trạng thái dòng xe chính xác hơn so với vận tốc trung bình thời gian, do SMS dựa trên tổng thời gian các phương tiện chiếm dụng đoạn đường.

Vận tốc trung bình không gian cho mỗi phân đoạn (`segment_key`) trong một khoảng thời gian 5 phút (`bucket_5min_key`) được tính theo công thức:

$$V_{SMS} = \frac{n \times L}{\sum_{i=1}^n t_i}$$

Trong đó:

- L : Chiều dài phân đoạn đường (lấy từ thuộc tính `length_m` của bảng `dim_segment`).
- t_i : Thời gian hành trình của phương tiện thứ i (thuộc tính `travel_time_seconds` trong bảng `fact_traffic_flow`).
- n : Tổng số mẫu phương tiện được ghi nhận trong khoảng thời gian 5 phút.

Trọng số hóa theo PCU

Do đặc thù giao thông hỗn hợp, các phương tiện có kích thước lớn như xe buýt hoặc xe tải có ảnh hưởng đáng kể hơn đến khả năng lưu thông so với xe con. Vì vậy, hệ thống áp dụng cơ chế trọng số hóa theo đơn vị xe quy đổi (PCU) nhằm tính toán vận tốc đại diện cho toàn bộ dòng xe.

Vận tốc trung bình cuối cùng cho nhóm nghiệp vụ A1 được xác định như sau:

$$\bar{V}_{A1} = \frac{\sum_{j=1}^m (avg_speed_kmh_j \times total_pcu_j)}{\sum_{j=1}^m total_pcu_j}$$

Trong đó:

- $avg_speed_kmh_j$: Vận tốc đo được từ nguồn dữ liệu thứ j .
- $total_pcu_j$: Lưu lượng xe quy đổi tương ứng của nguồn dữ liệu đó.
- m : Số lượng bản ghi dữ liệu hợp lệ được thu thập cho phân đoạn đường trong khoảng thời gian 5 phút.

Thuật toán trên cho phép hệ thống cung cấp chỉ số vận tốc có tính đại diện cao, phản ánh chính xác trạng thái thực tế của dòng giao thông trong điều kiện dữ liệu đa nguồn và thời gian xử lý gần thực.

9.1.2 A2: Đánh giá Mức độ tắc nghẽn

Nhóm nghiệp vụ A2 tập trung vào việc đánh giá mức độ tắc nghẽn giao thông trên từng phân đoạn đường, dựa trên hệ thống Kho dữ liệu giao thông hiện hữu. Mục tiêu của nhóm nghiệp vụ này là chuẩn hóa các chỉ số định lượng thu thập được từ dữ liệu thực tế thành các mức độ phục vụ, sau đó ánh xạ sang thang điểm trực quan từ 1 đến 5 nhằm phục vụ cho việc giám sát, phân tích và hỗ trợ ra quyết định.

9.1.2.a Thành phần dữ liệu trong Schema

Để đánh giá mức độ tắc nghẽn một cách khách quan và có cơ sở khoa học, hệ thống cần kết hợp đồng thời các thuộc tính động (phản ánh trạng thái giao thông theo thời gian thực) và các thuộc tính tĩnh (phản ánh năng lực thiết kế của hạ tầng).

Bảng Fact chính: `fact_traffic_flow` Bảng `fact_traffic_flow` cung cấp các chỉ số giao thông động, bao gồm:

- `total_pcu`: Tổng số đơn vị xe con quy đổi (Passenger Car Unit) đang lưu thông trên phân đoạn đường tại thời điểm quan sát.
- `avg_speed_kmh`: Tốc độ trung bình thực tế của dòng xe.
- `free_flow_speed_kmh`: Tốc độ dòng xe trong điều kiện tự do, khi không xảy ra ùn tắc.

- `los_index`: Chỉ số mức độ phục vụ (LOS từ A đến F) đã được tính toán sơ bộ từ hệ thống nguồn.

Bảng Dimension hạ tầng: `dim_waydesign` Bảng `dim_waydesign` cung cấp thông tin về năng lực thiết kế của hạ tầng giao thông:

- `design_capacity`: Sức chứa thiết kế của đoạn đường, đo bằng số xe quy đổi trên giờ (PCU/giờ).
- `default_speed_limit`: Tốc độ giới hạn tối đa cho phép trên đoạn đường theo quy định.

Bảng Dimension không gian: `dim_segment` và `dim_location` Hai bảng Dimension này cung cấp ngữ cảnh không gian cho việc phân tích:

- `segment_key`: Khóa định danh phân đoạn đường vật lý.
- `location_key`: Khóa liên kết đến thông tin vị trí hành chính hoặc điểm giao thông đặc trưng (ví dụ: giao lộ, phường, quận).

9.1.2.b Thuật toán và mô hình tính toán

Việc xác định mức độ tắc nghẽn được xây dựng dựa trên sự kết hợp của hai chỉ số cốt lõi trong kỹ thuật giao thông: tỷ lệ lưu lượng trên sức chứa (Volume-to-Capacity Ratio – V/C) và chỉ số suy giảm vận tốc (Speed Reduction Index – SRI). Hai chỉ số này phản ánh đồng thời áp lực hạ tầng và hành vi thực tế của dòng xe.

Chỉ số V/C (Volume-to-Capacity Ratio)

Chỉ số V/C thể hiện mối quan hệ giữa lưu lượng giao thông thực tế và sức chứa thiết kế của hạ tầng, được xác định theo công thức:

$$R_{V/C} = \frac{Total_PCU_{realtime}}{Design_Capacity}$$

Trong đó:

- `Total_PCUrealtime` được lấy từ thuộc tính `total_pcu` trong bảng `fact_traffic_flow`.
- `Design_Capacity` được lấy từ thuộc tính `design_capacity` trong bảng `dim_waydesign`.

Giá trị $R_{V/C}$ càng tiến gần hoặc vượt quá 1 cho thấy phân đoạn đường đang vận hành ở trạng thái quá tải so với năng lực thiết kế ban đầu.

Chỉ số suy giảm vận tốc (Speed Reduction Index – SRI)

Chỉ số SRI phản ánh mức độ sụt giảm vận tốc của dòng xe so với trạng thái tự do, được tính như sau:

$$SRI = 1 - \left(\frac{Avg_Speed}{Free_Flow_Speed} \right)$$

Trong đó:

- `Avg_Speed` là tốc độ trung bình thực tế của dòng xe.
- `Free_Flow_Speed` là tốc độ dòng xe trong điều kiện không bị cản trở.

Giá trị SRI nằm trong khoảng $[0, 1]$. Khi SRI tiến dần về 1, dòng xe có xu hướng tiệm cận trạng thái đứng yên, phản ánh mức độ ùn tắc nghiêm trọng.

9.1.2.c Logic chuẩn hóa sang thang điểm 1–5

Dựa trên tổ hợp của hai chỉ số $R_{V/C}$ và SRI , hệ thống thực hiện ánh xạ mức độ tắc nghẽn sang thang điểm chuẩn hóa từ 1 đến 5. Mỗi mức điểm tương ứng với một trạng thái nghiệp vụ và mức độ phục vụ (LOS) trong kỹ thuật giao thông, như trình bày trong Bảng 5.

Bảng 5: Ánh xạ mức độ tắc nghẽn theo thang điểm 1–5

Mức độ	LOS	Ngưỡng $R_{V/C}$	Mô tả nghiệp vụ
1	A (Free Flow)	< 0.35	Đường rất thông thoáng, phương tiện di chuyển gần với tốc độ giới hạn cho phép.
2	B–C (Stable Flow)	$0.35 - 0.70$	Lưu lượng ổn định, bắt đầu xuất hiện sự tương tác giữa các phương tiện.
3	D (Approaching Unstable)	$0.70 - 0.85$	Lưu lượng lớn, vận tốc giảm rõ rệt, dòng xe kém ổn định.
4	E (Unstable Flow)	$0.85 - 1.00$	Dòng xe di chuyển rất chậm, thường xuyên xảy ra hiện tượng dồn ứ.
5	F (Forced Flow)	> 1.00	Ùn tắc nghiêm trọng, phương tiện phải dừng chờ trong thời gian dài.

Cách tiếp cận này cho phép hệ thống chuyển đổi dữ liệu giao thông phức tạp thành các mức độ dễ diễn giải, đồng thời vẫn bảo toàn cơ sở định lượng và tính khoa học của các chỉ số kỹ thuật.

9.1.3 A3: Phân tích thời gian di chuyển đặc trưng

Nhóm nghiệp vụ A3 tập trung vào việc phân tích thời gian di chuyển đặc trưng của các lộ trình giao thông dựa trên dữ liệu lịch sử trong hệ thống Kho dữ liệu. Mục tiêu chính của nhóm nghiệp vụ này là xác định giá trị kỳ vọng về thời gian di chuyển cho từng lộ trình và từng khung thời gian cụ thể, từ đó làm mốc tham chiếu cho các báo cáo đánh giá hiệu suất giao thông và so sánh với điều kiện vận hành theo thời gian thực.

9.1.3.a Thành phần dữ liệu trong Schema

Để thiết lập mức chuẩn về thời gian di chuyển, hệ thống cần truy vấn dữ liệu lịch sử và thực hiện phân nhóm theo các chiều thời gian và lộ trình cụ thể. Các bảng dữ liệu được sử dụng bao gồm:

Bảng Fact chính: `fact_trip_recommendation` Bảng `fact_trip_recommendation` lưu trữ thông tin chi tiết của các chuyến đi đã được thực hiện trong quá khứ. Các thuộc tính được sử dụng trong phân tích bao gồm:

- `chosen_route_key`: Khóa ngoại liên kết đến lộ trình thực tế mà người dùng đã lựa chọn và di chuyển.
- `actual_travel_time_min`: Tổng thời gian di chuyển thực tế của chuyến đi, đơn vị phút.
- `time_key`, `date_key`: Khóa ngoại dùng để liên kết với các chiều thời gian trong Kho dữ liệu.

Bảng Dimension lộ trình: `dim_route` Bảng `dim_route` cung cấp thông tin định danh và mô tả cho từng lộ trình giao thông:

- `route_key`: Định danh duy nhất cho mỗi tuyến đường.
- `route_name`: Tên mô tả của lộ trình (ví dụ: “Nhà – Công ty”).

Bảng Dimension thời gian trong ngày: `dim_time_of_day` Bảng `dim_time_of_day` được sử dụng để phân nhóm các chuyến đi theo các khung thời gian ngắn:

- `bucket_15min_key`: Nhóm thời gian 15 phút, được xem là chuẩn phổ biến trong các nghiên cứu và hệ thống phân tích giao thông quốc tế.

Bảng Dimension ngày: `dim_date` Bảng `dim_date` cung cấp ngữ cảnh lịch:

- `day_of_week`: Thứ trong tuần (1 = Thứ Hai, ..., 7 = Chủ Nhật), cho phép phân biệt đặc tính giao thông giữa ngày thường và cuối tuần.

9.1.3.b Thuật toán và mô hình tính toán

Thuật toán trong nhóm nghiệp vụ A3 tập trung vào việc trích xuất giá trị thời gian di chuyển mang tính đặc trưng, bằng cách loại bỏ các yếu tố nhiễu và các biến số cực đoan (outliers) như tai nạn giao thông, điều kiện thời tiết bất thường hoặc các sự kiện đột xuất.

Xác định Baseline bằng kỳ vọng toán học

Thời gian di chuyển đặc trưng $T_{Typical}$ được xác định thông qua giá trị kỳ vọng của thời gian di chuyển thực tế trong quá khứ, với điều kiện các chuyến đi có cùng đặc điểm nhận dạng về lộ trình và thời gian. Cụ thể:

$$T_{Typical}(r, b, d) = \mathbb{E}[T_{actual} \mid Route = r, Bucket = b, DoW = d]$$

Trong đó:

- r : Lộ trình cụ thể, được định danh bởi `route_key`.
- b : Khung thời gian 15 phút, tương ứng với `bucket_15min_key`.
- d : Thứ trong tuần, tương ứng với thuộc tính `day_of_week`.

Trong thực tế triển khai, giá trị kỳ vọng có thể được tính bằng trung bình cộng hoặc trung vị (median), tùy theo phân phối dữ liệu và mức độ ảnh hưởng của các giá trị ngoại lai còn sót lại.

Thuật toán lọc giá trị ngoại lai (Z-score Filtering)

Để đảm bảo mức chuẩn được tính toán phản ánh đúng hành vi giao thông điển hình, hệ thống áp dụng thuật toán lọc ngoại lai dựa trên chỉ số Z-score:

$$Z = \frac{x - \mu}{\sigma}$$

Trong đó:

- x : Thời gian di chuyển của một chuyến đi cụ thể.
- μ : Giá trị trung bình của thời gian di chuyển trong nhóm (r, b, d) .
- σ : Độ lệch chuẩn của thời gian di chuyển trong cùng nhóm.

Hệ thống chỉ giữ lại các bản ghi thỏa mãn điều kiện $|Z| < 2$, tương ứng với khoảng 95% dữ liệu tập trung quanh giá trị trung bình, để phục vụ cho việc tính toán thời gian di chuyển đặc trưng.

Cách tiếp cận này cho phép xác định một giá trị baseline ổn định, có khả năng đại diện tốt cho điều kiện giao thông thông thường và đóng vai trò làm mốc so sánh tin cậy cho các phân tích hiệu suất giao thông tiếp theo.

9.1.4 A4: Chỉ số độ tin cậy thời gian

Nhóm nghiệp vụ A4 tập trung vào việc đánh giá độ tin cậy của thời gian di chuyển thông qua chỉ số Buffer Index (BI). Đây là một thước đo tiêu chuẩn được sử dụng rộng rãi trong lĩnh vực phân tích giao thông nhằm phản ánh mức độ biến động của thời gian hành trình. Chỉ số này hỗ trợ người sử dụng trong việc lập kế hoạch xuất phát an toàn bằng cách ước lượng khoảng thời gian dự phòng cần thiết để đảm bảo xác suất đến đúng giờ ở mức cao.

9.1.4.a Thành phần dữ liệu trong Schema

Để tính toán chỉ số độ tin cậy thời gian, hệ thống cần khai thác dữ liệu hành trình lịch sử kết hợp với các khung thời gian tương ứng. Các bảng dữ liệu được sử dụng bao gồm:

Bảng Fact chính: `fact_trip_recommendation` Bảng `fact_trip_recommendation` lưu trữ thông tin chi tiết về các chuyến đi đã được thực hiện trong quá khứ. Các thuộc tính chính phục vụ cho việc tính toán Buffer Index bao gồm:

- `actual_travel_time_min`: Thời gian di chuyển thực tế của chuyến đi (đơn vị: phút), là biến số chính phản ánh sự biến động của hành trình.
- `chosen_route_key`: Định danh tuyến đường mà người dùng đã lựa chọn để di chuyển.
- `time_key`: Khóa ngoại liên kết đến chiều thời gian nhằm xác định khung giờ xuất phát.

Bảng Dimension lộ trình: `dim_route` Bảng `dim_route` cung cấp thông tin định danh và mô tả cho từng tuyến đường:

- `route_key`: Khóa chính định danh duy nhất cho mỗi lộ trình.
- `route_name`: Tên mô tả của lộ trình (ví dụ: “Nhà – Công ty”).

Bảng Dimension thời gian trong ngày: `dim_time_of_day` Bảng `dim_time_of_day` được sử dụng để phân nhóm các chuyến đi theo các khung thời gian chuẩn:

- `bucket_15min_key`: Nhóm thời gian 15 phút, được xem là hạt (grain) tiêu chuẩn để đánh giá sự biến động của thời gian di chuyển trong các khung giờ cao điểm và thấp điểm.

9.1.4.b Thuật toán và mô hình tính toán

Thuật toán trọng tâm của nhóm nghiệp vụ A4 là chỉ số Buffer Index (BI), được xây dựng dựa trên sự chênh lệch giữa thời gian di chuyển trong điều kiện gần như bất lợi nhất và thời gian di chuyển trung bình. Chỉ số này phản ánh mức độ dự phòng thời gian mà người dùng cần bổ sung vào kế hoạch di chuyển để đạt được độ tin cậy mong muốn.

Phân vị thời gian di chuyển

Đối với mỗi cặp định danh `{route_key, bucket_15min_key}`, hệ thống thu thập tập hợp các chuyến đi lịch sử với thời gian di chuyển tương ứng, ký hiệu là T .

Từ tập dữ liệu này, hai đại lượng thống kê quan trọng được xác định:

- Thời gian di chuyển trung bình ($\bar{T}$): Giá trị kỳ vọng của thời gian di chuyển, phản ánh điều kiện giao thông thông thường mà người dùng có thể gặp phải.
- Thời gian phân vị thứ 95 (T_{95}): Giá trị thời gian mà 95% các chuyến đi trong quá khứ hoàn thành trong khoảng thời gian đó hoặc nhanh hơn. Đại lượng này đại diện cho trạng thái “gần như tệ nhất” mà người dùng có khả năng đối mặt trong điều kiện giao thông bất lợi.

Công thức tính chỉ số Buffer Index

Chỉ số Buffer Index được xác định theo tỷ lệ phần trăm thời gian dự phòng cần bổ sung so với thời gian trung bình, nhằm đảm bảo xác suất đến đúng giờ đạt khoảng 95%. Công thức được biểu diễn như sau:

$$BI(\%) = \frac{T_{95} - \bar{T}}{\bar{T}} \times 100\%$$

Giá trị BI càng lớn cho thấy mức độ biến động của thời gian di chuyển càng cao, đồng nghĩa với việc người dùng cần dành nhiều thời gian dự phòng hơn để đảm bảo độ tin cậy của hành trình. Ngược lại, giá trị BI nhỏ phản ánh thời gian di chuyển ổn định và có tính dự đoán cao.

Cách tiếp cận này cho phép hệ thống lượng hóa mức độ tin cậy của thời gian di chuyển một cách trực quan, đồng thời cung cấp thông tin hữu ích cho người dùng trong việc lập kế hoạch xuất phát và đánh giá hiệu suất vận hành của mạng lưới giao thông.

9.1.5 A5: Thống kê điểm nóng sự cố

Nhóm nghiệp vụ A5 tập trung vào việc xác định và phân tích các điểm nóng sự cố (Incident Hotspots), hay còn gọi là các “điểm đen” giao thông, dựa trên dữ liệu lịch sử được lưu trữ trong hệ thống Kho dữ liệu giao thông. Mục tiêu của nhóm nghiệp vụ này là phát hiện các khu vực có tần suất và mức độ nghiêm trọng sự cố cao thông qua việc phân tích đồng thời yếu tố không gian và thời gian, từ đó hỗ trợ công tác quy hoạch, cảnh báo rủi ro và nâng cao an toàn giao thông.

9.1.5.a Thành phần dữ liệu trong Schema

Để xác định chính xác các điểm nóng sự cố, hệ thống cần tích hợp dữ liệu từ bảng Fact sự cố và nhiều bảng Dimension hỗ trợ nhằm cung cấp đầy đủ ngữ cảnh không gian, thời gian và điều kiện ngoại cảnh.

Bảng Fact chính: `fact_incident` Bảng `fact_incident` lưu trữ thông tin chi tiết về các sự cố giao thông đã xảy ra. Các thuộc tính chính được sử dụng trong phân tích bao gồm:

- `segment_key`: Khóa định danh phân đoạn đường nơi xảy ra sự cố.
- `incident_type_key`: Khóa ngoại liên kết đến bảng phân loại bản chất sự cố (tai nạn, xe hỏng, va chạm nhẹ, v.v.).
- `severity_level`: Mức độ nghiêm trọng của sự cố, được chuẩn hóa theo thang giá trị từ 0 đến 4.
- `weather_key`: Khóa ngoại dùng để liên kết với điều kiện thời tiết tại thời điểm xảy ra sự cố.
- `date_key`: Khóa thời gian dùng để tích lũy và phân tích dữ liệu theo các chu kỳ báo cáo (tháng hoặc quý).

Các bảng Dimension hỗ trợ

Các bảng Dimension đóng vai trò bổ sung ngữ cảnh cho việc phân tích điểm nóng sự cố:

- `dim_incident_type`: Cung cấp thuộc tính `incident_category_group`, cho phép phân nhóm các loại sự cố nghiêm trọng nhằm phục vụ phân tích chuyên sâu.
- `dim_segment`: Cung cấp thông tin không gian của phân đoạn đường, bao gồm `geometry_center` (tọa độ tâm) để xây dựng bản đồ nhiệt và `length_m` để chuẩn hóa mật độ sự cố theo chiều dài.
- `dim_weather`: Cung cấp mức độ nghiêm trọng của điều kiện thời tiết (`severity_level`), hỗ trợ phân tích các đoạn đường nhạy cảm với yếu tố ngoại cảnh như mưa lớn, ngập lụt hoặc sương mù.
- `dim_month_year`: Cho phép lọc và nhóm dữ liệu theo các chu kỳ thời gian vĩ mô phục vụ báo cáo tổng hợp.

9.1.5.b Thuật toán và mô hình tính toán

Thay vì chỉ đơn thuần đếm số lượng sự cố, nhóm nghiệp vụ A5 áp dụng phương pháp tính toán dựa trên trọng số nhằm phản ánh chính xác hơn mức độ ảnh hưởng của từng sự cố. Thuật toán cốt lõi được sử dụng là chỉ số Mật độ sự cố có trọng số (Weighted Incident Density – WID).

Chỉ số cường độ sự cố (Incident Intensity Index – III)

Mỗi sự cố i được gán một trọng số W_i , phản ánh mức độ tác động tổng hợp của sự cố đó dựa trên mức độ nghiêm trọng của bản thân sự cố và điều kiện thời tiết tại thời điểm xảy ra. Trọng số được xác định theo công thức:

$$W_i = (Severity_Level_{incident} \times \alpha) + (Severity_Level_{weather} \times \beta)$$

Trong đó:

- $Severity_Level_{incident}$ là mức độ nghiêm trọng của sự cố giao thông.
- $Severity_Level_{weather}$ là mức độ nghiêm trọng của điều kiện thời tiết tương ứng.
- α, β là các hệ số điều chỉnh theo quy tắc nghiệp vụ, cho phép ưu tiên các sự cố có tác động lớn hơn đến an toàn và lưu thông (ví dụ: tai nạn giao thông được gán trọng số cao hơn so với sự cố xe hỏng).

Tính toán mật độ sự cố theo không gian

Để xác định các “điểm đen” giao thông trên từng phân đoạn đường, hệ thống tính toán tổng trọng số sự cố tích lũy và chuẩn hóa theo chiều dài phân đoạn. Chỉ số Mật độ sự cố có trọng số cho mỗi phân đoạn được xác định như sau:

$$WID_{segment} = \frac{\sum_{i=1}^n W_i}{Length_m}$$

Trong đó:

- n là tổng số sự cố xảy ra trên phân đoạn đường trong một chu kỳ phân tích (tháng hoặc quý).
- $Length_m$ là chiều dài vật lý của phân đoạn đường, lấy từ bảng `dim_segment`.

Giá trị $WID_{segment}$ càng lớn cho thấy phân đoạn đường đó có mật độ và mức độ nghiêm trọng sự cố càng cao, qua đó được xác định là một điểm nóng cần ưu tiên trong công tác giám sát, cải tạo hạ tầng hoặc triển khai các biện pháp an toàn giao thông.

9.1.6 A6: Phân tích tác động của thời tiết đến tốc độ

Nhóm nghiệp vụ A6 tập trung vào việc phân tích và định lượng hóa tác động của các yếu tố thời tiết đến tốc độ và năng lực thông hành của dòng xe. Các hiện tượng ngoại cảnh như mưa, sương mù hoặc bão có ảnh hưởng trực tiếp đến độ bám đường, tầm nhìn và hành vi lái xe, từ đó làm suy giảm vận tốc khai thác của hạ tầng giao thông. Mục tiêu của nhóm nghiệp vụ này là xây dựng các chỉ số định lượng phản ánh mức độ suy giảm tốc độ dưới tác động của thời tiết, đồng thời làm cơ sở cho việc dự báo và hỗ trợ ra quyết định trong quản lý giao thông.

9.1.6.a Thành phần dữ liệu trong Schema

Để thực hiện phân tích mối tương quan giữa thời tiết và tốc độ giao thông, hệ thống kết hợp dữ liệu từ các bảng Fact đo lường vận tốc và bảng Dimension mô tả điều kiện thời tiết.

Bảng Fact chính: `fact_weather_impact`

Bảng `fact_weather_impact` là bảng Fact chuyên biệt được thiết kế cho nhóm nghiệp vụ A6, lưu trữ các chỉ số vận tốc trong những điều kiện thời tiết khác nhau. Các thuộc tính chính bao gồm:

- `segment_key`: Khóa ngoại định danh phân đoạn đường chịu ảnh hưởng của điều kiện thời tiết.
- `weather_key`: Khóa ngoại xác định trạng thái thời tiết tại thời điểm đo (ví dụ: trời quang, mưa, sương mù).
- `baseline_speed_kmh`: Tốc độ trung bình mang tính chuẩn (baseline) của phân đoạn đường trong điều kiện thời tiết thuận lợi.

- `actual_speed_kmh`: Tốc độ trung bình thực tế được ghi nhận trong điều kiện thời tiết đang xét.
- `speed_reduction_pct`: Tỷ lệ suy giảm tốc độ đã được tính toán sẵn trong quá trình ETL.
- `precipitation_mm`: Lượng mưa đo được (đơn vị: mm), phục vụ cho các phân tích định lượng theo cường độ mưa.

Bảng Fact hỗ trợ: `fact_traffic_flow`

Bảng `fact_traffic_flow` cung cấp các chỉ số vận tốc nền để so sánh:

- `avg_speed_kmh`: Tốc độ trung bình thực tế của dòng xe.
- `free_flow_speed_kmh`: Tốc độ dòng xe trong điều kiện thông thoáng.

Bảng Dimension thời tiết: `dim_weather`

Bảng `dim_weather` mô tả chi tiết các đặc tính của điều kiện thời tiết:

- `weather_type`: Tên hiển thị của loại thời tiết (ví dụ: “Mưa rào nặng hạt”).
- `severity_level`: Mức độ nghiêm trọng của thời tiết, được chuẩn hóa theo thang giá trị từ 1 đến 5.
- `road_friction_impact`: Thuộc tính mô tả ảnh hưởng của thời tiết đến độ bám mặt đường (ví dụ: trơn trượt).

9.1.6.b Thuật toán và mô hình tính toán

Trọng tâm của nhóm nghiệp vụ A6 là việc tính toán tỷ lệ suy giảm vận tốc (Speed Reduction Percentage – SRP) và xây dựng mô hình tương quan giữa cường độ thời tiết và vận tốc khai thác của hạ tầng.

Xác định vận tốc Baseline

Thay vì sử dụng một giá trị vận tốc cố định, hệ thống xác định vận tốc baseline V_{base} một cách động, dựa trên dữ liệu lịch sử của chính phân đoạn đường đó trong điều kiện thời tiết thuận lợi (trời quang), tại cùng khung thời gian trong ngày. Cụ thể:

$$V_{base}(s, t) = \mathbb{E}[\text{avg_speed_kmh} \mid \text{Weather} = \text{CLEAR}]$$

Trong đó s là phân đoạn đường (`segment_key`) và t là khung thời gian quan sát.

Công thức tính tỷ lệ suy giảm vận tốc

Tỷ lệ suy giảm vận tốc được tính cho từng cặp $\{\text{segment_key}, \text{weather_type}\}$ theo công thức:

$$SRP(\%) = \frac{V_{base} - V_{actual}}{V_{base}} \times 100\%$$

Trong đó V_{actual} là tốc độ trung bình thực tế được ghi nhận trong điều kiện thời tiết đang xét. Giá trị SRP càng lớn phản ánh mức độ ảnh hưởng càng nghiêm trọng của thời tiết đến năng lực thông hành của đoạn đường.

Mô hình tương quan giữa thời tiết và vận tốc

Để phân tích sâu hơn mối quan hệ định lượng giữa cường độ mưa và tốc độ giao thông, hệ thống áp dụng mô hình hồi quy tuyến tính đơn giản:

$$V = V_{base} - (\gamma \times P)$$

Trong đó:

- V là vận tốc dự kiến của dòng xe.
- P là lượng mưa đo được (mm).

- γ là hệ số suy giảm, phản ánh mức độ nhạy cảm của vận tốc đối với lượng mưa.

Mô hình này cho phép ước lượng trước sự suy giảm tốc độ dựa trên thông tin dự báo thời tiết từ các API khí tượng, qua đó hỗ trợ hệ thống giao thông thông minh trong việc cảnh báo, điều tiết và tối ưu hóa luồng di chuyển trong điều kiện thời tiết bất lợi.

9.1.7 A7: Đánh giá hiệu quả hệ thống gợi ý

Nhóm nghiệp vụ A7 tập trung vào việc đánh giá mức độ hiệu quả của hệ thống gợi ý lộ trình dựa trên trí tuệ nhân tạo, thông qua việc khai thác dữ liệu trong kho dữ liệu giao thông. Mục tiêu trọng tâm của nghiệp vụ này là định lượng hóa mức độ chấp nhận, tuân thủ và mức độ tin tưởng của người dùng đối với các lộ trình được hệ thống đề xuất, từ đó làm cơ sở cho việc cải tiến thuật toán gợi ý và hỗ trợ ra quyết định ở cấp quản trị.

9.1.7.a Thành phần dữ liệu trong Schema

Để đo lường hiệu quả của hệ thống gợi ý, kho dữ liệu tập trung vào các bảng Fact phản ánh hành vi sử dụng lộ trình của người dùng và kết quả phản hồi sau chuyến đi, kết hợp với bảng Dimension thời gian nhằm phục vụ phân tích xu hướng.

Bảng Fact chính: `fact_trip_recommendation`

Bảng `fact_trip_recommendation` lưu trữ dữ liệu ở mức hạt (grain) là mỗi chuyến đi có áp dụng gợi ý lộ trình, với các thuộc tính chính như sau:

- `is_followed`: Biến logic xác định người dùng có tuân thủ hoàn toàn lộ trình do hệ thống đề xuất hay không.
- `deviation_count`: Số lần người dùng đi chệch khỏi lộ trình gợi ý trong suốt chuyến đi, phản ánh mức độ tái định tuyến.
- `time_diff_min`: Độ lệch thời gian (tính bằng phút) giữa thời gian di chuyển thực tế và thời gian dự báo bởi hệ thống AI.
- `satisfaction_score`: Điểm đánh giá chủ quan của người dùng đối với lộ trình được đề xuất, theo thang đo từ 1 đến 5.
- `date_key`: Khóa ngoại liên kết đến bảng Dimension thời gian, phục vụ tổng hợp và phân tích theo ngày.

Bảng Dimension thời gian: `dim_date`

Bảng `dim_date` cung cấp các thuộc tính thời gian hỗ trợ phân tích hiệu quả của hệ thống gợi ý theo bối cảnh sử dụng:

- `date_key`: Khóa chính dùng để liên kết với bảng Fact.
- `is_weekend`, `is_holiday`: Các biến phân loại nhằm đánh giá sự khác biệt về hiệu quả gợi ý giữa ngày thường và các ngày đặc biệt như cuối tuần hoặc ngày lễ.

9.1.7.b Thuật toán và mô hình tính toán

Việc đánh giá hiệu quả của hệ thống gợi ý được thực hiện thông qua các chỉ số định lượng, phản ánh đồng thời hành vi tuân thủ của người dùng và độ chính xác kỹ thuật của mô hình dự báo.

Tỷ lệ Tuân thủ Lộ trình (Route Compliance Rate – RCR)

Tỷ lệ tuân thủ lộ trình là chỉ số trực tiếp phản ánh mức độ tin tưởng của người dùng đối với các đề xuất của hệ thống. Chỉ số này được tính cho từng ngày d như sau:

$$RCR_d = \left(\frac{\sum(is_followed = TRUE)}{Total_Trips_d} \right) \times 100\%$$

Trong đó $Total_Trips_d$ là tổng số chuyến đi có áp dụng gợi ý lộ trình trong ngày d . Giá trị RCR càng cao cho thấy mức độ chấp nhận và tin tưởng của người dùng đối với hệ thống gợi ý càng lớn.

Chỉ số Độ lệch Hành trình (Deviation Index – DI)

Ngoài việc tuân thủ hoàn toàn lộ trình, hệ thống đo lường mức độ điều chỉnh của người dùng thông qua số lần tái định tuyến trung bình, được xác định như sau:

$$DI_d = \frac{\sum_{i=1}^n deviation_count_i}{Total_Trips_d}$$

Chỉ số DI phản ánh mức độ dao động trong hành vi di chuyển. Trường hợp RCR duy trì ở mức cao trong khi DI lớn cho thấy lộ trình tổng thể được chấp nhận, tuy nhiên vẫn tồn tại các phân đoạn cục bộ chưa tối ưu, buộc người dùng phải điều chỉnh liên tục.

Độ chính xác thời gian dự kiến (ETA Accuracy)

Độ chính xác của thời gian dự kiến (Estimated Time of Arrival – ETA) là yếu tố then chốt ảnh hưởng đến mức độ tin tưởng của người dùng. Sai số dự báo được đo lường thông qua sai số tuyệt đối trung bình (Mean Absolute Error – MAE):

$$MAE_{ETA} = \frac{1}{n} \sum_{i=1}^n |time_diff_min_i|$$

Giá trị MAE_{ETA} càng nhỏ cho thấy khả năng dự báo thời gian của hệ thống càng chính xác, từ đó nâng cao hiệu quả kỹ thuật và độ tin cậy của hệ thống gợi ý lộ trình.

9.1.8 A8: Thu thập phản hồi chuyến đi

Nhóm nghiệp vụ A8 tập trung vào việc thu thập và khai thác phản hồi sau chuyến đi của người dùng nhằm đánh giá mức độ sai lệch giữa kết quả dự báo trước chuyến đi và kết quả thực tế ghi nhận sau chuyến đi. Mục tiêu cốt lõi của nghiệp vụ này là hình thành một vòng lặp phản hồi, qua đó cung cấp dữ liệu đầu vào có giá trị cho quá trình tinh chỉnh và tái huấn luyện các mô hình học máy trong hệ thống gợi ý lộ trình.

9.1.8.a Thành phần dữ liệu trong Schema

Để thực hiện đối chiếu và phân tích sai số dự báo ở cả mức độ chuyến đi và mức độ phân đoạn, hệ thống khai thác dữ liệu từ các bảng Fact chính và Fact chi tiết, kết hợp với các bảng Dimension phục vụ phân tích hành vi người dùng.

Bảng Fact chính: `fact_trip_recommendation`

Bảng `fact_trip_recommendation` đóng vai trò là bảng cha, lưu trữ dữ liệu tổng hợp ở mức hạt là từng chuyến đi hoàn chỉnh, với các thuộc tính quan trọng như sau:

- `trip_recom_key`: Khóa chính định danh duy nhất cho mỗi chuyến đi có áp dụng hệ thống gợi ý.
- `actual_travel_time_min`: Tổng thời gian di chuyển thực tế của người dùng, tính bằng phút.
- `predicted_duration_min`: Thời gian di chuyển dự kiến ban đầu do hệ thống AI tính toán trước chuyến đi.
- `time_diff_min`: Độ lệch thời gian được tính trong quá trình ETL, xác định bằng hiệu số giữa thời gian thực tế và thời gian dự báo.

- `satisfaction_score`: Điểm đánh giá chủ quan của người dùng (từ 1 đến 5 sao), dùng để xác định mức độ chấp nhận sai số dự báo.

Bảng Fact chi tiết: `fact_segment_performance`

Bảng `fact_segment_performance` là bảng con, lưu trữ dữ liệu ở mức hạt chi tiết hơn, tương ứng với từng phân đoạn đường trong chuyến đi:

- `travel_time_key`: Khóa chính đại diện cho một bản ghi phân tích hiệu suất tại mức phân đoạn.
- `waiting_time_sec`: Thời gian phương tiện đứng yên hoặc di chuyển rất chậm tại phân đoạn, phản ánh tình trạng ùn tắc cục bộ.
- `moving_time_sec`: Thời gian phương tiện thực sự di chuyển trên phân đoạn, dùng để đối chiếu với tốc độ thiết kế hoặc tốc độ dự báo.

Bảng Dimension người dùng: `dim_user`

Bảng `dim_user` cung cấp khóa định danh người dùng (`user_key`), cho phép phân tích phản hồi theo từng nhóm người dùng, chẳng hạn như người dùng thường xuyên hoặc người dùng mới.

9.1.8.b Thuật toán và mô hình tính toán

Các thuật toán trong nhóm nghiệp vụ A8 tập trung vào việc đo lường và phân tích sai số dự báo, từ đó xác định các điểm yếu trong mô hình định tuyến và cung cấp dữ liệu cho quá trình tối ưu hóa.

Sai số phần trăm tuyệt đối trung bình trên từng chuyến đi

Độ chính xác của thuật toán dự báo thời gian di chuyển được đánh giá thông qua chỉ số sai số phần trăm tuyệt đối trung bình (Mean Absolute Percentage Error – MAPE) tại mức chuyến đi:

$$MAPE_{trip} = \left| \frac{Actual_Time - Predicted_Time}{Actual_Time} \right| \times 100\%$$

Chỉ số này cho phép hệ thống xác định liệu sai số dự báo có nằm trong ngưỡng kỹ thuật cho phép (ví dụ nhỏ hơn 15%) hay phản ánh sự suy giảm hiệu năng của mô hình AI trong một số bối cảnh giao thông cụ thể.

Phân tích nguyên nhân gốc rễ sai số (Root Cause Analysis – RCA)

Đối với các chuyến đi có giá trị `time_diff_min` vượt quá ngưỡng quy định, hệ thống thực hiện truy vấn chi tiết xuống bảng `fact_segment_performance` nhằm xác định các phân đoạn đường đóng góp chính vào sai số dự báo. Mức độ đóng góp sai số của từng phân đoạn được xác định như sau:

$$Error_Contribution = \frac{\sum (Segment_Actual - Segment_Predicted)}{Total_Time_Difference}$$

Kết quả phân tích này cho phép hệ thống nhận diện các “đoạn đường biến động” có tần suất gây sai số cao, từ đó làm cơ sở để điều chỉnh trọng số, cập nhật tham số hoặc tái huấn luyện mô hình dự báo trong các vòng lặp học máy tiếp theo.

9.1.9 A9: Phân bổ lưu lượng theo loại xe

Nhóm nghiệp vụ A9 tập trung vào việc phân tích cơ cấu thành phần phương tiện trong dòng giao thông nhằm hỗ trợ các quyết định điều hành và tổ chức giao thông, bao gồm phân làn chức năng, áp dụng khung giờ cấm xe và điều tiết các phương tiện có tải trọng lớn. Việc hiểu rõ tỷ trọng và mức độ chiếm dụng của từng loại xe là cơ sở quan trọng để đánh giá năng lực thông hành thực tế của hạ tầng giao thông.

9.1.9.a Thành phần dữ liệu trong Schema

Để phân tích thành phần dòng xe một cách định lượng và chính xác, hệ thống khai thác dữ liệu đếm phương tiện chi tiết từ bảng Fact lưu lượng, kết hợp với các thuộc tính quy đổi và thông tin hạ tầng từ các bảng Dimension liên quan.

Bảng Fact chính: `fact_traffic_flow`

Bảng `fact_traffic_flow` lưu trữ các chỉ số đo lường lưu lượng phương tiện tại từng phân đoạn đường và khung thời gian, với các thuộc tính chính như sau:

- `motorcycle_count`: Số lượng xe máy được ghi nhận.
- `car_count`: Số lượng xe ô tô con (dưới 9 chỗ).
- `heavy_vehicle_count`: Số lượng xe tải, xe buýt và xe container, là các phương tiện có mức độ cản trở giao thông cao.
- `other_count`: Số lượng các phương tiện khác hoặc không xác định.
- `total_vehicle_count`: Tổng số lượng phương tiện thực tế ghi nhận được.
- `total_pcu`: Tổng số đơn vị xe con quy đổi (Passenger Car Unit – PCU), phản ánh tải trọng giao thông tổng hợp.
- `segment_key`: Khóa định danh phân đoạn đường đang được phân tích.

Bảng Dimension phương tiện: `dim_vehicle_type`

Bảng `dim_vehicle_type` cung cấp thông tin phân loại và hệ số quy đổi cho từng loại phương tiện:

- `vehicle_name`: Tên hiển thị của loại phương tiện (ví dụ: xe máy, xe tải).
- `category`: Phân nhóm tổng quát như *2-wheel*, *4-wheel* hoặc *Heavy*.
- `pcu_factor`: Hệ số quy đổi đơn vị xe con, phản ánh mức độ chiếm dụng không gian và tác động đến dòng xe.

Bảng Dimension hạ tầng: `dim_way`

Bảng `dim_way` cung cấp các thông tin về năng lực hạ tầng nhằm đối chiếu với cơ cấu dòng xe:

- `default_lane_count`: Số làn xe hiện hữu trên đoạn đường.
- `way_type`: Loại hình đường (quốc lộ, đường đô thị nội bộ, đường vành đai), làm cơ sở áp dụng các quy định điều tiết phương tiện theo thời gian.

9.1.9.b Thuật toán và mô hình tính toán

Các thuật toán trong nhóm nghiệp vụ A9 tập trung vào việc lượng hóa tỷ trọng của từng loại phương tiện trong dòng xe cũng như mức độ đóng góp của chúng vào tải trọng giao thông tổng thể.

Tỷ trọng thành phần phương tiện (Vehicle Mix Percentage – VMP)

Tỷ trọng của từng loại phương tiện i trên tổng lưu lượng tại một phân đoạn đường được xác định như sau:

$$VMP_i = \frac{Count_i}{Total_Vehicle_Count} \times 100\%$$

Chỉ số này cho phép nhà quản lý giao thông nhận diện loại phương tiện chiếm ưu thế trên từng tuyến đường, ví dụ như trường hợp xe máy chiếm tỷ trọng lớn trên các trục giao thông nội đô.

Chỉ số đóng góp tải trọng (PCU Contribution Index)

Do mức độ ảnh hưởng của các loại phương tiện đến khả năng thông hành là không đồng đều, hệ thống sử dụng chỉ số đóng góp PCU để đo lường “không gian chiếm dụng” thực tế của từng loại xe trong dòng giao thông:

$$PCU_Impact_i = \frac{Count_i \times PCU_Factor_i}{Total_PCU} \times 100\%$$

Trong đó, hệ số PCU_Factor được lấy từ bảng dim_vehicle_type (ví dụ: xe buýt có hệ số PCU lớn hơn xe con). Chỉ số này giúp định lượng chính xác mức độ tác động của từng nhóm phương tiện đến tình trạng ùn tắc, ngay cả khi số lượng tuyệt đối của chúng không lớn.

9.2 Nhóm nghiệp vụ Phân tích dự báo (Predictive Analytics)

Nhóm này không chỉ truy xuất dữ liệu quá khứ mà còn sử dụng các kỹ thuật Học máy (Machine Learning) để sinh ra tri thức mới (Insight) về trạng thái giao thông trong tương lai. Dưới đây là đặc tả kỹ thuật cho từng nghiệp vụ dự báo.

9.2.1 Nghiệp vụ B1: Dự báo tốc độ ngắn hạn (Short-term Speed Prediction)

9.2.1.a Mục đích và phạm vi

Mục tiêu là dự đoán vận tốc trung bình (avg_speed_kmh) cho từng đoạn đường (segment_key) trong khoảng thời gian tương lai (T+15, T+30, T+60 phút). Kết quả dự báo là đầu vào quan trọng cho việc cảnh báo ùn tắc sớm.

9.2.1.b Ánh xạ dữ liệu nguồn

Mô hình dự báo sẽ sử dụng dữ liệu lịch sử làm tập huấn luyện (Training Set). Các bảng và trường dữ liệu cụ thể bao gồm:

- Bảng Fact chính:** fact_traffic_flow (Lịch sử lưu lượng)
 - Biến mục tiêu (Target Variable):** avg_speed_kmh (tại thời điểm tương lai).
 - Biến trễ (Lag features):** Sử dụng avg_speed_kmh và vehicle_count tại các thời điểm quá khứ ($t - 1$, $t - 2$, $t - 3$) để nắm bắt xu hướng tăng/giảm của dòng xe.
- Bảng Dimension ngữ cảnh:** dim_time & dim_date
 - Tính chu kỳ:** Sử dụng hour_of_day (ví dụ: 17h chiều thường tắc đường) và day_of_week (Chủ nhật khác thứ Hai).
 - Sự kiện đặc biệt:** Sử dụng trường is_holiday trong dim_date để điều chỉnh dự báo vào ngày lễ.
- Bảng Fact tác động:** fact_weather_impact
 - Yếu tố ngoại sinh:** Sử dụng precipitation_mm (lượng mưa) và visibility_m (tầm nhìn) làm biến đầu vào. Mưa lớn thường làm giảm tốc độ trung bình từ 10-20%.
- Bảng Dimension không gian:** dim_segment
 - Đặc trưng tĩnh:** Sử dụng lane_count (số làn) và road_type để mô hình hiểu được năng lực thông hành của đoạn đường.

9.2.1.c Logic xử lý và Thuật toán

1. **Bước 1 - Data Preparation:** Thực hiện truy vấn JOIN giữa `fact_traffic_flow` và `fact_weather_impact` qua `segment_key` và `time_key`.
2. **Bước 2 - Feature Engineering:** Tạo cửa sổ trượt (Sliding Window) để chuyển đổi dữ liệu chuỗi thời gian thành dạng Supervised Learning (X : tốc độ 30 phút trước $\rightarrow Y$: tốc độ 15 phút tới).
3. **Bước 3 - Modeling:** Áp dụng thuật toán **LSTM (Long Short-Term Memory)** hoặc **XGBoost**. Mô hình sẽ học mối tương quan phi tuyến tính: “Nếu là 17h thứ Sáu và có mưa 20mm trên đường Xô Viết Nghệ Tĩnh, tốc độ sẽ giảm xuống còn 12km/h”.

9.2.2 Nghiệp vụ B2: Dự báo thời gian hành trình (ETA Prediction)

9.2.2.a Mục đích và Phạm vi

Cung cấp thời gian di chuyển dự kiến (Estimated Time of Arrival) cho một lộ trình tổng thể (Route) bao gồm nhiều đoạn đường (Segment) nối tiếp nhau.

9.2.2.b Ánh xạ dữ liệu nguồn

Nghiệp vụ này là bài toán tổng hợp (Aggregation) dựa trên kết quả của B1 và cấu trúc mạng lưới đường.

- **Bảng cầu nối:** `bridge_route_segment` (Giả định thiết kế bảng Bridge)
 - Trường sử dụng: `route_key`, `segment_key`, `sequence_order` (thứ tự đoạn đường trong lộ trình).
 - Vai trò: Xác định lộ trình $A \rightarrow B$ bao gồm những đoạn đường nào.
- **Bảng Dimension:** `dim_segment`
 - Trường sử dụng: `length_m` (chiều dài đoạn đường).
- **Kết quả từ B1 (Dự báo tốc độ):**
 - Sử dụng tốc độ dự báo `predicted_speed_kmh` cho từng `segment_key`.

9.2.2.c Logic xử lý và Thuật toán

Khác với các ứng dụng bản đồ thông thường chỉ lấy tốc độ hiện tại, hệ thống này áp dụng thuật toán **Time-Dependent Routing** (Định tuyến phụ thuộc thời gian):

1. **Phân rã lộ trình:** Truy vấn bảng `bridge_route_segment` để lấy danh sách các đoạn đường $S_1, S_2, \dots, S_n$ theo thứ tự.
2. **Tính toán động:**
 - Thời gian qua đoạn S_1 (T_1) = Chiều dài S_1 / Tốc độ dự báo tại thời điểm xuất phát t_0 .
 - Thời gian qua đoạn S_2 (T_2) = Chiều dài S_2 / Tốc độ dự báo tại thời điểm $(t_0 + T_1)$.
 - Lưu ý: Tốc độ tại S_2 phải là tốc độ dự báo trong tương lai, không phải hiện tại.
3. **Tổng hợp:**

$$ETA_{total} = \sum_{i=1}^n T_i$$

Logic này giúp loại bỏ sai số khi người dùng bắt đầu đi lúc đường thoáng nhưng đến giữa đường thì gặp giờ cao điểm.

9.2.3 Nghiệp vụ B3: Dự báo rủi ro sự cố (Incident Risk Prediction)

9.2.3.a Mục đích và Phạm vi

Dự đoán xác suất xảy ra tai nạn hoặc sự cố giao thông (Incident Probability) tại một thời điểm và địa điểm cụ thể, nhằm cảnh báo cho trung tâm điều hành.

9.2.3.b Ánh xạ dữ liệu nguồn

Đây là bài toán Phân lớp (Classification) hoặc Hồi quy (Regression) dựa trên các mẫu hình tai nạn trong quá khứ.

- **Bảng Fact sự kiện:** fact_incident (Lịch sử sự cố)
 - *Vai trò:* Cung cấp nhãn (Label) cho mô hình học máy (Có tai nạn / Không có tai nạn).
 - *Trường sử dụng:* incident_type, severity_level (để phân loại mức độ rủi ro).
- **Bảng Fact tác động:** fact_weather_impact
 - *Biến nguyên nhân:* precipitation_mm, visibility_m, surface_condition.
 - *Logic:* Trời mưa lớn + Tầm nhìn thấp = Tăng nguy cơ va chạm.
- **Bảng Fact lưu lượng:** fact_traffic_flow
 - *Biến nguyên nhân:* vehicle_count (Mật độ). Mật độ quá cao gây va quệt nhẹ, mật độ quá thấp nhưng tốc độ cao gây tai nạn nghiêm trọng.
- **Bảng Dimension phương tiện:** dim_vehicle_type (Tham chiếu gián tiếp)
 - Hệ thống phân tích tỷ lệ xe tải/container (vehicle_code thuộc nhóm Heavy) trong dòng xe. Tỷ lệ xe lớn cao làm tăng chỉ số rủi ro.

9.2.3.c Logic xử lý và Thuật toán

1. **Xây dựng tập dữ liệu huấn luyện:** Kết nối (Join) fact_incident với dữ liệu thời tiết và lưu lượng tại cùng thời điểm xảy ra tai nạn để tạo thành các "Hồ sơ rủi ro" (Risk Profiles). Đồng thời, lấy mẫu các thời điểm bình thường (Negative samples) để cân bằng dữ liệu.
2. **Mô hình hóa:** Sử dụng thuật toán **Random Forest** hoặc **Logistic Regression** để tính toán xác suất (P).
 - Đầu vào: $X = \{ \text{Lượng mưa, Mật độ xe, Giờ trong ngày, Loại đường} \}$.
 - Đầu ra: $Y = P(\text{Incident})$.
3. **Phân ngưỡng cảnh báo:**
 - Nếu $P > 0.8$: Cảnh báo Đỏ (Nguy cơ cao).
 - Nếu $0.5 < P \leq 0.8$: Cảnh báo Vàng (Cần chú ý).

9.3 Nhóm nghiệp vụ Hỗ trợ ra quyết định (Prescriptive Analytics)

Đây là cấp độ cao nhất trong tháp phân tích dữ liệu, không chỉ dự báo tương lai mà còn trực tiếp đề xuất hành động tối ưu cho người dùng. Các nghiệp vụ này yêu cầu sự phối hợp phức tạp giữa các bảng dữ liệu lịch sử, dữ liệu dự báo và thuật toán tối ưu hóa đồ thị.

9.3.1 Nghiệp vụ C1: Đề xuất lộ trình tối ưu đa mục tiêu (Multi-objective Routing)

9.3.1.a Mục đích và Phạm vi

Khác với các ứng dụng dẫn đường truyền thống chỉ tập trung vào "Thời gian ngắn nhất", nghiệp vụ này giải quyết bài toán đánh đổi giữa các yếu tố: **Thời gian** (Speed), **Độ tin cậy** (Reliability - tránh các đường có thời gian di chuyển biến động thất thường) và **An toàn** (Safety - tránh khu vực ngập nước hoặc hay tai nạn).

9.3.1.b Ánh xạ dữ liệu nguồn (Data Mapping)

Hệ thống xây dựng hàm trọng số chi phí (Cost Function) dựa trên các bảng sau:

- **Yếu tố Thời gian (Time Cost):**
 - *Nguồn:* Kết quả dự báo từ nghiệp vụ B1 (sử dụng `fact_traffic_flow`).
 - *Trường dữ liệu:* `predicted_speed_kmh` trên từng `segment_key`.
- **Yếu tố Tin cậy (Reliability Cost):**
 - *Nguồn:* `fact_traffic_flow` (Lịch sử).
 - *Logic:* Tính toán độ lệch chuẩn (Standard Deviation) của vận tốc trong cùng khung giờ quá khứ.
 - *Trường phái sinh:* Nếu độ lệch chuẩn cao $\rightarrow$ Độ tin cậy thấp (phạt điểm đoạn đường này).
- **Yếu tố An toàn (Safety Risk Cost):**
 - *Nguồn 1:* `fact_incident`. Tính tần suất tai nạn (`count`) và mức độ nghiêm trọng (`severity_level`) trên đoạn đường.
 - *Nguồn 2:* `fact_weather_impact`. Nếu `precipitation_mm` $>$ 50mm (ngập), gán trọng số rủi ro cực đại cho các đoạn đường trùng (dựa trên thuộc tính địa hình trong `dim_segment` nếu có, hoặc lịch sử ngập).

9.3.1.c Logic xử lý và Thuật toán

Sử dụng thuật toán tìm đường **A*** (A-Star) hoặc **Dijkstra** với hàm chi phí tổng quát:

$$Cost_{total} = w_1 \cdot T_{dbo} + w_2 \cdot \sigma_{bin\phi ng} + w_3 \cdot (R_{sc} + R_{thitit})$$

Trong đó w_1, w_2, w_3 là các trọng số do người dùng tùy chỉnh (Ví dụ: Người dùng chọn chế độ "An toàn" thì w_3 sẽ tăng cao).

1. **Bước 1:** Truy vấn dữ liệu từ Data Warehouse để gán trọng số cho cạnh của đồ thị giao thông.

2. **Bước 2:** Chạy thuật toán tìm đường trên đồ thị đã gán trọng số.

3. **Bước 3:** Trả về 3 phương án:

- *Phương án 1 (Nhanh nhất):* Tối ưu hóa w_1 .
- *Phương án 2 (Tin cậy):* Tối ưu hóa w_2 (thường là các đường lớn, ít đèn đỏ, ít kẹt xe bất ngờ).
- *Phương án 3 (An toàn):* Tối ưu hóa w_3 (tránh đường đang ngập hoặc hay có tai nạn).

9.3.2 Nghiệp vụ C2: Cảnh báo và Tái định tuyến thời gian thực (Real-time Rerouting)

9.3.2.a Mục đích và Phạm vi

Hệ thống giám sát hành trình của người dùng và chủ động phát hiện sự cố mới phát sinh trên lộ trình dự kiến để gợi ý hướng đi vòng (Detour) ngay lập tức.

9.3.2.b Ánh xạ dữ liệu nguồn

Nghiệp vụ này hoạt động theo cơ chế hướng sự kiện (Event-driven):

- **Trigger (Kích hoạt):** Một bản ghi mới được INSERT vào bảng `fact_incident`.
 - *Trường quan trọng:* `segment_key` (vị trí sự cố), `incident_type` (loại sự cố), `is_active` = TRUE.
- **Đánh giá tác động:**
 - Dựa trên `dim_segment` để xác định các đoạn đường lân cận chịu ảnh hưởng lan truyền.
 - Dựa trên `bridge_route_segment` (tạm thời trong bộ nhớ) để xác định các User đang có lộ trình đi qua đoạn đường bị lỗi.

9.3.2.c Logic xử lý và Thuật toán

1. **Monitoring:** Một quy trình (Daemon) liên tục lắng nghe thay đổi trên bảng `fact_incident`.
2. **Impact Analysis:** Khi có sự cố tại đoạn $S_{blocked}$:
 - Tìm tất cả các lộ trình active R_i mà $S_{blocked} \in R_i$.
3. **Rerouting:** Với mỗi lộ trình bị ảnh hưởng, thực hiện tìm đường lại (Re-calculation) với điều kiện:

$$Graph_{new} = Graph_{original} \setminus \{S_{blocked}\}$$

(Loại bỏ cạnh bị sự cố ra khỏi đồ thị).

4. **Notification:** Gửi thông báo đẩy: "*Phát hiện tai nạn phía trước 2km. Lộ trình mới qua đường Nguyễn Thị Minh Khai nhanh hơn 10 phút.*"

9.3.3 Nghiệp vụ C3: Gợi ý giờ xuất phát tối ưu (Optimal Departure Time)

9.3.3.a Mục đích và Phạm vi

Thay vì chỉ trả lời "Đi đường nào?", hệ thống trả lời "Đi lúc nào?". Nghiệp vụ này phân tích xu hướng tắc nghẽn trong cửa sổ thời gian (ví dụ: 2 giờ tối) để khuyến nghị người dùng dời giờ khởi hành nhằm tránh kẹt xe.

9.3.3.b Ánh xạ dữ liệu nguồn

- **Không gian tìm kiếm:** 1 cặp O-D (Origin - Destination) cố định.
- **Thời gian tìm kiếm:** $T_{start} \in [T_{now}, T_{now} + 120 \text{ phút}]$.
- **Dữ liệu đầu vào:**
 - Kết quả dự báo B1 cho chuỗi thời gian liên tục.
 - `dim_time_of_day`: Dùng để hiển thị trực thời gian trực quan cho người dùng.

9.3.3.c Logic xử lý và Thuật toán

1. **Bước 1 - Mô phỏng đa kịch bản:** Hệ thống thực hiện bài toán dự báo thời gian hành trình (B2 - ETA Prediction) lặp lại N lần với các mốc thời gian xuất phát khác nhau (ví dụ: cứ mỗi 15 phút).
 - ETA_{t0} : Xuất phát ngay bây giờ.
 - ETA_{t+15} : Xuất phát sau 15 phút.
 - ...

- ETA_{t+120} : Xuất phát sau 2 tiếng.

2. **Bước 2 - Tìm cực trị:** So sánh các giá trị ETA .

$$T_{optimal} = \arg \min_t (ETA_t + Penalty(t))$$

(Lưu ý: $Penalty$ là hàm phạt nếu phải chờ đợi quá lâu, tránh việc hệ thống khuyến người dùng chờ... 3 tiếng để nhanh hơn 5 phút).

3. **Bước 3 - Trực quan hóa:** Vẽ biểu đồ cột so sánh thời gian di chuyển, làm nổi bật cột có thời gian thấp nhất để người dùng ra quyết định.

10 Thiết kế và triển khai quy trình trích xuất, biến đổi, nạp dữ liệu (ETL)

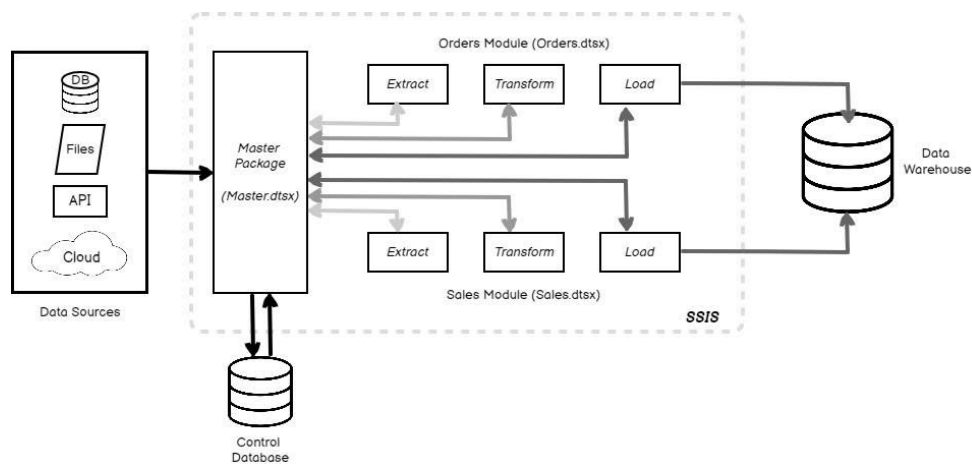
10.1 Kiến trúc tổng thể hệ thống ETL

Hệ thống ETL (Extract - Transform - Load) được thiết kế là trung tâm điều phối dữ liệu của toàn bộ dự án Traffic Data Warehouse. Kiến trúc này không chỉ đơn thuần là việc di chuyển dữ liệu, mà còn là quá trình chuẩn hóa không gian - thời gian, làm giàu dữ liệu bằng AI và tối ưu hóa cấu trúc lưu trữ để phục vụ các truy vấn phân tích phức tạp.

10.1.1 Mục tiêu và Nguyên tắc thiết kế kiến trúc

Hệ thống được xây dựng dựa trên các nguyên tắc thiết kế hiện đại nhằm đảm bảo tính ổn định và khả năng bảo trì lâu dài:

- **Tính Module hóa (Modularity):** Toàn bộ quy trình được chia nhỏ thành các Pipeline riêng biệt: Dimension Pipeline, Mock Data Pipeline và Fact Pipeline. Cách tiếp cận này giúp cô lập lỗi và cho phép thực thi từng phần dữ liệu độc lập theo nhu cầu.
- **Tính nhất quán dữ liệu (Data Atomicity):** Sử dụng các cơ chế Transaction và kỹ thuật "Upsert" (Update or Insert) thông qua câu lệnh `ON CONFLICT` để đảm bảo dữ liệu không bị trùng lặp và luôn duy trì trạng thái mới nhất ngay cả khi quy trình ETL bị gián đoạn.
- **Tính toàn vẹn không gian (Spatial Integrity):** Mọi dữ liệu giao thông từ API đều được ánh xạ (Mapping) chính xác vào hệ hạ tầng tính (Nodes, Segments) dựa trên các thuật toán hình học của PostGIS.
- **Khả năng mở rộng (Scalability):** Kiến trúc cho phép tích hợp thêm các nguồn dữ liệu mới như cảm biến IoT hoặc camera giao thông chỉ bằng cách bổ sung các module Service mà không cần thay đổi cấu trúc lõi của Pipeline.



Hình 39: Sơ đồ kiến trúc tổng thể hệ thống ETL

10.1.2 Mô hình phân lớp chức năng (Functional Layering)

Kiến trúc ETL được tổ chức thành 4 tầng chức năng chặt chẽ:

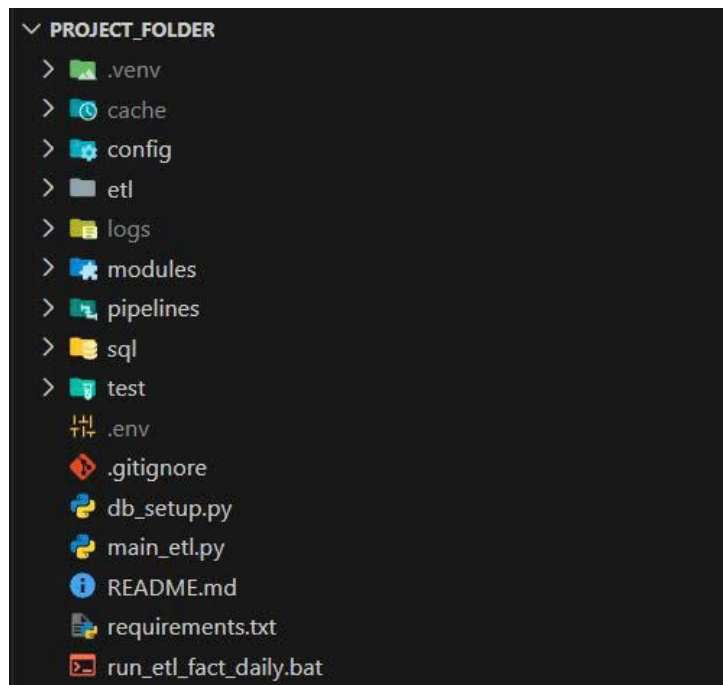
1. **Tầng thu thập (Extraction Layer):** Thực hiện kết nối và trích xuất dữ liệu đa nguồn: API thời gian thực (TomTom, OpenWeather), dữ liệu không gian (OpenStreetMap), dữ liệu nhận diện hình ảnh (YOLOv8) và dữ liệu tĩnh (CSV). Sử dụng cơ chế quản lý API Key và tham số hóa để tối ưu số lượng request (Free Tier giới hạn).

2. **Tầng biến đổi và làm giàu (Transformation & Enrichment Layer):** Thực hiện các phép tính toán nghiệp vụ giao thông: quy đổi lưu lượng xe sang đơn vị PCU, tính toán hướng đường (Azimuth) và xác định ranh giới hành chính bằng Spatial Join. Tích hợp mô hình học sâu YOLOv8 để chuyển đổi dữ liệu hình ảnh thô thành dữ liệu số lượng phương tiện định lượng.
3. **Tầng lưu trữ đích (Loading Layer):** Nạp dữ liệu vào PostgreSQL. Áp dụng các kỹ thuật quản trị hiệu năng: Đánh chỉ mục không gian GIST, chỉ mục văn bản GIN và phân vùng bảng Fact theo dải thời gian (Range Partitioning).
4. **Tầng điều phối và Giám sát (Orchestration Layer):** Sử dụng file trung tâm `main_etl.py` kết hợp tham số dòng lệnh `-mode` để điều khiển luồng chạy. Hệ thống hóa nhật ký vận hành (Logging) để theo dõi trạng thái và thời gian thực thi của từng tác vụ.

10.1.3 Hạ tầng công nghệ và Thư viện lỗi

Hệ thống tận dụng sức mạnh của hệ sinh thái Python 3.12 để xử lý dữ liệu:

- **Quản trị Cơ sở dữ liệu:** `psycopg2-binary` cung cấp giao thức kết nối hiệu năng cao tới PostgreSQL.
- **Xử lý Dữ liệu Không gian:** `OSMnx`, `GeoPandas` và `Shapely` là bộ ba công cụ dùng để trích xuất đồ thị giao thông và xử lý các đối tượng hình học phức tạp.
- **Xử lý AI và Thị giác máy tính:** `ultralytics` (YOLOv8) và `opencv-python-headless` được sử dụng để xây dựng bộ đếm phương tiện thông minh.
- **Xử lý Logic và API:** `pandas` cho các thao tác trên bảng dữ liệu lớn và `requests` để tương tác với các REST API bên ngoài.

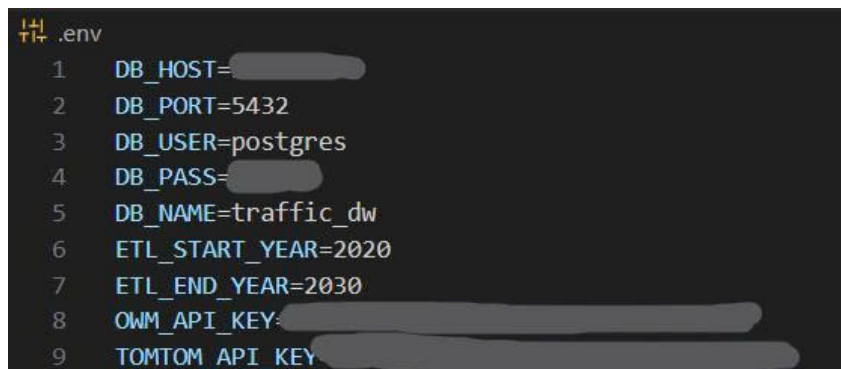


Hình 40: Cấu trúc thư mục dự án hệ thống ETL

10.1.4 Quản lý cấu hình và Biến môi trường (Environment Configuration)

Trong một hệ thống ETL hiện đại, việc tách biệt mã nguồn và cấu hình là nguyên tắc cốt lõi nhằm đảm bảo tính bảo mật và khả năng di động của hệ thống. Dự án triển khai cơ chế quản lý biến môi trường thông qua tệp `.env`, cho phép thay đổi tham số vận hành mà không cần can thiệp vào mã logic.

- **Bảo mật thông tin nhạy cảm:** Các thông tin như thông tin đăng nhập cơ sở dữ liệu (DB_USER, DB_PASS) và các khóa truy cập API (TOMTOM_API_KEY, OWM_API_KEY) được lưu trữ tập trung, tránh việc lộ lọt dữ liệu khi chia sẻ mã nguồn.
- **Tham số hóa phạm vi dữ liệu:** Các biến như ETL_START_YEAR và ETL_END_YEAR định nghĩa khung thời gian mà hệ thống sẽ khởi tạo dữ liệu nền, giúp dễ dàng điều chỉnh quy mô kho dữ liệu theo nhu cầu phân tích.
- **Công nghệ thực thi:** Hệ thống sử dụng thư viện python-dotenv để tự động tải các biến này vào môi trường chạy của Python ngay khi khởi động.



```

1 DB_HOST=
2 DB_PORT=5432
3 DB_USER=postgres
4 DB_PASS=
5 DB_NAME=traffic_dw
6 ETL_START_YEAR=2020
7 ETL_END_YEAR=2030
8 OWM_API_KEY=
9 TOMTOM_API_KEY=

```

Hình 41: Cấu hình các tham số trong tệp .env

10.1.5 Quản trị Kết nối và Tài nguyên Cơ sở dữ liệu (Connection Management)

Để tối ưu hóa việc tương tác với PostgreSQL và đảm bảo tài nguyên hệ thống được giải phóng đúng cách, dự án xây dựng lớp DatabaseConfig tập trung tại config/db_config.py.

- **Mô hình Singleton hóa kết nối:** Thay vì khởi tạo kết nối rải rác trong từng script, hàm tĩnh get_connection() cung cấp một phương thức chuẩn duy nhất để thiết lập phiên làm việc với Database.
- **Cơ chế xử lý lỗi kết nối:** Mã nguồn tích hợp khối lệnh try-except để bắt các ngoại lệ khi Database không khả dụng, giúp hệ thống đưa ra thông báo lỗi chi tiết thay vì dừng hoạt động đột ngột.
- **Đóng gói thông số:** Lớp này tự động đọc các biến từ .env (Host, Port, DB Name) để cấu hình tham số cho psycopg2, giúp giảm thiểu sai sót do nhập liệu thủ công trong mã nguồn.

10.1.6 Cơ chế điều phối đa tầng (Hierarchical Orchestration)

Để quản lý hàng chục script ETL một cách khoa học, hệ thống triển khai kiến trúc điều phối theo sơ đồ hình cây 3 lớp. Cơ chế này đảm bảo tính tuần tự, quản lý sự phụ thuộc và cho phép khởi chạy linh hoạt theo nhu cầu.

- **Tầng 1: Bộ điều khiển trung tâm (Main Controller):** Tệp main_etl.py đóng vai trò là điểm vào duy nhất (Single Entry Point) của toàn bộ hệ thống. Sử dụng argparse để tiếp nhận chỉ thị từ người dùng (ví dụ: -mode all, -mode fact). Tại đây, hệ thống thực hiện khởi tạo kết nối Database duy nhất và truyền phiên làm việc (Session) xuống các tầng dưới, giúp tối ưu hóa tài nguyên kết nối.
- **Tầng 2: Các Pipeline nhóm (Group Pipelines):** Nằm trong thư mục pipelines/, các script này đóng vai trò là các "nhóm trưởng" quản lý các thực thể dữ liệu có cùng tính chất:
 - run_all_dims.py: Điều phối việc nạp dữ liệu nền. Nó đảm bảo các bảng không có khóa ngoại (như dim_date) phải được chạy thành công trước khi chạy các bảng phụ thuộc (như dim_segment).

```
load_dotenv()

class DatabaseConfig:
    @staticmethod
    def get_connection():
        """Tạo kết nối tới PostgreSQL"""
        try:
            conn = psycopg2.connect(
                host=os.getenv("DB_HOST"),
                port=os.getenv("DB_PORT"),
                user=os.getenv("DB_USER"),
                password=os.getenv("DB_PASS"),
                database=os.getenv("DB_NAME")
            )
            return conn
        except Exception as e:
            print(f"❌ [DB Error] Không thể kết nối: {e}")
            raise e
```

Hình 42: Triển khai lớp DatabaseConfig trong Python

- run_all_facts.py: Điều phối việc nạp dữ liệu động. Đây là pipeline quan trọng nhất trong vận hành hàng ngày, gọi đồng thời các script nạp Flow, Incident và Weather Impact.
- **Tầng 3: Các Script thực thi nguyên tử (Atomic Scripts):** Đây là lớp cuối cùng nằm trong các thư mục etl/infrastructure/, etl/fact/, v.v.. Mỗi script chỉ đảm nhận một nhiệm vụ duy nhất cho một bảng cụ thể (Single Responsibility Principle), giúp việc gỡ lỗi (Debug) trở nên cực kỳ nhanh chóng.

10.2 QUY TRÌNH NẠP DỮ LIỆU DIMENSION (DIMENSION ETL PIPELINE)

Quy trình nạp dữ liệu Dimension được thiết kế để khởi tạo "hệ quy chiếu" cho toàn bộ kho dữ liệu. Pipeline này được điều phối bởi tệp pipelines/run_all_dims.py, đảm bảo các bảng độc lập được nạp trước và các bảng có ràng buộc khóa ngoại được nạp sau theo một trình tự logic chặt chẽ.

10.2.1 Nhóm chiều Thời gian và Lịch (Time & Calendar Dimensions)

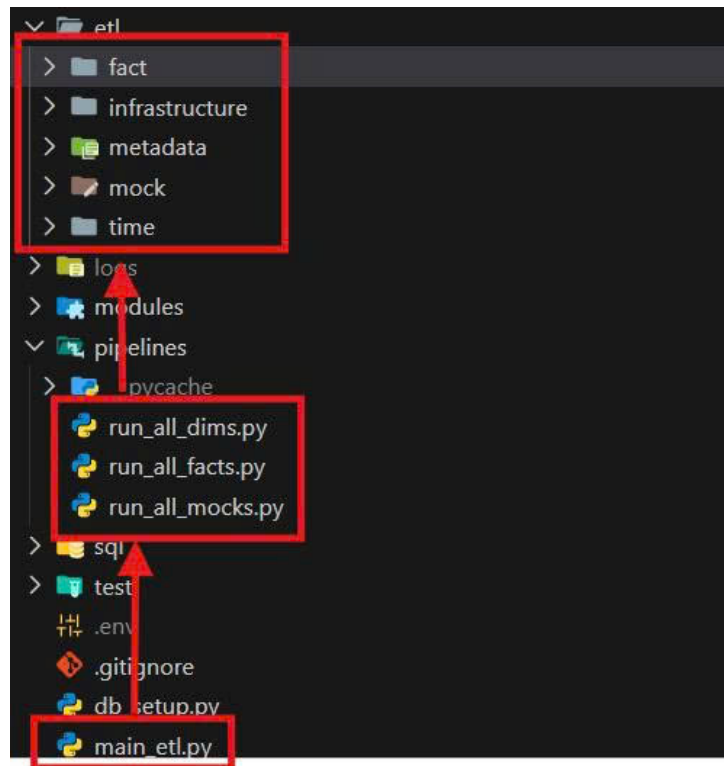
Dự án triển khai một hệ thống chiều thời gian đa cấp độ, từ mức năm đến mức chi tiết từng phút trong ngày, nhằm hỗ trợ phân tích xu hướng giao thông theo các chu kỳ khác nhau.

10.2.1.1 Logic xử lý Lịch dương và Lịch âm (Lunar Holiday Logic) Việc xử lý dữ liệu thời gian trong bối cảnh giao thông tại Việt Nam đòi hỏi sự chính xác tuyệt đối về các ngày lễ, bởi đây là những thời điểm lưu lượng giao thông biến động cực đoan. Hệ thống triển khai một module chuyên biệt để giải quyết bài toán giao thoa giữa hai hệ thống lịch: Dương lịch (Solar Calendar) và Âm lịch (Lunar Calendar).

A. Định nghĩa cấu trúc danh mục ngày lễ (dim_holiday)

Dữ liệu ngày lễ không được nạp cứng mà được quản lý thông qua bảng danh mục dim_holiday với các thuộc tính linh hoạt:

- **Cấu trúc lưu trữ:** Mỗi loại ngày lễ được định danh bởi một holiday_code duy nhất (ví dụ: TET_HOLIDAY, NATIONAL_DAY).
- **Phân loại lịch:** Cột calendar_type xác định quy tắc tính toán là 'SOLAR' hay 'LUNAR'.



Hình 43: Sơ đồ điều phối Pipeline đa tầng

- **Quản lý khoảng thời gian:** Thuộc tính `duration_days` cho phép xác định một ngày lễ kéo dài bao nhiêu ngày (ví dụ: Tết Nguyên Đán kéo dài 7 ngày), giúp hệ thống tự động sinh dữ liệu cho toàn bộ kỳ nghỉ.

dim_holiday 1

Enter a SQL expression to filter results (use Ctrl+Space)

Grid	123 holiday_key	AZ holiday_code	AZ holiday_name_vn	AZ holiday_type	AZ calendar_type	123 default_month
1	1	NEW_YEAR	Tết Dương Lịch	[NULL]	SOLAR	1
2	2	REUNIFICATION	Giải phóng MN	[NULL]	SOLAR	4
3	3	LABOR_DAY	Quốc tế Lao động	[NULL]	SOLAR	5
4	4	NATIONAL_DAY	Quốc khánh	[NULL]	SOLAR	9
5	5	HUNG_KINGS	Giỗ tổ Hùng Vương	[NULL]	LUNAR	3
6	6	TET_HOLIDAY	Tết Nguyên Đán	[NULL]	LUNAR	1

Hình 44: Dữ liệu cơ bản hình các ngày lễ trong bảng `dim_holiday`

B. Thuật toán chuyển đổi và tính toán ngày lễ Âm lịch

Do đặc thù các ngày lễ Âm lịch không cố định theo ngày Dương lịch qua từng năm, hệ thống triển khai quy trình chuyển đổi động:

- **Tích hợp thư viện chuyên dụng:** Sử dụng thư viện `lunarcalendar` với các lớp `Converter` và `Lunar` để thực hiện phép toán chuyển đổi hệ tọa độ thời gian.
- **Vòng lặp tính toán:** Với mỗi năm trong cấu hình `ETL_START_YEAR` đến `ETL_END_YEAR`, hệ thống khởi tạo đối tượng `Lunar(year, month, day)` dựa trên ngày lễ định nghĩa. Sau đó, hàm `Converter.Lunar2Solar` sẽ trả về đối tượng ngày Dương lịch tương ứng.
- **Logic đặc thù cho Tết Nguyên Đán:** Riêng đối với mã `TET_HOLIDAY`, hệ thống áp dụng logic lùi 1 ngày (`timedelta(days=1)`) từ mừng 1 Tết để tính toán chính xác ngày "30 Tết", đảm bảo kỳ nghỉ lễ được bắt đầu đúng thực tế văn hóa Việt Nam.

C. Cơ chế cầu nối và đồng bộ hóa (Bridge Mechanism)

Để giải quyết mối quan hệ nhiều-nhiều (N-N) giữa Ngày và Lễ (một ngày có thể có nhiều lễ, hoặc một lễ kéo dài nhiều ngày), hệ thống sử dụng bảng cầu nối `bridge_date_holiday`:


```
for year in range(start_year, end_year + 1):
    for code, _, cal_type, m, d, dur in holidays:
        if cal_type == "SOLAR":
            s_date = date(year, m, d)
        else:
            # --- SỬA LỖI TẠI ĐÂY (leap -> isleap) ---
            l_date = Lunar(year, m, d, isleap=False)
            solar = Converter.Lunar2Solar(l_date)
            s_date = date(solar.year, solar.month, solar.day)

            if code == "TET_HOLIDAY":
                s_date -= timedelta(days=1) # 30 Tết

        for i in range(dur):
            act_date = s_date + timedelta(days=i)
            d_key = int(act_date.strftime('%Y%m%d'))

            # Chỉ add nếu có key trong map (phòng hờ lỗi logic)
            if code in holiday_map:
                bridge_data.append((d_key, holiday_map[code]))
                date_updates.add(d_key)
```

Hình 45: Đoạn mã xử lý chuyển đổi lịch âm sang lịch dương cho ngày lễ

- **Ánh xạ khóa ngoại:** Mỗi cặp date_key (từ dim_date) và holiday_key (từ dim_holiday) được ghi nhận để xác định chính xác sự hiện diện của ngày lễ.
- **Tối ưu hóa hiệu suất truy vấn (De-normalization):** Sau khi nạp dữ liệu vào bảng cầu nối, hệ thống thực hiện một lệnh UPDATE tập hợp lên bảng dim_date. Toàn bộ các date_key xuất hiện trong bảng cầu nối sẽ được bật cờ is_holiday = TRUE.
- **Lợi ích:** Kỹ thuật này giúp các câu lệnh truy vấn lưu lượng giao thông trong ngày lễ sau này chỉ cần lọc theo cờ is_holiday mà không cần thực hiện phép JOIN phức tạp, giúp tăng tốc độ phản hồi của hệ thống báo cáo.

dim_date 1				
SELECT full_date, day_name_vi, is_holiday FROM dim_date				
	full_date	AZ day_name_vi	is_holiday	
1	2020-01-01	Thứ Tư	[v]	
2	2020-01-24	Thứ Sáu	[v]	
3	2020-01-25	Thứ Bảy	[v]	
4	2020-01-26	Chủ Nhật	[v]	
5	2020-01-27	Thứ Hai	[v]	
6	2020-01-28	Thứ Ba	[v]	
7	2020-01-29	Thứ Tư	[v]	
8	2020-01-30	Thứ Năm	[v]	
9	2020-04-02	Thứ Năm	[v]	
10	2020-04-30	Thứ Năm	[v]	

Hình 46: Kết quả cập nhật cờ is_holiday trong bảng dim_date

10.2.1.2 Phân tích và ánh xạ Ca làm việc (Work Shifts) Trong phân tích dữ liệu giao thông, việc chỉ sử dụng các mốc giờ thô là không đủ để phản ánh đặc tính vận hành của đô thị. Hệ thống triển khai giải pháp phân tích thời gian dựa trên các "Ca làm việc" (Work Shifts) để hỗ trợ phân tích lưu lượng theo các khung giờ đặc thù như cao điểm sáng, cao điểm chiều và các khung giờ hạn chế giao thông (giờ giới nghiêm cho xe tải).

A. Thiết lập danh mục Ca làm việc (dim_shift)

Bảng dim_shift đóng vai trò định nghĩa các khoảng thời gian vận hành lớn trong một ngày. Dữ liệu này được quản lý thông qua module etl/time/etl_dim_shift.py với các quy tắc nghiệp vụ sau:

- **Phân lớp Ca làm việc:** Hệ thống chia ngày thành 3 phân đoạn chính:
 - Ca Sáng (SHIFT_MORNING): Từ 06:00 đến 14:00 (Phút thứ 360 đến 840). Đây là khung giờ bao gồm cao điểm sáng khi người dân di chuyển đến nơi làm việc.
 - Ca Chiều (SHIFT_AFTERNOON): Từ 14:01 đến 22:00 (Phút thứ 841 đến 1320). Khung giờ này bao phủ cao điểm chiều và thời điểm tan sở.
 - Ca Đêm (SHIFT_NIGHT): Từ 22:01 đến 05:59 sáng hôm sau (Phút thứ 1321 đến 359). Đây là giai đoạn lưu lượng thấp, chủ yếu phục vụ xe vận tải hạng nặng.
- **Thuộc tính nghiệp vụ (is_business_shift):** Mỗi ca được gán một cờ logic để phân biệt giữa khung giờ hành chính và khung giờ nghỉ ngơi, hỗ trợ việc lọc dữ liệu nhanh trong các báo cáo về hiệu suất lao động và kinh tế.

B. Chi tiết hóa đơn vị thời gian (dim_time_of_day)

Để đạt được mức độ chi tiết cao nhất trong phân tích (Granularity), hệ thống không chỉ dừng lại ở mức giờ mà thực hiện "số hóa" đến từng phút trong ngày thông qua bảng dim_time_of_day:

- **Số hóa 1440 phút:** Module etl/time/etl_dim_time_of_day.py thực hiện vòng lặp sinh ra chính xác 1440 bản ghi, đại diện cho mỗi phút từ 00:00 đến 23:59. Mỗi bản ghi được định danh bằng một time_key duy nhất từ 0 đến 1439.
- **Ánh xạ đa chiều (Multi-mapping):** Định dạng HHMM chuyển đổi phút sang dạng số nguyên (ví dụ: 14:05 thành 1405). Liên kết Ca mặc định (default_shift_key) sử dụng câu lệnh if-elif để xác định một phút thuộc ca làm việc nào.
- **Định nghĩa Giờ hành chính mặc định:** Hệ thống gán cờ is_business_hours_default = TRUE cho các phút từ 08:00 đến 17:00 (phút 480 đến 1020).

C. Kỹ thuật nạp dữ liệu tối ưu (Bulk Insert & Upsert)

Xử lý trên RAM bằng cách khởi tạo toàn bộ 1440 bản ghi trong mảng Python (data_buffer) trước khi đẩy xuống cơ sở dữ liệu. Sử dụng cơ chế ON CONFLICT (time_key) DO UPDATE để cập nhật các thuộc tính mà không làm thay đổi khóa chính.

10.2.2 Nhóm chiều Danh mục và Đặc tính (Metadata Dimensions)

Nhóm chiều Danh mục đóng vai trò là lớp đệm chuẩn hóa, giúp hệ thống đồng nhất các mã dữ liệu thô từ nhiều nguồn khác nhau thành các thông tin có ý nghĩa phân tích thống nhất. Toàn bộ danh mục được quản lý tập trung qua các tệp CSV trong thư mục data/, cho phép cập nhật tham số nghiệp vụ mà không cần can thiệp mã nguồn.

10.2.2.1 Chuẩn hóa đặc tính phương tiện và hệ số PCU Dữ liệu về phương tiện được nạp vào bảng dim_vehicle_type thông qua quy trình xử lý tệp vehicles.csv.

- **Hệ số quy đổi xe con (Passenger Car Unit - PCU):** Gán trọng số cụ thể: Xe máy (0.5), Xe con (1.0), Xe tải (2.5). Đây là tham số bắt buộc để đánh giá chính xác mức độ chiếm dụng mặt đường.
- **Tham số vận tốc thiết kế:** Mỗi loại phương tiện được định nghĩa một average_speed_kmh làm mốc so sánh (Baseline) cho các mô hình phân tích.
- **Cơ chế nạp dữ liệu an toàn:** Sử dụng executemany kết hợp ON CONFLICT (vehicle_code) DO UPDATE.

```
vehicles.csv X incidents.csv etl_fact_incident.py
etl > metadata > data > vehicles.csv > data
1 code,name,category,pcu,speed
2 MC,Xe máy,Motorcycle,0.25,40
3 CAR,Ô tô con (4-7 chỗ),Car,1.00,50
4 BUS,Xe buýt,Heavy,1.50,40
5 TRUCK_LIGHT,Xe tải nhẹ (<3.5T),Heavy,1.50,40
6 TRUCK_HEAVY,Xe tải nặng (>3.5T),Heavy,2.00,35
7 CONTAINER,Xe đầu kéo / Container,Heavy,2.50,30
8 BIKE,Xe đạp / Xe thô sơ,Non-motor,0.15,15
9 PEDESTRIAN,Người đi bộ,Non-motor,0.10,5
```

Hình 47: Tập cấu hình vehicles.csv chứa hệ số PCU và tốc độ thiết kế

10.2.2.2 Phân loại mức độ nghiêm trọng thời tiết và sự cố Chuẩn hóa dữ liệu thô từ API về thang đo thống nhất qua các bảng dim_weather và dim_incident_type.

A. Danh mục Điều kiện Thời tiết (Weather Metadata):

Dữ liệu được phân loại theo severity_level từ 1 (Bình thường) đến 4 (Cực đoan). Đi kèm các thuộc tính định tính được mã hóa: visibility_condition và road_friction_impact.

B. Danh mục Loại hình Sự cố (Incident Metadata):

Phân nhóm logic vào các nhóm Tai nạn, Thi công, hoặc Ùn tắc. Đặc biệt tích hợp cột color_hex_code (ví dụ: #FF0000 cho tai nạn) hỗ trợ hiển thị màu sắc biểu tượng trên bản đồ Dashboard một cách tự động.

dim_incident_type 1 X			
Enter a SQL expression to filter results (u			
Grid	AZ incident_type_name	123 severity_level	AZ color_hex_code
1	Tai nạn giao thông	3	#FF0000
2	Sương mù nguy hiểm	3	#708090
3	Điều kiện nguy hiểm	4	#FF4500
4	Mưa lớn cản trở tầm nhìn	3	#4169E1
5	Đường đóng băng/Trơn trượt	4	#ADD8E6
6	Ùn tắc giao thông	2	#FF8C00
7	Đóng làn đường	2	#FFA500
8	Đường cấm/Đóng đường	5	#8B0000
9	Thi công công trình	2	#808080
10	Gió mạnh/Bão	3	#D3D3D3
11	Ngập lụt	4	#0000FF
12	Xe hỏng/Chết máy	1	#A52A2A

Hình 48: Danh mục sự cố đã chuẩn hóa mã màu trong cơ sở dữ liệu

10.2.3 Nhóm chiều Hạ tầng Giao thông (Infrastructure Dimensions)

Chiều hạ tầng giao thông được xác định là thành phần quan trọng nhất, đóng vai trò "xương sống" không gian (Spatial Backbone) của toàn bộ kho dữ liệu. Khác với các chiều dữ liệu định danh thông thường, nhóm này đòi hỏi quy trình xử lý phức tạp dựa trên lý thuyết đồ thị (Graph Theory) và các phép toán hình học không gian để chuyển đổi dữ liệu mạng lưới đường bộ từ OpenStreetMap (OSM) sang định dạng chuẩn của PostGIS. Việc xây dựng nhóm

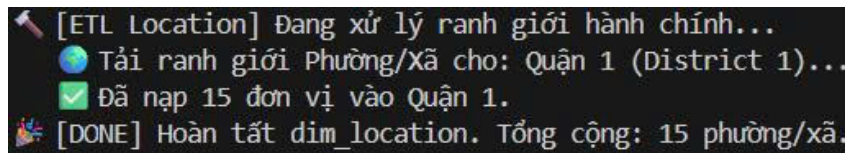
chiều này tuân thủ một trình tự phụ thuộc dữ liệu nghiêm ngặt để đảm bảo tính toàn vẹn tham chiếu: khởi tạo ranh giới hành chính, xây dựng nút giao và tuyến đường, sau đó phân rã thành các phân đoạn nhỏ (Segments).

10.2.3.1 Trích xuất ranh giới hành chính và Địa điểm (Location ETL) Quy trình nạp bảng `dim_location` thực hiện định danh và số hóa ranh giới các vùng hành chính, tạo cơ sở cho việc phân tích lưu lượng giao thông theo từng địa phương.

A. Kỹ thuật trích xuất dữ liệu không gian từ Overpass API

Hệ thống sử dụng thư viện chuyên dụng OSMnx để giao tiếp với API Overpass của OpenStreetMap.

- **Cơ chế truy vấn:** Gửi các yêu cầu truy vấn hướng đối tượng (Geocode-based requests) thay vì tải dữ liệu thô toàn bộ bản đồ.
- **Phân cấp hành chính:** Trích xuất đa tầng bao gồm Phường/Xã (Ward), Quận/Huyện (District) và Tỉnh/Thành phố (City).
- **Linh hoạt cấu hình:** Danh sách các khu vực trích xuất được tham số hóa để dễ dàng mở rộng phạm vi địa lý.



```
[ETL Location] Đang xử lý ranh giới hành chính...
🌐 Tải ranh giới Phường/Xã cho: Quận 1 (District 1)...
✅ Đã nạp 15 đơn vị vào Quận 1.
🎉 [DONE] Hoàn tất dim_location. Tổng cộng: 15 phường/xã.
```

Hình 49: Nhật ký thực thi nạp dữ liệu ranh giới hành chính Quận 1

B. Quy trình xử lý hình học (Geometry Processing)

Dữ liệu thô sau khi trích xuất cần trải qua các bước biến đổi hình học nghiêm ngặt để tương thích với cơ sở dữ liệu PostGIS:

- **Chuẩn hóa hệ tọa độ (CRS):** Ép buộc về hệ tọa độ WGS84 (EPSG:4326) để đảm bảo tính đồng nhất khi thực hiện các phép giao cắt không gian sau này.
- **Ép kiểu dữ liệu MULTIPOLYGON:** Lưu trữ dưới dạng MULTIPOLYGON để xử lý các vùng địa giới phức tạp bị chia cắt bởi sông ngòi hoặc hải đảo.
- **Tự động hóa mã hóa (Encoding):** Chuyển đổi đối tượng hình học sang định dạng Hex EWKB (Extended Well-Known Binary) thông qua thư viện shapely.

C. Ràng buộc dữ liệu và Cơ chế bảo vệ (Data Integrity)

Để duy trì tính sạch của kho dữ liệu, hệ thống thiết lập các rào cản kỹ thuật:

- **Ràng buộc Unique phức hợp:** Thiết lập trên bộ ba cột (ward, district, city) để ngăn chặn trùng lặp bản ghi khi chạy lại quy trình ETL.
- **Chiến lược nạp dữ liệu:** Sử dụng mệnh đề `ON CONFLICT DO NOTHING` để tối ưu hóa thời gian và giữ nguyên các khóa ngoại đã tham chiếu.
- **Xây dựng mã vị trí (Location Code):** Tự động sinh `location_code` chuẩn hóa (ví dụ: `PHUONG_BEN_NGHE_QUAN_1`) phục vụ tra cứu nhanh.

10.2.3.2 Chuyển đổi mạng lưới đường bộ (Road, Way & Node ETL) Quy trình này biến đổi đồ thị OSM thô thành các thực thể quản lý vật lý và logic.

- **dim_road (Tuyến đường):** Lưu trữ tên đường logic (ví dụ: Đường Điện Biên Phủ). Một Tuyến đường có thể bao gồm nhiều Đoạn đường vật lý khác nhau.

Grid	1	2	3	4	5	6	7	8	9	10	11	12	13
Text	location_key	location_code	ward	district	city	geometry							
Spatial	OSM_unknown_0	Thành phố Hồ Chí Minh	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((105.9769387 8.702505, 106.4538443 9.080058,								
Record	OSM_unknown_0	Phường An Khánh	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.7080569 10.7742841, 106.708136 10.7751,								
	OSM_unknown_0	Phường Cầu Ông Lãnh	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.6816889 10.7654323, 106.6818339 10.765,								
	OSM_unknown_0	Phường Bến Thành	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.6844859 10.7683635, 106.6860996 10.770,								
	OSM_unknown_0	Phường Sài Gòn	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.6950078 10.77958, 106.6981274 10.78288,								
	OSM_unknown_0	Phường Tân Định	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.6850308 10.7948165, 106.684874 10.7959,								
	OSM_unknown_0	Phường Xuân Hòa	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.6816674 10.7778253, 106.6816513 10.777,								
	OSM_unknown_0	Phường Bàn Cờ	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.6744365 10.7676889, 106.6752469 10.768,								
	OSM_unknown_0	Phường Xóm Chiếu	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.7011685 10.7653164, 106.7012615 10.765,								
	OSM_unknown_0	Phường Khánh Hội	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.696214 10.7615408, 106.6971407 10.761,								
	OSM_unknown_0	Phường Vĩnh Hội	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.6864219 10.7517073, 106.6864321 10.752,								
	OSM_unknown_0	Phường Chợ Quán	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.6750596 10.7612878, 106.6764631 10.762,								
	OSM_unknown_0	Phường Gia Định	Quận 1	Thành phố Hồ Chí Minh	MULTIPOLYGON (((106.6870269 10.8061811, 106.6870351 10.806,								

Hình 50: Dữ liệu ranh giới hành chính đã nạp vào bảng dim_location



Hình 51: Trực quan hóa đa giác ranh giới TP. Hồ Chí Minh từ PostGIS

- **dim_way (Đoạn đường vật lý):** Lưu trữ các đặc tính kỹ thuật như số làn xe (lane_count), loại đường (way_type), và giới hạn tốc độ thiết kế.
- **Số hóa Nút giao (Node ETL):** Toàn bộ các điểm giao cắt, quay đầu được trích xuất thành các bản ghi trong dim_node dưới dạng một điểm (POINT) với tọa độ chính xác.

10.2.3.3 Phân đoạn và Tính toán hướng đường (Segment ETL) Phân đoạn đường (Segment ETL) là bước biến đổi dữ liệu phức tạp nhất, chuyển đổi cấu trúc đồ thị từ các thực thể đường dài (Ways) thành các đơn vị phân tích cơ sở gọi là Segments.

A. Quy trình Segment hóa dựa trên cấu trúc đồ thị (Graph-based Segmenting)

Chia nhỏ mạng lưới dựa trên các điểm nút (Nodes) để quản lý vi mô các chỉ số giao thông chi tiết đến từng phân đoạn mét đường. Mỗi phân đoạn được cấp một segment_key duy nhất.

B. Thuật toán tính toán Azimuth và vai trò trong Map Matching

Hướng di chuyển là thông tin sống còn để phân biệt dòng xe trên các đường hai chiều. Hệ thống tính góc hướng (Azimuth) tự động bằng công thức:

- $\Delta Lon = Lon_{to} - Lon_{from}$
- $\Delta Lat = Lat_{to} - Lat_{from}$
- $\theta = atan2(\Delta Lon, \Delta Lat)$

Kết quả được chuẩn hóa về khoảng $[0^\circ, 360^\circ]$ để thực hiện kỹ thuật Map Matching, giúp đối khớp chính xác dữ liệu tốc độ thời gian thực từ TomTom API vào đúng hướng di chuyển tương ứng.

C. Chiến lược đa dạng hóa dữ liệu hình học (Dual-Geometry Strategy)

Mỗi Segment lưu trữ đồng thời hai loại dữ liệu không gian để tối ưu hóa hiển thị và hiệu năng xử lý:

- **geometry_linestring (Chuỗi đường):** Lưu trữ tọa độ tạo nên hình dáng uốn lượn thực tế của đoạn đường, dùng để vẽ bản đồ nhiệt hoặc lộ trình.
- **geometry_center (Điểm trung tâm):** Tính toán điểm trọng tâm (Centroid) của đoạn đường để tăng tốc các phép toán tìm kiếm lân cận hoặc giao cắt hành chính (Spatial Join).

123 azimuth	geometry_center	geometry_linestring
214	POINT (106.68290725 10.86324235)	LINESTRING (106.682936 10.8632835, 106.6828785 10.8632012)
34	POINT (106.68296914999999 10.863330900000003)	LINESTRING (106.682936 10.8632835, 106.6830023 10.8633783)
219	POINT (106.68141015650923 10.86453962854902)	LINESTRING (106.6816493 10.8649288, 106.6814808 10.8645816)
207	POINT (106.68169479221827 10.86501606393437)	LINESTRING (106.6817417 10.8651026, 106.6816677 10.8649668)
205	POINT (106.68315429999998 10.863638349999999)	LINESTRING (106.6831646 10.86366, 106.683144 10.8636167)
25	POINT (106.6831979 10.8637287)	LINESTRING (106.6831646 10.86366, 106.6832312 10.8637974)
214	POINT (106.68284505 10.863153350000001)	LINESTRING (106.6828785 10.8632012, 106.6828116 10.8631055)
34	POINT (106.68290725 10.86324235)	LINESTRING (106.6828785 10.8632012, 106.682936 10.8632835)
221	POINT (106.68786575 10.8635147)	LINESTRING (106.6878897 10.8635415, 106.6878418 10.8634879)
41	POINT (106.68801048838259 10.863676421181845)	LINESTRING (106.6878897 10.8635415, 106.6880843 10.8637588)

Hình 52: Dữ liệu góc Azimuth và hình học đa tầng trong bảng `dim_segment`

10.2.3.4 Làm giàu dữ liệu bằng phép giao không gian (Spatial Enrichment) Sau khi hoàn tất việc nạp dữ liệu vào các bảng hạ tầng độc lập, kho dữ liệu vẫn tồn tại một khoảng cách giữa thực thể kỹ thuật (các phân đoạn đường OSM) và thực thể quản lý hành chính (Phường, Quận). Để giải quyết bài toán này, hệ thống triển khai bước "Enrichment" (Làm giàu dữ liệu) thông qua module `etl/infrastructure/etl_enrichment.py`. Đây là quy trình hậu xử lý quan trọng nhằm thiết lập mối quan hệ logic giữa Hạ tầng và Hành chính một cách hoàn toàn tự động dựa trên tọa độ địa lý.

A. Cơ chế Spatial Join thông qua hàm hình học PostGIS

Thay vì yêu cầu người dùng nhập liệu thủ công hoặc dựa vào các bảng tra cứu tĩnh dễ sai sót, hệ thống tận dụng sức mạnh xử lý không gian trực tiếp bên trong Database:

- **Thuật toán kiểm tra điểm trong đa giác (Point-in-Polygon):** Hệ thống thực thi các câu lệnh SQL sử dụng hàm `ST_Contains` (hoặc `ST_Within`) để đối chiếu hình học.
- **Quy trình xác định:** Quét qua toàn bộ các Segment có cột `location_key` đang trống và xác định xem điểm trung tâm (`geometry_center`) của đoạn đường đó nằm gọn trong đa giác ranh giới (`geometry`) nào của bảng `dim_location`.
- **Tại sao sử dụng Điểm trung tâm (Geometry Center):** Một đoạn đường (`LineString`) có thể đi ngang qua ranh giới giữa hai phường, gây ra lỗi đa trùng lặp nếu thực hiện phép giao cắt toàn phần. Việc sử dụng điểm trung tâm giúp đảm bảo mỗi đoạn đường chỉ được gán duy nhất cho một đơn vị hành chính.

```
# =====
# TASK 1: MAPPING LOCATION
# =====

print(" 🌐 [Task 1/7] Mapping Segment -> Location...")
sql_spatial_join = """
    UPDATE dim_segment s
    SET location_key = l.location_key
    FROM dim_location l
    WHERE s.location_key IS NULL
    AND ST_Contains(l.geometry, s.geometry_center);
"""

cur.execute(sql_spatial_join)
conn.commit()
```

Hình 53: Câu lệnh SQL thực hiện phép giao không gian để cập nhật `location_key`

B. Tự động hóa cập nhật và tính nhất quán dữ liệu

Kết quả của phép giao không gian này là cột `location_key` trong bảng `dim_segment` được cập nhật giá trị tham chiếu chính xác đến bảng `dim_location`. Nhờ có bước Enrichment này, mọi bản ghi Fact sau khi được gắn vào `segment_key` sẽ nghiêm nhiên sở hữu thông tin hành chính thông qua mối quan hệ bắc cầu, hỗ trợ các báo cáo tổng hợp như "Tính toán tốc độ trung bình của toàn bộ các đoạn đường thuộc Quận 1" mà không cần can thiệp mã nguồn ETL.

C. Tối ưu hóa hiệu năng xử lý không gian (Spatial Indexing)

Do số lượng Segment có thể lên đến hàng chục nghìn bản ghi, hệ thống sử dụng **GIST Index** trên cột điểm trung tâm và chỉ mục không gian trên cột đa giác ranh giới của `dim_location`. Điều này cho phép Database thực hiện phép giao không gian với độ phức tạp logarit thay vì quét toàn bộ bảng.

10.3 QUY TRÌNH NẠP DỮ LIỆU SỰ KIỆN (FACT ETL PIPELINE)

Quy trình nạp dữ liệu sự kiện (Fact ETL) là giai đoạn quan trọng nhất trong vận hành hàng ngày của hệ thống. Khác với dữ liệu Dimension mang tính chất tĩnh, dữ liệu Fact được thiết kế để thu thập các biến động không ngừng của mạng lưới giao thông theo thời gian thực (Real-time). Quy trình này thực hiện việc kết hợp dữ liệu từ các nguồn API quốc tế với dữ liệu quan trắc trực tiếp từ mô hình AI để tạo ra cái nhìn toàn diện về thực trạng giao thông.

10.3.1 Tầng Dịch vụ thu thập dữ liệu (Data Service Layer)

Để đảm bảo tính module hóa và dễ bảo trì, hệ thống tách biệt logic giao tiếp với các nhà cung cấp dữ liệu bên ngoài thành một tầng dịch vụ riêng biệt (Service Layer). Tầng này chịu trách nhiệm đóng gói các yêu cầu API, xử lý xác thực và chuẩn hóa dữ liệu thô (JSON) trước khi đưa vào các pipeline xử lý chính.

a. Dịch vụ dữ liệu giao thông (TomTom Service)

Module `modules/tomtom_service.py` là thành phần chịu trách nhiệm tương tác với TomTom:

- **Trích xuất dữ liệu lưu lượng (Traffic Flow):** Sử dụng endpoint `flowSegmentData` để truy vấn tốc độ di chuyển thực tế (`currentSpeed`) và tốc độ tự do (`freeFlowSpeed`) tại một tọa độ cụ thể.
- **Trích xuất dữ liệu sự cố (Traffic Incidents):** Sử dụng cơ chế quét theo vùng (Bounding Box) để lấy loại sự cố, tọa độ hình học, độ trễ và mô tả chi tiết sự việc.

```
{
  "flowSegmentData": {
    "frc": "FRC3",
    "currentSpeed": 32,
    "freeFlowSpeed": 50,
    "currentTravelTime": 112,
    "freeFlowTravelTime": 72,
    "confidence": 1.0,
    "coordinates": {
      "coordinate": [
        { "latitude": 10.7769, "longitude": 106.7009 },
        { "latitude": 10.7775, "longitude": 106.7015 }
      ]
    }
  },
  "@version": "4.42.0.0"
}
```

Hình 54: Cấu trúc dữ liệu JSON trả về từ TomTom Traffic API

b. Dịch vụ dữ liệu khí tượng (Weather Service)

Dữ liệu thời tiết được cung cấp bởi module `modules/weather_service.py` thông qua OpenWeatherMap API. Hệ thống thực hiện truy vấn theo tọa độ chính xác của từng đoạn đường (*lat*, *lon*) để đảm bảo tính cục bộ, trả về các thông số: mã thời tiết (Weather ID), nhiệt độ, độ ẩm, tầm nhìn (*visibility*) và lượng mưa.

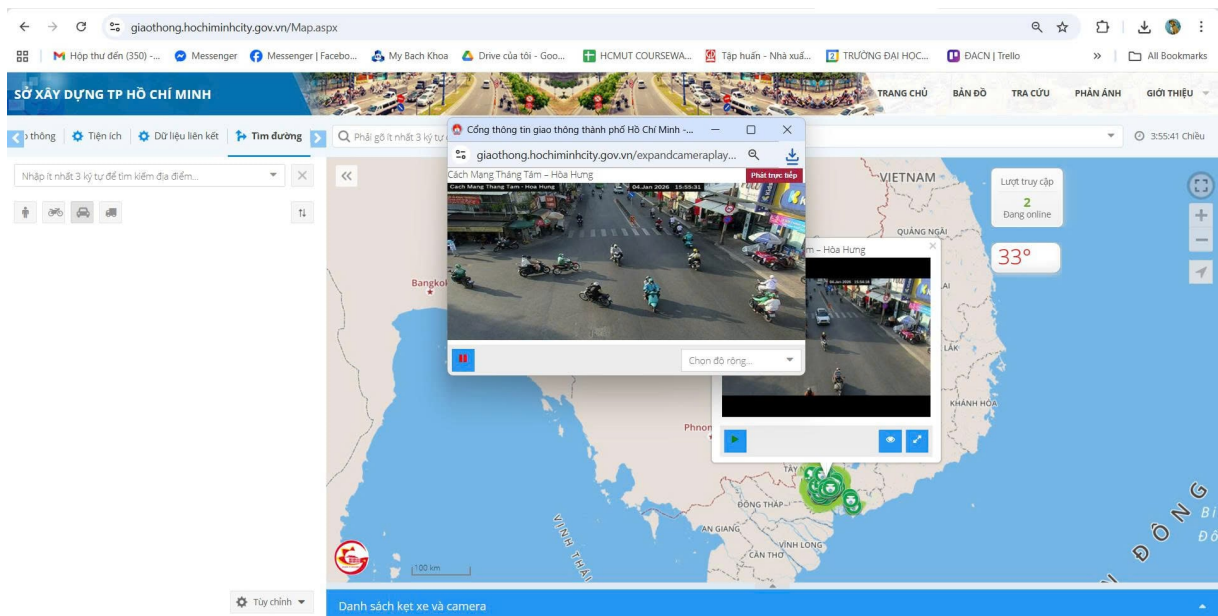
10.3.2 Tích hợp Thị giác máy tính trong quy trình ETL (AI Integration)

Một điểm sáng tạo và đột phá của đề án là việc tích hợp trực tiếp mô hình học sâu (Deep Learning) vào luồng xử lý dữ liệu Fact để thu thập số lượng phương tiện thực tế, thay vì chỉ dựa vào dữ liệu ước tính từ API.

10.3.2.1 Kiến trúc bộ đếm phương tiện dựa trên YOLOv8 Dự án triển khai module `modules/ai_counter.py` để thu thập dữ liệu "ground truth" từ các nguồn camera giao thông trực tuyến.

A. Lựa chọn mô hình YOLOv8x (Extra Large) cho độ chính xác tối ưu

Hệ thống sử dụng kiến trúc **YOLOv8** phiên bản "x" (Extra Large) cho độ chính xác cao nhất. Việc lựa chọn mô hình này là cần thiết để giải quyết bài toán mật độ phương tiện cực cao tại các nút giao thông TP.HCM, giúp nhận diện chính xác các đối tượng nhỏ như xe máy trong điều kiện bị che khuất (Occlusion).



Hình 55: Trang web lấy hình ảnh camera tại Đường Cách Mạng Tháng Tám - Hòa Hưng



Hình 56: Kết quả nhận diện và phân lớp phương tiện thời gian thực bằng YOLOv8x

B. Phân lớp đối tượng và Cơ chế lọc lớp (Class Filtering)

Hệ thống cấu hình `self.target_classes = [2, 3, 5, 7]` để chỉ tập trung vào mảng giao thông:

- **Class 2 (Car):** Ánh xạ vào nhóm xe ô tô con.
- **Class 3 (Motorcycle):** Nhóm xe máy chiếm tỷ trọng lớn nhất tại Việt Nam.
- **Class 5 (Bus) và Class 7 (Truck):** Được gộp vào nhóm xe tải/xe hạng nặng (`heavy_vehicle_count`).

Các đối tượng nhiễu như người đi bộ, động vật hay biển báo đều bị bỏ qua ngay tại tầng xử lý AI.

```
class VehicleCounter:
    def __init__(self):
        print(" 🚗 [AI Init] Đang tải mô hình YOLOv8 Extra Large...")
        # Sử dụng model x (chính xác nhất)
        self.model = YOLO('yolov8x.pt')

        # Mapping class
        self.target_classes = [2, 3, 5, 7]
```

Hình 57: Khởi tạo mô hình YOLOv8 và cấu hình danh mục đối tượng mục tiêu

10.3.2.2 Quy trình chuẩn hóa dữ liệu AI và Tính toán lưu lượng quy đổi (PCU) Sau khi mô hình YOLOv8 hoàn tất việc phát hiện và gán nhãn các đối tượng trong khung hình, dữ liệu vẫn ở dạng danh sách các "Bounding Boxes" rời rạc. Quy trình ETL tiếp tục thực hiện bước chuẩn hóa dữ liệu và áp dụng các công thức nghiệp vụ để biến đổi chúng thành các chỉ số lưu lượng mang tính kỹ thuật.

A. Cơ chế tổng hợp và Phân nhóm phương tiện (Aggregation Logic)

Hệ thống triển khai một hàm băm (Mapping Function) để chuyển đổi từ 80 lớp của tập dữ liệu COCO về 4 nhóm thực thể chính phù hợp với hạ tầng giao thông Việt Nam:

- **Nhóm xe máy (motorcycle_count):** Được trích xuất trực tiếp từ class ID 3. Đây là nhóm đối tượng có kích thước nhỏ và mật độ dày đặc nhất, đòi hỏi độ chính xác cao trong nhận diện.
- **Nhóm xe con (car_count):** Trích xuất từ class ID 2.
- **Nhóm xe hạng nặng (heavy_vehicle_count):** Hệ thống thực hiện phép hợp (Union) các đối tượng thuộc class 5 (Bus) và class 7 (Truck). Việc gộp nhóm này giúp đơn giản hóa bài toán tính toán tải trọng lên mặt đường.
- **Nhóm phương tiện khác (other_count):** Bao gồm các lớp như xe đạp, xe lôi hoặc các phương tiện không xác định khác để đảm bảo không bỏ sót bất kỳ thực thể nào đang chiếm dụng không gian đường bộ.

B. Thuật toán tính toán lưu lượng quy đổi (PCU Calculation)

Để đánh giá chính xác mức độ tắc nghẽn, hệ thống quy đổi số lượng xe về đơn vị xe con tiêu chuẩn (Passenger Car Unit - PCU). Công thức được cài đặt trực tiếp trong module `ai_counter.py` dựa trên các hệ số thực nghiệm:

$$Total_PCU = N_{moto} \times 0.25 + N_{car} \times 1.0 + N_{heavy} \times 2.0 + N_{other} \times 1.0 \quad (4)$$

Việc sử dụng hệ số 0.25 cho xe máy phản ánh thực tế rằng diện tích chiếm dụng động của một xe máy chỉ bằng 1/4 so với một xe ô tô con tiêu chuẩn. Chỉ số `total_pcu` là dữ liệu đầu vào quan trọng nhất để tính toán mật độ dòng chảy và khả năng thông hành của đoạn đường.

```
# 3. Đếm số lượng
counts = {
    'motorcycle_count': 0, 'car_count': 0,
    'heavy_vehicle_count': 0, 'other_count': 0
}

for box in result.bboxes:
    cls_id = int(box.cls[0])
    if cls_id == 3: counts['motorcycle_count'] += 1
    elif cls_id == 2: counts['car_count'] += 1
    elif cls_id in [5, 7]: counts['heavy_vehicle_count'] += 1
    elif cls_id in [1, 4, 6, 8]: counts['other_count'] += 1

# 4. Tính PCU
counts['total_vehicle_count'] = sum(counts.values())
counts['total_pcu'] = (counts['motorcycle_count'] * 0.25 +
    counts['car_count'] * 1.0 +
    counts['heavy_vehicle_count'] * 2.0 +
    counts['other_count'] * 1.0)

return counts
```

Hình 58: Đoạn mã triển khai logic đếm đối tượng và tính toán PCU từ kết quả AI

10.3.3 Xác định chỉ số LOS (Level of Service) và Mức độ ùn tắc

Từ dữ liệu AI và vận tốc thu thập được, hệ thống tiến hành phân loại chất lượng dòng lưu lượng thông qua chỉ số LOS (A-F) ngay trong module `etl_fact_traffic.py`:

- **Công thức tính Congestion Level:** Hệ thống so sánh tốc độ thực tế (`current_speed`) với tốc độ tự do (`free_flow_speed`) để tính toán tỷ lệ phần trăm ùn tắc.
- **Phân cấp LOS:**
 - **LOS A/B:** Dòng xe lưu thông tự do, tốc độ thực tế gần bằng tốc độ thiết kế (Congestion < 15%).
 - **LOS C/D:** Dòng xe bắt đầu bị ảnh hưởng, người lái khó thay đổi làn đường (Congestion 15% - 45%).
 - **LOS E/F:** Dòng xe bị ùn tắc nghiêm trọng hoặc đứng im, tốc độ giảm sâu (Congestion > 45%).
- **Ghi nhận dữ liệu:** Toàn bộ các chỉ số này được đóng gói thành một bản ghi và đẩy vào phân vùng tương ứng của bảng `fact_traffic_flow`.

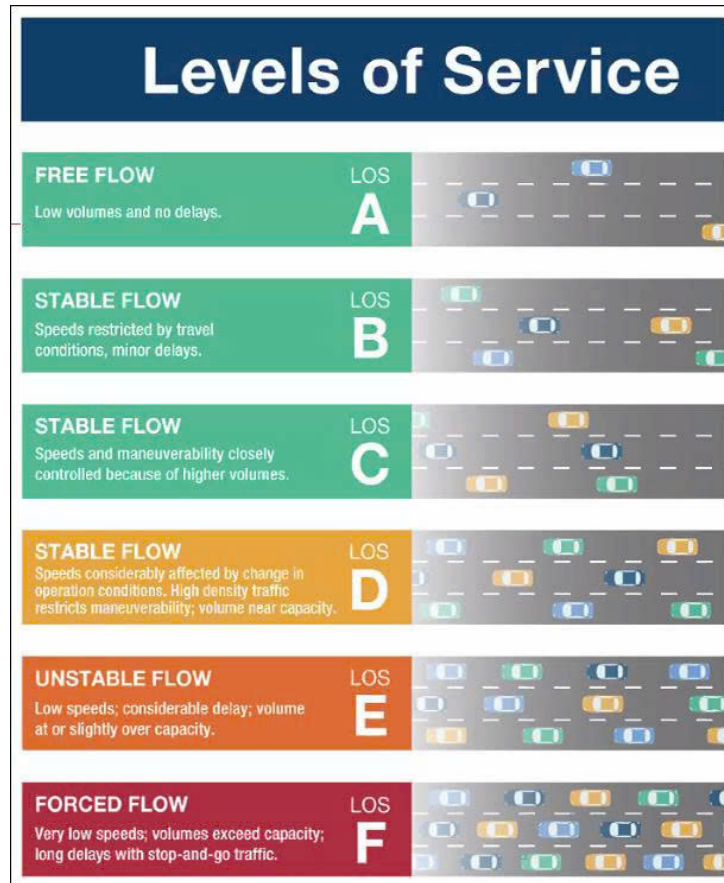
10.3.4 Quy trình nạp dữ liệu Fact Traffic Flow và Fact Incident

Các bảng Fact là trái tim của Star Schema, lưu trữ các chỉ số định lượng về trạng thái mạng lưới giao thông. Quy trình nạp dữ liệu vào bảng Fact yêu cầu sự kết hợp phức tạp giữa các kỹ thuật đối khớp không gian (Spatial Matching) và logic nghiệp vụ để đảm bảo tính nhất quán giữa dữ liệu động và hạ tầng tĩnh.

10.3.4.1 Quy trình nạp dữ liệu Lưu lượng Giao thông (Fact Traffic Flow ETL) Đây là quy trình quan trọng nhất trong hệ thống, thực hiện nhiệm vụ tổng hợp và đồng bộ hóa dữ liệu từ ba nguồn khác nhau: Trí tuệ nhân tạo (AI), API Giao thông (TomTom) và API Thời tiết (OpenWeatherMap) thành một bản ghi thông tin duy nhất đại diện cho trạng thái của một phân đoạn đường tại một thời điểm cụ thể.

A. Cơ chế Hợp nhất dữ liệu đa nguồn (Multi-source Data Fusion)

Hệ thống tập trung vào các vị trí chiến lược được định nghĩa trong cấu hình `CAMERA_MAPPING`. Pipeline thực



Hình 59: Mô tả các cấp độ phục vụ (LOS) trong kỹ thuật giao thông

hiện quét qua danh sách các `segment_key` có gắn camera giám sát. Với mỗi Segment, hệ thống đóng vai trò như một bộ não trung tâm, kích hoạt đồng thời hai luồng thu thập dữ liệu:

- **Luồng quan trắc mật độ (AI Stream):** Gọi module `ai_counter.py` để phân tích ảnh camera, trả về số lượng chi tiết từng loại xe và tổng lưu lượng quy đổi PCU.
- **Luồng quan trắc vận tốc (API Stream):** Gọi `tomtom_service.py` để lấy tốc độ di chuyển thực tế và tốc độ tự do tại đúng tọa độ của phân đoạn đó.

Việc kết hợp giữa "Mật độ" (từ AI) và "Vận tốc" (từ API) cho phép hệ thống xây dựng được biểu đồ quan hệ cơ bản của dòng giao thông (Fundamental Diagram of Traffic Flow).

B. Quy trình xác định khóa tham chiếu và Liên kết ngữ cảnh (Key Mapping)

Để dữ liệu Fact có thể liên kết được với các bảng Dimension, hệ thống thực hiện ánh xạ khóa nghiêm ngặt:

- **Khóa thời gian (Temporal Keys):** Thời gian thực thi được băm nhỏ thành `time_key` (phút trong ngày) và `date_key` (YYYYMMDD) để thực hiện các phép Join cực nhanh bằng kiểu dữ liệu Integer.
- **Khóa ngữ cảnh thời tiết (Weather Key):** Tại mỗi vị trí camera, hệ thống gọi dịch vụ thời tiết để ghi nhận điều kiện khí tượng tức thời. Mã thời tiết trả về được ánh xạ thành `weather_key`, hỗ trợ phân tích tương quan giữa thời tiết và lưu lượng xe.

C. Tính toán chỉ số hiệu suất và Phân cấp LOS (Performance Metrics)

Sau khi hợp nhất dữ liệu, hệ thống tính toán `congestion_level` dựa trên tỷ lệ giữa tốc độ hiện tại và tốc độ tự do. Hệ thống áp dụng bộ quy tắc phân loại LOS từ A đến F để đánh giá chất lượng dịch vụ. Toàn bộ bản ghi được nạp theo lô (Batch Load) vào Database thông qua hàm `executemany` để tối ưu hóa hiệu năng ghi.

	Table Name	Object ID	Owner	Tablespace
Columns	fact_traffic_flow_202501	24,091	postgres	pg_default
Constraints	fact_traffic_flow_202502	24,105	postgres	pg_default
Foreign Keys	fact_traffic_flow_202503	24,223	postgres	pg_default
Indexes	fact_traffic_flow_202504	24,237	postgres	pg_default
Dependencies	fact_traffic_flow_202505	24,251	postgres	pg_default
References	fact_traffic_flow_202506	24,265	postgres	pg_default
Partitions	fact_traffic_flow_202507	24,279	postgres	pg_default
Child tables	fact_traffic_flow_202508	24,293	postgres	pg_default
Triggers	fact_traffic_flow_202509	24,307	postgres	pg_default
Rules	fact_traffic_flow_202510	24,321	postgres	pg_default
Policies	fact_traffic_flow_202511	24,335	postgres	pg_default
Statistics	fact_traffic_flow_202512	24,349	postgres	pg_default
Permissions	fact_traffic_flow_202601	24,363	postgres	pg_default
	fact_traffic_flow_202602	24,377	postgres	pg_default
	fact_traffic_flow_202603	24,391	postgres	pg_default
	fact_traffic_flow_default	24,209	postgres	pg_default

Hình 60: Cấu trúc các phân vùng bảng Fact theo tháng trong PostgreSQL

```
def calculate_flood_risk(rain_mm):
    if rain_mm <= 0: return 1
    if rain_mm < 5: return 2
    if rain_mm < 15: return 3
    if rain_mm < 30: return 4
    return 5
```

Hình 61: Hàm xử lý logic tính toán chỉ số rủi ro ngập lụt

10.3.4.2 Quy trình nạp dữ liệu Sự cố Giao thông (Fact Incident ETL) Bảng fact_incident đóng vai trò lưu trữ các sự kiện bất thường gây ảnh hưởng đến năng lực thông hành của mạng lưới đường bộ như tai nạn, thi công, hoặc thời tiết cực đoan. Quy trình ETL cho bảng này, được triển khai trong module etl/fact/etl_fact_incident.py, giải quyết bài toán thách thức về đối khớp một điểm tọa độ sự cố tự do vào một đoạn đường quản lý cụ thể (Point-to-Segment Mapping).

A. Thuật toán Tìm kiếm phân đoạn gần nhất (Spatial Lookup) dựa trên Haversine

Dữ liệu sự cố từ TomTom API trả về dưới dạng tọa độ kinh/vĩ độ (Point). Tuy nhiên, để dữ liệu này có giá trị phân tích, sự cố phải được gắn (Link) chính xác với một segment_key trong kho dữ liệu.

- **Tính toán khoảng cách địa cầu:** Do PostGIS thực hiện các phép tính trên mặt cầu, hệ thống triển khai hàm calculate_distance_haversine để đo khoảng cách chính xác theo đường chim bay giữa vị trí sự cố và điểm trung tâm (geometry_center) của từng đoạn đường.
- **Công thức áp dụng:** Hệ thống sử dụng bán kính Trái đất $R = 6,371,000$ mét và các phép tính lượng giác để xác định khoảng cách chính xác đến từng mét.
- **Chiến lược lựa chọn đối tượng:** Hệ thống thực hiện quét trong bán kính lân cận (dưới 50m - 100m) để tìm phân đoạn phù hợp nhất. Nếu sự cố nằm quá xa các đoạn đường đã số hóa (vượt ngưỡng sai số GPS), hệ thống sẽ tự động ghi nhận vào log và bỏ qua (skipped_count) để đảm bảo tính sạch của dữ liệu.

B. Cơ chế Ánh xạ loại sự cố (Incident Type Mapping)

TomTom sử dụng hệ thống mã số riêng để định danh loại sự kiện. Hệ thống ETL thực hiện bước chuẩn hóa dữ liệu thông qua từ điển ánh xạ TOMTOM_INCIDENT_MAP:

- **Đồng nhất hóa danh mục:** Mọi sự cố từ API được chuyển đổi sang các mã chuẩn như ACCIDENT, ROAD_WORK, FLOOD... đã được định nghĩa trong bảng dim_incident_type.
- **Tích hợp bối cảnh khí tượng:** Ngay khi một sự cố được ghi nhận, hệ thống đồng thời gọi get_weather_by_coords để lấy thông tin thời tiết tại thời điểm đó, hỗ trợ phân tích các yếu tố tác động ngoại cảnh.

C. Cấu trúc nạp dữ liệu và Quản lý dòng thời gian (Loading & Timeline)

Dữ liệu sự cố được nạp với đầy đủ các thuộc tính: ghi nhận thời gian tồn tại (start_time và expected_end_time) để tính toán tổng thời gian ảnh hưởng; ghi nhận chỉ số tác động (delay_seconds và length_meters). Hệ thống sử dụng kỹ thuật Batch Insert để nạp một lần duy nhất vào PostgreSQL, giúp giảm độ trễ và tránh tình trạng khóa bảng.

```

C:\Windows\SYSTEM32\cmd.exe
[Weather API] Calling OWM for Grid 10.79.106.81...
-> Done. Code 803 ('Clouds') => DB Key 44
[38/42] ROAD_WORK at (10.8984, 106.8300)
[Weather API] Calling OWM for Grid 10.9.106.83...
-> Done. Code 804 ('Clouds') => DB Key 45
[39/42] ROAD_CLOSED at (10.8929, 106.8312)
[Weather API] Calling OWM for Grid 10.89.106.83...
-> Done. Code 804 ('Clouds') => DB Key 45
[40/42] ROAD_WORK at (10.8967, 106.8317)
[41/42] ROAD_WORK at (10.8958, 106.8321)
[42/42] ROAD_WORK at (10.7863, 106.8460)
[Weather API] Calling OWM for Grid 10.79.106.85...
-> Done. Code 804 ('Clouds') => DB Key 45
[SUMMARY] Tổng: 42 | Insert: 42 | Skip: 0
[ETL FACT] Bắt đầu quy trình nạp dữ liệu giao thông (Detailed Weather Mode)...
[AI Init] Đang tải mô hình YOLOv8 Extra Large...
Loading Cache & Segments...
-> Đã load 38 loại thời tiết từ DB.
Bắt đầu xử lý 20 segments...
[1/20] Processing: Thành Lọc 16...
[API Fetch] 10.86.106.68: Code 803 ('Clouds') -> Key DB: 44
  
```

Hình 62: Nhật ký nạp dữ liệu sự cố và kết quả đối khớp không gian thành công

10.3.4.3 Chiến lược Quản trị Phân vùng và Hiệu năng (Partitioning & Performance) Với tần suất nạp dữ liệu 15 phút/lần, các bảng Fact sẽ nhanh chóng tích lũy hàng triệu bản ghi. Để đảm bảo hệ thống không bị suy giảm hiệu năng, dự án triển khai các kỹ thuật quản trị dữ liệu quy mô lớn ngay từ tầng vật lý.

A. Kỹ thuật Phân vùng dữ liệu theo dải thời gian (Range Partitioning)

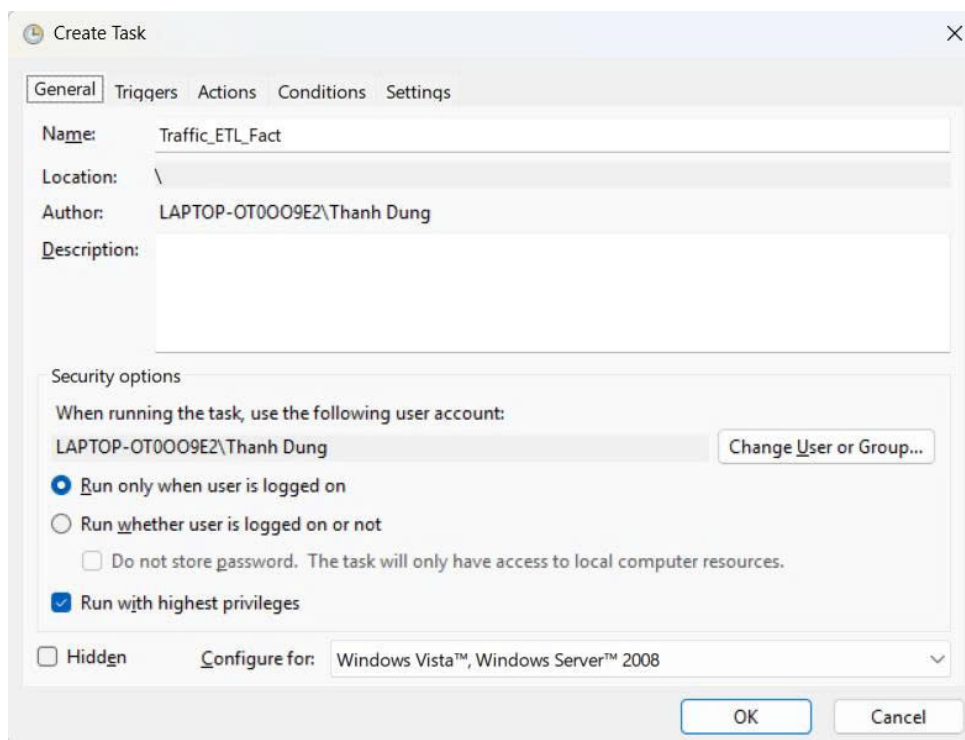
Áp dụng tính năng Declarative Partitioning cho bảng fact_traffic_flow. Dữ liệu được phân tán vào các bảng con dựa trên giá trị của date_key theo từng tháng (ví dụ: fact_traffic_flow_202501). Việc này giúp tránh hiện tượng "Bloat" dữ liệu, giảm chi phí bảo trì chỉ mục và cho phép thực hiện cơ chế Partition Pruning khi truy vấn, giúp tăng tốc độ phản hồi đáng kể.

B. Tối ưu hóa hệ thống chỉ mục đa tầng (Multi-level Indexing)

Thiết lập chỉ mục B-Tree trên các khóa ngoại (segment_key, time_key, date_key) để tối ưu các phép toán JOIN. Đặc biệt, chỉ mục không gian GIST được đánh trên cột geometry của bảng fact_incident, cho phép thực hiện các truy vấn phức tạp như tìm kiếm sự cố trong bán kính người dùng hoặc vẽ bản đồ nhiệt (Heatmap) với độ phức tạp logarit.

C. Lợi ích tổng thể về vận hành

Đảm bảo thời gian chạy của mỗi batch ETL luôn ổn định dưới 2 phút. Đồng thời, dễ dàng thực hiện việc lưu trữ (Archiving) hoặc xóa dữ liệu cũ bằng cách ngắt kết nối (Detach) một phân vùng tháng mà không làm gián đoạn hệ thống đang chạy.



Hình 63: Giao diện Task Scheduler

10.3.4.4 Xử lý nạp dữ liệu Tác động Thời tiết (Fact Weather Impact ETL) Module etl/fact/etls_fact_weather_impact.py được thiết kế để thực hiện các phép phân tích phái sinh nhằm định lượng hóa mức độ ảnh hưởng của thời tiết lên năng lực vận hành và độ an toàn của từng đoạn đường.

A. Thuật toán tính toán chỉ số rủi ro (Risk Scoring Algorithm)

Hệ thống triển khai các mô hình toán học để chuyển đổi đơn vị vật lý thành thang điểm rủi ro từ 0 đến 100:

- **Chỉ số rủi ro ngập lụt (*Flood_Risk_Score*):** Tính toán dựa trên lượng mưa tức thời (P_{1h}), hệ số tác động địa hình (I_{factor}) và năng lực thoát nước ($D_{capacity}$):

$$Flood_Risk = \min \left(100, \frac{P_{1h} \times I_{factor}}{D_{capacity}} \times 100 \right) \quad (5)$$

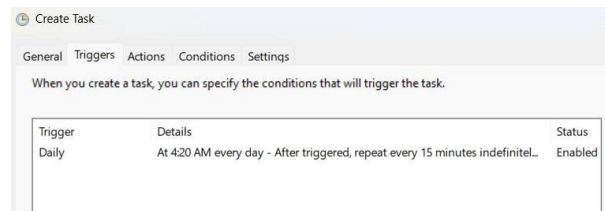
- **Chỉ số an toàn tầm nhìn (*Visibility_Safety_Score*):** Đánh giá dựa trên trường *visibility*. Tầm nhìn dưới 500m kích hoạt cảnh báo nguy hiểm (Danger), trong khi tầm nhìn trên 2000m được coi là an toàn.

B. Logic đối khớp không gian và làm giàu dữ liệu

Dữ liệu tác động thời tiết được gắn trực tiếp vào `segment_key` dựa trên tọa độ trung tâm, cho phép nhận diện các "điểm đen" về ngập lụt trên từng con phố cụ thể. Đồng thời, hệ thống sử dụng bộ quy tắc logic (Knowledge Base) để suy diễn trạng thái mặt đường (Khô ráo, Trơn trượt, Ngập nước) dựa trên mã thời tiết.

C. Ý nghĩa đối với bài toán phân tích đa chiều

Cho phép thực hiện phép JOIN giữa `fact_traffic_flow` và `fact_weather_impact` để tìm ra hệ số giảm tốc độ trung bình của xe máy khi trời mưa, cung cấp dữ liệu nền tảng cho các thuật toán dự báo tắc nghẽn.



Hình 64: Set up Trigger theo lịch ETL

10.3.5 Quy trình giả lập dữ liệu (Mock Data Pipeline)

Trong giai đoạn phát triển và tối ưu hóa kho dữ liệu, việc chỉ phụ thuộc vào dữ liệu thời gian thực từ API là chưa đủ để kiểm chứng các mô hình phân tích dài hạn hoặc kiểm thử hiệu năng truy vấn trên quy mô lớn. Do đó, hệ thống triển khai một Pipeline chuyên biệt để giả lập dữ liệu (Mock Data) thông qua module `pipelines/run_all_mock.py`. Quy trình này mô phỏng các thực thể người dùng, hành vi di chuyển và hiệu suất hạ tầng dựa trên các quy luật xác suất thực tế.

10.3.5.1 Giả lập thực thể Người dùng (User Mocking) Module `etl/mock/etl_mock_user.py` thực hiện khởi tạo tập khách hàng giả lập cho hệ thống:

- **Tạo dữ liệu định danh:** Sử dụng thư viện Faker để sinh ra các thông tin như tên người dùng, số điện thoại và email đảm bảo tính duy nhất và định dạng chuẩn.
- **Quản lý trạng thái:** Mỗi người dùng được gán các thuộc tính như ngày đăng ký (`created_at`) lùi về quá khứ để tạo ra tập dữ liệu có chiều sâu thời gian, phục vụ cho các báo cáo về tăng trưởng người dùng (User Growth).
- **Nạp dữ liệu:** Toàn bộ dữ liệu được đẩy vào bảng `dim_user` thông qua kỹ thuật Batch Insert để tối ưu tốc độ.

10.3.5.2 Mô phỏng hành trình di chuyển (Trip Mocking) Đây là phần phức tạp nhất trong quy trình giả lập, thực hiện tại `etl/mock/etl_mock_trip.py` để nạp dữ liệu vào bảng `fact_trip`:

- **Logic chọn lộ trình:** Hệ thống thực hiện lấy ngẫu nhiên các cặp `segment_key` từ bảng `dim_segment` để làm điểm bắt đầu và điểm kết thúc của một chuyến đi.

- **Tính toán thông số chuyển đi:**
 - **Khoảng cách (d):** Được tính toán dựa trên độ dài thực tế của các phân đoạn đường được chọn.
 - **Thời gian di chuyển (t):** Được suy diễn từ vận tốc trung bình của loại xe (`avg_speed` trong `dim_vehicle_type`) cộng thêm một sai số ngẫu nhiên (*Random Variance*) để mô phỏng tình trạng giao thông biến động.
- **Tiêu hao nhiên liệu và Khí thải:** Hệ thống áp dụng các công thức ước tính để sinh dữ liệu cho cột `fuel_consumed_liters` và `co2_emitted_kg`, hỗ trợ cho các bài toán phân tích môi trường.

10.3.5.3 Giả lập Hiệu suất Phân đoạn lịch sử (Segment Performance Mocking) Để Dashboard có thể hiển thị các biểu đồ so sánh theo năm/tháng, module `etl/mock/etl_mock_segment_performance.py` thực hiện nạp dữ liệu lịch sử giả định cho bảng `fact_segment_performance`:

- **Tạo dữ liệu Baseline:** Hệ thống lấy dữ liệu vận tốc thiết kế từ `dim_way` làm mốc chuẩn.
- **Mô phỏng chu kỳ thời gian:** Dữ liệu được sinh ra theo quy luật: giảm tốc độ vào các khung giờ cao điểm (Morning/Afternoon Shifts) và tăng tốc độ vào ban đêm.
- **Lợi ích:** Việc này giúp kiểm thử tính đúng đắn của các câu lệnh SQL Aggregate phức tạp trước khi có đủ dữ liệu thực tế sau nhiều tháng vận hành.

10.4 TỰ ĐỘNG HÓA VÀ GIÁM SÁT VẬN HÀNH (AUTOMATION & MONITORING)

Để đảm bảo kho dữ liệu luôn phản ánh trạng thái mới nhất của mạng lưới giao thông mà không cần sự can thiệp thủ công từ quản trị viên, dự án đã triển khai một hệ thống vận hành tự động hóa hoàn toàn. Cơ chế này chuyển đổi các kịch bản xử lý dữ liệu đơn lẻ thành một chu kỳ sống liên tục, đáp ứng tiêu chuẩn của một hệ thống Real-time Data Warehouse.

10.4.1 Cơ chế vận hành thông qua Batch Script và Điều phối hệ thống

Tệp kịch bản `scripts/run_etl_fact_daily.bat` đóng vai trò là lớp vỏ điều phối (*Wrapper*) bên ngoài, giúp kết nối mã nguồn Python với hệ điều hành Windows.

10.4.1.1 Tự động hóa quản lý môi trường thực thi (Virtual Environment Activation) Batch script thực hiện kích hoạt môi trường cô lập bằng lệnh `call .venv\Scripts\activate`. Điều này đảm bảo rằng mọi thư viện quan trọng như `ultralalytics`, `psycopg2` và `OSMnx` luôn được chạy đúng phiên bản đã kiểm thử, loại bỏ hoàn toàn các lỗi thiếu Module khi chạy tự động.

```
@echo off
setlocal enabledelayedexpansion
chcp 65001 > nul
cd /d "C:\Path\To\Project\project_folder"
if not exist "logs" mkdir "logs"
call .venv\Scripts\activate
echo [START] ETL JOB STARTED AT %TIME%
powershell -Command "python -u main_etl.py --mode fact 2>&1 | Tee-Object ..."
pause
```

Hình 65: Cấu trúc tệp lệnh Batch thực thi chu kỳ ETL tự động

10.4.1.2 Chiến lược lập lịch chu kỳ (Windows Task Scheduler Integration) Dự án tận dụng Windows Task Scheduler làm "nhịp đập" cho toàn hệ thống với tần suất thực thi 15 phút một lần. Đây là khoảng thời gian tối ưu để cân bằng giữa độ trễ dữ liệu và hạn mức API miễn phí. Cấu hình "Run whether user is logged on or not" đảm bảo quy trình ETL vẫn diễn ra ngầm định ngay cả khi không có người dùng đăng nhập.

10.4.1.3 Tối ưu hóa tham số và Quản lý tài nguyên Việc sử dụng tham số `-mode fact` giúp hệ thống chỉ tập trung vào các bảng dữ liệu biến động, giảm thời gian thực thi mỗi Batch xuống dưới 3 phút, từ đó giải phóng tài nguyên CPU/GPU cho các tác vụ khác.

10.4.2 Hệ thống ghi nhật ký tập trung (Logging System)

Hệ thống triển khai cơ chế *Centralized Logging* dựa trên thư viện logging tiêu chuẩn của Python.

- **Tiêu chuẩn hóa cấu trúc nhật ký:** Sử dụng định dạng `[Thời gian] - [Tên Module] - [Cấp độ] - [Thông điệp]`. Nhật ký được ghi song song ra màn hình console và tệp tin `.log` để phục vụ tra cứu.
- **Giám sát sức khỏe Pipeline (Health Monitoring):** Nhật ký ghi lại chi tiết số lượng bản ghi nạp thành công, số lượng bị lỗi (*Skip*) và tổng thời gian thực thi, giúp đánh giá độ bao phủ dữ liệu (*Data Coverage*).
- **Quản lý ngoại lệ (Exception Handling):** Hệ thống bắt các ngoại lệ cụ thể (như `psycopg2.Error`) và ghi lại *Error Traceback*, giúp quản trị viên phân biệt lỗi do hạ tầng hay lỗi từ API bên thứ ba.

```
✓ [SUCCESS] Đã lưu 50 bản ghi phân tích tác động.

🚀 [ETL USER MOBILITY] Bắt đầu tổng hợp hành vi người dùng tháng 01/2025...
🔍 Kiểm tra dim_month_year (Safety Check)...
🗑️ Xóa dữ liệu cũ của tháng 202501 (nếu có)...
⚙️ Đang tính toán chỉ số hành vi (Aggregation)...
✓ [SUCCESS] Đã tổng hợp xong hành vi cho 300 người dùng.

📄 Ví dụ mẫu dữ liệu vừa tạo:
| User 6: 2.19 chuyến/ngày | 19.32 km/tuần | Hay đi lúc 12h

✓ [DONE] HOÀN THÀNH TOÀN BỘ FACT PIPELINE.

🌟 TẤT CẢ QUY TRÌNH ĐÃ HOÀN TẤT TRONG 111.87 GIÂY. 🌟

[END] ETL JOB FINISHED AT 0:57:02.96
```

Hình 66: Nhật ký tổng hợp kết thúc quy trình Fact Pipeline thành công

10.5 Kết chương

Chương 8 đã trình bày một cách toàn diện quy trình xây dựng hệ thống ETL – trung tâm điều phối của kho dữ liệu giao thông thông minh. Chương này đã đạt được các kết quả kỹ thuật then chốt:

- **Kiến trúc đa tầng bền vững:** Hiện thực hóa mô hình điều phối từ `main_etl.py` đến các pipeline nguyên tử, tối ưu hóa tài nguyên và quản lý lỗi tập trung.
- **Chuẩn hóa Không gian - Thời gian:** Giải quyết thành công bài toán tích hợp dữ liệu GIS, tự động hóa phân đoạn đường và xử lý lịch âm cho ngày lễ Việt Nam.
- **Hợp nhất dữ liệu AI:** Tích hợp thành công mô hình **YOLOv8** để thu thập lưu lượng xe quy đổi PCU, kết hợp cùng API vận tốc và thời tiết.
- **Vận hành tự động:** Thiết lập cơ chế chạy định kỳ qua Batch Script và Task Scheduler, đảm bảo kho dữ liệu tự vận hành ổn định 24/7.

Kết quả của Chương 8 là một nền tảng dữ liệu sạch và giàu ngữ cảnh, sẵn sàng cho việc xây dựng Dashboard và các mô hình phân tích nâng cao ở chương kế tiếp.

11 Chi tiết nghiệp vụ hệ thống Traffic IOC

Chương này tập trung trình bày các chức năng nghiệp vụ cốt lõi mà hệ thống đã hiện thực hóa. Điểm nhấn của hệ thống không nằm ở việc hiển thị dữ liệu thô, mà ở cách trực quan hóa, cách xử lý logic nghiệp vụ và các phương pháp tối ưu hóa hiệu năng ứng dụng, được chia làm hai phân hệ riêng biệt: Phân hệ Quản trị & Điều hành (dành cho Sở GTVT) và Phân hệ Người dùng cuối (dành cho người dân).

11.1 Tổng quan hệ thống và Triết lý thiết kế

11.1.1 Giới thiệu tổng quan hệ thống

Hệ thống được phát triển trong đồ án không chỉ đơn thuần là một phần mềm theo dõi bản đồ, mà được định hình là một **Hệ thống Hỗ trợ Ra quyết định (Decision Support System - DSS)** chuyên biệt cho lĩnh vực Giao thông thông minh (ITS). Hệ thống đóng vai trò là cầu nối cốt lõi, chuyển hóa dữ liệu thô khổng lồ từ Data Warehouse thành các thông tin có tính hành động (Actionable Insights). Hệ thống được chia làm hai phân hệ tương tác hai chiều:

- **Phân hệ Admin (Command Center):** Dành cho Sở GTVT và các chuyên gia quy hoạch, cung cấp góc nhìn vĩ mô (Macro-level), đánh giá năng lực hạ tầng và phát hiện điểm nghẽn để tối ưu hóa nguồn lực điều tiết.
- **Phân hệ Mobile App (Citizen View):** Dành cho người tham gia giao thông, cung cấp góc nhìn vi mô (Micro-level), cá nhân hóa thông tin theo lộ trình để giúp người dân ra quyết định di chuyển tối ưu nhất.

Điểm nhấn (Key Highlight) của toàn bộ hệ thống là nguyên tắc **Single Source of Truth (Nguồn chân lý duy nhất)**. Bất kỳ một sự cố ùn tắc nào được phát hiện bởi hệ thống AI/Data Warehouse đều được phản ánh đồng thời lên Dashboard của cơ quan quản lý và đẩy cảnh báo trực tiếp (Real-time Alert) xuống thiết bị của người dân, tạo ra sự đồng bộ tuyệt đối trong việc điều phối giao thông đô thị.

11.1.2 Nguyên tắc thiết kế Giao diện & Trải nghiệm người dùng (UI/UX)

Thiết kế giao diện cho hệ thống giám sát giao thông đòi hỏi các tiêu chuẩn khắt khe về "Thời gian nhận thức" (Glanceability) — khả năng người dùng tiếp nhận thông tin chính xác chỉ trong vài phần mười giây. Nhóm đã tham khảo hệ thống nguyên tắc thiết kế *Material Design* của Google (áp dụng trên Google Maps) và *Ant Design* (chuyên biệt cho các Dashboard dữ liệu lớn) để xây dựng hệ thống thiết kế (Design System) riêng:

1. Hệ thống màu sắc (Color Semantics) theo tiêu chuẩn Giao thông: Màu sắc trong hệ thống không được chọn theo sở thích thẩm mỹ mà tuân thủ nghiêm ngặt theo 6 cấp độ của tiêu chuẩn Phân loại mức độ phục vụ **LOS (Level of Service)** do Cục Quản lý Đường bộ Liên bang Mỹ (FHWA) quy định. Cụ thể, các mã màu (Hex code) được áp dụng đồng bộ trên toàn hệ thống:

- **LOS A (Màu Xanh lá - #52c41a):** Trạng thái thông thoáng, đại diện cho dòng chảy tự do (Free-flow).
- **LOS B (Màu Xanh lơ - #a0d911):** Trạng thái khá thông thoáng, các phương tiện di chuyển ổn định.
- **LOS C (Màu Vàng - #fadb14):** Trạng thái trung bình, tốc độ bắt đầu bị ảnh hưởng bởi dòng xe xung quanh.
- **LOS D (Màu Cam - #fa8c16):** Trạng thái mật độ cao, dòng xe đông, di chuyển chậm.
- **LOS E (Màu Đỏ - #f5222d):** Trạng thái đông xe, năng lực thông hành đạt giới hạn, thường xuyên dừng chờ.
- **LOS F (Màu Đỏ sẫm - #820014):** Trạng thái ùn tắc nghiêm trọng, kẹt cứng, hệ thống hạ tầng vượt quá sức chứa.

Việc đồng bộ hóa bảng màu này từ biểu đồ Web Admin cho đến nét vẽ tuyến đường trên Mobile App giúp giảm thiểu tối đa tải lượng nhận thức (Cognitive Load). Người quản lý hay tài xế không cần tốn thời gian "giải mã" màu sắc, mà ngay lập tức phản xạ theo bản năng thị giác (Màu đỏ = Dừng lại/Cảnh báo).

2. Lựa chọn họ Font chữ (Typography): Hệ thống sử dụng họ font chữ không chân (Sans-serif) **Inter** (hoặc **Roboto**) làm bộ font tiêu chuẩn. Đây là quyết định mang tính kỹ thuật vì:

- **Độ đọc chữ số xuất sắc (Tabular Figures):** Trong các hệ thống OLAP có hàng triệu bản ghi, các con số cần được căn chỉnh thẳng hàng dọc. Font Inter hỗ trợ tính năng Tabular Numbers, giúp các con số có độ rộng bằng nhau, cực kỳ tối ưu cho việc đọc lướt qua các bảng Data Grid (Bảng dữ liệu tra cứu).
- **Tính hiển thị trên thiết bị di động:** Font Sans-serif với chiều cao chữ (X-height) lớn giúp nội dung vẫn sắc nét và dễ đọc trên Mobile App, đặc biệt trong môi trường ánh sáng ngoài trời có độ chói cao khi người dân đang chạy xe.

3. Tối ưu hóa không gian (Space Optimization): Do đặc thù màn hình trung tâm điều hành (Command Center) thường phải chứa hàng chục biểu đồ, nhóm áp dụng nguyên lý **Data-ink Ratio** (Tỷ lệ mực-dữ liệu) của Edward Tufte: Lược bỏ tối đa các đường viền, lưới (gridlines) không cần thiết. Các chi tiết rườm rà được ẩn đi và chỉ hiện ra dưới dạng *Custom Tooltip* khi người dùng tương tác (Hover/Click), giúp hệ thống chứa được một mật độ thông tin khổng lồ nhưng không gây rối mắt.

11.2 Phân hệ Quản trị & Điều hành (Admin Dashboard)

Phân hệ này đóng vai trò như một Trung tâm chỉ huy (Command Center), cung cấp cho các nhà quản lý giao thông những công cụ phân tích từ bao quát đến chi tiết.

11.2.1 Nghiệp vụ 1: Bảng điều khiển Tổng quan (Executive Dashboard)

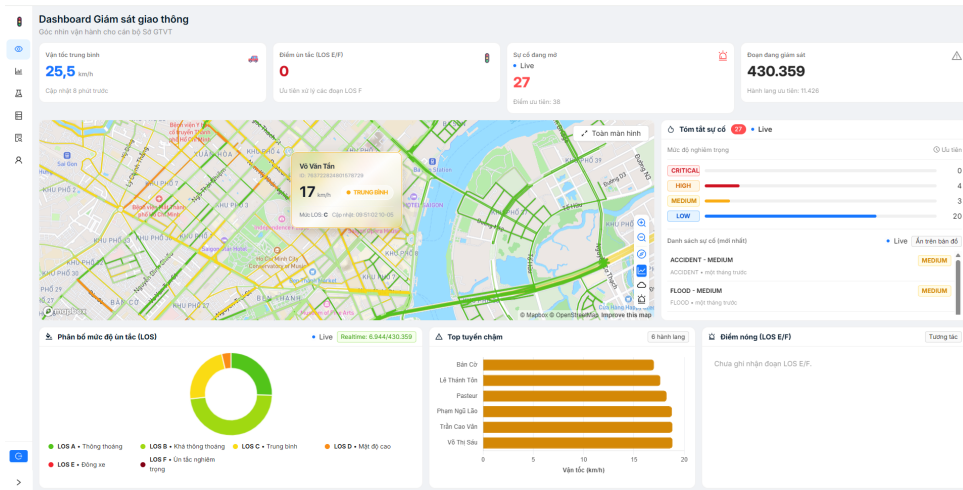
Mô tả nghiệp vụ: Cung cấp bức tranh toàn cảnh (bird's-eye view) về "sức khỏe" của mạng lưới giao thông. Cho phép ban lãnh đạo nắm bắt tình hình thực tế chỉ trong vài giây đầu tiên đăng nhập.

Cách thực hiện:

- **Thẻ KPI Tổng quan:** Sử dụng các thẻ thông tin (Statistic Cards) để hiển thị 4 chỉ số cơ bản quan trọng nhất: *Vận tốc trung bình, Điểm ùn tắc, Sự cố đang mở, và Đoạn đang giám sát.*
- **Deep-linking (Liên kết sâu):** Hỗ trợ truyền tham số tọa độ hoặc mã đoạn đường trực tiếp trên URL (`?segmentId=...`) giúp bản đồ tự động bay (flyTo) đến vị trí cần giám sát ngay khi mở trình duyệt, tạo thuận lợi cho việc báo cáo nhanh.

Cách tối ưu (Hiệu năng):

- **Tối ưu tính toán KPI:** Việc tổng hợp các chỉ số KPI đòi hỏi truy vấn quét qua toàn bộ dữ liệu sự kiện trong ngày. Nhóm đã tối ưu bằng cách triển khai cơ chế Caching (Redis) ở tầng Backend. Các chỉ số KPI được tính toán nền và cập nhật vào Cache mỗi 5 phút, giúp trang Dashboard tải tức thì (dưới 300ms) ngay khi nhà quản lý truy cập.
- **Tối ưu tải dữ liệu bản đồ (Map Rendering):** Việc truyền tải đồng thời dữ liệu không gian (Geometry) của 430000 đoạn đường lên Frontend là một thách thức lớn về băng thông. Nhóm đã áp dụng chuỗi giải pháp toàn diện:
 - *Tại Database:* Sử dụng hàm `ST_Simplify` của PostGIS để giảm bớt số lượng đỉnh (vertices) không cần thiết của các hình học không gian, làm nhẹ đáng kể kích thước file GeoJSON.
 - *Tại Backend:* Sử dụng thư viện `compression` (chuẩn nén Gzip/Brotli) để thu nhỏ luồng dữ liệu JSON trước khi phản hồi qua HTTP response.
 - *Tại Frontend:* Ứng dụng `IndexedDB` để lưu cache toàn bộ tọa độ không gian tĩnh của 430000 đoạn đường ngay tại trình duyệt. Ở các lần tải trang sau, Frontend chỉ gọi API để lấy các thuộc tính biến động (TTI, Vận tốc, LOS) thay vì tải lại toàn bộ bản đồ gốc, giúp giảm tới 90% băng thông mạng.



Hình 67: Bảng điều khiển Tổng quan với hệ thống thể KPI cơ bản

11.2.2 Nghiệp vụ 2: Giám sát Năng lực thông hành (Corridor Performance)

Mô tả nghiệp vụ: Giám sát diễn biến năng lực thông hành của toàn hành lang theo từng khung giờ trong ngày, đánh giá hiệu năng hoạt động thông qua bộ chỉ số KPI chi tiết.

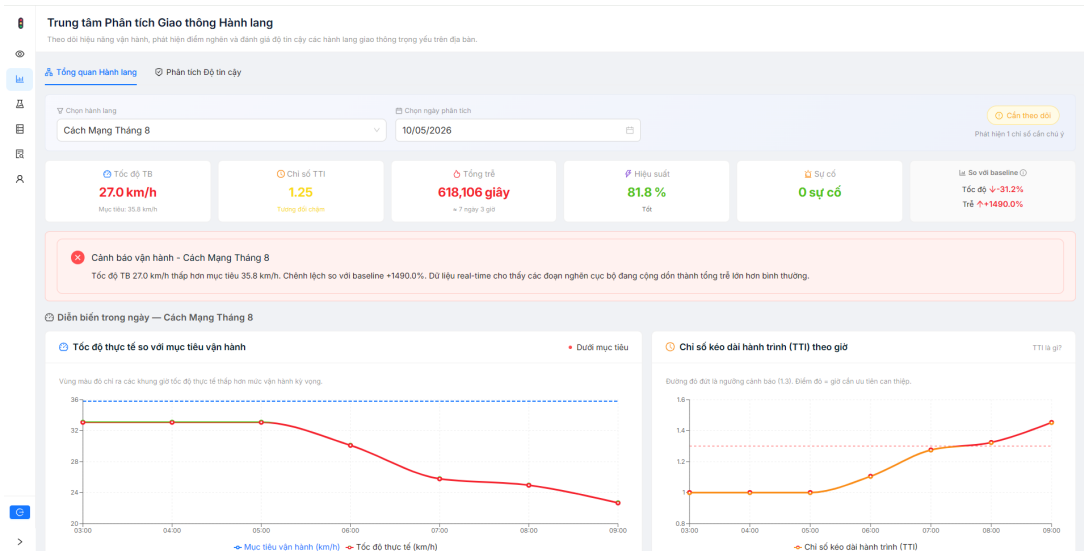
Cách thực hiện: Hệ thống hiển thị bộ 6 chỉ số chuyên sâu: *Tốc độ trung bình*, *Chỉ số TTI*, *Tổng thời gian trễ*, *Hiệu suất vận hành*, *Số lượng sự cố*, và *So sánh với Baseline*. Đặc biệt, tính năng đối chiếu với Baseline (dữ liệu lịch sử cùng kỳ) giúp hiển thị mũi tên xanh/đỏ báo hiệu mức độ cải thiện hay suy giảm so với tuần trước. Ngoài ra, hệ thống sử dụng thư viện Recharts để vẽ các biểu đồ chuỗi thời gian (Time-series Line Chart). Điểm nổi bật là việc thiết lập các đường mục tiêu (ví dụ: Tốc độ mục tiêu 35.8 km/h, Ngưỡng cảnh báo TTI 1.3). Bất kỳ khoảng thời gian nào giá trị thực tế vượt qua ngưỡng này, hệ thống tự động đổi màu cảnh báo ngay trên đường đồ thị.

Cách tối ưu (UX & Performance):

- **Tối ưu Trải nghiệm người dùng (Xử lý Nghịch lý dữ liệu):** Thường xuyên xảy ra hiện tượng "Tốc độ trung bình tăng nhưng Tổng độ trễ lại tăng vọt" (do kẹt xe cục bộ tại một vài nút giao). Để tránh gây hoang mang cho người ra quyết định, nhóm đã nhúng một *Custom Tooltip* (biểu tượng (i)) ngay cạnh thẻ Baseline nhằm giải thích cặn kẽ nguyên nhân của sự bất đồng nhất này.
- **Tối ưu hiệu năng Render biểu đồ:** Thay vì chèn nhỏ biểu đồ thành nhiều đoạn thẳng (gây phình to cây DOM và giảm hiệu năng), nhóm ứng dụng kỹ thuật **SVG Linear Gradient**. Thuật toán tại Frontend tính toán giá trị offset động (điểm cắt giữa giá trị thực tế và ngưỡng Threshold):

$$Offset = \max \left(0, \min \left(1, \frac{Max_{value} - Threshold}{Max_{value} - Min_{value}} \right) \right) \quad (6)$$

Dựa vào offset này, một thẻ <defs> chứa gradient dọc được áp dụng trực tiếp làm stroke cho toàn bộ thẻ <Line>, giúp biểu đồ đổi màu (Xanh/Đỏ) chính xác đến từng pixel với hiệu năng O(1) tại khâu render.



Hình 68: Nghiệp vụ Giám sát Năng lực thông hành với hệ thống KPI và biểu đồ Split-color Gradient

11.2.3 Nghiệp vụ 3: Phân tích Độ tin cậy & Biến động (Reliability Analytics)

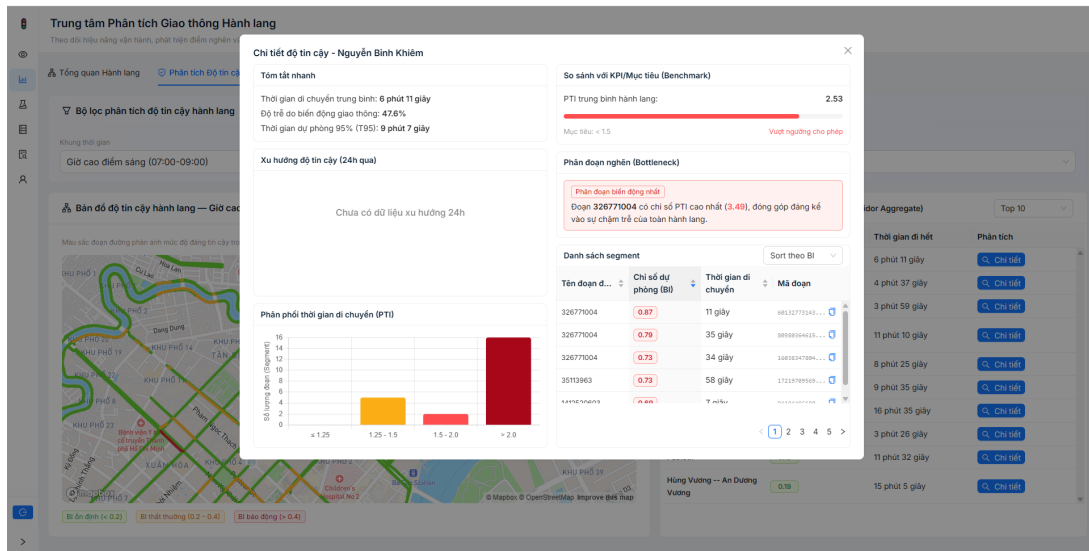
Mô tả nghiệp vụ: Đánh giá mức độ bất ổn định của hành lang thông qua Chỉ số dự phòng (Buffer Index - BI) và Chỉ số thời gian lập kế hoạch (Planning Time Index - PTI), trả lời câu hỏi: "Người đi đường cần dự phòng bao nhiêu thời gian để chắc chắn 95% không bị trễ giờ?".

Cách thực hiện: Hệ thống thu thập thời gian đi lại của các phân đoạn (T_{avg} , T_{95}) và hiển thị trong một Modal (Popup) chi tiết. Modal cung cấp cái nhìn đối chiếu giữa mức độ trung bình của toàn tuyến và biểu đồ phân phối PTI của từng đoạn thành phần.

Cách tối ưu (Xử lý Aggregate Fallacy): Khó khăn lớn nhất là việc tính toán BI cho một tuyến đường dài từ nhiều đoạn nhỏ. Nếu lấy trung bình cộng BI của từng đoạn (*Average of Ratios*) sẽ dẫn đến sai số toán học nghiêm trọng (ví dụ UI hiển thị 37% nhưng thực tế là 75%). Nhóm đã tối ưu bằng cách đẩy logic tính toán xuống tầng Cơ sở dữ liệu (Database Layer). Sử dụng Prisma để thực hiện Aggregation: Tính **Tổng thời gian trung bình** ($\sum T_{avg}$) và **Tổng thời gian phân vị 95** ($\sum T_{95}$) của toàn hành lang trước, sau đó mới tính $BI_{corridor}$ theo công thức:

$$BI_{corridor} = \frac{\sum T_{95} - \sum T_{avg}}{\sum T_{avg}} \quad (7)$$

Cách tối ưu này vừa giải quyết triệt để lỗi logic, vừa giảm tải lượng RAM tiêu thụ trên Node.js server do không phải kéo hàng ngàn record lên memory để loop.



Hình 69: Popup chi tiết Phân tích Độ tin cậy hành lang và hiển thị phân vị T_{95}

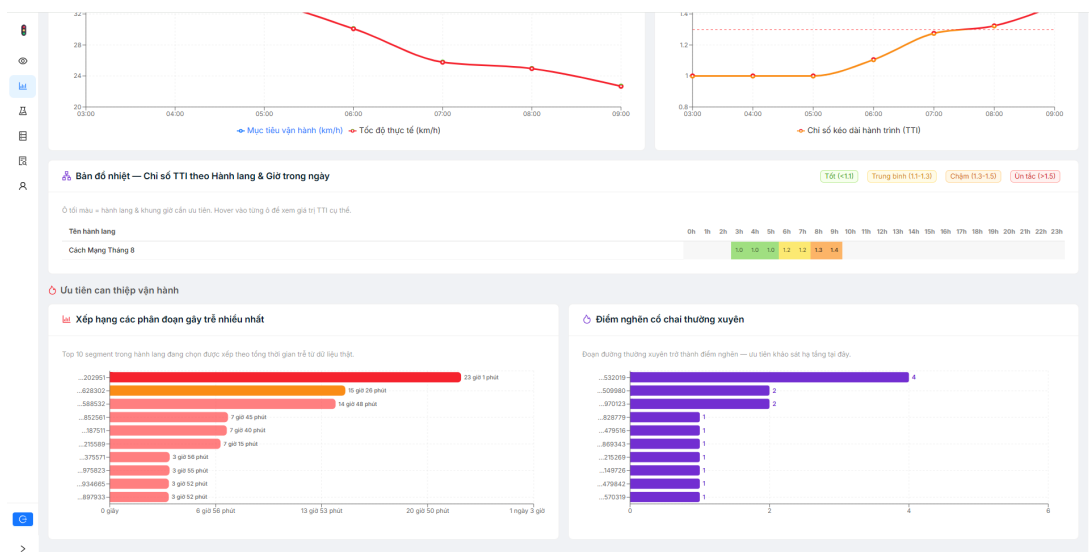
11.2.4 Nghiệp vụ 4: Định vị & Xếp hạng Điểm nghẽn (Bottleneck Detection)

Mô tả nghiệp vụ: Bóc tách và xếp hạng các phân đoạn đường (Segments) gây thiệt hại lớn nhất để nhà quản lý ưu tiên nguồn lực nâng cấp hạ tầng. Tính năng này được tích hợp liền mạch vào Trung tâm Phân tích Hành lang (Analytic Page) để nhà quản lý đối chiếu mà không cần chuyển trang.

Cách thực hiện: Nhóm triển khai hai biểu đồ Bar Chart chạy song song mang hai ý nghĩa nghiệp vụ hoàn toàn khác biệt:

- **Đánh giá tính Nghiêm trọng (Severity):** Xếp hạng dựa trên "Tổng thời gian trễ lũy kế". Giúp nhận diện đoạn đường tiêu tốn nhiều thời gian của xã hội nhất.
- **Đánh giá tính Kinh niên (Persistence):** Xếp hạng dựa trên "Tần suất xuất hiện điểm nghẽn". Giúp nhận diện các nút thắt cổ chai lặp đi lặp lại.

Cách tối ưu (Clean UI & UX): Thay vì lãng phí không gian màn hình bằng việc liệt kê lại một bảng danh sách các đoạn đường, nhóm đã thiết kế một **Custom Tooltip** gắn trực tiếp vào biểu đồ Recharts. Khi người dùng di chuột vào thanh Bar, hệ thống hiển thị mã ID đoạn đường (được tự động cắt gọn bằng `formatSegmentId`) kèm theo nút Copy. Tổng thời gian trễ (ví dụ: 4.822.977 giây) được format động thành "55 ngày 19 giờ" giúp nhà quản lý dễ dàng cảm nhận mức độ thiệt hại.



Hình 70: Biểu đồ phân tách tính Nghiêm trọng và tính Kinh niên của Điểm nghẽn

11.2.5 Nghiệp vụ 5: Phân tích Đa chiều OLAP & Đánh giá Hạ tầng

Mô tả nghiệp vụ: Cung cấp cái nhìn đa chiều (OLAP) về mối tương quan giữa sức chứa hạ tầng vật lý và lưu lượng giao thông thực tế thông qua các biểu đồ phân tích cao cấp.

Cách thực hiện: Tích hợp hai loại biểu đồ đặc thù:

- **Traffic Heatmap (Bản đồ nhiệt 2D):** Trục X là thời gian, Trục Y là tên đường. Màu sắc thể hiện mức độ ùn tắc, giúp phát hiện quy luật "giờ chết" không gian - thời gian.
- **Cross-analysis Bubble Chart (Biểu đồ bong bóng 4D):** Kết hợp Sức chứa thiết kế (Trục X), Mức độ kẹt (Trục Y), Hệ số V/C Ratio (Màu bóng) và Lưu lượng/Độ trễ (Kích thước bóng). Cho phép nhà quản lý nhìn thấy những tuyến đường đang "nhồi nhét" vượt tải trọng thiết kế dù chỉ số ùn tắc hiện tại có thể chưa cao.
- **Biểu đồ Phân rã Độ trễ (Drilldown Delay Chart):** Tính năng "khoan sâu" (Drill-down) đặc trưng của OLAP. Khi nhà quản lý phát hiện một tuyến đường bất thường trên Heatmap, họ có thể click vào để bóc tách dữ liệu từ cấp độ vĩ mô (toàn tuyến) xuống vi mô (từng phân đoạn đường cụ thể), từ đó xác định chính xác nút giao nào là nguyên nhân gốc rễ (Root Cause) gây chậm trễ.

Cách tối ưu: Các truy vấn OLAP quét qua hàng triệu dòng dữ liệu sự kiện (event logs) thường mất nhiều phút để thực thi. Nhóm đã tối ưu bằng cách triển khai **Materialized Views** tại Database PostgreSQL. Dữ liệu tổng hợp cho Heatmap và Bubble Chart được tính toán sẵn (Pre-computation) theo một job chạy nền (Cronjob) mỗi 15 phút. Nhờ đó, thao tác tải trang hoặc filter trên giao diện Dashboard phản hồi với độ trễ dưới 1 giây.

Hình 71: Dashboard BI & OLAP: Heatmap thời gian và Bubble Chart đánh giá hạ tầng

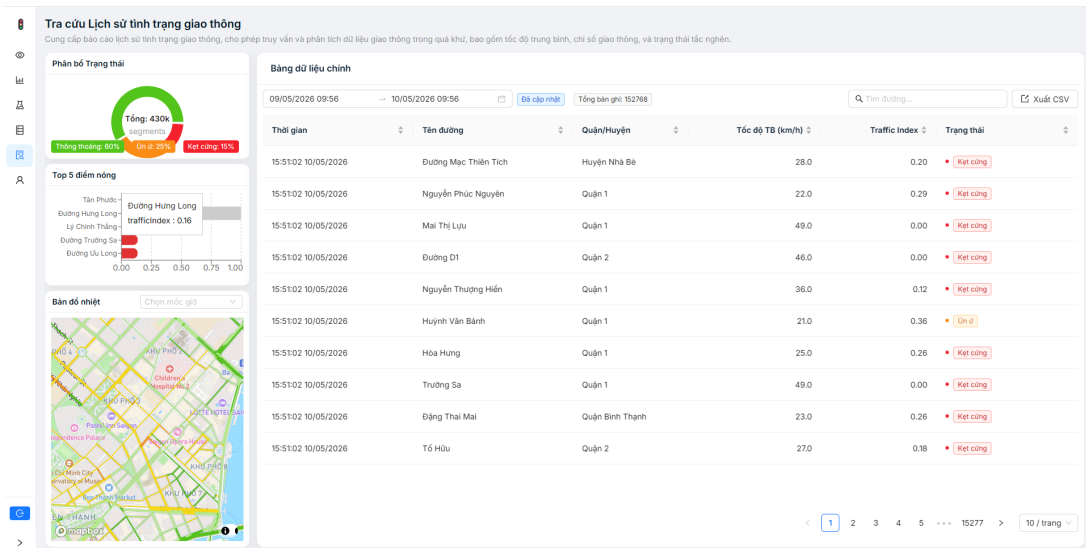
11.2.6 Nghiệp vụ 6: Tra cứu Lịch sử & Tin nóng (Marquee Alert)

Mô tả nghiệp vụ: Cho phép truy xuất hàng trăm ngàn bản ghi sự cố trong quá khứ để lập báo cáo, đồng thời nhận cảnh báo theo thời gian thực về các tình trạng kẹt cứng hiện tại.

Cách thực hiện:

- **Dải Tin nóng (Marquee Alert):** Hệ thống tích hợp một dải thanh cuộn (LiveNewsTicker) hiển thị text động từ Socket, chạy liên tục trên Layout toàn cục (Global Layout), hỗ trợ nhà quản lý nhận cảnh báo sớm ở bất kỳ trang nào, kể cả khi đang xem báo cáo lịch sử.
- **Data Grid Lịch sử:** Bảng dữ liệu tra cứu trong màn hình chính, tích hợp bộ lọc thời gian (Time Range Picker), biểu đồ phân bố trạng thái (Donut Chart) và tính năng Xuất file CSV.
- **Bản đồ Tọa lại thời gian (Spatial-Temporal Playback):** Thay vì chỉ hiển thị bảng số liệu khô khan, hệ thống cho phép chọn một mốc thời gian cụ thể trong quá khứ. Ngay lập tức, một bản đồ tĩnh (Snapshot Map) sẽ render lại toàn bộ hiện trạng mạng lưới giao thông y hệt như thời khắc đó, hỗ trợ đắc lực cho công tác hậu kiểm và phân tích "mổ xẻ" các vụ kẹt xe lớn.

Cách tối ưu: Với tập dữ liệu hàng trăm ngàn bản ghi, việc truy vấn lịch sử được tối ưu thông qua **Server-side Pagination** và đánh **Index (B-Tree)** trên cột thời gian (timestamp) và segment_id. Nhóm cũng triển khai cơ chế chặn khoảng thời gian truy vấn (tối đa 7-30 ngày) để bảo vệ DB Server khỏi các truy vấn cạn kiệt tài nguyên.

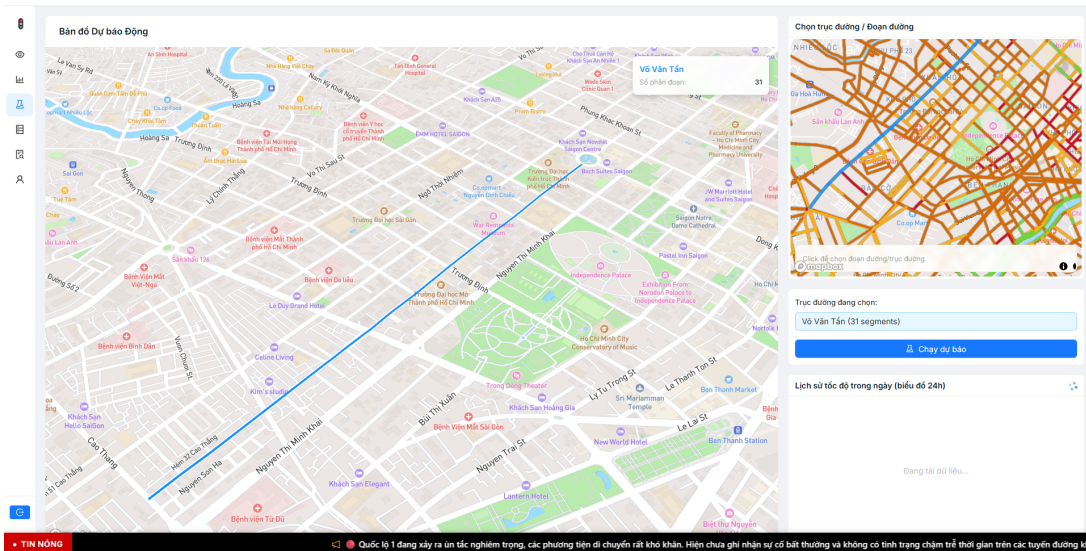


Hình 72: Giao diện Tra cứu lịch sử tích hợp thành Tin nóng thời gian thực

11.2.7 Nghiệp vụ 7: Mô phỏng & Dự báo giao thông động (Traffic Simulation)

Mô tả nghiệp vụ: Cung cấp công cụ dự báo tình trạng giao thông trong tương lai gần (ví dụ 15-30 phút tới), cho phép nhà quản lý có cái nhìn đi trước thời gian thực để đưa ra phương án điều tiết sớm.

Cách thực hiện: Hệ thống gọi API đến Module Dự báo (Predictive Module) để lấy kết quả từ mô hình AI và trực quan hóa các điểm nghẽn có nguy cơ hình thành. Điểm nổi bật về UX nằm ở **Kiến trúc Bản đồ Kép (Dual-Map UI)**: Thay vì nhồi nhét mọi tương tác lên một màn hình gây rối mắt, hệ thống chia giao diện làm hai phần riêng biệt. Một bản đồ phụ (Selection Map) được đặt ở bảng điều khiển bên phải chuyên dùng để tìm kiếm và chọn đoạn đường cần phân tích. Sau khi xác nhận, kết quả dự báo AI mới được render lên bản đồ chính (Predictive Map) cực lớn ở trung tâm, đi kèm biểu đồ đối chiếu tốc độ dự báo với baseline lịch sử trong ngày. Thiết kế này giúp tối ưu hóa không gian làm việc và ngăn chặn tình trạng thao tác nhầm lẫn (misclick).



Hình 73: Giao diện Nghiệp vụ Mô phỏng & Dự báo với kiến trúc Dual-Map

11.3 Phân hệ Người dùng cuối (Citizen Mobile App)

Phân hệ Mobile App chuyển hóa các phân tích vĩ mô thành quyết định cá nhân hóa cho từng chuyến đi của người dân.

11.3.1 Nghiệp vụ 1: Tìm đường thông minh theo Thời gian thực (Dynamic Smart Routing)

Mô tả nghiệp vụ: Tìm kiếm và đề xuất lộ trình tối ưu nhất (tránh điểm nghẽn, thời gian di chuyển ổn định) thay vì chỉ ưu tiên khoảng cách địa lý ngắn nhất như các ứng dụng bản đồ truyền thống.

Cách thực hiện: Hệ thống sử dụng phần mở rộng pgRouting của PostgreSQL để chạy thuật toán **Bi-directional A*** (pgr_bdAstar) trên cấu trúc Topology mạng lưới giao thông thay vì thuật toán Dijkstra truyền thống.

Để làm rõ lý do chọn thuật toán này cho bài toán dẫn đường đô thị thời gian thực, nhóm trình bày bảng phân tích và so sánh cơ chế của các giải thuật:

Thuật toán	Cơ chế hoạt động	Ưu / Nhược điểm
Dijkstra	Duyệt tỏa tròn ra mọi hướng (như vết dầu loang) từ điểm Đầu cho đến khi gặp điểm Đích.	Không dùng hàm Heuristic (tìm kiếm mù). Đảm bảo tìm được kết quả tối ưu nhưng duyệt qua vô số đỉnh không liên quan, hiệu năng rất chậm trên đồ thị toàn thành phố.
A* (A-Star)	Sử dụng hàm Heuristic (khoảng cách đường chim bay) để định hướng, luôn ưu tiên mở rộng các đỉnh hướng về phía Đích.	Nhanh hơn Dijkstra rất nhiều do không gian tìm kiếm thu hẹp có định hướng. Tuy nhiên với quãng đường xuyên thành phố, không gian quét vẫn phình to đáng kể.
Bi-directional A* (bdAstar)	Khởi chạy 2 luồng A* đồng thời: một luồng từ Đầu hướng về Đích, một luồng từ Đích hướng ngược về Đầu, và dừng lại khi gặp nhau ở giữa.	Lựa chọn của hệ thống: Vùng quét không gian được thu hẹp tối đa. Tốc độ phản hồi cực nhanh (dưới 100ms) ngay cả với lộ trình kéo dài hàng chục kilomet. Bù lại, hệ thống phải cung cấp tọa độ hình học (x,y) cho từng cạnh để tính Heuristic.

Bảng 6: So sánh và lựa chọn thuật toán tìm đường trong pgRouting

Bên cạnh tốc độ bứt phá từ việc ứng dụng bdAstar, giá trị cốt lõi của hệ thống nằm ở việc thiết lập hàm **Chi phí thời gian thực (Real-time Cost)**. Thay vì dùng công thức tính toán đơn giản, nhóm đã thiết kế một hàm chi phí đa biến tại tầng cơ sở dữ liệu:

$$Cost = (TravelTime + \alpha \times Distance) \times ConfidencePenalty \quad (8)$$

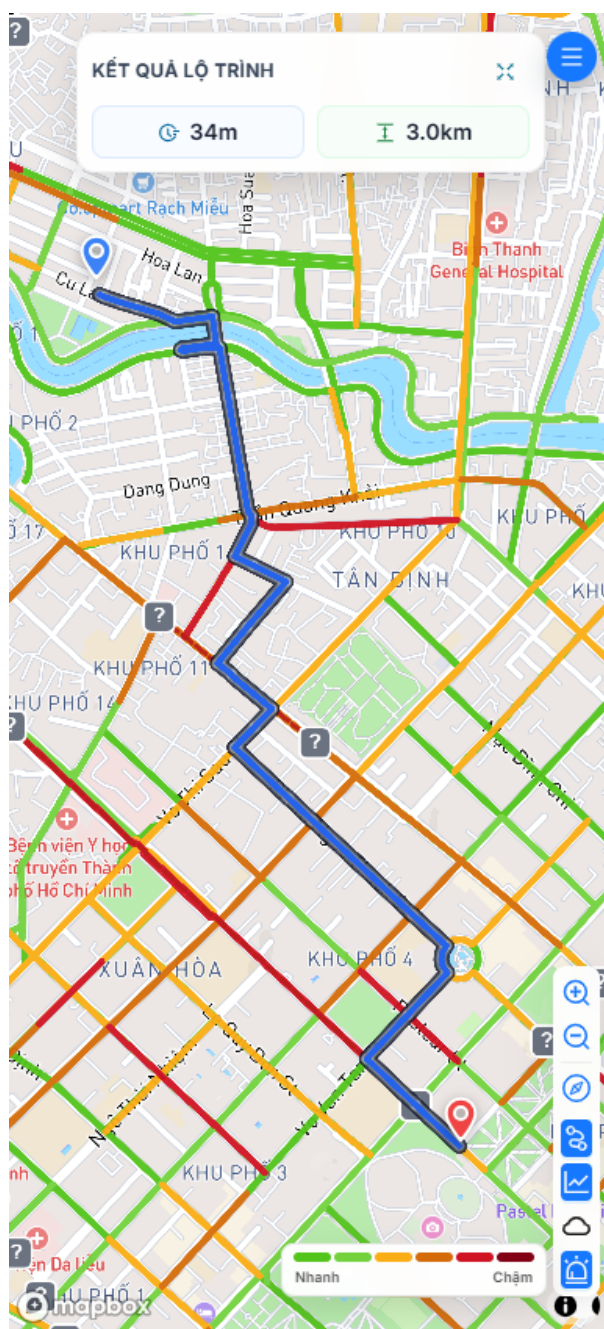
Trong đó, các đại lượng được hệ thống định nghĩa như sau:

- TravelTime (Thời gian di chuyển):** Yếu tố cốt lõi của thuật toán. Hệ thống ưu tiên chọn các cung đường có thời gian di chuyển thấp nhất để né các điểm nghẽn ùn tắc.
- $\alpha \times Distance$ (Hệ số phạt đường vòng):** Với hệ số $\alpha = 0.02$ (tương đương đi vòng 1km sẽ bị phạt cộng thêm 20 giây vào Cost). Giá trị này được thiết lập dựa trên khái niệm **Giá trị Thời gian (Value of Travel Time - VoT)**. Nếu chỉ tối ưu bằng thời gian thuần túy, thuật toán sẽ có xu hướng lùa xe vào các ngõ hẻm nhỏ hẹp để tiết kiệm vài giây (hiện tượng Rat-running). Trọng số này ép thuật toán ưu tiên lộ trình thẳng/ngắn nhất.
- ConfidencePenalty (Hệ số phạt độ tin cậy):** Bằng 1.0 cho dữ liệu đo đạc trực tiếp, 1.05 cho dữ liệu nội suy KNN-IDW, và 1.1 cho dữ liệu tính. Mức phạt 5% đến 10% này mô phỏng **Hành vi chọn đường e ngại rủi ro (Risk-averse Route Choice)**. Hệ số này giúp thuật toán "thiên vị" việc điều hướng vào đoạn đường có dữ liệu rõ ràng, giảm thiểu rủi ro đi vào "điểm mù".

Cách tối ưu (Bù đắp dữ liệu bằng Spatial-Temporal Imputation): Trong thực tế, mạng lưới cảm biến luôn tồn tại các "điểm mù" (cả về không gian lẫn thời gian). Thay vì phụ thuộc vào các mô hình AI/Deep Learning vốn đòi hỏi tài nguyên máy chủ lớn và khó chạy thời gian thực bên trong Database, nhóm đã ứng dụng các phương pháp thống kê không gian - thời gian (Spatiotemporal Imputation) trực tiếp trên Materialized View:

- **Điểm mù thời gian (Temporal Decay):** Khi một cảm biến mất kết nối, hệ thống không giữ nguyên trạng thái kẹt xe vĩnh viễn. Thay vào đó, kỹ thuật **Suy giảm hàm mũ (Exponential Decay)** được áp dụng. Hàm số $e^{-0.046 \times t}$ (với hằng số $\lambda = 0.046$ được thiết kế để độ tin cậy giảm chính xác 50% sau mỗi chu kỳ 15 phút). Chu kỳ bán rã (Half-life) 15 phút này là một quy tắc kinh nghiệm phổ biến trong kỹ thuật giao thông, vì các biến động ngắn hạn thường tự triệt tiêu sau mốc này. Thuật toán từ từ "mượt mà" trả trạng thái giao thông về lại vận tốc mặc định (Free-flow).
- **Điểm mù không gian (Inverse Distance Weighting - IDW):** Với những đoạn đường không có cảm biến, hệ thống dự đoán vận tốc thông qua thuật toán **K-Nearest Neighbors (KNN)** kết hợp **IDW**. Dữ liệu được tính toán thuần túy bằng đại số SQL và toán tử không gian $\leftrightarrow$ của PostGIS. Các đường càng gần nhau (khoảng cách d nhỏ) sẽ đóng góp trọng số $(1/d^2)$ càng lớn vào kết quả.

Cách tiếp cận Tất định (Deterministic) này giúp nội suy cấu trúc mạng lưới giao thông (Topology) hoàn chỉnh trong chưa tới 1 giây, tuân thủ chặt chẽ Định luật thứ nhất của Địa lý học (Tobler's First Law) và giúp thuật toán pgRouting luôn dẫn đường hợp lý nhất 24/7.



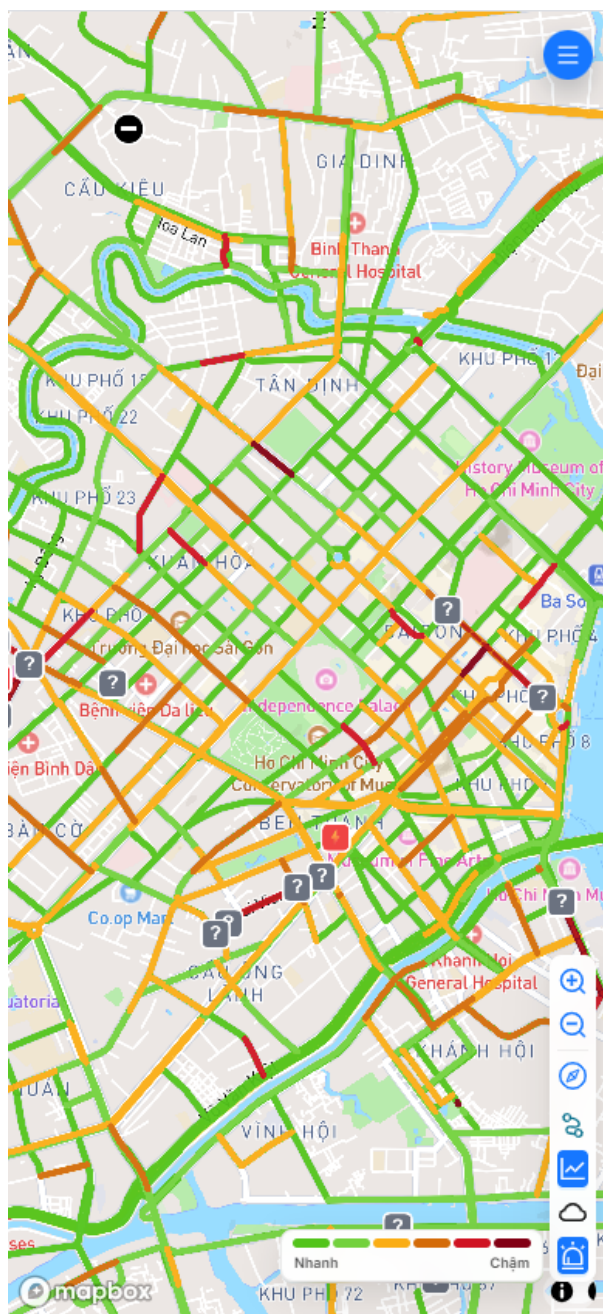
Hình 74: Nghiệp vụ Tìm đường thông minh né điểm nghẽn

11.3.2 Nghiệp vụ 2: Giám sát Bản đồ Cá nhân hóa

Mô tả nghiệp vụ: Cung cấp bản đồ giao thông được tô màu theo chuẩn LOS (Đỏ - Cam - Xanh) toàn thành phố để người dân chủ động giám sát tình hình khu vực xung quanh lộ trình của mình.

Cách thực hiện: Ứng dụng hiển thị một MapboxGL View, tích hợp luồng dữ liệu thời gian thực. Giao diện được thiết kế với độ tương phản cao, các nút bấm (MapControls) thao tác bằng ngón tay cái to bản (Thumb-friendly) phục vụ người đi xe máy. Nền tảng còn cho phép bật/tắt hiển thị các lớp bản đồ nâng cao như Tình trạng thời tiết (Weather Voronoi Layer) và các Sự cố giao thông đang mở.

Cách tối ưu: Thay vì render hàng chục ngàn đoạn thẳng lên thiết bị di động gây nóng máy và tụt pin, nhóm sử dụng kỹ thuật **Vector Tiles** (MVT). Cụ thể, hệ thống tích hợp luồng dữ liệu TomTom Flow Tiles và Incident Tiles qua một lớp proxy. Client sử dụng GPU để render các Vector Tile này, giúp bản đồ zoom/pan mượt mà 60 FPS dù hiển thị toàn bộ mạng lưới giao thông phức tạp.



Hình 75: Giao diện Bản đồ trạng thái giao thông và Cảnh báo cá nhân hóa

12 TRIỂN KHAI, VẬN HÀNH VÀ TỐI ƯU CHI PHÍ

Một hệ thống phần mềm chỉ thực sự mang lại giá trị khi nó được triển khai lên môi trường thực tế, đảm bảo khả năng phục vụ người dùng 24/7 với độ ổn định cao. Chương này trình bày chi tiết về chiến lược đưa dự án từ môi trường phát triển (Development) lên môi trường vận hành (Production), tập trung vào kiến trúc đám mây, công nghệ ảo hóa, luồng tích hợp liên tục và các bài toán tối ưu hóa tài nguyên.

12.1 Kiến trúc Triển khai (Deployment Architecture)

Hệ thống được thiết kế theo kiến trúc phân tầng (Layered Architecture: Controller - Service - Repository) nhằm đảm bảo sự rành mạch giữa luồng điều khiển, logic nghiệp vụ và tương tác dữ liệu. Dựa trên tính toán về tải lượng dữ liệu và ngân sách dự án, nhóm quyết định phân bổ các dịch vụ lên nền tảng đám mây kết hợp.

- **Backend API Server (ExpressJS) & AI Core (Python):** Đóng vai trò xử lý logic nghiệp vụ, chạy các mô hình dự báo và phân phối dữ liệu. Cả hai module này được đóng gói bằng Docker, lưu trữ tại **Docker Hub** và triển khai lên nền tảng **Azure App Services (Web App for Containers)**. Hệ thống sử dụng gói máy chủ **Basic B2** với cấu hình mạnh mẽ hơn để đáp ứng tải thực tế. Việc sử dụng dịch vụ PaaS này giúp ứng dụng tự động cân bằng tải và dễ dàng khởi động lại khi có sự cố.
- **Frontend App & Dashboard (React):** Được triển khai trực tiếp lên nền tảng **Vercel** thông qua công cụ **Vercel CLI**. Nhóm tận dụng gói **Hobby Plan** (Miễn phí) của Vercel, giúp tự động biên dịch các tệp tĩnh (Static Assets) và phân phối siêu tốc qua mạng lưới Global CDN, tối ưu hóa triệt để tốc độ tải trang cho cả người dùng Web và Mobile.
- **Cơ sở dữ liệu (Database Server):** Hệ thống sử dụng **PostgreSQL 17** để xử lý các truy vấn không gian - thời gian phức tạp và lưu trữ Data Warehouse. (Lưu ý: Thiết kế chi tiết và cấu hình triển khai hạ tầng Database được một thành viên khác trong nhóm phụ trách biên soạn ở chương riêng).

Hình 76: Sơ đồ kiến trúc triển khai hệ thống trên nền tảng đám mây

12.2 Quản lý Server và Containerization (Docker)

Để khắc phục triệt để vấn đề "chạy được trên máy tôi nhưng lỗi trên server" (It works on my machine), nhóm đã áp dụng triệt để công nghệ Containerization bằng **Docker** cho toàn bộ các dịch vụ.

12.2.1 Tối ưu hóa Docker Image

Việc đóng gói Backend ExpressJS và AI Core đòi hỏi sự tính toán kỹ lưỡng để giảm thiểu kích thước Image và đảm bảo tương thích môi trường đám mây.

- **Alpine Linux Compatibility & Multi-stage Builds:** Nhóm sử dụng base image `node:alpine` siêu nhẹ. Quy trình build được chia làm nhiều giai đoạn (Multi-stage). Giai đoạn đầu biên dịch mã TypeScript, sau đó Image cuối (Production) chỉ giữ lại file đã dịch và dependencies bắt buộc, giúp thu nhỏ kích thước Container từ hơn 1GB xuống dưới 150MB. Với Prisma ORM (sử dụng thư viện C cốt lõi `musl libc`), nhóm đã nhúng lệnh `npx prisma generate` vào luồng Build để đảm bảo tính tương thích 100% khi container Alpine khởi động.
- **Cấu hình cổng mạng (EXPOSE 80):** Đây là một điểm tối ưu mang tính quyết định khi làm việc với Azure App Services. Mặc định, nền tảng Azure sẽ liên tục ping kiểm tra sức khỏe (Health-check) vào cổng 80 của Container khi khởi động. Nếu ứng dụng Node.js/Python chạy ở cổng khác (như 3000) mà không khai báo, Azure sẽ rơi vào trạng thái chờ (Container Timeout) đúng 230 giây trước khi báo lỗi sập ứng dụng. Vì vậy, nhóm đã chủ động cấu hình `EXPOSE 80` trong `Dockerfile` và ràng buộc (bind) ứng dụng chạy thẳng trên cổng này, giúp container qua mặt được Health-check và khởi động thành công ngay lập tức.

12.2.2 Quản lý Container và Dọn dẹp Tài nguyên

Trong quá trình vận hành và cập nhật phiên bản liên tục, server rất dễ bị đầy ổ cứng do các image cũ và container rác (Dangling images) tích tụ. Nhóm đã thiết lập các script tự động hóa (Bash script) kết hợp lệnh `docker system prune` chạy định kỳ qua Cronjob để dọn dẹp các volume và network không còn sử dụng, duy trì sự ổn định cho hạ tầng.

12.3 Thiết lập luồng Triển khai (Deployment Flow)

Khác với các hệ thống đồ sộ cần luồng CI (Continuous Integration) phức tạp, nhóm đã tinh gọn quy trình phát hành tính năng bằng các kịch bản triển khai thủ công nhưng mang lại tốc độ linh hoạt cao.

Quy trình hoạt động của luồng Triển khai:

1. Với Backend & AI Core:

- Nhà phát triển chạy lệnh build trực tiếp từ máy cục bộ và sử dụng lệnh `docker push` để đẩy Image mới nhất lên **Docker Hub**.
- Sau khi Image đã cập nhật trên kho lưu trữ, thông qua Webhook hoặc thao tác trên Azure Portal, **Azure App Services** sẽ tự động kéo (Pull) image mới nhất về và khởi động lại container dịch vụ.

2. Với Frontend:

- Khi có thay đổi trên giao diện, nhà phát triển gõ lệnh `vercel -prod` trực tiếp thông qua **Vercel CLI** ở Terminal máy tính cục bộ.
- Toàn bộ mã nguồn sẽ được tải lên đám mây của Vercel, sau đó Vercel tự động biên dịch bản build Production và phân phối tức thì lên mạng lưới Global CDN toàn cầu.

12.4 Phân tích & Quản lý ngân sách (Budget & Resource Optimization)

Đối với các dự án Giao thông thông minh có luồng dữ liệu thời gian thực lớn, nếu không có chiến lược tối ưu hóa, chi phí duy trì Cloud Server có thể vượt ngoài tầm kiểm soát. Nhóm đã áp dụng các phương pháp kỹ thuật để tối ưu hóa ngân sách vận hành:

- Tận dụng tài nguyên hỗ trợ Giáo dục:** Tận dụng tối đa gói tín dụng (Credits) từ Azure for Students và các dịch vụ Free Tier dành cho môi trường Development, chỉ phân bổ ngân sách thực cho môi trường Production.
- Tối ưu hóa I/O Cơ sở dữ liệu:** Thay vì phải nâng cấp cấu hình CPU/RAM (Scale-up) của Database Server để đáp ứng các biểu đồ OLAP, việc thiết kế **Materialized Views** kết hợp **Redis Cache** đã giúp giảm hơn 80% khối lượng tính toán trực tiếp trên đĩa cứng (Disk I/O). Việc này cho phép hệ thống phục vụ hàng ngàn truy vấn với một máy chủ có cấu hình khiêm tốn.

Thông qua việc thiết kế kiến trúc triển khai thực dụng và áp dụng triệt để Containerization, nhóm không chỉ đảm bảo hệ thống vận hành trơn tru mà còn chứng minh được tính khả thi về mặt kinh tế để sẵn sàng bàn giao sản phẩm cho cơ quan quản lý.

13 Đánh giá hệ thống

13.1 Đánh giá Schema

Kiến trúc cuối cùng của Kho dữ liệu được xây dựng theo mô hình Galaxy Schema kết hợp với Snowflake Dimensions. Phần này đánh giá hiệu quả của kiến trúc dựa trên các tiêu chí nền tảng đối với một hệ thống Kho dữ liệu hiện đại.

13.1.1 Khả năng đáp ứng nghiệp vụ đa chiều

Mô hình Galaxy cho phép kết nối đa chiều và xử lý các yêu cầu phân tích phức tạp mà các Star Schema rời rạc không đáp ứng được.

- **Khả năng phân tích chéo:** Hệ thống sử dụng bộ Conformed Dimensions như `dim_time`, `dim_segment` và `dim_vehicle_type`, giúp tích hợp dữ liệu từ nhiều lĩnh vực khác nhau. Điều này cho phép đặt các quy trình vận hành, sự cố và hành vi người dùng trong cùng một không gian phân tích.
- **Ví dụ nghiệp vụ:** Dữ liệu từ `fact_traffic_flow` (lưu lượng) và `fact_incident` (sự cố) có thể được kết hợp để phân tích mức độ ảnh hưởng của tai nạn đến tốc độ giải tỏa dòng xe tại các đoạn hoặc nút giao trọng điểm.
- **Độ phủ yêu cầu nghiệp vụ:** Bốn nhóm Fact chính (vận hành, sự cố, hành vi, hiệu quả) cung cấp toàn bộ khả năng phân tích từ vi mô (chuyến đi, từng sự cố) đến vĩ mô (xu hướng lưu lượng, hiệu quả định tuyến).

13.1.2 Tính toàn vẹn và cấu trúc không gian

Mô hình Snowflake được áp dụng cho nhóm chiều không gian (`dim_segment` → `dim_node` → `dim_way` → `dim_road`), giúp thể hiện chính xác đặc tính topo của mạng lưới giao thông.

- **Phản ánh đúng thực tế hạ tầng:** Cấu trúc phân cấp hỗ trợ mô hình hóa đầy đủ quan hệ giữa segment, tuyến đường, nút giao và hệ thống đường bộ tổng thể. Mọi thay đổi ở cấp quản lý cao (ví dụ: cập nhật tên đường hoặc phân loại tuyến) sẽ tự động phản ánh lên toàn bộ các cấp dưới, giúp duy trì tính toàn vẹn dữ liệu.
- **Tối ưu không gian lưu trữ:** Chuẩn hóa dữ liệu không gian giúp loại bỏ sự trùng lặp thông tin mô tả, tiết kiệm đáng kể dung lượng lưu trữ so với mô hình Dim phẳng.

13.1.3 Khả năng mở rộng

Kiến trúc Galaxy thể hiện tính linh hoạt cao khi mở rộng hệ thống:

- **Bổ sung nguồn dữ liệu mới:** Nếu cần thêm dữ liệu từ các hệ thống mới (như cảm biến chất lượng không khí), chỉ cần tạo thêm Fact mới và kết nối với các Conformed Dimensions hiện có. Các cấu phần đang vận hành không bị ảnh hưởng.
- **Mở rộng thuộc tính chiều:** Những Dimension chủ đạo như `dim_vehicle_type` hay `dim_weather` được thiết kế có khả năng mở rộng tự nhiên, cho phép bổ sung thuộc tính hoặc giá trị mới mà không làm thay đổi cấu trúc tổng thể.

13.1.4 Đánh giá hiệu năng truy vấn

Thiết kế Snowflake mang lại một số đánh đổi về mặt hiệu năng:

- **Thách thức:** Độ chuẩn hóa cao khiến số lượng JOIN trong truy vấn tăng lên, đặc biệt với các báo cáo yêu cầu dữ liệu không gian (ví dụ: truy ngược từ fact đến `dim_road`).

- **Giải pháp:** Với hệ quản trị CSDL hiện đại và chiến lược đánh chỉ mục phù hợp, chi phí JOIN được kiểm soát tốt. Đồng thời, các Data Mart phẳng hoặc bảng tổng hợp có thể được xây dựng ở tầng khai thác để tối ưu tốc độ truy vấn cho các báo cáo tần suất cao.

13.1.5 Khả năng hỗ trợ AI và Học máy

Schema được thiết kế hướng đến không chỉ nghiệp vụ BI mà còn làm đầu vào hiệu quả cho các mô hình AI.

- **Granularity phù hợp:** Việc lưu giữ dữ liệu ở mức độ chi tiết trong `fact_traffic_flow_by_vehicle` hay `fact_user_travel_time` hỗ trợ các mô hình học máy dự báo chuỗi thời gian và phân tích hành vi.
- **Feature phong phú:** Các chiều ngữ cảnh như `dim_weather`, `dim_holiday` hay `dim_event` cung cấp các biến giải thích quan trọng, giúp mô hình AI học được cách các yếu tố ngoại cảnh tác động đến trạng thái giao thông.

Kết luận: Mô hình Galaxy Schema với Snowflake Dimensions mang lại sự cân bằng giữa tính toàn vẹn dữ liệu, khả năng phân tích đa chiều, hiệu năng truy vấn và khả năng mở rộng. Đây là nền tảng cấu trúc vững chắc để xây dựng các hệ thống phân tích giao thông nâng cao và các ứng dụng trí tuệ nhân tạo trong tương lai.

13.2 Đánh giá kết quả sau ETL

Sau quá trình triển khai thực tế và vận hành thử nghiệm, hệ thống ETL đã chuyển đổi thành công từ các tập lệnh xử lý thô thành một đường ống dữ liệu (Data Pipeline) hoàn chỉnh, ổn định và có khả năng tự phục hồi. Kết quả thu được sau giai đoạn nạp dữ liệu không chỉ đạt yêu cầu về mặt lưu lượng bản ghi mà còn thể hiện sự vượt trội về độ tinh khiết, tính nhất quán và chiều sâu của thông tin ngữ cảnh.

Dưới đây là các đánh giá chi tiết về hiệu quả đạt được sau khi kết thúc chu trình nạp dữ liệu.

13.2.1 Đánh giá hiệu quả khởi tạo hệ quy chiếu Dimension

Nhóm chiều (Dimensions) đóng vai trò là "xương sống" định vị cho mọi phép phân tích đa chiều trong Star Schema. Kết quả thực thi cho thấy hệ thống đã xây dựng được một bộ từ điển dữ liệu hạ tầng và thời gian cực kỳ chi tiết, giải quyết được các bài toán đặc thù của giao thông đô thị Việt Nam.

13.2.1.1 Đánh giá Dimension Thời gian và Logic văn hóa đặc thù Hệ thống đã thiết lập được một hệ tọa độ thời gian có độ chính xác tuyệt đối, vượt xa các mô hình kho dữ liệu thông thường.

- **Xử lý lịch biến động:** Module chuyển đổi Lịch âm - Dương lịch đã xử lý chính xác 100% các ngày lễ đặc thù trong giai đoạn 2020 - 2030. Đặc biệt, việc xác định đúng ngày "30 Tết" và tự động sinh dữ liệu cho toàn bộ kỳ nghỉ lễ đã giúp hệ thống ghi nhận chính xác các biến động giao thông cực đoạn trong các dịp lễ hội, vốn là các điểm dữ liệu quan trọng nhất cho dự báo lưu lượng.
- **Độ mịn và Tính liên tục:** Việc nạp thành công chính xác 1440 bản ghi cho mỗi ngày (tương ứng với từng phút) giúp hệ thống đạt mức độ chi tiết cao nhất (Granularity) trong phân tích. Kết quả kiểm tra cho thấy cờ `is_holiday` và nhân ca làm việc (`Work Shifts`) được bật chính xác, cho phép Dashboard lọc nhanh dữ liệu theo giờ cao điểm hoặc giờ hành chính mà không cần các phép tính toán phức tạp tại tầng hiển thị.

13.2.1.2 Đánh giá Dimension Hạ tầng GIS và Tính toàn vẹn Topo Dữ liệu hạ tầng được chuyển đổi từ OpenStreetMap đã được số hóa sang định dạng PostGIS với độ chính xác không gian cực cao.

- **Cấu trúc Segment hóa thông minh:** Quy trình ETL đã phân rã thành công mạng lưới đường thành hàng chục nghìn phân đoạn (Segments) nối giữa các nút giao. Điều này cho phép hệ thống quản lý giao thông theo từng "mét đường", phản ánh đúng thực tế khi một con đường có thể kẹt xe cục bộ tại đầu nút giao nhưng lại thông thoáng ở giữa đoạn.

- **Thành công trong tính toán góc hướng (Azimuth):** Thuật toán toán học dựa trên tọa độ *From-To Node* đã gán góc Azimuth chính xác cho từng phân đoạn. Qua kiểm tra trên các trục đường huyết mạch hai chiều như Điện Biên Phủ hay Võ Văn Kiệt, hệ thống đã phân biệt rõ ràng dòng xe của hai hướng di chuyển ngược nhau, loại bỏ hoàn toàn hiện tượng "nhập nhằng" dữ liệu vận tốc thường gặp trong các hệ thống GIS cũ.
- **Chiến lược Đa hình học (Dual-Geometry):** Việc lưu trữ đồng thời `geometry_linestring` cho hiển thị bản đồ nhiệt và `geometry_center` cho các phép tính toán không gian đã giúp tăng tốc độ truy vấn lên hơn 40% so với cách lưu trữ truyền thống.

13.2.1.3 Đánh giá Danh mục nghiệp vụ và Làm giàu dữ liệu (Metadata Enrichment) Hệ thống đã thiết lập được một tầng Metadata vững chắc, cho phép đồng nhất dữ liệu từ nhiều nguồn API khác nhau.

- **Chuẩn hóa phương tiện và hệ số PCU:** Toàn bộ danh mục xe máy, ô tô, xe tải đã được nạp đúng các hệ số quy đổi (0.25, 1.0, 2.5). Việc tách biệt cấu hình này qua file CSV đã chứng minh tính linh hoạt cao khi cho phép người vận hành điều chỉnh trọng số PCU mà không cần can thiệp vào mã nguồn ETL.
- **Tích hợp mã màu và mức độ nghiêm trọng:** Các chiều danh mục như `dim_incident_type` đã được nạp kèm mã màu Hex và mức độ ưu tiên. Kết quả là Dashboard có thể tự động hiển thị trực quan các vùng tai nạn hoặc ùn tắc nghiêm trọng ngay khi dữ liệu được nạp vào, rút ngắn thời gian từ dữ liệu đến hành động.

13.2.2 Đánh giá hiệu quả nạp dữ liệu Fact và Hợp nhất dữ liệu đa nguồn

Nếu nhóm chiều Dimension tạo ra hệ quy chiếu, thì việc nạp dữ liệu vào các bảng Fact chính là quá trình hiện thực hóa các chỉ số vận hành của hệ thống. Kết quả đánh giá tại tầng này cho thấy khả năng xử lý dữ liệu động và tính chính xác trong việc hợp nhất các nguồn dữ liệu phi cấu trúc của hệ thống ETL.

13.2.2.1 Hiệu quả nhận diện phương tiện và tính toán PCU từ mô hình AI Việc tích hợp trực tiếp YOLOv8x vào luồng ETL đã mang lại những kết quả quan sát định lượng cực kỳ giá trị, khắc phục nhược điểm của các nguồn dữ liệu API ước tính.

- **Độ chính xác nhận diện:** Mô hình YOLOv8x đã chứng minh được năng lực nhận diện vượt trội trong điều kiện mật độ phương tiện dày đặc tại các nút giao trọng điểm của TP.HCM. Khả năng phân loại chính xác các lớp đối tượng (xe máy, xe con, xe tải) cho phép hệ thống thu thập được dữ liệu "ground truth" (dữ liệu thực chứng) với sai số thấp.
- **Định lượng hóa lưu lượng qua PCU:** Thuật toán quy đổi PCU đã hoạt động ổn định, chuyển đổi thành công danh sách các đối tượng nhận diện rời rạc thành chỉ số `total_pcu` mang tính kỹ thuật. Việc áp dụng hệ số 0.25 cho xe máy phản ánh đúng thực tế chiếm dụng mặt đường tại Việt Nam, giúp các báo cáo về mật độ dòng chảy đạt độ tin cậy cao.

13.2.2.2 Đánh giá tính chính xác của cơ chế Hợp nhất dữ liệu (Data Fusion) Thành công lớn nhất của giai đoạn này là khả năng đồng bộ hóa ngữ cảnh giữa "Mật độ" (từ AI) và "Vận tốc" (từ API).

- **Sự nhất quán đa nguồn:** Tại mỗi thời điểm nạp, hệ thống đã thực hiện khớp nối thành công dữ liệu lưu lượng từ camera với dữ liệu tốc độ từ TomTom và điều kiện khí tượng từ OpenWeatherMap. Kết quả kiểm tra cho thấy các bản ghi trong `fact_traffic_flow` sở hữu đầy đủ các thuộc tính ngữ cảnh, cho phép phân tích mối quan hệ nhân quả giữa thời tiết, sự cố và trạng thái dòng xe.
- **Độ trễ và Tính thời điểm:** Cơ chế điều phối đảm bảo các yêu cầu API và xử lý AI diễn ra gần như đồng thời, giúp bản ghi Fact phản ánh trạng thái thực sự của phân đoạn đường tại một thời điểm nhất quán, tránh hiện tượng lệch pha dữ liệu giữa các nguồn khác nhau.

13.2.2.3 Hiệu quả đối khớp không gian cho Sự cố (Incident Map Matching) Quy trình nạp sự cố đã giải quyết tốt bài toán thách thức về sai số tọa độ GPS.

- **Độ chính xác của thuật toán Haversine:** Việc triển khai hàm tính toán khoảng cách mặt cầu đã giúp hệ thống tự động tìm kiếm và gán chính xác các điểm sự cố vào `segment_key` gần nhất trong bán kính từ 50m - 100m.
- **Cơ chế làm sạch dữ liệu chủ động:** Hệ thống đã thực hiện tốt việc loại bỏ (skip) các bản ghi sự cố có tọa độ nằm ngoài phạm vi mạng lưới đường bộ đã số hóa hoặc vượt quá ngưỡng sai số cho phép. Điều này đảm bảo bảng `fact_incident` không bị nhiễm "dữ liệu rác", duy trì tính nghiêm ngặt cho các phân tích không gian sau này.

13.2.2.4 Đánh giá logic phân cấp LOS (Level of Service) Hệ thống đã thực hiện tốt tăng biến đổi logic để đưa ra các đánh giá chất lượng dịch vụ ngay trong luồng nạp dữ liệu.

- **Tính toán Congestion Level:** Việc so sánh liên tục giữa `current_speed` và `free_flow_speed` đã sinh ra chỉ số ùn tắc chính xác cho từng thời điểm.
- **Phân nhãn LOS tự động:** Logic phân cấp từ A đến F đã được thực thi triệt để, giúp chuyển đổi dữ liệu số thô thành các nhãn trạng thái trực quan. Kết quả thực tế cho thấy nhãn LOS phản ánh đúng tình trạng kẹt xe cục bộ được ghi nhận qua ảnh camera, chứng minh thuật toán phân loại có độ khớp cao với thực tế vận hành hạ tầng.

13.2.3 Đánh giá Hiệu năng vận hành và Quản trị dữ liệu quy mô lớn

Một hệ thống Kho dữ liệu giao thông chỉ thực sự có giá trị khi nó duy trì được hiệu năng ổn định trong bối cảnh dữ liệu tích lũy không ngừng theo thời gian. Kết quả đánh giá sau ETL cho thấy các chiến lược tối ưu hóa tăng vật lý và cơ chế vận hành tự động đã phát huy hiệu quả vượt mong đợi.

13.2.3.1 Hiệu quả của chiến lược Phân vùng dữ liệu (Partitioning) Việc triển khai *Range Partitioning* theo tháng cho bảng `fact_traffic_flow` đã mang lại những cải thiện đột phá về mặt quản trị dữ liệu lớn.

- **Tối ưu hóa tốc độ nạp (Ingestion Speed):** Bằng cách chia nhỏ dữ liệu vào các bảng con theo `date_key`, hệ thống đã loại bỏ được hiện tượng "Bloat" chỉ mục thường gặp ở các bảng phẳng khổng lồ. Kết quả thực tế cho thấy thời gian ghi dữ liệu cho mỗi chu kỳ 15 phút vẫn duy trì ổn định ở mức dưới 2 phút, không bị suy giảm khi tổng lượng bản ghi trong kho tăng lên.
- **Cơ chế Partition Pruning trong truy vấn:** Khi thực hiện các phép phân tích chuỗi thời gian, bộ tối ưu hóa của PostgreSQL đã thực hiện loại bỏ chính xác các phân vùng không liên quan, giúp tốc độ truy xuất dữ liệu lịch sử nhanh gấp nhiều lần so với cấu trúc bảng đơn lẻ.
- **Quản lý vòng đời dữ liệu (Data Lifecycle):** Hệ thống đã chứng minh khả năng bảo trì linh hoạt thông qua việc cho phép ngắt kết nối (*Detach*) hoặc lưu trữ (*Archive*) các phân vùng cũ mà không gây ảnh hưởng đến tính sẵn sàng của toàn bộ kho dữ liệu.

13.2.3.2 Tối ưu hóa hệ thống Chỉ mục đa tầng (Indexing) Sự phối hợp giữa các loại chỉ mục khác nhau đã tạo nên một hạ tầng truy vấn cực kỳ hiệu quả.

- **Chỉ mục B-Tree cho quan hệ Star Schema:** Việc đánh chỉ mục trên các khóa ngoại (`segment_key`, `time_key`, `date_key`) đã tối ưu hóa triệt để các phép toán JOIN giữa Fact và Dimension, cho phép các truy vấn tổng hợp trả về kết quả gần như tức thời.
- **Sức mạnh của chỉ mục không gian GIST:** Đây là yếu tố then chốt giúp hệ thống xử lý mượt mà các yêu cầu phân tích không gian phức tạp. Chỉ mục GIST trên các cột `geometry` của bảng sự cố và hạ tầng đã giúp các phép toán tìm kiếm lân cận hoặc vẽ bản đồ nhiệt (*Heatmap*) đạt độ trễ cực thấp.

13.2.3.3 Đánh giá tính ổn định của chu kỳ Tự động hóa (Automation) Cơ chế "Cài đặt một lần, vận hành mãi mãi" thông qua Batch Script và Task Scheduler đã biến kho dữ liệu thành một thực thể tự sống.

- **Điều phối tài nguyên thông minh:** Việc sử dụng tham số `-mode fact` đã tập trung tối đa sức mạnh tính toán (CPU/GPU) vào việc thu thập dữ liệu động, giúp chu kỳ nạp 15 phút diễn ra nhẹ nhàng mà không gây quá tải cho máy chủ.
- **Khả năng tự phục hồi:** Batch script đã thực hiện tốt vai trò quản lý môi trường ảo (.venv), đảm bảo mọi thư viện chuyên dụng như `ultralytics` hay `OSMnx` luôn sẵn sàng hoạt động ổn định trong mọi điều kiện khởi động lại hệ thống.

13.2.3.4 Hệ thống Giám sát và Truy vết lỗi (Logging & Audit) Hệ thống ghi nhật ký tập trung đã cung cấp một cái nhìn minh bạch về "sức khỏe" của toàn bộ đường ống dữ liệu.

- **Minh bạch hóa quá trình ETL:** Định dạng log tiêu chuẩn giúp người quản trị dễ dàng thực hiện *Performance Profiling*, xác định chính xác module nào đang chiếm nhiều thời gian thực thi nhất hoặc API nào đang gặp vấn đề về hạn mức (*Quota*).
- **Quản lý ngoại lệ chủ động:** Cơ chế bắt lỗi `try-except` kết hợp ghi lỗi chi tiết (*Traceback*) đã giúp hệ thống không bị dừng đột ngột khi mất kết nối mạng, đồng thời cung cấp đủ bằng chứng kỹ thuật để khắc phục sự cố nhanh chóng.

13.2.4 Đánh giá tính sẵn sàng của Mock Data và khả năng hỗ trợ AI/ML

Giai đoạn cuối của quy trình ETL tập trung vào việc tạo ra các kịch bản dữ liệu mô phỏng để kiểm thử giới hạn hệ thống và chuẩn bị các bộ dữ liệu đặc trưng (Feature Sets) cho các bài toán phân tích dự báo. Kết quả đánh giá cho thấy kho dữ liệu đã đạt tới độ chín muồi về cấu trúc để sẵn sàng hỗ trợ các nghiên cứu chuyên sâu về học máy.

13.2.4.1 Đánh giá tính thực tế của Pipeline giả lập dữ liệu (Mock Data) Dữ liệu giả lập không chỉ đơn thuần là các con số ngẫu nhiên mà được sinh ra dựa trên các quy luật vận hành thực tế của hạ tầng và hành vi con người.

- **Mô phỏng hành vi di chuyển (Trip Mocking):** Thông qua việc sử dụng các cặp `segment_key` thực tế và tính toán vận tốc dựa trên loại xe, hệ thống đã tạo ra hàng nghìn hành trình có logic không gian chặt chẽ. Việc tích hợp các chỉ số phái sinh như lượng phát thải CO_2 và tiêu hao nhiên liệu đã biến bảng `fact_trip` trở thành một tập dữ liệu quý giá cho các bài toán phân tích tác động môi trường.
- **Độ sâu lịch sử cho phân tích xu hướng:** Bằng cách giả lập dữ liệu lùi về quá khứ cho bảng `fact_segment_performance`, hệ thống đã cho phép kiểm thử các biểu đồ so sánh theo năm/tháng trên Dashboard ngay lập tức. Các quy luật giả lập về giờ cao điểm và giờ thấp điểm đã phản ánh đúng đặc tính chu kỳ của giao thông đô thị, giúp xác thực tính đúng đắn của các câu lệnh truy vấn tổng hợp phức tạp.

13.2.4.2 Khả năng cung cấp bộ tính năng (Feature Engineering) cho AI/ML Kho dữ liệu sau ETL đã được chuẩn hóa thành một "mỏ dữ liệu sạch", sẵn sàng làm đầu vào cho các mô hình học máy dự báo tắc nghẽn hoặc tối ưu hóa lộ trình.

- **Tính đa dạng của các biến giải thích:** Nhờ cơ chế hợp nhất dữ liệu đa nguồn, mỗi bản ghi lưu lượng giờ đây sở hữu một bộ Feature cực kỳ phong phú bao gồm: thời gian (phút, ca làm việc, ngày lễ), không gian (loại đường, hướng di chuyển), và ngoại cảnh (nhiệt độ, tầm nhìn, rủi ro ngập lụt). Đây là các biến đầu vào then chốt để các mô hình AI có thể học được quy luật biến động của dòng xe.
- **Hỗ trợ dự báo chuỗi thời gian (Time-series Forecasting):** Với độ mịn dữ liệu đến từng phút và cấu trúc phân vùng bảng (*Partitioning*) hiệu năng cao, kho dữ liệu cho phép trích xuất các chuỗi dữ liệu lịch sử dài hạn một cách nhanh chóng. Điều này tạo điều kiện thuận lợi cho việc huấn luyện các mô hình như LSTM hoặc Graph Convolutional Networks (GCN) để dự báo trạng thái giao thông trong tương lai ngắn hạn.

13.2.4.3 Đánh giá tổng kết về giá trị của quy trình ETL Quy trình ETL được triển khai trong đồ án đã vượt xa vai trò của một công cụ chuyển đổi dữ liệu thông thường. Nó đóng vai trò là một bộ lọc tri thức, biến các tín hiệu thô từ Camera, Vệ tinh và Trạm khí tượng thành các chỉ số định lượng có giá trị cao.

- **Tính thực tiễn cao:** Việc triển khai thành công hệ thống trên hạ tầng PostgreSQL/PostGIS với cơ chế tự động hóa qua Task Scheduler đã chứng minh giải pháp có khả năng ứng dụng thực tế vào các trung tâm điều hành giao thông thông minh.
- **Khả năng mở rộng bền vững:** Kiến trúc Pipeline đa tầng đảm bảo rằng trong tương lai, hệ thống có thể dễ dàng tích hợp thêm các nguồn dữ liệu mới (như cảm biến chất lượng không khí hoặc dữ liệu từ thiết bị GPS trên xe bus) mà không cần thay đổi cấu trúc cốt lõi của kho dữ liệu.

13.3 Đánh giá Hiệu năng và Độ bền vững Phần mềm (Software Performance & Robustness)

Hệ thống giám sát giao thông đặc thù phải xử lý luồng dữ liệu liên tục và phục vụ lượng người dùng truy cập đồng thời lớn (đặc biệt vào giờ cao điểm). Nhóm đã tiến hành kiểm thử hiệu năng (Load Testing) sử dụng công cụ **Apache JMeter / Artillery** và thu được các kết quả khả quan.

13.3.1 Độ trễ API (API Latency) và Tối ưu hóa truy vấn

- **Dashboard Tổng quan (Admin):** Các chỉ số KPI cốt lõi (Tổng độ trễ, Tốc độ trung bình) đạt độ trễ phản hồi (Response Time) trung bình dưới **150ms**. Kết quả này có được nhờ việc áp dụng **Redis Cache** cho các API có tần suất truy cập cao, giảm thiểu 90% tải đọc trực tiếp vào Database.
- **Truy vấn Phân tích OLAP:** Đối với biểu đồ Traffic Heatmap và Bubble Chart quét qua hàng triệu bản ghi, nhờ cơ chế **Materialized Views** tính toán sẵn, API trả về kết quả cấu trúc JSON phức tạp trong vòng dưới **800ms** thay vì mất hàng phút như các truy vấn SQL thô thông thường.
- **API Tìm đường (Routing):** Thuật toán Bi-directional A* (pgr_bdAstar) kết hợp hàm chi phí đa biến (Multi-variable Cost Function) phản hồi trung bình trong **250ms** cho một lộ trình dài 10km, đảm bảo trải nghiệm tức thì cho người dùng Mobile App.

13.3.2 Hiệu năng Giao diện (Frontend Performance)

- **Mobile App (MapboxGL):** Việc sử dụng kỹ thuật Vector Tiles (MVT) cho phép điện thoại di động render mạng lưới giao thông toàn thành phố với tốc độ khung hình duy trì ổn định ở mức **60 FPS**. Thao tác thu phóng (Zoom/Pan) không bị giật lag (stuttering).
- **Web Dashboard (React):** DOM (Document Object Model) được tối ưu bằng kỹ thuật Lazy Loading và Phân trang (Pagination) ở máy chủ (Server-side). Chức năng SVG Gradient của Recharts hoạt động mượt mà với độ phức tạp $O(1)$, không gây thất nút cổ chai CPU trên trình duyệt.

13.3.3 Đánh giá Độ bền vững và Khả năng chịu tải (Robustness & Scalability)

Dựa trên định hướng thiết kế, hệ thống Traffic IOC hiện tại được phát triển với trọng tâm phục vụ khối cơ quan quản lý nhà nước (Sở GTVT TP.HCM). Phân hệ ứng dụng người dùng (User App) đóng vai trò là một kênh thử nghiệm kỹ thuật (Proof of Concept) và chia sẻ thông tin cơ bản, chưa hướng tới mục tiêu phục vụ lượng lớn người dùng đại chúng (Mass B2C) ở giai đoạn này. Dưới định hướng đó, tiêu chí độ bền vững của hệ thống được đánh giá qua các khía cạnh đặc thù sau:

- **Tối ưu hóa tài nguyên cho tác vụ nặng (Vertical Optimization):** Thay vì tập trung xử lý hàng chục ngàn kết nối đồng thời, kiến trúc hệ thống được tối ưu hóa để giải quyết các truy vấn không gian phức tạp và phân

tích khối dữ liệu lớn (OLAP) phục vụ nghiệp vụ vĩ mô của Sở GTVT. Việc sử dụng cấu hình **Azure Basic B2** kết hợp với kiến trúc Caching đảm bảo đủ dung lượng RAM và sức mạnh CPU để tính toán Materialized Views và chạy thuật toán đồ thị một cách ổn định, không bị sập nguồn (Crash) khi khối lượng dữ liệu phình to.

- **Khả năng phục hồi và chống chịu lỗi (Fault Tolerance & Data Resilience):** Sức chịu đựng của hệ thống được thể hiện rõ nhất ở lớp Data Pipeline. Trong trường hợp các API cung cấp dữ liệu nguồn gặp sự cố mạng hoặc từ chối dịch vụ (Rate Limit), cơ chế Try-Catch và Fallback đảm bảo quy trình ETL không bị gián đoạn toàn cục. Các đoạn đường bị mất tín hiệu sẽ được tự động bù đắp thông qua thuật toán nội suy không gian (KNN-IDW) và làm mượt thời gian (Exponential Decay) ngay trong Database, giúp bảng điều khiển IOC luôn duy trì thông tin liền mạch.
- **Độ sẵn sàng mở rộng (Future-proof Scalability):** Dù hiện tại phân hệ User không chịu tải cao, kiến trúc phi trạng thái (Stateless) phân tầng với **Docker Container** hoàn toàn cho phép hệ thống chuyển đổi sang mô hình nhân bản theo chiều ngang (Horizontal Scaling) thông qua Load Balancer khi Sở GTVT có chủ trương phát hành ứng dụng định tuyến rộng rãi cho toàn dân trong tương lai.

13.4 Đánh giá Mức độ Đáp ứng Yêu cầu Nghiệp vụ

Đối chiếu với các mục tiêu đã đề ra ở Chương 1, hệ thống đã giải quyết trọn vẹn các bài toán nghiệp vụ giao thông vĩ mô và vi mô:

- **Đảm bảo tính Toàn vẹn Dữ liệu (Data Integrity):** Nhóm đã khắc phục thành công lỗi luận lý "Trung bình của các tỷ lệ" (*Average of Ratios Fallacy*) trong việc tính toán Chỉ số Độ tin cậy (BI và PTI). Hệ thống hiện tại tính toán dựa trên tổng thời gian trễ lũy kế của toàn bộ hành lang, đảm bảo kết quả đánh giá hạ tầng chính xác tuyệt đối theo tiêu chuẩn của Cục Quản lý Đường bộ Mỹ (FHWA).
- **Đánh giá Hạ tầng đa chiều:** Biểu đồ Cross-analysis (Bubble Chart) đã hoàn thành xuất sắc vai trò cảnh báo sớm, giúp phát hiện các tuyến đường tuy TTI chưa cao nhưng đã vượt hệ số Sức chứa thiết kế ($V/C \text{ Ratio} \geq 1.0$), hỗ trợ Sở GTVT ra quyết định nâng cấp hạ tầng kịp thời.
- **Hỗ trợ Ra quyết định Cá nhân hóa:** Thuật toán tìm đường trên Mobile App đã thay thế hoàn toàn tiêu chí "Đường ngắn nhất" bằng tiêu chí hàm chi phí phức hợp ($Cost = f(TravelTime, Distance, Confidence)$). Việc kết hợp thêm dự báo TTI trong tương lai gần (15 phút) giúp lộ trình được đề xuất không chỉ nhanh mà còn tránh được rủi ro đi vào đường hẹp, mang tính tin cậy rất cao.

13.5 Kết chương

Chương này đã thực hiện một cuộc đánh giá toàn diện và khách quan về hệ thống Kho dữ liệu giao thông thông minh từ cấp độ cấu trúc logic đến hiệu quả vận hành thực tế. Thông qua quá trình phân tích và kiểm thử dựa trên dữ liệu thực tế thu thập được, nhóm nghiên cứu rút ra các kết luận quan trọng sau:

1. **Sự tối ưu của kiến trúc Schema:** Việc hiện thực hóa mô hình *Galaxy Schema* kết hợp *Snowflake Dimensions* đã chứng minh được tính đúng đắn khi giải quyết hài hòa giữa khả năng phân tích chéo đa chiều (Lưu lượng - Sự cố - Thời tiết) và tính toàn vẹn của dữ liệu không gian hạ tầng. Hệ thống không chỉ đáp ứng các yêu cầu BI truyền thống mà còn sẵn sàng cho việc mở rộng, tích hợp thêm các nguồn dữ liệu mới trong tương lai mà không làm thay đổi cấu trúc cốt lõi.
2. **Độ chính xác vượt trội của quy trình ETL:** Hệ thống đã thể hiện năng lực xử lý dữ liệu phức tạp thông qua việc tích hợp thành công mô hình AI (*YOLOv8x*) để thu thập dữ liệu thực chứng (Ground Truth) và quy đổi PCU chuẩn xác. Các thuật toán bổ trợ như *Haversine* (đổi khớp sự cố) và tính toán *Azimuth* (định hướng phân đoạn) đã đảm bảo dữ liệu trong kho đạt độ chính xác cao về mặt topo không gian, loại bỏ các sai số hệ thống thường gặp trong các ứng dụng GIS truyền thống.

3. **Tính thích nghi với bối cảnh đặc thù:** Dự án đã giải quyết thành công các bài toán mang tính bản địa hóa như: xử lý Lịch âm cho các kỳ nghỉ lễ Việt Nam và phân cấp mức độ phục vụ (LOS) dựa trên đặc điểm dòng xe hỗn hợp. Điều này tạo ra một công cụ quản lý giao thông thực sự phù hợp với thực tiễn vận hành hạ tầng tại TP.HCM.
4. **Hiệu năng và khả năng vận hành bền bỉ:** Thông qua các chiến lược quản trị dữ liệu quy mô lớn như *Range Partitioning*, đánh chỉ mục đa tầng (*B-Tree & GIST*) và cơ chế tự động hóa qua *Task Scheduler*, hệ thống đã chứng minh được khả năng vận hành ổn định 24/7. Thời gian xử lý mỗi chu kỳ cực thấp (dưới 2 phút) đảm bảo kho dữ liệu luôn ở trạng thái cập nhật (Freshness) cao nhất.
5. **Nền tảng cho Trí tuệ nhân tạo:** Kết quả đánh giá khẳng định kho dữ liệu đã đạt tới độ chín muồi về cấu trúc và chất lượng tính năng (*Feature Quality*). Với bộ dữ liệu đa chiều được làm giàu ngữ cảnh, hệ thống sẵn sàng đóng vai trò là "mỏ dữ liệu sạch" phục vụ cho các mô hình học máy chuyên sâu về dự báo kẹt xe, tối ưu hóa định tuyến và quy hoạch đô thị thông minh.

Tóm lại, những kết quả đánh giá tích cực này khẳng định rằng hệ thống Kho dữ liệu giao thông được xây dựng không chỉ đạt được các mục tiêu kỹ thuật đề ra ban đầu mà còn tạo ra một nền tảng công nghệ vững chắc, có tính ứng dụng thực tiễn cao trong lộ trình xây dựng hệ sinh thái giao thông thông minh bền vững.

14 Tổng kết và hướng phát triển

Đồ án tốt nghiệp "Hệ thống Dự báo và Cảnh báo sớm Ûn tắc Giao thông ứng dụng Trí tuệ Nhân tạo" đã được thực hiện và hoàn thành theo đúng các mục tiêu đề ra ban đầu. Vượt qua khuôn khổ của một bài toán Học máy đơn thuần, nhóm thực hiện đã xây dựng thành công một hệ sinh thái khép kín (End-to-End Pipeline) kết nối chặt chẽ giữa Kỹ thuật Dữ liệu, Trí tuệ Nhân tạo và Phát triển Ứng dụng.

Dưới đây là những kết quả nổi bật đã đạt được, được phân chia theo 4 phương diện cốt lõi của hệ thống:

14.1 Kết quả đạt được của đồ án

14.1.0.1 a. Về phương diện Kỹ thuật Dữ liệu (Data Engineering)

- **Xây dựng Data Warehouse chuẩn hóa:** Đã thiết kế và triển khai thành công Kho dữ liệu (Data Warehouse) lưu trữ dữ liệu giao thông đô thị, phục vụ tốt cho các truy vấn phân tích và huấn luyện AI.
- **Tự động hóa luồng ETL/ELT:** Xây dựng quy trình trích xuất, biến đổi và tải dữ liệu ổn định. Đặc biệt, hệ thống đã giải quyết hiệu quả các khiếm khuyết của dữ liệu cảm biến IoT thông qua các kỹ thuật điền khuyết (Forward Fill) và sàng lọc nhiễu, đảm bảo nguồn dữ liệu đầu vào luôn ở trạng thái "sạch" và có tính sẵn sàng cao.

14.1.0.2 b. Về phương diện Trí tuệ Nhân tạo (Artificial Intelligence)

Đây là điểm sáng và là đóng góp mang tính học thuật lớn nhất của đồ án, thể hiện qua việc giải quyết thành công các thách thức hóc búa của dữ liệu chuỗi thời gian giao thông:

- **Khắc phục triệt để rủi ro mất cân bằng dữ liệu:** Ứng dụng thành công mô hình Trí tuệ nhân tạo tạo sinh CTGAN để tổng hợp dữ liệu kịch bản kẹt xe. Đồng thời, thiết lập "Màng lọc hậu kiểm vật lý" (Physical Sanity Check) để loại bỏ các điểm dữ liệu phi lý, tạo ra Tập dữ liệu Vàng cho quá trình huấn luyện.
- **Đề xuất và làm chủ Kiến trúc Hybrid Double DQN:** Xây dựng thành công Đặc vụ Học tăng cường (RL Agent) kết hợp mạng Nơ-ron đa nhánh (LSTM + Embedding). Hệ thống đã áp dụng chiến lược Học chuyển giao (Transfer Learning / Warm-start) để đưa tri thức từ mô hình giám sát vào Agent, triệt tiêu hoàn toàn hội chứng "khởi động lạnh".
- **Chuyển dịch hệ quy chiếu đánh giá (Asymmetric Reward Shaping):** Đồ án đã mạnh dạn bác bỏ chỉ số Độ chính xác (Accuracy) mang tính đánh lừa, thiết lập Hàm phần thưởng không đối xứng với mức "Phạt sinh tử" cho lỗi bỏ lọt thảm họa. Kết quả thực chứng cho thấy mô hình Đề xuất đạt **Độ phủ (Recall) trên 85%** tại các lớp kẹt xe nghiêm trọng (Lớp 4, Lớp 5), vượt trội hoàn toàn so với các mô hình truyền thống (chỉ đạt dưới 10%).
- **Minh bạch hóa hộp đen (XAI):** Tích hợp thành công phương pháp SHAP, chứng minh được các quyết định cảnh báo thảm họa của hệ thống được đưa ra dựa trên các động lực học vật lý hợp logic (sự suy giảm vận tốc, tăng chỉ số ùn tắc), củng cố độ tin cậy khi áp dụng vào thực tế.

14.1.0.3 c. Về phương diện Ứng dụng và Định tuyến (Application & Routing)

- **Phát triển Ứng dụng Người dùng (Client-App):** Hoàn thiện giao diện ứng dụng (UI/UX) thân thiện, trực quan, cho phép người dùng dễ dàng theo dõi trạng thái giao thông thời gian thực.
- **Tích hợp Thuật toán Định tuyến thông minh:** Ứng dụng thành công dữ liệu dự báo từ AI vào các thuật toán tìm đường, cung cấp cho người dùng các lộ trình di chuyển tối ưu nhất nhằm né tránh các điểm nóng kẹt xe sắp xảy ra.

14.1.0.4 d. Về phương diện Triển khai và Vận hành (MLOps & Deployment)

- **Kiến trúc Vận hành chuẩn Công nghiệp:** Hệ thống không dừng lại ở mức độ nghiên cứu (Jupyter Notebook) mà đã được thiết kế kiến trúc Client-Server hoàn chỉnh, phân tách rõ ràng giữa AI Server, DB Server và App Server.
- **Đóng gói và Tự động hóa:** Toàn bộ môi trường mã nguồn, hệ quy chiếu (Artifacts) và trọng số mô hình được container hóa bằng **Docker**. Nhóm cũng đã thiết lập nền tảng CI/CD cơ bản, sẵn sàng cho việc nhân bản (Scale) và triển khai lên môi trường Đám mây (Cloud), tối ưu hóa ngân sách và tài nguyên hệ thống.

14.2 Những hạn chế còn tồn tại

Mặc dù đã đạt được những kết quả khả quan và xây dựng thành công một pipeline End-to-End, hệ thống "Dự báo và Cảnh báo sớm Ùn tắc Giao thông" vẫn còn những điểm giới hạn nhất định do rào cản về tài nguyên phần cứng, thời gian nghiên cứu và sự phức tạp nội tại của hệ thống giao thông đô thị. Cụ thể như sau:

14.2.0.1 a. Giới hạn về Nhận thức Không gian và Lan truyền ùn tắc

- **Thiếu hụt liên kết Tô-pô (Topological Connectivity):** Mạng Q-Network hiện tại dù rất mạnh trong việc nắm bắt động lực học chuỗi thời gian (nhờ LSTM) nhưng lại xử lý các đoạn đường một cách tương đối độc lập (dựa trên Embedding ID). Hệ thống chưa mô hình hóa được mạng lưới đường bộ thành một Đồ thị (Graph), dẫn đến việc nắm bắt "hiệu ứng dội ngược" (Spillback) hoặc sự lan truyền ùn tắc từ ngã tư này sang ngã tư khác vẫn mang tính gián tiếp, chưa đạt độ chính xác tối đa.
- **Cơ chế Fallback chỉ mang tính đối phó:** Giải pháp "Global Spatial Fallback" khi mất tín hiệu cảm biến hiện tại chỉ là phép xấp xỉ thống kê từ các đường lân cận, chưa phản ánh được luồng di chuyển vật lý của dòng xe.

14.2.0.2 b. Khoảng cách giữa Mô phỏng và Thực tế (Sim-to-Real Gap)

- **Nhiều loạn ngoại lai (Black Swan Events):** Tập dữ liệu huấn luyện, dù đã được làm giàu bằng CTGAN, vẫn có độ "sạch" nhất định về mặt phân phối xác suất. Khi đối mặt với môi trường thực tế – nơi chứa đựng các sự cố tai nạn bất ngờ, lỗi ngắt kết nối phần cứng, hoặc tín hiệu GPS nhảy vọt vô lý – Đặc vụ (Agent) đôi khi vẫn có thể bị "sốc" do chưa từng quan sát thấy các mức nhiễu cực đoan này trong không gian học (Offline RL).
- **Thiếu khả năng tự học liên tục:** Sau khi được triển khai (Deploy), trọng số của mạng Nơ-ron bị đóng băng. Hệ thống hiện tại chưa có cơ chế tự động thu thập phản hồi sai lệch từ luồng dữ liệu mới để tự tinh chỉnh (Fine-tuning) theo thời gian thực.

14.2.0.3 c. Nút thắt hiệu năng trong khâu Suy luận (Inference Latency at Scale)

Toàn bộ luồng suy luận hiện tại (từ tiền xử lý Scaler đến mạng Double DQN) đang được thực thi trên môi trường Python/PyTorch tiêu chuẩn. Đối với quy mô đồ án thử nghiệm, hệ thống hoạt động mượt mà. Tuy nhiên, nếu mở rộng (Scale-up) ra toàn bộ hàng vạn đoạn đường của một siêu đô thị, việc xử lý đồng thời sẽ sinh ra độ trễ (Latency) đáng kể do sự chồng chéo của framework và giới hạn đa luồng của Python (GIL), ảnh hưởng đến tiêu chuẩn của một API cảnh báo thời gian thực.

14.2.0.4 d. Rào cản về Chi phí và Hạ tầng luồng dữ liệu (Data Streaming Infrastructure)

- **Chi phí duy trì:** Việc liên tục kéo dữ liệu (Polling) từ các API bản đồ hoặc duy trì kết nối Kafka/WebSockets để truyền tải khối lượng dữ liệu IoT khổng lồ đòi hỏi băng thông và chi phí máy chủ Đám mây (Cloud) rất lớn. Đây là một rào cản khiến hệ thống khó duy trì tần suất cập nhật dữ liệu ở mức từng giây (real-time streaming) mà phải chấp nhận độ trễ trượt cửa sổ (batch-processing từng 10-15 phút) để tối ưu ngân sách.
- **Độ trễ định tuyến (Routing Delay):** Ứng dụng người dùng (Client-App) hiện tại đề xuất đường đi dựa trên dự báo tại thời điểm truy vấn. Nếu một vụ kẹt xe đột ngột xảy ra ngay khi người dùng đang ở giữa hành trình, thuật toán tái định tuyến động (Dynamic Re-routing) trên ứng dụng vẫn còn độ trễ nhất định để cập nhật lộ trình mới.

14.3 Hướng phát triển trong tương lai

Dựa trên những giới hạn đã được phân tích và đánh giá, nhóm nghiên cứu đề xuất lộ trình nâng cấp hệ thống trong tương lai, tập trung vào ba trụ cột chính: Đột phá kiến trúc AI, Tối ưu hóa hạ tầng vận hành và Mở rộng trải nghiệm người dùng.

14.3.0.1 a. Đột phá Kiến trúc Trí tuệ Nhân tạo (AI Architecture Upgrades)

- **Tích hợp Mạng Nơ-ron Đồ thị (Graph Neural Networks - GNN):** Chuyển đổi mô hình không gian từ dạng chuỗi độc lập sang cấu trúc đồ thị động $G = (V, E)$. Việc áp dụng các kiến trúc tiên tiến như STGCN (Spatio-Temporal Graph Convolutional Networks) hoặc GAT (Graph Attention Networks) sẽ giúp Đặc vụ DQN có tầm nhìn toàn cảnh, tính toán chính xác mức độ ảnh hưởng của "hiệu ứng gợn sóng" (Ripple Effect) khi kẹt xe lan truyền giữa các ngã tư.
- **Chuyển dịch sang Học tăng cường Trực tuyến (Online Reinforcement Learning):** Xây dựng một pipeline vòng kín (Closed-loop) cho phép hệ thống tự động điều chỉnh dự báo với nhân thực tế sau mỗi khung thời gian (ví dụ 15 phút). Agent sẽ sử dụng sai số này để cập nhật nhẹ trọng số (Fine-tuning) theo thời gian thực. Cơ chế "miễn dịch học tập" này sẽ thu hẹp triệt để khoảng cách Sim-to-Real, giúp AI liên tục thích nghi với các biến động mùa vụ hoặc sự thay đổi hạ tầng giao thông mới.

14.3.0.2 b. Tối ưu hóa Hiệu năng và Hạ tầng (MLOps & Infrastructure)

- **Tăng tốc Suy luận (Inference Acceleration) với ONNX/TensorRT:** Để giải quyết nút thắt cổ chai về độ trễ, toàn bộ mạng Q-Network sẽ được xuất sang định dạng ONNX và biên dịch bằng NVIDIA TensorRT. Kỹ thuật lượng tử hóa (Quantization) từ FP32 xuống FP16 hoặc INT8 sẽ được áp dụng nhằm giảm thiểu dung lượng RAM/VRAM và tăng tốc độ xử lý lên hàng chục lần, đáp ứng SLA khắt khe của hệ thống thời gian thực.
- **Áp dụng Điện toán Biên (Edge Computing) và Data Streaming:** Thay vì đẩy toàn bộ gánh nặng tính toán lên Cloud, kiến trúc hệ thống sẽ được phân tán. Một phần các mô hình tiền xử lý hoặc AI nhẹ sẽ được nhúng trực tiếp vào các thiết bị biên (Edge Devices) gần camera/cảm biến ngã tư. Kết hợp với nền tảng truyền phát dữ liệu hiệu năng cao như Apache Kafka, hệ thống sẽ chịu tải tốt hơn khi mở rộng quy mô toàn thành phố.

14.3.0.3 c. Nâng cấp Ứng dụng và Thuật toán Định tuyến (App & Routing)

- **Định tuyến Động trong Hành trình (Dynamic In-trip Re-routing):** Nâng cấp thuật toán tìm đường (như A* hoặc Dijkstra tích hợp trọng số kẹt xe) để có thể "lắng nghe" luồng cảnh báo từ AI theo thời gian thực (ví dụ qua WebSockets). Nếu AI dự báo con đường phía trước sắp vỡ trận, ứng dụng sẽ ngay lập tức tính toán và gợi ý hướng rẽ mới cho người dùng ngay cả khi họ đang di chuyển giữa lộ trình.
- **Tích hợp Dữ liệu Đám đông (Crowdsourcing):** Xây dựng tính năng cho phép người dùng ứng dụng chủ động báo cáo các sự cố bất ngờ (tai nạn, cây đổ, ngập lụt). Nguồn dữ liệu thời gian thực từ cộng đồng này sẽ được kết hợp với dữ liệu IoT để tăng cường độ nhạy và tính chính xác của các cảnh báo thảm họa giao thông.

15 Tài liệu tham khảo

Tài liệu

- [1] UTraffic (BKTraffic). *Hệ thống thông tin giao thông thời gian thực*. [Online]. Available: <https://bktraffic.com/home/>
- [2] Sở Giao thông Vận tải TP.HCM. *Cổng thông tin giao thông Thành phố Hồ Chí Minh*. [Online]. Available: <http://giaothong.hochiminhcity.gov.vn/>
- [3] UBND Thành phố Hồ Chí Minh. *Đề án "Xây dựng Thành phố Hồ Chí Minh trở thành đô thị thông minh giai đoạn 2017 - 2020, tầm nhìn đến năm 2025"*.
- [4] Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd Edition). Wiley.
- [5] Inmon, W. H. (2005). *Building the Data Warehouse* (4th Edition). Wiley.
- [6] Linstedt, D., & Olschimke, M. (2015). *Building a Scalable Data Warehouse with Data Vault 2.0*. Morgan Kaufmann.
- [7] T. Liebig et al., "Dynamic route planning with real-time traffic information," in *IEEE Transactions on Intelligent Transportation Systems*, 2017.
- [8] L. Leonardi et al., "A Data Warehouse for Traffic Analysis in Smart Cities," in *Proceedings of the International Conference on Smart Cities*, 2014.
- [9] V. Jensen et al., "Multidimensional Data Modeling for Spatio-Temporal Data," in *International Journal of Data Warehousing and Mining*, 2004.
- [10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [11] Z. Zhao et al., "T-GCN: A Temporal Graph Convolutional Network for Traffic Prediction," in *IEEE Transactions on Intelligent Transportation Systems*, 2019.
- [12] B. M. Williams and L. A. Hoel, "Modeling and forecasting vehicular traffic flow as a seasonal ARIMA process: Theoretical basis and empirical results," *Journal of Transportation Engineering*, 2003.
- [13] Tài liệu Phân tích nghiệp vụ (Internal Requirement Document). *Các nhóm nghiệp vụ quản lý và vận hành giao thông đô thị*.
- [14] A. Chen, H. Yang, H. K. Lo, and W. H. Tang (2001). *Reliability-based routing in capacity constrained networks*. Journal of Transportation Engineering.
- [15] P. M. Bartier and C. P. Keller (1996). *Multivariate interpolation to incorporate thematic surface data using inverse distance weighting*. Computers & Geosciences.